ETAS RTA-BSW v12.7.0

RTA-Mem Stack Reference Guide

www.etas.com

Copyright

The data in this document may not be altered or amended without special notification from ETAS GmbH. ETAS GmbH undertakes no further obligation in relation to this document. The software described in it can only be used if the customer is in possession of a general license agreement or single license. Using and copying is only allowed in concurrence with the specifications stipulated in the contract.

Under no circumstances may any part of this document be copied, reproduced, transmitted, stored in a retrieval system or translated into another language without the express written permission of ETAS GmbH.

© Copyright 2025 ETAS GmbH, Stuttgart

The names and designations used in this document are trademarks or brands belonging to the respective owners.

ETAS RTA-BSW 12.7.0 - RTA-Mem Stack Reference Guide R01 EN – 05.2025

Contents

1 EEPROM Abstraction . . . 7
- 1.1 Introduction . . . 7
- 1.2 Supported Features . . . 8
- 1.3 Extensions . . . 8
- 1.4 Limitations . . . 18
- 1.5 Exclusions . . . 19
- 1.6 Deviations . . . 19
- 1.7 API Definition . . . 21
- 1.8 Configuration Advice . . . 22
- 1.9 Integration Advice . . . 22
- 1.10 Hardware Requirements . . . 24

2 Flash EEPROM Emulation . . . 25
- 2.1 Introduction . . . 25
- 2.2 Supported Features . . . 26
- 2.3 Extensions . . . 33
- 2.4 Limitations . . . 69
- 2.5 Exclusions . . . 70
- 2.6 Deviations . . . 70
- 2.7 API Definition . . . 75
- 2.8 Configuration Advice . . . 75
- 2.9 Integration Advice . . . 83
- 2.10 Hardware Requirements . . . 87

3 MemAcc . . . 88
- 3.1 Introduction . . . 88
- 3.2 Supported Features . . . 89
- 3.3 Extensions . . . 90
- 3.4 Limitations . . . 133
- 3.5 Exclusions . . . 134
- 3.6 Deviations . . . 135
- 3.7 API Definition . . . 136
- 3.8 Configuration Advice . . . 137
- 3.9 Integration Advice . . . 143
- 3.10 Hardware Requirements . . . 147

4 Memory Abstraction Interface . . . 148
4.1 Introduction . . . 148
4.2 Supported Features . . . 149
4.3 Extensions . . . 150
4.4 Limitations . . . 154
4.5 Exclusions . . . 154
4.6 Deviations . . . 154
4.7 API Definition . . . 155
4.8 Configuration Advice . . . 156
4.9 Integration Advice . . . 156
4.10 Hardware Requirements . . . 159
5 NVRAM Manager . . . 160
5.1 Introduction . . . 160
5.2 Supported Features . . . 162
5.3 Extensions . . . 187
5.4 Limitations . . . 245
5.5 Exclusions . . . 249
5.6 Deviations . . . 250
5.7 API Definition . . . 284
5.8 Configuration Advice . . . 285
5.9 Integration Advice . . . 303
5.10 Hardware Requirements . . . 308
6 rba_EaFs2 . . . 309
6.1 Introduction . . . 309
6.2 Supported Features . . . 309
6.3 Extensions . . . 310
6.4 Limitations . . . 310
6.5 Exclusions . . . 310
6.6 Deviations . . . 310
6.7 API Definition . . . 310
6.8 Configuration Advice . . . 310
6.9 Integration Advice . . . 311
6.10 Hardware Requirements . . . 312
7 rba_FeeFs1x . . . 313

7.1 Introduction . . . 313
7.2 Supported Features . . . 313
7.3 Extensions . . . 317
7.4 Limitations . . . 317
7.5 Exclusions . . . 317
7.6 Deviations . . . 317
7.7 API Definition . . . 318
7.8 Configuration Advice . . . 318
7.9 Integration Advice . . . 318
7.10 Hardware Requirements . . . 319
8 rba_FeeFs2 . . . 320
8.1 Introduction . . . 320
8.2 Supported Features . . . 321
8.3 Extensions . . . 322
8.4 Limitations . . . 322
8.5 Exclusions . . . 323
8.6 Deviations . . . 323
8.7 API Definition . . . 323
8.8 Configuration Advice . . . 323
8.9 Integration Advice . . . 325
8.10 Hardware Requirements . . . 328
9 rba_FeeFs3 . . . 329
9.1 Introduction . . . 329
9.2 Supported Features . . . 329
9.3 Extensions . . . 330
9.4 Limitations . . . 330
9.5 Exclusions . . . 331
9.6 Deviations . . . 331
9.7 API Definition . . . 333
9.8 Configuration Advice . . . 333
9.9 Integration Advice . . . 334
9.10 Hardware Requirements . . . 336
10 rba_MemLib . . . 337
10.1 Introduction . . . 337

10.2 Supported Features . . . 337
10.3 Extensions . . . 338
10.4 Limitations . . . 352
10.5 Exclusions . . . 352
10.6 Deviations . . . 352
10.7 API Definition . . . 352
10.8 Configuration Advice . . . 352
10.9 Integration Advice . . . 353
10.10 Hardware Requirements . . . 356
11 rba_MemWrprMemAcc . . . 357
11.1 Introduction . . . 357
11.2 Supported Features . . . 357
11.3 Extensions . . . 360
11.4 Limitations . . . 362
11.5 Exclusions . . . 362
11.6 Deviations . . . 362
11.7 API Definition . . . 362
11.8 Configuration Advice . . . 362
11.9 Integration Advice . . . 362
11.10 Hardware Requirements . . . 363
12 Glossary . . . 364

1 EEPROM Abstraction

Module name	Ea
Package	RTA-MEM
AUTOSAR Module ID	40
AUTOSAR Version	23-11

1.1 Introduction

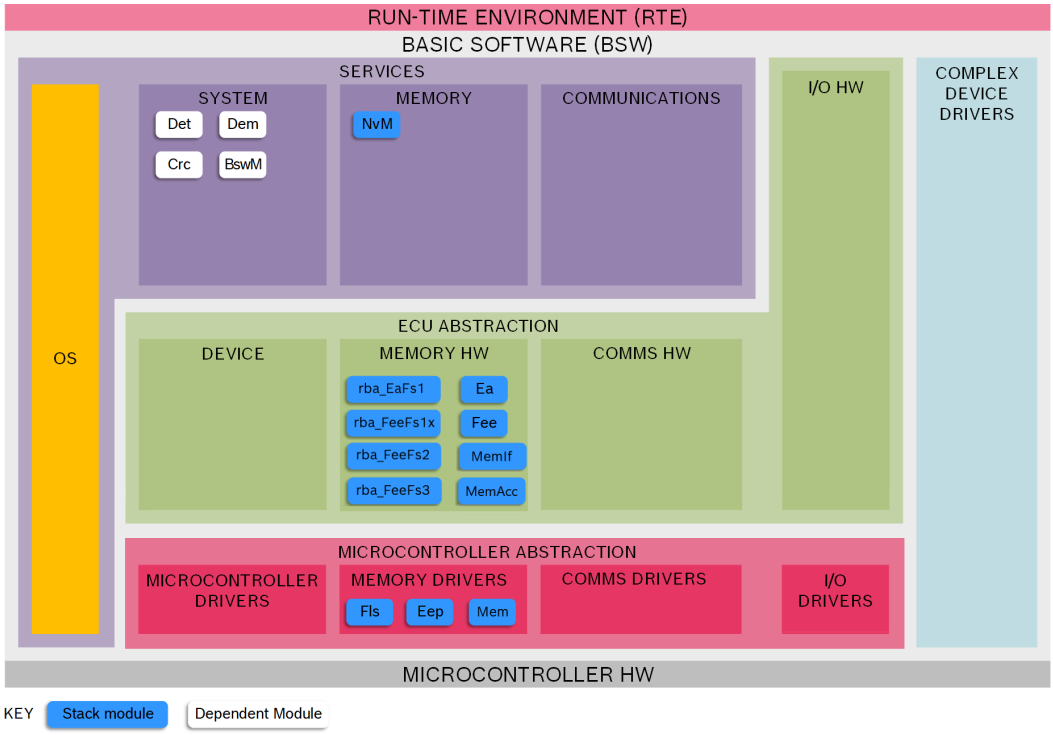


Figure 1.1.Memory Stack

The EEPROM Abstraction module (Ea) is a basic software module in the Memory Hardware Abstraction layer, and it forms part of the RTA-MEM stack.

Ea provides mechanisms for accessing micro-controller internal and external EEPROM devices, and it organizes and routes different tasks for reading and writing data to and from internal/external EEPROM.

Ea abstracts from the device-specific addressing scheme and segmentation, and it provides the upper layers with a virtual addressing scheme and segmentation, and a "virtually unlimited" number of erase cycles.

Ea receives orders sequentially from NvM/MemIf (without queues), checks for block inconsistency, calculates the checksum, and routes the orders to the EEPROM driver.

Module Configuration

This module can be generated by the RTA-BSW Configuration Generator.

For any unsupported automatic configuration, the module can be manually configured by the user.

1.2 Supported Features

The following features are supported by the Ea module:

- Reading of data/blocks out of the EEPROM to a user buffer.
- Writing of data/blocks to an EEPROM.
- Invalidation of blocks.
- Cancellation of running orders.
- Erasing of immediate data.

1.2.1 Migration (via rba_EaFs2)

An additional Migration feature, which supports layout changes after flashing, is provided through the Flash EEPROM Abstraction 2.0 module (rba_EaFs2).

The data that is to be stored on EEPROM is organized in blocks, and these blocks have configurable settings that are set by the code generator before compiling.

During the startup procedure after every flash, the block configuration is checked against the real layout on EEPROM and, if any difference is detected, a layout "migration" is necessary.

rba_EaFs2 is a sub-component of the Ea module, and an implementation variant that adds the aforementioned migration feature to the standard features available in Ea.

When the EaRbFsSelection configuration parameter (in /Ea/EaGeneral) is configured to FS2, the Ea module's source code is disabled and rba_EaFs2 provides the implementation of the Ea functions.

See the rba_EaFs2 section for more details.

1.3 Extensions

1.3.1 Configuration Extensions

The following configuration parameters are available in version v12.7.0 in addition to those defined in AR v22-11

1.3.1.1 Ea/EaBlockConfiguration/EaRbBlockPersistentId

Short Name	EaRbBlockPersistentId
Description	BlockIndex of corresponding NvM-Block, Link between NvM-Block and Ea-Block: min: 1 max: 65534
Multiplicity	1..1
Value Range	1..65534
Default Value	-

Type	ECUC-INTEGER-PARAM-DEF

1.3.1.2 Ea/EaBlockConfiguration/EaRbCallbackEnd

Short Name	EaRbCallbackEnd
Description	Name of callback function which is called after successful finished order.
Introduction	Possible values are: Name of the CallBack-Function or EA_NULL if no Call-Back-Function is available
Multiplicity	0..1
Default Value	-
Type	ECUC-STRING-PARAM-DEF

1.3.1.3 Ea/EaBlockConfiguration/EaRbCallbackError

Short Name	EaRbCallbackError
Description	Name of callback function which is called if order couldn't be finished successful.
Introduction	Possible values are: Name of the CallBack-Function or EA_NULL if no Call-Back-Function is available
Multiplicity	0..1
Default Value	-
Type	ECUC-STRING-PARAM-DEF

1.3.1.4 Ea/EaBlockConfiguration/EaRbCallbackStart

Short Name	EaRbCallbackStart
Description	Name of callback function which is called before job is started.
Introduction	Possible values are: Name of the CallBack-Function or EA_NULL if no Call-Back-Function is available
Multiplicity	0..1
Default Value	-
Type	ECUC-STRING-PARAM-DEF

1.3.1.5 Ea/EaBlockConfiguration/EaRbDeviceIndex

Short Name	EaRbDeviceIndex
Description	Rb special device index for different EEPROM devices.
Introduction	Range: 1...254 for Ea.
Multiplicity	0..1
Value Range	1..254
Default Value	-

Type	ECUC-INTEGER-PARAM-DEF

1.3.1.6 Ea/EaBlockConfiguration/EaRbFirstReadDataMigration

Short Name	EaRbFirstReadDataMigration
Long Name	First Read Data Migration
Description	This tag activates or deactivates the First Read Data Migration feature.
Introduction	False: First Read Data Migration is disabled (default).True: First Read Data Migration is enabled.
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

1.3.1.7 Ea/EaBlockConfiguration/EaRbOrigin

Short Name	EaRbOrigin
Long Name	Responsible module for this configuration.
Description	Only NVM supported
Multiplicity	0..1
Default Value	-
Enumeration Literals	NVM
Type	ECUC-ENUMERATION-PARAM-DEF

1.3.1.8 Ea/EaBlockConfiguration/EaRbSecLevel

Short Name	EaRbSecLevel
Description	Security Level
Introduction	Range: 1 = Block has one copy in Eeprom; 2 = Block has two identical copies in Eeprom for security
Multiplicity	1..1
Value Range	1..2
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

1.3.1.9 Ea/EaBlockConfiguration/EaRbSurvival

Short Name	EaRbSurvival
Long Name	Block survival attribute.
Description	This tag changes the migration behaviour for this block in case it is not within the layout.

Introduction	true: Migration will not delete this block, even if it is not part of the layout. false: Migration will delete this block if it is not part of the layout.
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

1.3.1.10 Ea/EaGeneral/EaBufferAlignmentValueRef

Short Name	EaBufferAlignmentValueRef
Long Name	Buffer Alignment
Description	Parameter determines the alignment of the start address that Ea buffers need to have.Value shall be inherited from MemAccBufferAlignmentValue. This parameter is not supported by the Ea. Instead the MemMap section EA_START_SEC_VAR_SECURED_CACHED_RAM_ALIGNED_CLEARED_UNSPECIFIED can be used to specifically align Ea buffers containing data which is forward to lower layers.
Multiplicity	0..0
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

1.3.1.11 Ea/EaGeneral/EaIndex

Short Name	EaIndex
Description	Specifies the InstanceId of this module instance. If only one instance is present it shall have the Id 0.
Multiplicity	1..1
Value Range	0..254
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

1.3.1.12 Ea/EaGeneral/EaPushedEepSize

Short Name	EaPushedEepSize
Description	The EEPROM size in bytes which is pushed from EEPROM driver. In case of Ea integrated with Eep this parameter value will be pushed from Eep to Ea automatically.In case of Ea integrated with 3rd party Eep driver then this value should be configured manually.This parameter correspond to the Eep driver Autosar Configuration parameter “EepSize”

Multiplicity	0..1
Value Range	0..4294967295
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

1.3.1.13 Ea/EaGeneral/EaRbActSurvivalEval

Short Name	EaRbActSurvivalEval
Long Name	Activation of the “partial afterburner”.
Description	Pre-processor switch to enable or disable an additional block specific migration for unknown blocks with an active survival setting (“EaRbSurvival”) in the medium.
Introduction	true: Feature enabled (“block specific afterburner” on for blocks with active survival setting) false: Feature disabled (“block specific afterburner” off for blocks with active survival setting). default: true Hint: If activated, this block will be overtaken independent from the “EaRbActUnknownBlockMig” setting (“general afterburner”). To delete all unknown survival blocks, it should be deactivated.
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

1.3.1.14 Ea/EaGeneral/EaRbActUnknownBlockMig

Short Name	EaRbActUnknownBlockMig
Long Name	Activation of the “global afterburner”.
Description	Pre-processor switch to enable or disable an additional migration for any unknown block in the medium. Only the CB (Customer Block) is allowed to enable this feature, because this is because this is affecting the thresholds of migration start. It must be deactivated (default) for the ASW!
Introduction	true: Feature enabled (“general afterburner” on for all unknown blocks) false: Feature disabled (“general afterburner” off for all unknown blocks). default: false. Hints: If activated, all unknown blocks will be overtaken independent from the “RbSurvival” setting (“block specific afterburner”). Only the CB (Customer Block) is allowed to enable this feature, because this is affecting the thresholds of migration start.It must be deactivated (default) for the ASW!

Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

1.3.1.15 Ea/EaGeneral/EaRbDevRuntimeErrorDetect

Short Name	EaRbDevRuntimeErrorDetect
Description	Pre-processor switch to enable and disable development runtime error detection. In case the parameter is not configured, it shall be enabled
Introduction	true: Development runtime error detection enabled. false: Development runtime error detection disabled.
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

1.3.1.16 Ea/EaGeneral/EaRbEnterStopModeApi

Short Name	EaRbEnterStopModeApi
Long Name	Activation of EnterStopMode API
Description	Pre-processor switch to enable or disable Stop Mode ("Stop Mode", "Write address based", "Read address based").
Introduction	true: Add-ons for CC-AS enabled. false: Add-ons for CC-AS disabled. default: false.
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

1.3.1.17 Ea/EaGeneral/EaRbErrorHandlingCallback

Short Name	EaRbErrorHandlingCallback
Long Name	Error handling callback in Ea module
Description	Name of the module-specific callback routine which shall be invoked if critical error in Ea occurs, for example migration not successful or not possible, so that some block cann't been written any more.
Multiplicity	0..1
Default Value	-
Type	ECUC-FUNCTION-NAME-DEF

1.3.1.18 Ea/EaGeneral/EaRbFastModeCCPS

Short Name	EaRbFastModeCCPS
Long Name	Activation of CC-PS specific Fast Mode
Description	Pre-processor switch to Enable or Disable CC-PS Specific Fast Mode feature for writing Crash Data. It must be activated only for CC-PS productline. Eep_ Mode should be Set to required MemIf_ModeType by calling Ea_SetMode function.
Introduction	True : CC-PS Fast mode enabled. False: CC-PS Fast mode disabled. default:False.
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

1.3.1.19 Ea/EaGeneral/EaRbFsSelection

Short Name	EaRbFsSelection
Long Name	Selection of Filesystem
Description	Specifies the actual layout of the data on EEPROM
Introduction	If left unconfigured the component will not be activated.
Multiplicity	0..1
Default Value	-
Enumeration Literals	EaFs2
Type	ECUC-ENUMERATION-PARAM-DEF

1.3.1.20 Ea/EaGeneral/EaRbMainFunctionScheduling

Short Name	EaRbMainFunctionScheduling
Long Name	MainFunction Scheduling Type
Description	Selection on type of scheduling required for the abstraction layer
Multiplicity	0..1
Default Value	EA_RB_MAINFUNC_SCHEDULING_RTE_TIMED
Enumeration Literals	EA_RB_MAINFUNC_SCHEDULING_RTE_BACKGROUND, EA_RB_MAIN-FUNC_SCHEDULING_RTE_TIMED, EA_RB_MAINFUNC_SCHEDULING_ MANUAL
Type	ECUC-ENUMERATION-PARAM-DEF

1.3.1.21 Ea/EaGeneral/EaRbMemAccAddressArea

Short Name	EaRbMemAccAddressArea
Long Name	Reference to MemAccAddressAreaConfiguration
Description	Reference to the MemAccAddressAreaConfiguration.
Introduction	This must be configured in order to use Ea with MemAcc. If this is not configured, Eep is used.
Multiplicity	0..1
Default Value	-
Type	ECUC-SYMBOLIC-NAME-REFERENCE-DEF

1.3.1.22 Ea/EaGeneral/EaRbMigEraseBufferSize

Short Name	EaRbMigEraseBufferSize
Description	This parameter defines the size of an internal array (of uint32) used during migration. The array holds the address of the blocks to be removed from the Eeprom. This array is introduced to reduce the Initialization time during migration. Larger the size of the array, faster is the initialization time. For optimal performance the size of this array must be equal to or more than the number of blocks that are removed from old layout to new layout.
Multiplicity	0..1
Value Range	1..65535
Default Value	50
Type	ECUC-INTEGER-PARAM-DEF

1.3.1.23 Ea/EaGeneral/EaSetModeSupported

Short Name	EaSetModeSupported
Description	Compiler switch to enable/disable the 'SetMode' functionality of the EA module.
Introduction	TRUE: SetMode functionality supported / code present, FALSE: SetMode functionality not supported / code not present. Note: This configuration setting has to be consistent with that of all underlying EEPROM device drivers.
Multiplicity	1..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

1.3.1.24 Ea/EaPublishedInformation/EaMaximumBlockingTime

Short Name	EaMaximumBlockingTime
Description	The maximum time the EA module API routines shall be blocked (delayed) by internal operations.
Introduction	Note: Internal operations in that case means operations that are not explicitly invoked from the upper layer module but need to be handled for proper operation of this module or the underlying memory driver.
Multiplicity	1..1
Value Range	0..Inf
Default Value	-
Type	ECUC-FLOAT-PARAM-DEF

1.3.2 Additional Services

The following API are available in version v12.7.0 in addition to those defined in AR v22-11

1.3.2.1 Ea_Rb_EndInit

Syntax	`Ea_Rb_EndInit`
Description	Service To indicate the end of the system's init phase and therefore allow the Ea to start migration if necessary.

1.3.2.2 Ea_Rb_BlockMaintenance

Syntax	`Ea_Rb_BlockMaintenance(uint16 BlockNumber)`
Parameter	`Blocknumber` Number of logical block, also denoting start address of that block in EEPROM.
Returns	E_OK - If the requested job has been accepted by the module. E_NOT_OK - Possible reasons are that the given BlockNumber is invalid, or the module has not yet been initialized.
Description	Takes the redundant block number and repairs the corresponding data of both two copies.

1.3.2.3 Ea_Rb_MigrationStatus

Syntax	`Ea_Rb_MigrationStatus`

Returns	EA_RB_MIGRATION_NOTNEEDED - The configured layout, and the layout present in EEPROM Hardware, is the same. EA_RB_MIGRATION_RUNNING - The configured layout, and the layout present in EEPROM Hardware is different, and layout change is ongoing. EA_RB_MIGRATION_FINISHED - Migration is completed successfully. EA_RB_MIGRATION_ERROR - Migration did not complete successfully.
Description	Returns the Migration status.

1.3.2.4 Ea_Rb_GetAdapterJobResult

Syntax	`Ea_Rb_GetAdapterJobResult(void)`
Return Values	`MEMIF_JOB_OK` The last job has finished successfully. `MEMIF_JOB_PENDING` The last job is waiting for execution or is currently being executed. `MEMIF_JOB_FAILED` The last job did not finished successfully.
Description	Queries the result of the last accepted job issued by the upper layer software.

1.3.2.5 Ea_Rb_EnterStopMode

Syntax	`Ea_Rb_EnterStopMode(void)`
Description	Service to enter stop mode.

1.3.2.6 Ea_Rb_ForcedMigration

Syntax	`Ea_Rb_ForcedMigration(void)`
Description	Service to force a migration.

1.3.2.7 Ea_Rb_MainFunctionAndDependency

Syntax	`void Ea_Rb_MainFunctionAndDependency(void)`
Description	Invokes Ea MainFunction and its dependencies.

1.3.2.8 Ea_Rb_GetDetailedBlkInfo

Syntax	`MemIf_Rb_DetailedBlkInfo_ten Ea_Rb_GetDetailedBlkInfo(uint16 blockNumber_u16)`
Parameter	`blockNumber_u16` BlockNumber.

Returns	EA_RB_DETAILED_BLK_INFO_INIT_E - Block was never read. EA_RB_DETAILED_BLK_INFO_INCONSISTENT_MISSING_E - Valid block (header) was not found on the medium. EA_RB_DETAILED_BLK_INFO_INCONSISTENT_DATA_E - Data CRC of block is wrong. EA_RB_DETAILED_BLK_INFO_INCONSISTENT_LENGTH_E - Length is not supported (if a greater length is requested, the excessive buffer is filled with erase pattern).
Description	Returns detailed error information of a specific block.

1.4 Limitations

1.4.1 Ea Block Configuration

All block configuration in Ea will be ignored (SWS ECUC_Ea_00130). Block configuration should be set in NvM. Any references from NvM block configurations to Ea/Fee block configurations are superfluous and effectively removed from NvM's ECU parameter definition file (by setting the upper multiplicity to zero).

1.4.2 Dem Events

The triggering of Dem events for production errors has not been implemented in Ea because block-independent information about such errors is not generally of any use for production error tracking. NvM/Ea users should trigger appropriate application-specific Dem events wherever applicable. Ea provides block-specific flags indicating comparable error conditions, as well as block-independent summary information on these errors, for debugging purposes.

1.4.3 EaBufferAlignmentValueRef and EaMinimumReadPageSize

The parameters EaBufferAlignmentValueRef and EaMinimumReadPageSize are not supported. Alternatively the MemMap section EA_START_SEC_VAR_SECURED_CACHED_RAM_ALIGNED_CLEARED_UNSPECIFIED can be used to specifically align Ea buffers containing data which is forwarded to lower layers. Affected SWS items: SWS_Ea_00197, SWS_Ea_00198, ECUC_Ea_00136, ECUC_Ea_00135.

1.4.4 Ea_EraseImmediateBlock

The Ea_EraseImmediateBlock function doesn't currently check whether or not the ordered block has been configured with immediate data (EaImmediateData set to TRUE) because NvM doesn't currently support immediate data. (SWS_Ea_00065).

1.4.5 Ea Interaction with Lower Layer Drivers

Interaction of Ea with the lower layer drivers is done by cyclic polling in Ea's main function. Autosar suggests a possible alternative of using a callback interface between NvM and Ea, and Ea and low level drivers (SWS_Ea_00114 and others). This alternative implementation has not been made available in Ea because thor-

oughtesting indicates that it is unlikely to bring a considerable performance boost.

1.4.6 EA_VIRTUAL_PAGE_SIZE

The configuration of EA_VIRTUAL_PAGE_SIZE (SWS_Ea_00005) currently has currently no effect. All the blocks have been addressed with ECC check unit (4 Bytes).

1.4.7 Polling Mode

Polling Mode (SWS ECUC_Ea_00123) is not currently supported. The API interface from Eep has no polling mode, so it is not necessary to configure EA_POLLING_MODE. External EEPROM via SPI is currently supported.

1.4.8 rba_EaFs1

Ea (high-level) is no longer backwards compatible with rba_EaFs1.

1.5 Exclusions

The following configuration parameters from AR v22-11 are not supported in version v12.7.0.

- Ea/EaBlockConfiguration/EaBlockNumber
- Ea/EaGeneral/EaBufferAlignmentValue
- Ea/EaGeneral/EaMinimumReadPageSize

1.6 Deviations

1.6.1 Configuration Deviations

Configuration parameters which differ from the AR v22-11 specification are listed below.

1.6.1.1 Ea

[REMOVED] Introduction	The module shall abstract from the device specific addressing scheme and segmentation and provide the upper layers with a virtual addressing scheme and segmentation as well as a 'virtually' unlimited number of erase cycles.

1.6.1.2 Ea/EaBlockConfiguration

[RTA-BSW] Description	Configuration of block specific parameters for the EEPROM Abstraction module.
[AUTOSAR] Description	Configuration of block specific parameters for the EEPROM abstraction module.
[RTA-BSW] Upper Multiplicity	65535

[AUTOSAR] Upper Multiplicity	*

1.6.1.3 Ea/EaBlockConfiguration/EaDeviceIndex

[RTA-BSW] Description	Device index (handle).
[AUTOSAR] Description	Reference to the device this block is stored in.
[RTA-BSW] Introduction	Not used.
[AUTOSAR] Introduction	This reference is mutually exclusive to EaMemAccAddressArea.
[RTA-BSW] Upper Multiplicity	0
[AUTOSAR] Upper Multiplicity	1

1.6.1.4 Ea/EaBlockConfiguration/EaMemAccAddressArea

[RTA-BSW] Description	Reference to the MemAccAddressAreaConfiguration. This reference is mutually exclusive to EaDeviceIndex
[AUTOSAR] Description	Reference to the MemAccAddressAreaConfiguration. Tags: atp.Status=draft
[RTA-BSW] Introduction	Reference to the MemAccAddressAreaConfiguration. This reference is mutually exclusicve to EaDeviceIndex
[AUTOSAR] Introduction	This reference is mutually exclusive to EaDeviceIndex.

1.6.1.5 Ea/EaBlockConfiguration/EaNumberOfWriteCycles

[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1

1.6.1.6 Ea/EaGeneral

[RTA-BSW] Description	Container for general parameters. These parameters are not specific to a block.
[AUTOSAR] Description	General configuration of the EEPROM abstraction module. This container lists block independent configuration parameters.

1.6.1.7 Ea/EaGeneral/EaDevErrorDetect

[RTA-BSW] Description	Pre-processor switch to enable and disable development error detection.
[AUTOSAR] Description	Switches the development error detection and notification on or off.
[RTA-BSW] Introduction	true: Development error detection enabled. false: Development error detection disabled.

[AUTOSAR] Introduction	* true: detection and notification is enabled. * false: detection and notification is disabled.
[REMOVED] Default Value	false

1.6.1.8 Ea/EaGeneral/EaMainFunctionPeriod

[RTA-BSW] Description	Call cycle time of the EEPROM Abstraction layer's main function. Unit: [s]
[AUTOSAR] Description	The period between successive calls to the main function in seconds.
[RTA-BSW] Maximum Value	1
[AUTOSAR] Maximum Value	INF

1.6.1.9 Ea/EaGeneral/EaPollingMode

[RTA-BSW] Introduction	true: Polling mode enabled, callback functions (provided to Eep module) disabled. false: Polling mode disabled, callback functions (provided to Eep module) enabled.
[AUTOSAR] Introduction	true: Polling mode enabled, callback functions (provided to EEP module) disabled. false: Polling mode disabled, callback functions (provided to EEP module) enabled.

1.6.1.10 Ea/EaGeneral/EaVersionInfoApi

[REMOVED] Default Value	false

1.7 API Definition

Table 1.44.Ea Supported API functions

`Ea_Init`	Initializes the EEPROM abstraction module.
`Ea_SetMode`	Sets the mode.
`Ea_Read`	Reads Length bytes of block [Blocknumber] at offset [BlockOffset] into the buffer DataBufferPtr.
`Ea_Write`	Writes the contents of the DataBufferPtr to the block.
`Ea_Cancel`	Cancels the ongoing asynchronous operation.
`Ea_GetStatus`	Returns the status.
`Ea_GetJobResult`	Returns the job result.
`Ea_InvalidateBlock`	Invalidates the block.
`Ea_GetVersionInfo`	Gets the version information of this module.
`Ea_EraseImmediateBlock`	Erases the block.

`Ea_MainFunction`	Handles the requested jobs and the internal management operations.

For further information on these functions, please refer to AUTOSAR_SWS_EEPROMAbstraction.pdf

1.8 Configuration Advice

> **NOTE**
>
> Before configuring this module, please note that some of the default values used by ConfGen to produce the default ECU Configuration can be changed, and this is done by either adding a Conf-Gen Enhancer Mapping (CEM) or by adding a rule to an *algo.properties* file. For more information, see the "Configuration Generation" section of the RTA-CAR User Guide.

1.8.1 Ea_Rb_BlockMaintenance

The Ea_Rb_BlockMaintenance function detects corrupted data, and copies and repairs it with valid data from another copy. Nothing is changed if both copies are corrupted, or if both copies contain the same data. Ea_Rb_BlockMaintenance should be executed asynchronously within Ea's main function.

1.9 Integration Advice

1.9.1 Init Functions

The initialisation of Ea is achieved through Ea_Init and Ea_Rb_Endinit, as described in the sub-sections below.

1.9.1.1 Ea_Init

Ea_Init is called at system startup, and this is typically done by invocation from one of EcuM's "DriverInit" call-outs. Ea_Init has to be called after the EEPROM driver initialization routine (Eep_Init) and before any user initialization routines are executed (e.g. NvM_Init, NvM_ReadBlock etc.).

1.9.1.2 Ea_Rb_EndInit

Ea_Rb_EndInit is called at the end of initialisation, and after any user routines have been executed and completed (e.g. NvM_Init, NvM_ReadBlock) to ensure that a migration isn't started during initialisation. Ea_Rb_EndInit should run after any fast startup has passed (e.g. after calling NvM_ReadAll).

Ea_Rb_EndInit should immediately follow Ea_Init if the system uses the initialization function inside a power-down service (e.g. Shutdown), otherwise migration will not be formed inside this system state. Note that a Service Id is given for this function to avoid a validation error.

Ea_Rb_EndInit is not applicable for bootmode - a migration will be started only after the call of the Ea_Rb_EndInit function.

1.9.2 DeInit Functions

The Ea module does not have a DeInit function.

1.9.2.1 Migration during Shutdown

As described in the Supported Features section, Ea (through rba_EaFs2) includes an additional Migration feature, which supports layout changes after flashing.

During shutdown, and before calling NvM_WriteAll, unless there is a strict timing restriction you should first call Ea_Rb_MigrationStatus to check that a migration is not still running. See the rba_EaFs2 Integration Advice for more details.

1.9.2.2 MemIf and Ea with 3rd Party MCAL Internal Eep Driver

There is no SPI driver between the Ea and Eep modules when using a third-party MCAL internal Eep driver.

This leads to a compilation issue, in case the internal EEPROM is used. The Spi_-MainFunction_Handling function should be stubbed as integration code and return nothing to avoid an error. For example, in Eep.h:

```
extern void Spi_MainFunction_Handling(void);
```

And in Eep.c:

```
void Spi_MainFunction_Handling(void)
{
    return;
}
```

1.9.3 Scheduled Functions

The EaRbMainFunctionScheduling parameter is provided to configure the type of scheduling required for Ea_MainFunction. The following options are available:

- **RTE Timed Task**: The EaMainFunctionPeriod parameter is used to generate a timing event in the BSWMD file. Ea_MainFunction is provided as a schedulable entity, and a Timing Event is generated with the interval defined by EaMainFunctionPeriod. This is the default if EaRbMainFunctionScheduling is not configured.
- **RTE Background Task**: Ea_MainFunction is provided as a schedulable entity with a background event. With this option, EaMainFunctionPeriod should be configured as 0.0.
- **Manual Task**: Entities and Events are not generated with BSWMD. With this option, EaMainFunctionPeriod should be configured as 0.0.

If Ea_MainFunction is schedulable, it should be called periodically, and it should ideally be scheduled in the same task or container as the main functions of NvM, MemIf and Fee, and before Eep_MainFunction.

Since all the main activities of Ea are done as part of NvM_MainFunction, Ea's

performance directly correlates to the invocation frequency of this function.

It is also important that there is smooth transition while going to shutdown (from cyclic execution to linear execution) - Ea_MainFunction should not be pre-empted by another operation before control goes to shutdown. A solution is to use Shutdown 2.0 or greater, and to schedule Ea_MainFunction in the same core where the task OS_ReInitModeM_Task (which decides to go to shutdown) is scheduled.

Scheduling for Ea is set by the parameter EaRbMainFunctionScheduling = EA_RB_MAINFUNC_SCHEDULING_MANUAL or EA_RB_MAINFUNC_SCHEDULING_RTE_TIMED or EA_RB_MAINFUNC_SCHEDULING_BACKGROUND. Scheduling mode RTE_TIMED will configure Ea_MainFunction to be periodically called by RTE with a time period set by the parameter EaMainFunctionPeriod. This parameter must be set if timed scheduling is used. Scheduling mode BACKGROUND will configure Ea_MainFunction to be called by RTE in the background task. Scheduling mode MANUAL will not configure any calling of Ea_MainFunction with any RTE events or entries. Ea_MainFunction will have to be manually called by the integrator.

1.9.4 OS Resources

The number of the active OS resources, and their usage, depends on the user configuration of the AUTOSAR System and Basic Software. It is the user's responsibility to ensure that the `ENTER` and `EXIT` resource API are implemented to disable/enable interrupts as required. Alternatively they must be configured in the OS with the correct task/ISR allocation.

The following is a list of all exclusive areas that this module might use regardless of the user configuration. It is the job of the integrator to ensure that the implementation of these exclusive areas is correctly defined. This implementation can be done using the RTE exclusive area configuration wizard or by manual implementation of the corresponding BSW API calls to ensure exclusivity.

The Ea module uses the following OS resources:

- Ea_Order

1.10 Hardware Requirements

Information relating to the hardware resources required by this module is not currently available.

2 Flash EEPROM Emulation

Module name	Fee
Package	RTA-MEM
AUTOSAR Module ID	21
AUTOSAR Version	23-11

2.1 Introduction

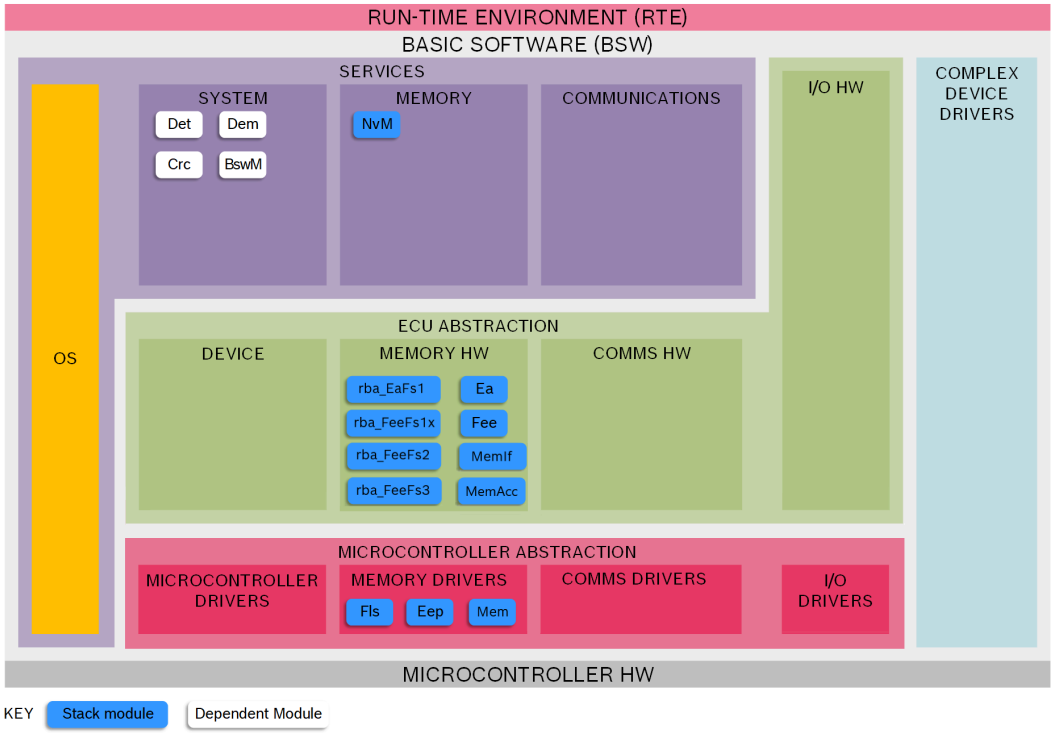


Figure 2.1.Memory Stack

The Flash EEPROM Emulation module (Fee) is a basic software module in the Memory Hardware Abstraction layer, and it forms part of the RTA-MEM stack.

Fee emulates the functionality of a serial EEPROM inside a flash memory by organizing and saving different types of data to non-volatile memory media.

This non-volatile memory can be DFLASH in some controllers; on external controllers it can be just a normal FLASH and in some controllers it is PCM. The term *Data-NvM* is used to refer to all such areas where the Fee driver is used to manage data.

The data to be stored is organized in blocks with configurable characteristics. Unlike with serial EEPROM, which overwrites already existing blocks, Fee attaches new blocks to free pages (the smallest programmable HW unit) inside a flash sector (the smallest erasable HW unit) of an existing block.

This means that there is no fixed physical address location for emulation data. If

the amount of data reaches a certain level, data that is only available inside the "oldest sector" is copied to the latest ("active") sector via a process called "sector reorganization". The oldest sector can then be erased.

Fee can also contribute to an extension to the lifespan of the ECU. An increase in write/erase cycles leads to the eventual destruction of the oxide layer of the transistor's floating gate, which in turn leads to a reduction in data retention of the flash cells. Fee's emulation automatically reduces the unnecessary programming access to these cells, and its efficient utilization of available memory areas allows for a maximum ECU lifespan.

In summary, Fee's emulation principle offers several advantages compared to a serial EEPROM, including reduced HW costs and improved access times.

Module Configuration

This module can be generated by the RTA-BSW Configuration Generator.

For any unsupported automatic configuration, the module can be manually configured by the user.

2.1.1 Fee Variants

The Flash EEPROM Emulation sub-components rba_FeeFs1x, rba_FeeFs2 and rba_FeeFs3 are implementation variants of the Fee module that enable further features. See Chapter 7, Chapter 8 and Chapter 9 respectively for more details.

2.2 Supported Features

Fee supports the following key features (for configured blocks), each of which is described in more detail in the sub-sections below.

- Job Processing and Result.
- Setting/Getting the Job Mode.
- Writing blocks of user data to a non-volatile memory.
- Reading blocks of data out of the flash to a user buffer.
- Invalidation of blocks by writing to the flash with a block length of zero.
- Block Maintenance.
- Stress Measurement.

Note: See also rba_FeeFs1x for the Sector Reorganization feature.

2.2.1 Job Processing and Result

The user's job processing is done via the Fee_MainFunction, and the result can be checked via the Fee_GetJobResult/Fee_Rb_GetAdapterJobResult/Fee_GetSystemJobResult APIs).

- The way the job result is triggered (where the FeeRbOrigin is Fee, Adapter or NvM) must be identical within one type of block, and cannot be mixed.
- Only the NvM, if activated, can use the global job result mechanism. All other users must configure callback functions to synchronize with the result.

After an acceptance of user jobs, they are added to an internal order array which provides one entry per user (NvM/Adapter). These jobs are handled alternately by Fee's main function.

2.2.2 Setting/Getting the Job Mode

By default, all the steps are performed for a job. However, with the Fee_Rb_SetJobMode API, it is possible to skip some steps of a job type.

Different job types can be in different modes. Once the mode is set for a job type, it will continue to perform all the jobs in the new job mode. The mode can be changed again by calling Fee_Rb_SetJobMode and setting to another mode.

Fee_Rb_SetJobMode can be called when Fee is busy, either due to a job requested by the user or because of an internal operation, but the new mode will be applied only for the next job. The ongoing job will continue to work in the previous mode.

The Fee_Rb_GetJobMode API can be used to return the current mode of a specified job.

Each job type has one or more job modes. The different modes that are supported is file system specific. As some of the steps are skipped, there are risks and/or limitations when operating in certain modes, as described below.

FEE_RB_ALLJOBS_ALLSTEPS (All jobs)

This is the default mode for all jobs. In this mode all the standard steps are performed for the job.

FEE_RB_WRITEJOB_WRITE_VERIFY (Block write jobs)

In this mode, the following steps are skipped for a write job triggered via Fee_WriteBlock.

- The check to see if the data is different from the existing data copy. Since the write is always performed, there is a risk that the life of data flash is reduced. This is the compromise for getting fast performance, so the recommendation is to use this mode sparingly.
- The destroying of old copies for nofallback block. If the latest copies are lost due to bit flips, the old values might be returned.
- The copying of the data into internal buffer, thereby removing the limitation of writing the data only in maximum chunk length = medium buffer size. The block robustness feature (configured via the FeeRbRobustness parameter) will not work. If the user data is changed during the write, the new CRC will not be calculated, so the data that will be written is not returned at the next read attempt.

FEE_RB_WRITEJOB_WRITE_ONLY (Block write jobs)

This is the fastest mode in which a write job can be finished. In this mode, the same steps are skipped as in the FEE_RB_WRITEJOB_WRITE_VERIFY mode (with the same risks), and the verification after write step is also skipped. The driver just writes the new data and does the bare minimum of steps, such as updating

the cache with the new address.

As there is no validation after writing, if there are any errors during the write operation (e.g. bit flips during programming) the operation is not repeated and the newly written data will not be available on the next read operation.

FEE_RB_INVALIDATEJOB_WRITE_VERIFY (Block invalidate jobs)

This mode is similar to FEE_RB_WRITEJOB_WRITE_VERIFY, but is for invalidate requests triggered via Fee_InvalidateBlock.

FEE_RB_INVALIDATEJOB_WRITE_ONLY (Block invalidate jobs)

This mode is similar to FEE_RB_WRITEJOB_WRITE_ONLY, but for invalidate requests triggered via Fee_InvalidateBlock.

2.2.3 Writing blocks

Whenever Fee_Write is called, and assuming the user's parameters are valid, Fee adds the write order to the respective order array position. Depending on the system state (i.e. Scheduling or Non-scheduling mode), Fee_MainFunction will either be called automatically, or it must be called manually by the user, in which case it will execute the job immediately if no other job is currently being processed (internal or external), and the amount of data will not exceed the "hard sector reorganization threshold". Otherwise, all other necessary steps are finished before the new order is processed.

The write execution itself consists of the following steps:

- Identification of the latest valid block header inside all sectors.
- Write sequence (only if necessary).
- Verification of the written data.

As described above, the result of this user operation can be checked with Fee_GetJobResult/Fee_Rb_GetAdapterJobResult or the FeeRbCallbackEnd/FeeRbCallbackError callbacks.

2.2.4 Reading blocks

Whenever the read interface is called, and assuming the user's parameters are valid, Fee adds the read order to the respective order array position.

Depending on the system state (i.e. Scheduling or Non-scheduling mode), Fee_MainFunction will either be called automatically, or it must be called manually by the user, in which case it will execute the job immediately if no other job is currently being processed (internal or external). Otherwise, all other necessary steps are finished first, before the new order is processed.

The read sequence itself consists of the following steps:

- Identification of the latest valid block header inside all sectors (using cache, if possible).
- Verification of the read data. Only valid data will be read to the user buffer. If the data is corrupt, and if this data must still be returned, then the extension parameter FeeRbReadForce must be enabled.

- Continue searching, if necessary.
- Returning of the data.

As described above, the result of this user operation can be checked with the Fee_GetJobResult/Fee_Rb_GetAdapterJobResult interfaces.

Fee_Read will read the block data directly to the given user buffer, where data consistency is checked.

In the case of an unsuccessful read request (i.e. the job result is not ok) the user buffer will contain invalid/corrupted data. The user cannot rely on the assumption that old data from the last successful read request will be kept in the user buffer.

It is mandatory, therefore, to use the Fee_GetJobResult/Fee_Rb_GetAdapterJobResult interfaces and evaluate the return value carefully before accessing the data in the user buffer.

Depending on the return value, the user is allowed to do the following:

- Read Job OK: The buffer can be accessed and the data used without restrictions.
- Read Block Inconsistent: The buffer contains corrupted data which cannot be used.
- Read Job Failed/Read Block Invaild: The buffer contains either corrupted data from the read request or old data. The old data can only be used if additional measures were taken (e.g. it contains CRC, Hash, encryption, …) to detect data corruption.
- Read pending: Read ongoing - do not access the user buffer data.

2.2.5 Invalidation of blocks

Whenever Fee_InvalidateBlock is called, and assuming the user's parameters are valid, Fee adds the order to the respective order array position.

Depending on the system state (i.e. Scheduling or Non-scheduling mode), Fee_MainFunction will either be called automatically, or it must be called manually by the user, in which case it will execute the job immediately if no other job is currently being processed (internal or external), and the amount of data will not exceed the "hard sector reorganization threshold". Otherwise, all other necessary steps are finished before the new order is processed.

The invalidation execution itself consists of the following steps:

- Identification of the latest valid block header inside all sectors.
- Write sequence (if necessary).
- Verification of the written block header.

As described above, the result of this user operation can be checked with Fee_GetJobResult/Fee_Rb_GetAdapterJobResult or the FeeRbCallbackEnd/FeeRbCallbackError callbacks.

2.2.6 Block Maintenance

The Fee_Rb_BlockMaintenance function repairs a redundant block with security level setting 2 if one of the copies has corrupted data. The order is accepted only if the specified block number is valid. If Development Error Detection is enabled, the function also checks whether the Fee module has been initialized, and reports an error in the event of a failure.

Depending on the origin of the job (see "FeeRbOrigin" in the Configuration Advice), the order is added to the respective position inside the order array. Only one order per user is allowed at the same time. The order processing state can be checked using Fee_GetJobResult/Fee_Rb_GetAdapterJobResult. A block is not invalidated twice.

The time required for the order processing depends on various factors, including:

- The scheduling of the Fee and Fls/MemAcc Main Functions.
- Cache availability for the respective block.
- Underlying HW.
- Free space available in the current sector.

The Fee_Rb_BlockMaintenance function should not be called in the init phase of the ECU because this might trigger a sector reorganization.

Note that Fee_Rb_BlockMaintenance has to be specifically enabled (via the FeeRbMaintainRedundantBlocks configuration parameter).

2.2.7 Effective Layout Size Determination (at build time)

The effective layout size i.e. the space required for the configured blocks, plus the management information, is calculated and generated into *Fee_Report.txt*. Depending on the file system, various validations are executed during build time in order to ensure that this size doesn't exceed any physical or logical contraints.

> **NOTE**
>
> This feature is only available if the size of all blocks (`FeeBlockSize`) is known at the time of code generation. If the NvM component doesn't provide this, there will be no corresponding output in *Fee_Report.txt*. For further details, please refer to the Auto Block Length feature (Section 5.2.25) in the NvM documentation.

For *Fee1x* and *Fee2*, the calculation includes:

- The size of internal sector marker areas;
- Known block (payload) sizes;
- The size of internal block management headers (*Fee1x*), or the size of block management area/FAT (*Fee2*) respectively.

The values above are rounded up to the next page. Where necessary, the block

attributes *no-fallback* and *redundancy (double storage)* are considered. The effective layout size must be small enough to allow an efficient EEPROM emulation.

2.2.8 Driving Cycle and Stress Prediction (at build time)

If supported by the file system, this feature is active by default and executed during build time. It provides a prediction of the reachable driving cycles and the percentage lifetime stress, which the data-NvM is exposed to, in *Fee_Report.txt*. Both values are best-guess simulation predictions only; the real values might differ depending on real usage and temperatures.

> **NOTE**
>
> This feature is only available if the size of all blocks (`FeeBlockSize`) is known at the time of code generation. If the NvM component doesn't provide this, there will be no corresponding output in *Fee_Report.txt*. For further details, please refer to the Auto Block Length feature (Section 5.2.25) in the NvM documentation.

Reachable Driving Cycles

This value is a very rough indication of the (program/erase) stress which the data-Nvm is exposed to. It predicts the number of driving cycles that can be reached over ECU lifetime, where the term *driving cycle* denotes the time between two resets (usually between two ignition cycles). The calculation assumes that every block in the configured layout is written once per driving cycle i.e. independent of the configured number of block write cycles over ECU lifetime (`FeeNumberOfWriteCycles`).

e.g. Reachable driving cycles = 100,000. After 100,000 driving cycles, the data-NvM is operated outside its specified program/erase cycles (i.e. each configured block can be written 100,000 times before the data-NvM is operated outside its specified limits).

Lifetime Stress Ratio

Compared to reachable driving cycles, this percentage value is a more precise estimation for the (program/erase) stress that the data-NvM is exposed to. Its calculation considers the configured number of block write cycles over ECU lifetime (`FeeNumberOfWriteCycles`). A value above 100 per cent indicates that the data-NvM is operating outside of its specified limits (program/erase cycles), before all blocks have been written with the configured number of cycles.

e.g. 40 per cent. When all blocks have been written with the configured number of write cycles (`FeeNumberOfWriteCycles`), 40 per cent of the specified program/erase cycles are used up. Another 60 per cent are available before the data-NvM is operating outside its specified limits.

A meaningful value can only be calculated if the user correctly configures the block's write cycles over ECU lifetime (`FeeNumberOfWriteCycles`). A configured value of 1 means that the corresponding block is written only once in the whole ECU lifetime. If the user's intention is to write a block once per driving cycle, then

the number of expected driving cycles has to be configured (e.g. 250,000).

2.2.9 Stress Measurement and Driving Cycle Prediction (at run time)

Fee offers the possibility to track the (program/erase) stress that the NvM is exposed to by various operations at run time. The data-NvM must survive ignition cycles, and is therefore contained inside a Fee-internal persistent data block.

When using this feature, it is essential that the Fee-internal persistent data block is written once in every shutdown, after all the other NvM/Fee blocks have been written (via `Fee_Rb_WritePersistentData`, which must be called once in every shutdown, after all NvM/Fee blocks have been written).

The following data is tracked:

- The amount programmed over ECU lifetime.
- The amount programmed since measurement (re-)start.
- The number of driving cycles over ECU lifetime.
- The number of driving cycles since measurement (re-)start.
- The number of Fee-internal persistent data block updates over ECU lifetime.

The term “driving cycle” denotes the time between two resets (usually the time between two ignition cycles). Based on the tracked data, the Fee also offers the possibility to predict the maximum number of reachable driving cycles. The prediction is done both for the data tracked over ECU lifetime, and since measurement (re-)start.

A measurement (re-)start can be triggered by calling the C interface `Fee_Rb_RestartStressMeasmt`. This will copy the ECU lifetime values into the values taken at measurement (re-)start.

The prediction is a simple rule of three, assuming that the data-NvM is evenly stressed:

- Amount of data programmed per driving cycle = [data programmed over ECU lifetime] / [driving cycles over ECU lifetime] :prediction for data tracked over ECU lifetime and [data programmed over ECU lifetime - data programmed since measurement (re-)start] / [driving cycles over ECU lifetime - driving cycles since measurement (re-)start] :prediction for data tracked since measurement (re-)start
- Maximum number of reachable driving cycles = [data-NvM’s program and erase cycles * data-NvM’s size] / [amount of data programmed per driving cycle]

2.2.10 Multiple Instances of Fee

Fee supports the use of multiple instances in the same software for providing flash EEPROM emulation on different devices e.g. internal and external flash areas using different settings and configurations.

Currently, only following the combination of multi-instance is supported:

rba_FeeFs1x (with FeeRbGeneralDeviceIndex = 0) + rba_FeeFs2

Please see Intgration Advice (Section 2.9.6) for more details.

2.2.11 Multiple Machines

Fee supports multiple machine types, meaning that the same software can be used on different controllers. It is possible, therefore, to configure different set of Blocks for each different machine type. Please see Configuration Advice (Section 2.8.5) for details of how to configure this feature.

2.3 Extensions

2.3.1 Configuration Extensions

The following configuration parameters are available in version v12.7.0 in addition to those defined in AR v22-11

2.3.1.1 Fee/FeeBlockConfiguration/FeeDeviceIndex

Short Name	FeeDeviceIndex
Long Name	Block Device index.
Description	This information is defined but not used.
Introduction	This tag has currently no effect.
Multiplicity	0..0
Default Value	-
Type	ECUC-STRING-PARAM-DEF

2.3.1.2 Fee/FeeBlockConfiguration/FeeRbBlockPersistentId

Short Name	FeeRbBlockPersistentId
Long Name	Permanent block ID.
Description	Permanent block ID which is stored in the medium (“UUID”).
Introduction	Permanent block id which is used for storage in medium (“UUID”).
Multiplicity	1..1
Value Range	1..65534
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

2.3.1.3 Fee/FeeBlockConfiguration/FeeRbFirstReadDataMigration

Short Name	FeeRbFirstReadDataMigration
Long Name	First Read Data Migration
Description	Pre-processor config parameter to activate or deactivate the First Read Data Migration feature.

Introduction	false: First Read Data Migration is disabled (default). true: First Read Data Migration is enabled.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

2.3.1.4 Fee/FeeBlockConfiguration/FeeRbNoFallback

Short Name	FeeRbNoFallback
Long Name	NoFallback attribute
Description	Pre-processor config parameter to activate or deactivate a fall back mechanism to older copies of the same block in case the latest block is corrupted. By default a fall back to old copies is possible. It is activated only when at least one block is configured as NoFallback. The API Fee_Rb_BlockMaintenance() needs to be activated and called during shutdown when this feature is used. This is necessary to repair double storage NoFallback blocks which are corrupted due to write interruptions.
Introduction	false: If the latest copy is corrupted an older valid copy of the same block is read (default). true: If the latest copy is corrupted all available older copies of the same block are ignored. Behavior during layout change: Parameter gets activated in the new layout (true): Old copies are still readable since the evaluation is done on the block properties from the data flash. New copies are written with an activated NoFallback setting. Parameter gets deactivated in the new layout (false) but the feature itself is still activated: Only the last valid copy which was also readable with an activated NoFallback setting is readable. New copies are written with a deactivated NoFallback setting.Parameter gets deactivated in the new layout (false) and the feature itself is deactivated: All old copies are readable.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

2.3.1.5 Fee/FeeBlockConfiguration/FeeRbOrigin

Short Name	FeeRbOrigin
Long Name	Responsible module for this configuration.

Description	NVM / Adapter / Fee-internal block type.
Introduction	Origin (owner) of the block. NVM: Block belongs to NvM, query its job result with Fee_GetJobResult API. ADAPTER: Block belongs to Eep_Adapter / PDM_ Adapter; query its job result with Fee_Rb_GetAdapter-JobResult API. FEE: Block belongs to Fee; there is no public API to query its job result (DO NOT USE THIS SETTING)!
Multiplicity	1..1
Default Value	-
Enumeration Literals	NVM, ADAPTER, FEE
Type	ECUC-ENUMERATION-PARAM-DEF

2.3.1.6 Fee/FeeBlockConfiguration/FeeRbRobustness

Short Name	FeeRbRobustness
Long Name	Block robustness attribute.
Description	This tag improves the robustness against a data modification during write executions by adding an additional checksum to the block data which is calculated over an internal buffer that is used for writing the block data step by step.
Introduction	true: Feature enabled (block will be increased by 4 bytes). false: Feature disabled.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

2.3.1.7 Fee/FeeBlockConfiguration/FeeRbSecLevel

Short Name	FeeRbSecLevel
Long Name	Redundant data storage.
Description	Security Level of the block
Introduction	Range: 1 = Block has one copy in flash, 2 = Block has two identical copies in flash
Multiplicity	1..1
Value Range	1..2
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

2.3.1.8 Fee/FeeBlockConfiguration/FeeRbSurvival

Short Name	FeeRbSurvival
Long Name	Block survival attribute.
Description	This tag changes the reorganization behaviour for this block in case it is not within the layout.
Introduction	true: Sector reorganization will not delete this block, even if it is not part of the layout. false: Sector reorganization will delete this block if it is not part of the layout.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

2.3.1.9 Fee/FeeGeneral/FeeBufferAlignmentValueRef

Short Name	FeeBufferAlignmentValueRef
Long Name	Buffer Alignment
Description	Parameter determines the alignment of the start address that Fee buffers need to have.Value shall be inherited from MemAccBufferAlignmentValue. This parameter is not supported by the Fee. Instead the MemMap section FEE_START_SEC_VAR_SECURED_CACHED_RAM_ALIGNED_CLEARED_UNSPECIFIED can be used to specifically align Fee buffers containing data which is forward to lower layers.
Multiplicity	0..0
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

2.3.1.10 Fee/FeeGeneral/FeeRbActSurvivalEval

Short Name	FeeRbActSurvivalEval
Long Name	Activation of the “partial afterburner”.
Description	Pre-processor config parameter to enable or disable an additional block specific reorganization for unknown blocks with an active survival setting (“FeeRbSurvival”) in the medium.

Introduction	true: Feature enabled ("block specific afterburner" on for blocks with active survival setting). false: Feature disabled ("block specific afterburner" off for blocks with active survival setting). Hint: If activated, this block will be overtaken independent from the "FeeRbActUnknownBlockReorg" setting ("general afterburner").
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

2.3.1.11 Fee/FeeGeneral/FeeRbActUnknownBlockReorg

Short Name	FeeRbActUnknownBlockReorg
Long Name	Activation of the "global afterburner".
Description	Pre-processor config parameter to enable or disable an additional reorganization for any unknown block in the medium. Only the CB (Customer Block) is allowed to enable this feature, because this is affecting the thresholds of the sector reorganization to a special safety level. It must be deactivated (default) for the ASW!
Introduction	true: Feature enabled ("general afterburner" on for all unknown blocks). false: Feature disabled ("general afterburner" off for all unknown blocks). Hints: If activated, all unknown blocks will be overtaken independent from the "FeeRbActSurvivalEval" setting ("block specific afterburner"). If deactivated and with active survival evaluation ("FeeRbActSurvivalEval" == TRUE), only blocks with an active survival setting ("FeeRbSurvival" == TRUE) will be overtaken. Only the CB (Customer Block) is allowed to enable this feature, because this is affecting the thresholds of the sector reorganization to a special safety level. It must be deactivated (default) for the ASW!
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

2.3.1.12 Fee/FeeGeneral/FeeRbActUnknownBlockReorgCacheSize

Short Name	FeeRbActUnknownBlockReorgCacheSize
Long Name	Available cache size for faster sector reorganisation for "global afterburner" feature (Fee1.x).

Description	Pre-processor config parameter to enable or disable an additional optimisation in reorganization for any unknown block in the dflash. This feature fastens the sector reorganisation for unknown blocks by scanning the sector and building up a cache for unknown block. In this config parameter, project can configure the amount of RAM space that can be used for the unknown block cache. Feature is to be used either in Boot mode with global afterburner feature activated (FeeRbActUnknownBlockReorg = TRUE) or in application software when using survival feature is used (FeeRbActSurvivalEval = TRUE). Currently only Fee1.x supports this parameter. For Fee2 please use parameter FeeRbUnknownBlockCacheSize.
Introduction	Hints: 1. Default value of the tag when only survival feature is activated is the space needed for one unknown block cache entry (Fee1x = 12bytes per cache entry). 2. Default value when global after burner feature is activated is 1800 bytes. 3. For best performance project must allocate enough cache size to store information of all unknown blocks (one more then actual number of unknown blocks). Example: Project has 250 unknown blocks then for Fee1x: 251*12 = 3012 bytes to be allocated. 4. The unknown block reorganisation will also work when the cache size is smaller then actual number of unknown blocks in dflash. But will be slower.
Multiplicity	0..1
Value Range	0..393210
Default Value	1800
Type	ECUC-INTEGER-PARAM-DEF

2.3.1.13 Fee/FeeGeneral/FeeRbBlockTableLocation

Short Name	FeeRbBlockTableLocation
Long Name	Selected Location for Block Properties Table
Description	Select Fee Block Properties Location - Keep default setting “RAM” to optimize the runtime of the code. Use “ROM” setting in case of reduction of RAM consumption only.
Introduction	RAM: RAM Location of Fee Block Properties. ROM: ROM Location of Fee Block Properties. default: RAM
Multiplicity	0..1
Default Value	RAM

Enumeration Literals	RAM, ROM
Type	ECUC-ENUMERATION-PARAM-DEF

2.3.1.14 Fee/FeeGeneral/FeeRbCancel

Short Name	FeeRbCancel
Long Name	Activation for cancel feature
Description	Pre-processor config parameter to enable/disable the cancel feature for different jobs (internal and externally requested user jobs). Its purpose is to save code space when the feature is not needed. When the feature is activated and the cancel API is called, the ongoing job is canceled and Fee state machines are reset to start with a new job. Refer to documentation for more details on how cancel feature works. Note: To be backward + AUTOSAR compatible, the API for cancel feature is always available for NvM user Jobs. But the actual functionality is available only when this config parameter is set to true.
Introduction	true: Feature to cancel ongoing job enabled. false: Feature to cancel ongoing job disabled.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

2.3.1.15 Fee/FeeGeneral/FeeRbChunkJobApis

Short Name	FeeRbChunkJobApis
Long Name	API activation for chunk-wise job processing
Description	Pre-processor config parameter to enable/disable the APIs for chunk-wise job processing. Note: When the feature is used to write unknown blocks (example for restoring the blocks from rba_MemBckp in Boot mode), the unknown block reorganisation feature should also be activated (FeeRbActUnknownBlockReorg to be set to true). This is done to avoid risk of losing the unknown blocks during reorganisation. To write survival unknown block in application mode, the survival feature should be activated (FeeRbActSurvivalEval to be set to true). If the unknown block reorg or survival reorg is not activated, requests on unknown block are rejected and error is reported to DET.

Introduction	true: APIs for chunk-wise job processing enabled. false: APIs for chunk-wise job processing disabled.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

2.3.1.16 Fee/FeeGeneral/FeeRbChunkJobFeatureRequested

Short Name	FeeRbChunkJobFeatureRequested
Long Name	Features requested via chunk API
Description	This configuration parameter selects the different features that the chunk APIs shall support. It is introduced for early identification (during BCT build time) whether or not the requested features are supported by the selected file system. Build aborts when the requested feature is not yet supported.
Introduction	This feature can be configured only when chunk API is activated (config parameter FeeRbChunkJobAPIs is set to TRUE). Single chunk read and write operations for known blocks are possible by default (FEE_RB_CHUNK_FEATURE_LEVEL1).
Multiplicity	0..1
Default Value	FEE_RB_CHUNK_FEATURE_LEVEL1
Enumeration Literals	FEE_RB_CHUNK_FEATURE_LEVEL1, FEE_RB_CHUNK_FEATURE_LEVEL2, FEE_RB_CHUNK_FEATURE_LEVEL3, FEE_RB_CHUNK_FEATURE_LEVEL4, FEE_RB_CHUNK_FEATURE_LEVEL5
Type	ECUC-ENUMERATION-PARAM-DEF

2.3.1.17 Fee/FeeGeneral/FeeRbConfirmManualHwRelease

Short Name	FeeRbConfirmManualHwRelease
Long Name	Confirmation: Manual HW release
Introduction	TRUE: Confirmation - Measures for verification and testing Fee module on target hardware taken successfully FALSE: Confirmation - Measures for verification and testing Fee module on target hardware were not taken or failed
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

Description: Background: Fee module is hardware independent an can be used on

different flash devices and SoCs ("Devices"). Nevertheless Fee does not fit to all devices out-of-the-box and it has to be configured and verified based on the used hardware - for config, analysis and knowledge of hardware specification is mandatory. Typically this configuration is provided and verified by Fee module when selecting a machine type using parameter "FeeRbMachineType" which was released by Fee module. For released devices, a signature is provided in "rba_MemLibSign" container. There might be use cases where a project, user or customer want to run Fee module on a custom device which was either not released by Fee module or using the full manual configuration machine type (machine type "FEE_RB_FULL_MANUAL_CFG") by adding configuration in "rba_FeeRefDeviceCfg" container using the device's device hardware specifications containing information such as: - Flash technology: Single-Ended, Complement-Sensing or PCM - Flash sectorization including page size, smallest programmable and erasable units defined by flash HW - Support of HW related features like overprogramming of flash cells, erase pattern, HW-based blank check - Special error handling related to HW like wordline errors In this case a confirmation is necessary that correctness of hardware config was ensured and measures to verify and test Fee module together with the target device were taken by the user and not by Fee module. These measures may vary depending on the needs and quality requirements. Fee module is not liable for problems on non-released devices which are related to wrong or unsupported settings or missing verifications which need advanced knowledge of the device hardware. For example suitable measures are: - Testing of all flash emulation operations on HW: e.g. Block read/write, invalidation, reorganization - Endurance and reset testing: HW behavior on power loss and interruption on programming and erase operations

2.3.1.18 Fee/FeeGeneral/FeeRbConfirmSingleStorageBlockloss

Short Name	FeeRbConfirmSingleStorageBlockloss
Long Name	Confirmation: Weakness Single Storage Blockloss
Description	If FS1x or FS2 is selected as filesystem: Blocks configured with single storage have a very low probability to get lost even during regular operation. Background: In case an abort (reset or power loss) happens during physically charging the cells of the last page of a block then the data can be left weakly programmed and be lost during later operation. Because of this reason it is recommended that critical data is configured as double storage. By setting this tag to true you confirm that the weakness is considered in the block configuration. If this is not done the build will abort. This is done due to a quality measure to ensure the knowledge about this information is spread in the organization.

Introduction	true: FS1X and FS2 reset weakness for single storage blocks is considered in the block configuration. false: FS1X and FS2 reset weakness for single storage blocks is not considered in the block configuration
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

2.3.1.19 Fee/FeeGeneral/FeeRbConfirmUseGetSystemJobResult

Short Name	FeeRbConfirmUseGetSystemJobResult
Long Name	Confirmation: Use Fee_Rb_GetSystemJobResult for system jobs
Description	If FS1x is selected as filesystem and a forced sector reorganization shall be triggered (FeeRbTriggerReorgApi = true) or driver shall be stopped (FeeRbEnterStopModeApi = true). Then for getting the actual job status the API Fee_Rb_GetSystemJobResult() has to be used. Previously used Fee_Rb_GetAdapterJobResult() will return the wrong status of job execution for these jobs.
Introduction	true: Yes, Fee_Rb_GetSystemJobResult() is used for getting the status of system jobs. false: It is not yet checked, that Fee_Rb_GetSystemJobResult() is used for getting the status of system jobs.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

2.3.1.20 Fee/FeeGeneral/FeeRbDetailedBlkInfoApi

Short Name	FeeRbDetailedBlkInfoApi
Long Name	API activation Detailed Block Info API
Description	Activation of Detailed Block Info API. In case of Block inconsistency, detailed block info API can be used to get a more detailed reason for block inconistency.
Introduction	True: API for cDetailed Block Info enabled. False: API for cDetailed Block Info disabled.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

2.3.1.21 Fee/FeeGeneral/FeeRbEnableRuntimeErrorReporting

Short Name	FeeRbEnableRuntimeErrorReporting
Long Name	Enable reporting of run time error
Description	Pre-processor switch to enable or disable run time error reporting. By default run time errors are reported. However, by this switch the reporting can be disabled by project. Note: The switch is applicable only for AUTOSAR version greater then and equal to AR4.4. For older AUTOSAR version, this parameter is ignored.
Introduction	true: run time error reporting enabled. false: run time error reporting disabled.
Multiplicity	0..1
Default Value	true
Type	ECUC-BOOLEAN-PARAM-DEF

2.3.1.22 Fee/FeeGeneral/FeeRbEnterAndExitHighSpeedModeApis

Short Name	FeeRbEnterAndExitHighSpeedModeApis
Long Name	API activation to Enter and Exit high speed mode
Description	Pre-processor config parameter to enter or exit high speed execution mode API functionality of the FEE module. If the enter high speed execution API is called, Fee state machines are set to execute the job in fastest possible way. Refer documentation for more details about what the API does.
Introduction	true: Enter and Exit high speed mode API enabled. false: Enter and Exit high speed mode API disabled.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

2.3.1.23 Fee/FeeGeneral/FeeRbEnterStopModeApi

Short Name	FeeRbEnterStopModeApi
Long Name	API activation for Fee_Rb_EnterStopMode()

Description	Pre-processor config parameter to enable/disable the stop mode functionality of the FEE module. If the stop mode API is called, the Fee is finishing the current job and afterwards locking itself. As soon as the Fee is locked, no further Fls/MemAcc requests are given until the Fee is initialized again (the stop mode cannot be left).
Introduction	true: Stop mode feature enabled. false: Stop mode feature disabled.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

2.3.1.24 Fee/FeeGeneral/FeeRbFsSelect

Short Name	FeeRbFsSelect
Long Name	Selected Filesystem.
Description	Specifies the actual layout of the data on the flash.
Multiplicity	0..1
Default Value	FS1X
Enumeration Literals	FS1X, FS2, FS3
Type	ECUC-ENUMERATION-PARAM-DEF

2.3.1.25 Fee/FeeGeneral/FeeRbGeneralDeviceIndex

Short Name	FeeRbGeneralDeviceIndex
Long Name	Device id when multi instance of Fee is used
Description	Specifies the InstanceId of this module instance. Note: This parameter is mandatory when more than one instance of FEE module is configured. For single instance when only one instance of FEE module is present, the device id will be ignored and will be always 0.
Multiplicity	0..1
Value Range	0..254
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

2.3.1.26 Fee/FeeGeneral/FeeRbGeneralDeviceNickName

Short Name	FeeRbGeneralDeviceNickName
Long Name	Device nick name when multi instance of Fee is used

Description	Specifies the nick name of the FEE module instance which is used for the API/Interface-name generation. Note: This parameter is mandatory when more than one instance of FEE module is configured. The instance with FeeRbGeneralDeviceIndex = 0 will not use the nick name for the API/Interface-name generation. For single instance when only one instance of FEE module is present, the nick name will be ignored.
Multiplicity	0..1
Default Value	-
Type	ECUC-STRING-PARAM-DEF

2.3.1.27 Fee/FeeGeneral/FeeRbGetFreeSpaceApi

Short Name	FeeRbGetFreeSpaceApi
Long Name	API activation to get number of free bytes in DFlash
Description	Pre-processor config parameter to enable/disable the API to return amount of free bytes in the DFLASH area.
Introduction	true: Free space API enabled. false: Free space API disabled.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

2.3.1.28 Fee/FeeGeneral/FeeRbGetNrFreeBytesAndFatEntriesApi

Short Name	FeeRbGetNrFreeBytesAndFatEntriesApi
Long Name	API activation to get number of free bytes and FAT entries in Dflash
Description	Pre-processor config parameter to enable/disable the API to return the amount of free bytes and FAT entries in the DFLASH area until a hard reorganization or an erase operation starts. This API applies only to Fee2.
Introduction	true: Number free bytes and FAT entries API enabled. false: Number free bytes FAT entries API disabled.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

2.3.1.29 Fee/FeeGeneral/FeeRbGetNrSectorErases

Short Name	FeeRbGetNrSectorErases

Long Name	Calculate the number of performed sector specific erase cycles
Description	Pre-processor config parameter to enable/disable the determination of the sector specific number of erase cycles. Only available for Fee2. number of erase cycles is stored to Dflash in background. Default: disabled
Introduction	true: determine sector specific number of erase cycles. false: do not determine sector specific number of erase cycles.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

2.3.1.30 Fee/FeeGeneral/FeeRbGetSectStateApi

Short Name	FeeRbGetSectStateApi
Long Name	API activation to get the state of sector/row
Description	Pre-processor config parameter to enable/disable the API to return the state of sector/row. In Fee1x and Fee2, the API returns the state of the sector and for Fee3, it returns the state of the row.
Introduction	true: get sector/row state API enabled. false: get sector/row state API disabled.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

2.3.1.31 Fee/FeeGeneral/FeeRbHighSpeedModeWriteDetails

Short Name	FeeRbHighSpeedModeWriteDetails
Long Name	Details of number of bytes to be reserved during high speed mode operation

Description	Pre-processor config parameters to to configure number of bytes to be reserved during high speed mode exeuction (ex. during crash). The soft and hard reorganisation thresholds are shifted based on the configured values below. Using this feature could reduce the total layout size that can be used (because now a bigger area is reserved to store the data during crash). One option to overcome this limitation is to reconfigure the sector size (using config parameter FeeRbDeviceFsXSectorList) and shift to smaller but more number of loglcal sectors. This feature is currently supported for Fee1x only. This sub-container will be ignored when configured for other file system.
Introduction	If these config parameters are not configured but high speed mode is used, then, there is a risk that crash data will be written in last logical sector, which could be lost in case memory gets full.
Multiplicity	0..1
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

2.3.1.32 Fee/FeeGeneral/FeeRbHighSpeedModeWriteDetails/FeeRbNrBytesInHighSpeedMode

Short Name	FeeRbNrBytesInHighSpeedMode
Long Name	Total number of bytes to be stored during high speed mode
Description	Total number of bytes calculated over all write request that will be written during high speed mode operation. This size shall also include the additional bytes needed per block for redundant blocks (FeeRbSecLevel = 2 or NvMBlockManagementType = NVM_BLOCK_REDUNDANT) and round off of the data to logical page size.
Multiplicity	1..1
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

2.3.1.33 Fee/FeeGeneral/FeeRbHighSpeedModeWriteDetails/FeeRbNrWritesInHighSpeedMode

Short Name	FeeRbNrWritesInHighSpeedMode
Long Name	Total number of writes during high speed mode
Description	Total number of write request during high speed mode operation

Multiplicity	1..1
Value Range	0..65535
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

2.3.1.34 Fee/FeeGeneral/FeeRbInclPrjHeader

Short Name	FeeRbInclPrjHeader
Long Name	Activate optional project extension header include
Description	This switch can be used to activate the optional #include Fee_Rb_Prj.h (see template folder) to activate/modify internal functionality within the Fee and the corresponding file system(s). Note: when activating this switch, you have to provide this header file in your project.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

2.3.1.35 Fee/FeeGeneral/FeeRbMachineType

Short Name	FeeRbMachineType
Long Name	Machine Type
Description	Controller type to be configured in case no MCU module is present.
Multiplicity	0..1
Default Value	-
Type	ECUC-ENUMERATION-PARAM-DEF

Enumeration Literals: FEE_RB_IFX_TC26X, FEE_RB_IFX_TC27X, FEE_RB_IFX_TC27X_HSM, FEE_RB_IFX_TC29X, FEE_RB_IFX_TC29X_HSM, FEE_RB_IFX_TC33X_CMPL, FEE_RB_IFX_TC33X_SE, FEE_RB_IFX_TC35X_CMPL, FEE_RB_IFX_TC35X_SE, FEE_RB_IFX_TC36X_CMPL, FEE_RB_IFX_TC36X_SE, FEE_RB_IFX_TC37X_CMPL, FEE_RB_IFX_TC37X_SE, FEE_RB_IFX_TC38X_CMPL, FEE_RB_IFX_TC38X_SE, FEE_RB_IFX_TC39X_CMPL, FEE_RB_IFX_TC39X_SE, FEE_RB_IFX_TC3EX_CMPL, FEE_RB_IFX_TC3EX_SE, FEE_RB_IFX_TC3XX_SE_HSM, FEE_RB_IFX_TC3XX_CMPL_HSM, FEE_RB_IFX_TC49X, FEE_RB_IFX_TC4XX_HSM, FEE_RB_JDP_MPC5726, FEE_RB_JDP_MPC5744, FEE_RB_JDP_MPC5746B, FEE_RB_JDP_MPC5746B_HSM, FEE_RB_JDP_MPC5777, FEE_RB_JDP_MPC5777_HSM, FEE_RB_ST_SPC58XNX, FEE_RB_ST_SPC58XNX_HSM, FEE_RB_ST_SPC58XBX, FEE_RB_ST_SPC58XCX, FEE_RB_ST_SPC58X_GH_X, FEE_RB_ST_SPC58XXX_HSM, FEE_RB_ST_SR6P3, FEE_RB_ST_SR6P6, FEE_RB_ST_SR6P7, FEE_RB_ST_SR6Px_HSM, FEE_RB_NXP_MPC5746C, FEE_RB_NXP_MPC5746C_HSM, FEE_RB_REN_RH850F1KM_S1_4S_16KB, FEE_RB_REN_RH850F1KM_S1_4S_15_5KB, FEE_R-

B_REN_RH850F1KM_S1_4S_15KB,FEE_RB_REN_RH850F1KM_S4_6S_17_375KB, FEE_RB_REN_RH850F1KM_S4_4S_5_9375KB_HSM, FEE_RB_REN_RH850F1KM_S4_6S_16KB, FEE_RB_REN_RH850F1KM_S4_4S_8KB_HSM, FEE_RB_REN_RH850F1KM_S2_6S_17_375KB, FEE_RB_REN_RH850F1KM_S2_4S_5_9375KB_HSM, FEE_RB_REN_RH850F1KM_S2_6S_16KB, FEE_RB_REN_RH850F1KM_S2_4S_8KB_HSM, FEE_RB_CYP_CYT2B6X_6S_8K, FEE_RB_CYP_CYT2B6X_4S_4K_HSM, FEE_RB_CYP_CYT2B6X_6S_8K_HSM, FEE_RB_CYP_CYT2B6X_4S_4K, FEE_RB_CYP_CYT2B7X_4S_18K, FEE_RB_CYP_CYT2B7X_4S_6K_HSM, FEE_RB_CYP_CYT2B7X_4S_18K_HSM, FEE_RB_CYP_CYT2B7X_4S_6K, FEE_RB_CYP_CYT2B9X_6S_16K, FEE_RB_CYP_CYT2B9X_4S_8K_HSM, FEE_RB_CYP_CYT2B9X_6S_16K_HSM, FEE_RB_CYP_CYT2B9X_4S_8K, FEE_RB_CYP_CYT2BLX_6S_16K, FEE_RB_CYP_CYT2BLX_4S_8K_HSM, FEE_RB_CYP_CYT2BLX_6S_16K_HSM, FEE_RB_CYP_CYT2BLX_4S_8K, FEE_RB_CYP_CYT3BBX_12S_16K, FEE_RB_CYP_CYT3BBX_8S_8K_HSM, FEE_RB_CYP_CYT3BBX_12S_16K_HSM, FEE_RB_CYP_CYT3BBX_8S_8K, FEE_RB_CYP_CYT4BFX_12S_16K, FEE_RB_CYP_CYT4BFX_8S_8K_HSM, FEE_RB_CYP_CYT4BFX_12S_16K_HSM, FEE_RB_CYP_CYT4BFX_8S_8K, FEE_RB_REN_RH850F1M_4S_16KB, FEE_RB_REN_RH850F1M_4S_15_5KB, FEE_RB_REN_RH850F1M_4S_15KB, FEE_RB_EXT_CYP_S25FS256, FEE_RB_EXT_CYP_S25FS512, FEE_RB_EXT_CYP_S25HS01GT, FEE_RB_EXT_CYP_S25HS01GT_HSM, FEE_RB_EXT_CYP_S26Kx256S, FEE_RB_EXT_CYP_S70FS01GS, FEE_RB_EXT_CYP_S70FS01GS_HSM, FEE_RB_EXT_MAC_MX25L32, FEE_RB_EXT_MAC_MX66U1G, FEE_RB_EXT_MAC_MX66U2G, FEE_RB_EXT_MIC_MT25QU01G, FEE_RB_EXT_MIC_MT25QU02G, FEE_RB_EXT_MIC_MT35XU512, FEE_RB_EXT_MIC_MT35XU01G, FEE_RB_EXT_MIC_MT35XU02G, FEE_RB_EXT_WIN_W25Q64JV, FEE_RB_EXT_WIN_W25Q128JV, FEE_RB_EXT_WIN_W25Q256JV, FEE_RB_FLSEXTSX6XX_SX60X, FEE_RB_REN_RH850U2A16_4S_60KB, FEE_RB_REN_RH850U2A16_4S_16KB_HSM, FEE_RB_REN_RH850U2A8_4S_60KB, FEE_RB_REN_RH850U2A8_4S_16KB_HSM, FEE_RB_REN_RH850U2A6_4S_44KB, FEE_RB_REN_RH850U2A6_4S_16KB_HSM, FEE_RB_REN_RH850U2B2X_4S_120KB, FEE_RB_REN_RH850U2B24_4S_40KB_HSM, FEE_RB_REN_RH850U2B20_4S_24KB_HSM, FEE_RB_REN_RH850U2B10_4S_60KB, FEE_RB_REN_RH850U2B10_4S_16KB_HSM, FEE_RB_REN_RH850U2B6_6S_16KB, FEE_RB_REN_RH850U2B6_4S_16KB_HSM, FEE_RB_REN_RH850U2B24E_4S_120KB, FEE_RB_REN_RH850U2B24E_4S_40KB_HSM, FEE_RB_REN_RH850U2B12E_4S_60KB, FEE_RB_REN_RH850U2B12E_4S_16KB_HSM, FEE_RB_REN_RH850U2B6E_6S_16KB, FEE_RB_REN_RH850U2B6E_4S_16KB_HSM, FEE_RB_REN_RH850U2C8_6S_60KB, FEE_RB_REN_RH850U2CX_4S_16KB_HSM, FEE_RB_REN_RH850U2C4_4S_60KB, FEE_RB_REN_RH850U2C2_4S_24KB, FEE_RB_REN_RH850E2GUH_4S_60KB, FEE_RB_REN_RH850E2GUH_4S_16KB_HSM, FEE_RB_REN_RH850E2M_6S_24KB, FEE_RB_REN_RH850E2M_4S_8KB_HSM, FEE_RB_REN_RH850P1LC_D1_4S_7_75KB, FEE_RB_REN_RH850P1LC_D2_4S_15_75KB, FEE_RB_REN_RH850P1M_4S_8KB, FEE_RB_REN_RH850P1XC_D3_4S_15_75KB, FEE_RB_REN_RH850P1XC_D3_6S_10_5KB, FEE_RB_REN_RH850P1XC_D3_4S_14_75KB, FEE_RB_REN_RH850P1XC_D3_6S_9_8125KB, FEE_R-

B_REN_RH850P1XC_D3_4S_9_75KB,FEE_RB_REN_RH850P1XC_D3_2S_15_5KB_HSM, FEE_RB_REN_RH850P1XC_D4_4S_31_75KB, FEE_RB_REN_RH850P1XC_D4_4S_30_75KB, FEE_RB_REN_RH850P1XC_D4_2S_15_5KB_HSM, FEE_RB_REN_RH850P1XC_D5_8S_23_875KB, FEE_RB_REN_RH850P1XC_D5_8S_23_375KB, FEE_RB_REN_RH850P1XC_D5_2S_15_5KB_HSM, FEE_RB_PC_3S_16KB, FEE_RB_PC_6S_64KB, FEE_RB_PC_6S_64KB_LowAddress, FEE_RB_PC_6S_64KB_SE, FEE_RB_PC_6S_64KB_16BP, FEE_RB_PC_2S_64KB, FEE_RB_PC_4S_24KB, FEE_RB_PC_4S_64KB, FEE_RB_PC_4S_256KB, FEE_RB_PC_6S_128KB, FEE_RB_PC_8S_8KB, FEE_RB_PC_8S_16KB, FEE_RB_PC_8S_32KB, FEE_RB_PC_8S_64KB, FEE_RB_FULL_MANUAL_CFG, FEE_RB_MPLDEV

2.3.1.36 Fee/FeeGeneral/FeeRbMainFunctionScheduling

Short Name	FeeRbMainFunctionScheduling
Long Name	MainFunction Scheduling Type
Description	Selection on type of scheduling required for the Fee_MainFunction
Multiplicity	0..1
Default Value	FEE_RB_MAINFUNC_SCHEDULING_RTE_TIMED
Enumeration Literals	FEE_RB_MAINFUNC_SCHEDULING_RTE_BACKGROUND, FEE_RB_MAINFUNC_SCHEDULING_RTE_TIMED, FEE_RB_MAINFUNC_SCHEDULING_MANUAL
Type	ECUC-ENUMERATION-PARAM-DEF

2.3.1.37 Fee/FeeGeneral/FeeRbMaintainRedundantBlocks

Short Name	FeeRbMaintainRedundantBlocks
Long Name	API activation for Fee_Rb_BlockMaintenance().
Description	Pre-processor config parameter to enable/disable the API to maintain the redundant blocks. Hint: If NvM is used this switch must be set to “TRUE”!
Introduction	true: Maintain API enabled. false: Maintain API disabled.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

2.3.1.38 Fee/FeeGeneral/FeeRbNrBytesBtwSSRAndHSR

Short Name	FeeRbNrBytesBtwSSRAndHSR
Long Name	Number of bytes between Soft Reorganisation and Hard Reorganisation Threshold
Description	Pre-processor config parameter to determine the gap between Soft Sector Reorganisation and Hard Sector Reorganisation threshold.

Introduction	Note: 1. When the parameter is not configured, soft reorganisation threshold are calculated internally as a factor of hard reorganisation threshold. Ex. SSR = 1.5 * HSR, when no unknown block handling is needed or SSR = 1.2 * 1.5 * HSR, when unknown block handling is activated. 2. Two sector emulation has no softreorganisation. so this tag is ignored for such dflash layout. 3. A higher SSR threshold value would mean lesser layout size that project can configure and earlier reorganization. A lower SSR value would mean higher chances of getting into hard reorganization. So please check the impact of the threshold change. 4. If unknown blocks are to be handled (when config parameter FeeRbActSurvivalEval or FeeRbActUnknownBlockReorg is set to true). Consider the additional space needed for the unknown blocks.
Multiplicity	0..1
Minimum Value	0
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

2.3.1.39 Fee/FeeGeneral/FeeRbNrOfUnknownBlockCacheEntries

Short Name	FeeRbNrOfUnknownBlockCacheEntries
Long Name	Cache size for “global afterburner” and “partial afterburner” feature (Fee2).
Introduction	Hints: 1. Default value when global afterburner feature is activated is 300 blocks. 2. When survival feature is activated exclusively the configuration of this parameter is mandatory, the default value will not be taken. 3. For best performance project must allocate space for all available unknown blocks + 1 (one more then actual number of unknown blocks). 4. The size of one cache entry for Fee2 is 4 bytes. This means if 250 is configured the unknown block cache will consume 250*4=1000 bytes. 5. The unknown block handling will also work when the cache size is smaller then the actual number of unknown blocks in the dflash. But it will slow down which each additional cache buildup iteration.
Multiplicity	0..1
Value Range	1..65534
Default Value	300
Type	ECUC-INTEGER-PARAM-DEF

Description: Pre-processor config parameter to determine the size of the cache used for the reorganization of unknown blocks which are stored in the flash eeprom emulation area. By scanning the flash content and building up the cache with the location information of the identified unknown blocks, the reorganizations of these blocks will essentially speed up. With this configuration parameter the project can configure the number of unknown blocks which can be managed within one cache build iteration. To reach the best performance set this value to the number of unknown blocks defined in your project + 1. This means that all blocks can be covered without intermediate cache build ups. This parameter has to be used either in Boot mode with global afterburner feature activated (FeeRbActUnknownBlockReorg = TRUE) or in application software when using partial afterburner (survival) feature (FeeRbActSurvivalEval = TRUE). Currently only Fee2 supports this parameter. For Fee1.x please use parameter FeeRbActUnknownBlockReorgCacheSize.

2.3.1.40 Fee/FeeGeneral/FeeRbNvMBlockHeaderInclude

Short Name	FeeRbNvMBlockHeaderInclude
Long Name	NvM header file to include for block size macros
Description	Header file providing the block size macros for NvM blocks with an undefined block-specific parameter FeeBlockSize. Only the header file name without the extension has to be entered here.
Introduction	The feature is enabled when a name is defined for this parameter, i.e. there is at least one NvM block with an empty block-specific parameter FeeBlockSize. The header with the given name is included by Fee to resolve the macro(s).. The feature is disabled when this parameter is empty, i.e. the block-specific parameter FeeBlockSize is defined for all blocks.
Multiplicity	0..1
Default Value	-
Type	ECUC-STRING-PARAM-DEF

2.3.1.41 Fee/FeeGeneral/FeeRbPageBufferSize

Short Name	FeeRbPageBufferSize
Long Name	Size of internal page buffer.
Description	Size of buffer for communication between flash driver and Fee.If there is no configuration value for this parameter, the default (1024) will be automatically used.
Introduction	The value has to be a multiple of virtual page size for writing into flash and a multiple of cache line size for reading from an external flash.
Multiplicity	0..1

Minimum Value	256
Default Value	1024
Type	ECUC-INTEGER-PARAM-DEF

2.3.1.42 Fee/FeeGeneral/FeeRbRestrictDirectMemoryAccess

Short Name	FeeRbRestrictDirectMemoryAccess
Description	Pre-processor config parameter to enable/disable direct memory access.
Introduction	true: direct memory access not allowed, always use flash driver. false: direct memory access allowed, provided the Fee and the hardware supports it
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

2.3.1.43 Fee/FeeGeneral/FeeRbSchedTriggerFeeRbEndInit

Short Name	FeeRbSchedTriggerFeeRbEndInit
Long Name	ETO for Fee_Rb_EndInit
Description	In case the function Fee_Rb_EndInit shall be scheduled using AUTOSAR the function Fee_Rb_EndInit will be generated as External Trigger Occured function. The name the external trigger can be configured here. If left empty then Fee_Rb_EndInit will not be mentioned in BSWMD. In this case manual scheduling of Fee_Rb_EndInit has to be done. This feature is supported only in case of exactly one Fee instance (no multi instance).
Introduction	1. Create extern trigger outside of the Mem stack. 2. Select trigger name here
Multiplicity	0..1
Default Value	-
Type	ECUC-STRING-PARAM-DEF

2.3.1.44 Fee/FeeGeneral/FeeRbSecondPageBufferSize

Short Name	FeeRbSecondPageBufferSize
Long Name	Size of second internal page buffer.

Description	Size of second buffer for communication between flash driver and Fee to reduce the duration of a reorg in Fee2, or of a defragmentation in Fee3. This parameter does not apply to other file systems. If no value is set in Fee2, the second buffer is enabled and generated with its default size 1024. In Fee3, the second buffer is only beneficial in systems, where Fee_MainFunction is not scheduled in background, but in a slower cyclic process. Therefore, it is disabled by default - independent of the default value. I.e. if no value is set in Fee3, only one buffer is used. If the value 0 is set, only one buffer is used for either file system.
Introduction	In Fee2, the value has to be a multiple of the virtual page size for writing into flash, and a multiple of the cache line size for reading from an external flash. In Fee3, the value has to be equal to the logical row size.
Multiplicity	0..1
Minimum Value	0
Default Value	1024
Type	ECUC-INTEGER-PARAM-DEF

2.3.1.45 Fee/FeeGeneral/FeeRbSetAndGetJobModeApi

Short Name	FeeRbSetAndGetJobModeApi
Long Name	Activation of set and get job mode APIs
Description	Pre-processor config parameter to enable/disable the APIs to set and get the job modes.
Introduction	true: Job mode set and get APIs are enabled. false: Both the job mode and set APIs are disabled.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

2.3.1.46 Fee/FeeGeneral/FeeRbStressMeasmt

Short Name	FeeRbStressMeasmt
Long Name	Activation of stress measurement and driving cycle prediction
Description	Pre-processor config parameter to enable/disable stress measurement and driving cycle prediction.
Introduction	true: Stress measurement and driving cycle prediction are enabled. false: Stress measurement and driving cycle prediction are both disabled.

Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

2.3.1.47 Fee/FeeGeneral/FeeRbSyncFlsMainLoopHook

Short Name	FeeRbSyncFlsMainLoopHook
Long Name	Hook into synchronous Fls/MemAcc_MainFunction loop
Description	A hook function can be configured, which will be called in the synchronous Fls/MemAcc_MainFunction loop used for all flash access operations during Fee initialization. This can be useful especially for projects, where the flash is accessed concurrently by different cores at initialization time. HW limitations due to shared logic can block one user until the job of the other user finishes. This impacts the runtime of the loop and consequently extends the runtime of the entire Fee initialization. No other task can be scheduled in between, which may cause a Watchdog reset or impacts the system behaviour in a different way. The user can use the hook function to take all necessary actions to keep the system stable.
Introduction	1. Integrate the function into your build environment. 2. Write the function name of your hook function to the configuration parameter. 3. Build the software 4. Run the software, the hook function will be called once per loop cycle
Multiplicity	0..1
Default Value	-
Type	ECUC-FUNCTION-NAME-DEF

2.3.1.48 Fee/FeeGeneral/FeeRbTriggerReorgApi

Short Name	FeeRbTriggerReorgApi
Long Name	API activation for Fee_Rb_TriggerReorg(), Fee_Rb_TriggerReorgFat(), Fee_Rb_TriggerReorgDat()

Description	Pre-processor config parameter to enable/disable the “Forced reorganisation followed by a sector erase” functionality for Fee1.x. For Fee2 the “Forced reorganization of the FAT and DAT sector” functionality is enabled. Succeeding sector erase of the reorganized sector is not part of the FAT, DAT sector reorganization.
Introduction	true: Forced reorganisation feature enabled (one force reorg per call). false: Forced reorganisation feature disabled.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

2.3.1.49 Fee/FeeGeneral/FeeRbUseRelativeAddressing

Short Name	FeeRbUseRelativeAddressing
Long Name	Activation of the relative Addressing Mode
Description	This mode is used for AR compatibility check of the Fee. It is only working if all users of one API interface have the relative mode activated
Introduction	True: The relative addressing is used False: Absolute Addressing is used
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

2.3.1.50 Fee/FeeGeneral/FeeSetModeSupported

Short Name	FeeSetModeSupported
Long Name	API activation for Fee_SetMode().
Description	Pre-processor config parameter to enable/disable the “SetMode” functionality of the FEE module.
Introduction	true: SetMode functionality enabled / code present. false: SetMode functionality disabled / code not present. Note: If Fls is used instead of MemAcc, this configuration setting has to be consistent with that of all underlying flash device drivers (configuration parameter FlsSetModeApi).
Multiplicity	1..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

2.3.1.51 Fee/FeeRbDebug

Short Name	FeeRbDebug
Long Name	Fee Debugging
Description	Container for debugging switches.
Multiplicity	0..1
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

2.3.1.52 Fee/FeeRbDebug/FeeRbDebugBlock

Short Name	FeeRbDebugBlock
Long Name	Own debugging block to keep debugging information over resets.
Description	Enables a Fee block owned by the Fee itself. This block holds the information about timing measurements and erase counters over resets.
Introduction	true: Fee debug block enabled. false: Fee debug block disabled.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

2.3.1.53 Fee/FeeRbDebug/FeeRbDebugDump

Short Name	FeeRbDebugDump
Long Name	Dump the DFLASH on unexpected errors.
Description	Whenever internally an unexpected error happens the DFLASH is dumped to a text file. This feature makes only sense when the Fee is running in a simulation on the PC.
Introduction	true: Dump on unexpected error enabled false: Dump on unexpected error disabled.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

2.3.1.54 Fee/FeeRbDebug/FeeRbDebugTime

Short Name	FeeRbDebugTime
Long Name	Enable timing measurements.
Description	Pre-processor config parameter to enable and disable timing measurements.

Introduction	true: Timing measurements enabled. false: Timing measurements disabled. In case timing measurement is enabled the following operations are measured: Main Function duration (Init, Read, Write, Invalidate, Data sector change, FAT sector change)
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

2.3.1.55 Fee/FeeRbDebug/FeeRbDebugTimeHistogramSize

Short Name	FeeRbDebugTimeHistogramSize
Long Name	Size of the timing measurement histogram.
Description	Number of slots in the histogram for timing measurement.
Introduction	When time measurement is enabled a histogram for timing measurement will be maintained for all measured operations. The unit in this histogram is microseconds and it is log(2) scaled where the first basket has an offset of 0. The bigger this histogram is, the bigger are the maximum values it can hold. On the other hand more RAM is consumed and the debug block (in case it is enabled) will also grow bigger to store the histogram. Every slot consumes 4 bytes (uint32). When timing measurement is turned off then this value has no effect.
Multiplicity	0..1
Value Range	1..50
Default Value	1
Type	ECUC-INTEGER-PARAM-DEF

2.3.1.56 Fee/FeeRbDebug/FeeRbGrayBoxTest

Short Name	FeeRbGrayBoxTest
Long Name	Enable gray box test
Description	Pre-processor config parameter to enable/disable gray box test.
Introduction	TRUE : Gray box test is active FALSE : Gray box test is not active
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

2.3.1.57 Fee/FeeRbMplDevCfg

Short Name	FeeRbMplDevCfg
Long Name	Fee Multiple Devices
Description	Container to configure multiple devices under Fee, during initialization underlying device can be detected and correct configuration can be loaded. This container is Mandatory when FeeRbMachineType is configured as Multiple devices
Multiplicity	0..1
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

2.3.1.58 Fee/FeeRbMplDevCfg/FeeRbMplDev

Short Name	FeeRbMplDev
Long Name	Configured Device
Description	Devices list for Multi-Device Configuration
Multiplicity	2..254
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

2.3.1.59 Fee/FeeRbMplDevCfg/FeeRbMplDev/FeeRbBlockConfigurationRef

Short Name	FeeRbBlockConfigurationRef
Long Name	Fee Blocks
Description	Reference to the Fee Blocks used on the device
Multiplicity	1..65535
Default Value	-
Type	ECUC-REFERENCE-DEF

2.3.1.60 Fee/FeeRbMplDevCfg/FeeRbMplDev/FeeRbMplDevRef

Short Name	FeeRbMplDevRef
Long Name	Reference to Device configuration
Description	Reference to the Device configuration in rba_FeeRefDeviceCfg. Currently, this feature is not supported for Fs2 and Fs3.
Introduction	Reference to the device configuration
Multiplicity	1..1
Default Value	-

Type	ECUC-CHOICE-REFERENCE-DEF

2.3.2 Additional Services

The following API are available in version v12.7.0 in addition to those defined in AR v22-11

2.3.2.1 Fee_Rb_EndInit

Syntax	`Fee_Rb_EndInit`
Description	Marks end of the init. After calling this function, erasing in the background is allowed. Also the Fee is set in a more cooperative mode to make sure task duration constraints are not violated.
Note 1	This function should be placed when the period of a necessary fast start-up is finished, e.g. after the ECU is ready to accept CAN messages.
Note 2	This function should also be called immediately after Fee_Init if the system uses the initialization function inside a power down service (e.g. Shutdown). Otherwise erasing and sector reorganizations will not be performed inside this system state anymore.

2.3.2.2 Fee_Rb_GetSectChngCnt

Syntax	`Fee_Rb_GetSectChngCnt`
Description	Returns the number of sector changes (sector reorganizations) which is represented by the highest sector counter. The average number of erases per sector can be calculated by dividing the delivered result with the number of sectors in the system.

2.3.2.3 Fee_Rb_GetAdapterJobResult

Syntax	`Fee_Rb_GetAdapterJobResult`
Return Values	`MEMIF_JOB_OK` Last job executed successfully. `MEMIF_JOB_FAILED` Last job failed unexpectedly. `MEMIF_JOB_PENDING` Last job is still running. `MEMIF_BLOCK_INCONSISTENT` Last job was a read and either not a single instance of all instances has consistent data, or the block is not present at all. `MEMIF_BLOCK_INVALID` Last job was a read and the block was invalidated intentionally.
Description	Queries the result of the last accepted job issued by the upper layer software coming from adapter layer (e.g. DGS EEP adapter).

Note	Only one user ("FeeRbOrigin") is allowed to use the job result functionality because the different user jobs might influence one another. All other users have to use the callback mechanism ("FeeRbCallbackEnd", "FeeRbCallbackError"). The AUTOSAR interface Fee_GetJobResult could not be used by the adapter layer because it's not possible for Adapter layer to know if the last job completed by Fee was NvM, the adapter or an internal job.

2.3.2.4 Fee_Rb_GetMigrationResult

Syntax	`Fee_Rb_GetMigrationResult(VAR(uint16, AUTOMATIC) Blocknumber)`
Parameter In	`Blocknumber` Logical block number as generated by Fee and saved in the auto-generated file Fee_Cfg.h (corresponds to the persistent ID).
Return Values	`MEMIF_RB_MIGRATION_RESULT_INIT_E` Initialization value, result not yet available. `MEMIF_RB_MIGRATION_RESULT_NOT_NECESSARY_E` RAM size == EEPROM size (migration not necessary). `MEMIF_RB_MIGRATION_RESULT_TO_SMALLER_SIZE_E` RAM size < EEPROM size (remaining data discarded). `MEMIF_RB_MIGRATION_RESULT_TO_BIGGER_SIZE_E` RAM size > EEPROM size (remaining data filled with zero). `MEMIF_RB_MIGRATION_RESULT_NOT_DONE_E` Migration not done, because data were not read. `MEMIF_RB_MIGRATION_RESULT_DEACTIVATED_E` Migrations is deactivated by configuration.
Description	Returns the migration result of the first read operation.
Note	Since the migration result can change asynchronously, a time consuming spin lock is needed. Consider this when creating do/while wait loops for the Fee.

2.3.2.5 Fee_Rb_DisableBackgroundOperation

Syntax	`Fee_Rb_DisableBackgroundOperation`
Description	Stops the execution of new background operations (i.e. Soft Sector Reorganisation and Sector Erase). Existing background tasks that are already running are not stopped or suspended. For example, if a sector reorganisation operation has started and one block is currently being transferred then the current block operation will be completed and next block will not be transferred. This interface should be called only when Fee Main functions are not running
Note 1	This interface might be used to delay background operations while critical writes take place.
Note 2	Application requests (read, write etc) and HSR will be processed as usual without any delay.
Note 3	Background operations can only be delayed till HSR is reached.

2.3.2.6 Fee_Rb_EnableBackgroundOperation

Syntax	`Fee_Rb_EnableBackgroundOperation`
Description	Resumes the execution of background operations (i.e. Soft Sector Reorganisation and Sector Erase) that have previously been halted by Fee_Rb_DisableBackgroundOperation. Should be called only when Fee Main functions are not running.

2.3.2.7 Fee_Rb_TriggerReorg

Syntax	`Fee_Rb_TriggerReorg(void)`
Description	Triggers a forced reorganisation.

2.3.2.8 Fee_Rb_TriggerReorgDat

Syntax	`Std_ReturnType Fee_Rb_TriggerReorgDat(void)`
Return Values	`E_OK` The requested job has been accepted by the module. `E_NOT_OK` The requested job has not been accepted by the module.
Description	Triggers one or more DAT sector reorganizations. It is configurable and only present if the configuration switch FeeRbTriggerReorg API is activated.

2.3.2.9 Fee_Rb_TriggerReorgFat

Syntax	`Std_ReturnType Fee_Rb_TriggerReorgFat(void)`
Return Values	`E_OK` The requested job has been accepted by the module. `E_NOT_OK` The requested job has not been accepted by the module.
Description	Triggers one or more FAT sector reorganizations. It is configurable and only present if the configuration switch FeeRbTriggerReorg API is activated.

2.3.2.10 Fee_Rb_EnterStopMode

Syntax	`Fee_Rb_EnterStopMode(void)`
Description	Puts the request for placing Fee into stop mode at the next possible synchronization point.
Note 1	The ongoing job gets completed, after which Fee is put into stop mode and no further jobs would be executed in this drive cycle.
Note 2	As soon as Fee enters stop mode it cannot be left without reinitializing the Fee. The stop mode can (and is supposed to) alter the Fee emulation area in the flash, thus Fee needs to analyse the DFLASH state again.

2.3.2.11 Fee_Rb_GetNrFreeBytes

Syntax	`Fee_Rb_GetNrFreeBytes(void)`
Description	Gets the amount of free space in DFLASH (in bytes). Returns 0 if the Fee is not initialized.

2.3.2.12 Fee_Rb_GetNrFreeBytesAndFatEntries

Syntax	`void Fee_Rb_GetNrFreeBytesAndFatEntries(uint32 * nrFreeBytes_pu32, uint32 * nrFreeFatEntries_pu32, boolean isEraseAccepted_b)`
Parameter In/Out	`nrFreeBytes_pu32` Pointer to the address where the number of free data bytes information shall be written. `nrFreeFatEntries_pu32` Pointer to the address where the number of free FAT entries information shall be written. `isEraseAccepted` TRUE: Free non-erased sectors shall be considered, erase operations may occur when writing the given number of bytes. FALSE: Free non-erased sectors shall not be considered, no erase operations when writing the given number of bytes.
Description	Provides the amount of data and the number of FAT entries in Data-NvM before a hard reorganization of the user. If the Fee is not initialized, the function sets the parameters to 0. Please note that not all Fee file systems are providing this feature. If not supported, a BCT error will be generated.

2.3.2.13 Fee_Rb_GetInternalJobResult

Syntax	`MemIf_JobResultType Fee_Rb_GetInternalJobResult (void)`
Return Values	`MEMIF_JOB_OK` The last job has been finished successfully. `MEMIF_JOB_PENDING` The last job is waiting for execution or currently being executed. `MEMIF_JOB_CANCELED` The last job has been cancelled (which means it failed). This return type is not currently used by Fee because the feature itself is not supported. `MEMIF_JOB_FAILED` The last job has not been finished successfully (it failed because non-reachable states were called, or the Fee/Fls/MemAcc are not yet initialized yet, or a HW failure occurred). `MEMIF_BLOCK_INCONSISTENT` The requested block is inconsistent, it may contain corrupted data. `MEMIF_BLOCK_INVALID` The requested block has been invalidated (intentionally by the user), the requested read operation cannot be performed (no error).
Description	Service to query the result of the last accepted job issued internally.

2.3.2.14 Fee_Rb_GetSystemJobResult

Syntax	`Fee_Rb_JobResultType_ten Fee_Rb_GetSystemJobResult(void)`
Return Values	`FEE_RB_JOB_OK` The last job has been finished successfully. `FEE_RB_JOB_PENDING` The last job is waiting for execution or currently being executed. `FEE_RB_JOB_CANCELED` This return type is currently not used by the Fee. `FEE_RB_JOB_FAILED` The last job has not been finished successfully (it failed e.g. because non-reachable states were called or the Fee/Fls/MemAcc was not initialized or a HW failure occurred). `FEE_RB_JOB_INCONSISTENT` This return type is currently not used by the Fee. `FEE_RB_JOB_INVALID` This return type is currently not used by the Fee. `FEE_RB_BLOCK_PART_OK` Processing of last chunk has finished successfully, ready for next chunk.
Description	This function is used for retrieving the system job status when using the file system FS1X and either the forced system reorganisation API (FeeRbTriggerReorgApi = TRUE) or the driver stop mode API (FeeRbEnterStopMod-eApi = TRUE) is enabled.
Note	To enable this feature, the parameter FeeRbConfir-mUseGetSystemJobResult = TRUE must be set.

2.3.2.15 Fee_Rb_GetStressMeasmtResults

Syntax	`Std_ReturnType Fee_Rb_GetStressMeasmtResults (uint32 *nrDrvCycMaxReaTot_pu32, uint32 *nrDrvCycMaxReaPredStrt_pu32)`
Parameter In	`nrDrvCycMaxReaTot_pu32` Pointer to location where the first result shall be stored: Maximum reachable driving cycles over ECU lifetime. `nrDrvCycMaxReaPredStrt_pu32` Pointer to location where the second result shall be stored: Maximum reachable driving cycles since measurement (re-)start.
Return Values	`E_OK` Both requested results have been provided in the given pointers. `E_NOT_OK` Any requested result couldn't be provided because of some error.
Description	Gets results of stress measurement and driving cycle prediction.

Note	The request is accepted in case the parameters are valid, and the execution conditions are met. A NULL_PTR for nrDrvCycMaxReaTot_pu32/nrDrvCycMaxReaPredStrt_pu32, or a user call for an uninitialized driver are not accepted. If Development Error Detection is enabled, the function reports an error in any failure case. This service is only available, if the stress measurement and driving cycle prediction is enabled.

2.3.2.16 Fee_Rb_RestartStressMeasmt

Syntax	`Std_ReturnType Fee_Rb_RestartStressMeasmt (void)`
Return Values	`E_OK` The request has been accepted; stress measurement and driving cycle prediction are (re-)started. `E_NOT_OK` The request has been erroneous and therefore been rejected.
Description	Restarts the stress measurement and driving cycle prediction. This service is only available if the stress measurement and driving cycle prediction is enabled.

2.3.2.17 Fee_Rb_WritePersistentData

Syntax	`Std_ReturnType Fee_Rb_WritePersistentData(void)`
Description	Service to trigger a write request of Fee internal data to flash.

2.3.2.18 Fee_Rb_GetSectEraseCntr

Syntax	`uint32 Fee_Rb_GetSectEraseCntr (uint8 physSectIdx_u8)`
Parameter In	`physSectIdx_u8` Number of the physical sector.
Description	Returns the number of sector erases of the requested physical sector.

2.3.2.19 Fee_Rb_GetNrOfSuprtdErases

Syntax	`uint32 Fee_Rb_GetNrOfSuprtdErases (void)`
Description	Returns the hardware dependent maximum number of supported erase cycles of the data flash.

2.3.2.20 Fee_Rb_SetJobMode

Syntax	`Std_ReturnType Fee_Rb_SetJobMode(Fee_Rb_JobType_ten jobType_en, Fee_Rb_JobMode_ten jobMode_en)`
Parameters In	`jobType_en` Type of the job for which mode has to be set. `jobMode_en` Mode which has to be set for the job.
Description	Changes the algorithm of a job.

2.3.2.21 Fee_Rb_GetJobMode

Syntax	`Fee_Rb_JobMode_ten Fee_Rb_SetJobMode(Fee_Rb_JobType_ten jobType_en)`
Parameter In	`jobType_en` Type of the job for which the current mode is requested.
Description	Returns the current mode for the specified job type.

2.3.2.22 Fee_Rb_GetMode

Syntax	`MemIf_ModeType Fee_Rb_GetMode(void)`
Description	Returns the actual active driver mode, which was set by Fee_SetMode(). This API is only available if Fee_SetMode() is available, otherwise the driver mode is always MEMIF_MODE_SLOW.

2.3.2.23 Fee_Rb_GetDetailedBlkInfo

Syntax	`MemIf_Rb_DetailedBlkInfo_ten Fee_Rb_GetDetailedBlkInfo (uint16 blockNumber_u16)`
Parameter In	`blockNumber_u16` Number of logical block, generated by Fee and saved in the auto-generated file "Fee_Cfg.h" and/or "Fee_Cfg_BlockDefines.h" (this is not the persistent ID).
Return Values	`MEMIF_RB_DETAILED_BLK_INFO_NOT_AVAILABLE_E` Detailed block information feature not available: Disabled or not supported by filesystem (FS1). `MEMIF_RB_DETAILED_BLK_INFO_INIT_E` Block was never read inconsistent - result not available OR unknown/invalid block identifier OR Fee in UNINIT state. `MEMIF_RB_DETAILED_BLK_INFO_INCONSISTENT_MISSING_E` Block was read with result MEMIF_BLOCK_INCONSISTENT - A valid block header was not found on the medium (Header CRC error, Header missing). `MEMIF_RB_DETAILED_BLK_INFO_INCONSISTENT_DATA_E` Block was read with result MEMIF_BLOCK_INCONSISTENT - A valid block header was found but Data-CRC error occurred when reading.
Description	Returns detailed information for the specified block.

2.3.2.24 Fee_Rb_BlockMaintenance

Syntax	`Std_ReturnType Fee_Rb_BlockMaintenance(uint16 BlockNumber)`
Parameter In	`BlockNumber` Number of logical block, generated by Fee and saved in the auto-generated file "Fee_Cfg.h" (corresponds to the persistent ID).
Description	Repairs a redundant block with security level setting 2, if one of the copies has corrupted data.

2.3.2.25 Fee_Rb_GetWorkingState

Syntax	`Fee_Rb_WorkingStateType_ten Fee_Rb_GetWorkingState(void)`
Return Values	`FEE_RB_IDLE_E` Fee is in idle state. `FEE_RB_WRITE_MODE_E` Write order is processed by Fee. `FEE_RB_READ_MODE_E` Read order is processed by Fee. `FEE_RB_INVALIDATE_MODE_E` Block invalidation is processed by Fee. `FEE_RB_MAINTAIN_MODE_E` Block maintenance is processed by Fee. `FEE_RB_SOFT_SECTOR_REORG_MODE_E` Soft sector reorganization is processed by Fee in the background. `FEE_RB_HARD_SECTOR_REORG_MODE_E` Hard sector reorganization is processed by Fee. `FEE_RB_SECTOR_ERASE_E` Sector erase is processed by Fee. `FEE_RB_STOPMODE_E` Fee is in stop mode.
Description	Provides the current order which is processed by Fee. If Fee is not initialized, the function returns FEE_RB_IDLE_E.

2.3.2.26 Fee_Rb_MainFunctionAndDependency

Syntax	`void Fee_Rb_MainFunctionAndDependency (void)`
Description	Service to invoke the Fee MainFunction and its lower layer dependencies (FIs/MemAcc).

2.3.2.27 Fee_Rb_IsBlockDoubleStorageActiveByBlockNr

Syntax	`boolean Fee_Rb_IsBlockDoubleStorageActiveByBlockNr(uint16 blkNr_u16)`
Parameter In	`blkNr_u16` Number of logical block, generated by Fee and saved in the auto-generated file “Fee_Cfg.h” and/or “Fee_Cfg_BlockDefines.h” (this is not the persistent ID).
Description	Returns whether or not the specified block is configured by the FeeRbSecLevel parameter as a double storage block.

2.3.2.28 Fee_Rb_GetSectState

Syntax	`Fee_Rb_SectState_ten Fee_Rb_GetSectState(uint16 idxFeePhys_u16)`
Parameter In	`uint16` Fee sector/row number for which state shall be returned.

Return Values	`FEE_RB_SECT_STATE_UNDEFD_E` Sector or row is undefined state. Has no valid markers and user data yet. `FEE_RB_SECT_STATE_ERASED_E` Sector or row has valid marker but no user data yet. `FEE_RB_SECT_STATE_USED_E` Sector or row has valid marker and valid user data. `FEE_RB_SECT_STATE_DEFECTIVE_E` Sector or row is defective and cannot be used (e.g. read, erase, program failure).
Description	Returns the simplified state for the requested sector passed as parameter.

2.3.2.29 Fee_Rb_EnterHighSpeedMode

Syntax	`Std_ReturnType Fee_Rb_EnterHighSpeedMode(void)`
Returns	`E_OK` Order was successfully placed. `E_NOT_OK` Order was rejected. Either by Fls/MemAcc function or high-speed mode already set for another user (e.g. Adapter user). Only one user can be in high-speed mode. Exit the high-speed mode for the other user and try again for the NvM user.
Description	Sets Fee to high-speed mode for the NvM user and also calls the corresponding Fls/MemAcc function. Can be called both when a job is ongoing or there are no jobs being processed. The mode change will be reflected only for new jobs and not for ongoing jobs.

2.3.2.30 Fee_Rb_ExitHighSpeedMode

Syntax	`Std_ReturnType Fee_Rb_EnterHighSpeedMode(void)`
Returns	`E_OK` Order was successfully placed. `E_NOT_OK` Order was rejected. Either by Fls/MemAcc function or high-speed mode is set for another user (e.g. Adapter user). Only the user currently set in high-speed mode can exit the mode. All other users should wait until the current user has been exited.
Description	Enables all jobs and exits high-speed mode for the NvM user and also calls the corresponding Fls/MemAcc function. This API is called to reverse the actions done by the function Fee_Rb_Idx_EnterHighSpeedMode().

2.3.2.31 Fee_Rb_SetCurMachine

Syntax	`Std_ReturnType Fee_Rb_SetCurMachine(Fee_Rb_Machine_ten curMachine_en)`
Parameters In	`Fee_Rb_Machine_ten` Machine type to be set. `curMachine_en`
Returns	`E_OK`: Machine type is set successfully. `E_NOT_OK`

Description	Sets the current machine type when multiple devices are configured using the `FeeRbMplDevCfg` container.

2.3.2.32 Fee_Rb_GetFlashProp

Syntax	`Std_ReturnType Fee_Rb_GetFlashProp(Fee_Rb_FlashEmulnProp_tst *flashProp_pst)`
Parameters Out	`flashProp_pst` Pointer to the Flash properties structure.
Returns	`E_OK`: Flash properties are fetched successfully. `E_NOT_OK`: Request is rejected, either Fee is not initialized or Fee is in stop mode.
Description	Fetches the flash emulation properties (Size, Number of sectors, Start and End Address).

2.4 Limitations

- The concept of a virtual address space and "spread mechanism" for data items and their storage (SWS_Fee_00005 and SWS_Fee_00103) is not supported because this would lead to an inefficient implementation.
- Restrictions deriving from a single user concept (i.e. NvM only) are not relevant to Fee because it allows access from different types of "block administrators" (i.e. the Multi-User concept) in order to be useable in different product lines. (Related SWS items include SWS_Fee_00133, SWS_Fee_00144, SWS_Fee_00142, SWS_Fee_00055, SWS_Fee_00098 and SWS_Fee_00056)
- Fee_Cancel is deactivated by default. It can be activated by setting the configuration parameter FeeRbCancel = TRUE. (SWS_Fee_00124, SWS_Fee_00080, SWS_Fee_00081, SWS_Fee_00164, SWS_Fee_00184, SWS_Fee_00157).
- Fee_EraseImmediateBlock is not supported.
- The setting of the Init status inside the Fee_MainFunction (SWS_Fee_00169) is not supported.
- Nvm Job end notification (`FeeNvmJobEndNotification`) is not supported - use the `Fee_GetJobResult/Fee_Rb_GetAdapterJobResult` APIs instead. (Affected SWS items: FEE098, FEE055. Open: FEE112_Conf).
- Nvm Job error notification (`FeeNvmJobErrorNotification`) is not supported - use the `Fee_GetJobResult/Fee_Rb_GetAdapterJobResult` APIs instead. (Affected SWS items: FEE099, FEE056. Open: FEE113_Conf.)
- The parameters FeeBufferAlignmentValueRef and FeeMinimumReadPageSize are not supported. Instead, the MemMap section FEE_START_SEC_VAR_SECURED_CACHED_RAM_ALIGNED_CLEARED_UNSPECIFIED can be used to align Fee buffers containing data which is sent to lower layers. The recommendation is an alignment of 64 bytes. (Affected SWS items:

ECUC_Fee_00156, ECUC_Fee_00157.)

- (SWS_Fls_00106) Fls shall not return MEMIF_JOB_FAILED for `Fls_Read` when the job has detected bit flips (or ECC failures). It shall always return the data as is, with return value MEMIF_JOB_OK (the validation will be performed by Fee).
- (SWS_Fls_00361) Fls shall return MEMIF_JOB_FAILED for any job when the timeout supervision has failed (AUTOSAR is unclear regarding this failure).

2.5 Exclusions

The following features from AR v22-11 are not supported in version v12.7.0.

API functions not currently implemented:

`Fee_JobEndNotification`

The following configuration parameters from AR v22-11 are not supported in version v12.7.0.

- Fee/FeeGeneral/FeeBufferAlignmentValue
- Fee/FeeGeneral/FeeMinimumReadPageSize

2.6 Deviations

2.6.1 Configuration Deviations

Configuration parameters which differ from the AR v22-11 specification are listed below.

2.6.1.1 Fee

[RTA-BSW] Description	Configuration of the Fee (Flash eeprom emulation) module.
[AUTOSAR] Description	Configuration of the Fee (Flash EEPROM Emulation) module.
[RTA-BSW] Upper Multiplicity	254
[AUTOSAR] Upper Multiplicity	1

2.6.1.2 Fee/FeeBlockConfiguration

[RTA-BSW] Upper Multiplicity	65535
[AUTOSAR] Upper Multiplicity	*

2.6.1.3 Fee/FeeBlockConfiguration/FeeBlockNumber

[RTA-BSW] Description	This tag has currently no effect, the FeeRbBlockPersistentId has to be configured instead. The FeeBlockNumber will be calculated by the Fee automatically. This can be used as a parameter in the user interfaces in order to identify the relevant block.

[AUTOSAR] Description	Block identifier (handle).
[RTA-BSW] Introduction	0x0000 and 0xFFFF shall not be used for block numbers (see FEE006). Range: min = 2^NVM_DATASET_SELECTION_BITS, max = 0xFFFF - 2^NVM_DATASET_SELECTION_BITS. Note: Depending on the number of bits set aside for dataset selection several other block numbers shall also be left out to ease implementation.
[AUTOSAR] Introduction	0x0000 and 0xFFFF shall not be used for block numbers (see FEE006). Range: min = 2^NVM_DATASET_SELECTION_BITS max = 0xFFFF -2^NVM_DATASET_SELECTION_BITS Note: Depending on the number of bits set aside for dataset selection several other block numbers shall also be left out to ease implementation.
[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1

2.6.1.4 Fee/FeeBlockConfiguration/FeeBlockSize

[ADDED] Introduction	If no value is defined for this parameter, NvM provides the block's size by a C-macro. In that case, the general parameter FeeRbNvMBlock HeaderInclude must indicate where to find the macro(s), and the block-specific parameter FeeRbOrigin must be "NVM".
[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1

2.6.1.5 Fee/FeeBlockConfiguration/FeeImmediateData

[RTA-BSW] Description	If the Fee's queue contains more than one order, high priority data will be favored.
[AUTOSAR] Description	Marker for high priority data.

2.6.1.6 Fee/FeeBlockConfiguration/FeeMemAccAddressArea

[RTA-BSW] Description	Reference to the MemAccAddressAreaConfiguration.
[AUTOSAR] Description	Reference to the MemAccAddressAreaConfiguration. Tags: atp.Status=draft

[ADDED] Introduction	Hints on configuration: - Do not configure if Fls is used instead of MemAcc - If parameter is not configured but MemAcc is used, Fee expects a MemAccAddressAreaConfiguration named “Fee” to be present in arxml config which is configured by user or MemAcc_AddressArealdType value named “MemAccConf_MemAccAddressAreaConfiguration_Fee” defined in the code and exported by public MemAcc header - If parameter is configured, all configured blocks in FeeBlockConfiguration container need to have the same reference. Different addess areas are not supported by Fee. Instead multiple instances of Fee can be configured using multiple Fee modules

2.6.1.7 Fee/FeeBlockConfiguration/FeeNumberOfWriteCycles

[RTA-BSW] Description	Number of write cycles (app.) required for this block (during the complete ECU lifetime). This tag is used for calculating the reachable driving cycles in some product lines.
[AUTOSAR] Description	Number of write cycles required for this block.

2.6.1.8 Fee/FeeGeneral/FeeDevErrorDetect

[RTA-BSW] Description	Pre-processor config parameter to enable or disable development error detection.
[AUTOSAR] Description	Switches the development error detection and notification on or off.
[RTA-BSW] Introduction	true: Development error detection enabled. false: Development error detection detection disabled.
[AUTOSAR] Introduction	* true: detection and notification is enabled. * false: detection and notification is disabled.
[REMOVED] Default Value	false

2.6.1.9 Fee/FeeGeneral/FeeMainFunctionPeriod

[RTA-BSW] Maximum Value	1
[AUTOSAR] Maximum Value	INF
[ADDED] Default Value	0

2.6.1.10 Fee/FeeGeneral/FeeNvmJobEndNotification

[RTA-BSW] Description	This feature is not supported.

[AUTOSAR] Description	Mapped to the job end notification routine provided by the upper layer module (NvM_JobEndNotification).
[RTA-BSW] Upper Multiplicity	0
[AUTOSAR] Upper Multiplicity	1

2.6.1.11 Fee/FeeGeneral/FeeNvmJobErrorNotification

[RTA-BSW] Description	This feature is not supported.
[AUTOSAR] Description	Mapped to the job error notification routine provided by the upper layer module (NvM_JobErrorNotification).
[RTA-BSW] Upper Multiplicity	0
[AUTOSAR] Upper Multiplicity	1

2.6.1.12 Fee/FeeGeneral/FeePollingMode

[RTA-BSW] Description	Pre-processor config parameter to enable or disable the polling mode for this module (usage of the job result functionality).
[AUTOSAR] Description	Pre-processor switch to enable and disable the polling mode for this module.
[RTA-BSW] Introduction	true: Polling mode enabled, callback functions (provided to Fls/MemAcc) disabled. false: Polling mode disabled, callback functions (provided to Fls/MemAcc) enabled.
[AUTOSAR] Introduction	true: Polling mode enabled, callback functions (provided to MemAcc module) disabled. false: Polling mode disabled, callback functions (provided to MemAcc module) enabled.

2.6.1.13 Fee/FeeGeneral/FeeVersionInfoApi

[RTA-BSW] Description	Pre-processor config parameter to enable/disable the API to read out the modules version information.
[AUTOSAR] Description	Pre-processor switch to enable / disable the API to read out the modules version information.
[REMOVED] Default Value	false

2.6.1.14 Fee/FeeGeneral/FeeVirtualPageSize

[RTA-BSW] Description	This parameter doesn't have any effect as the correct value is determined internally by the Fee. The size in bytes, to which logical blocks shall be aligned. Configure instead the actual virtual page-size within the FeeFsX parameters: rba_FeeRefFs1xLogPageSize, rba_FeeRefFs2LogPageSize or rba_FeeRefFs3LogPageSize.
[AUTOSAR] Description	The size in bytes to which logical blocks shall be aligned.
[ADDED] Introduction	Set to 0 to avoid any code generator aborts.
[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1
[ADDED] Default Value	0

2.6.1.15 Fee/FeePublishedInformation

[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1

2.6.1.16 Fee/FeePublishedInformation/FeeBlockOverhead

[RTA-BSW] Description	Management overhead per logical block in bytes (block header size).
[AUTOSAR] Description	Management overhead per logical block in bytes.
[RTA-BSW] Introduction	Note: If the management overhead depends on the block size or block location a formula has to be provided that allows the configurator to calculate the management overhead correctly. This tag is currently ignored.
[AUTOSAR] Introduction	Note: If the management overhead depends on the block size or block location a formula has to be provided that allows the configurator to calculate the management overhead correctly.
[RTA-BSW] Multiplicity	0..0
[AUTOSAR] Multiplicity	1..1

2.6.1.17 Fee/FeePublishedInformation/FeePageOverhead

[RTA-BSW] Introduction	Note: If the management overhead depends on the block size or block location a formula has to be provided that allows the configurator to calculate the management overhead correctly. This tag is currently ignored.
[AUTOSAR] Introduction	Note: If the management overhead depends on the block size or block location a formula has to be provided that allows the configurator to calculate the management overhead correctly.
[RTA-BSW] Multiplicity	0..0
[AUTOSAR] Multiplicity	1..1

2.7 API Definition

Table 2.111.Fee Supported API functions

`Fee_Init`	Initialises the Fee module.
`Fee_Read`	Initiates a read job.
`Fee_Write`	Initiates a write job.
`Fee_GetStatus`	Returns the status.
`Fee_GetJobResult`	Queries the result of the last accepted job issued by the upper layer software.
`Fee_InvalidateBlock`	Invalidates a logical block.
`Fee_GetVersionInfo`	Returns the version information of the Fee module.
`Fee_MainFunction`	Main function of the Fee module.
`Fee_EraseImmediateBlock`	Erases a logical block.
`Fee_Cancel`	Calls the cancel function of the underlying Fls/MemAcc.

For further information on these functions, please refer to AUTOSAR_SWS_FlashEEPROMEmulation.pdf

2.8 Configuration Advice

NOTE

Before configuring this module, please note that some of the default values used by ConfGen to produce the default ECU Configuration can be changed, and this is done by either adding a Conf-Gen Enhancer Mapping (CEM) or by adding a rule to an *algo.properties* file. For more information, see the “Configuration Generation” section of the RTA-CAR User Guide.

2.8.1 Fee interaction with Crc

Fee uses the Crc interfaces “Crc_CalculateCRC16” and “Crc_CalculateCRC32”.

The Crc module provides two different kinds of Crc calculation modes: “TABLE” and “RUNTIME”. In order to speed up the Fee behaviour, especially during the ECU’s startup routines, the “TABLE” variant should be configured and used by the system.

2.8.2 FeeRbMaintainRedundantBlocks

In order to save ROM resources when the Maintain Redundant Blocks feature is not used, the default setting of FeeRbMaintainRedundantBlocks (in FeeGeneral) has been changed from TRUE to FALSE. If required, this feature must now be explicitly enabled, with the following considerations:

- For an FS1 filesystem without NvM, set FeeRbMaintainRedundantBlocks to TRUE.
- For an FS1 filesystem with NvM, set FeeRbMaintainRedundantBlocks to TRUE (if not forwarded by NvM).

2.8.3 FeeRbOrigin

Support for the Job End and Job Error callbacks (using the FeeRbCallbackEnd and FeeRbCallbackError parameters) has been removed and replaced with the FeeRbOrigin API, as follows:

- For Nvm jobs (FeeRbOrigin == “NVM”): The order processing state can be checked by polling the job result using the Fee_GetJobResult API.
- For Adapter jobs (FeeRbOrigin == “ADAPTER”): The order processing state can be checked by polling the job result using the Fee_Rb_GetAdapterJobResult API.

2.8.4 FeeRbMachineType

The naming scheme for Fee machine types has been changed. The list below summarizes the old and new values. Note that if FeeRbMachineType is not configured, the machine type will be obtained from the Mcu module and mapped to the correct Fee machine type (if supported).

You can make any required changes manually, or you can use a provided Java Model Updater action (FeeUpdateModel.java), which will migrate your configuration and automatically update your Fee EcuCValues ARXML file.

- MCU_RB_IFX_UC1_DEV2_B → FEE_RB_IFX_TC26X
- MCU_RB_IFX_UC1_DEV3_C → FEE_RB_IFX_TC27X
- MCU_RB_IFX_UC1_DEV3_D → FEE_RB_IFX_TC27X
- MCU_RB_IFX_UC1_DEV3_C_HSM → FEE_RB_IFX_TC27X_HSM
- MCU_RB_IFX_UC1_DEV3_D_HSM → FEE_RB_IFX_TC27X_HSM

- MCU_RB_IFX_UC1_DEV4_B → FEE_RB_IFX_TC29X
- MCU_RB_IFX_UC1_DEV4_B_HSM → FEE_RB_IFX_TC29X_HSM
- FEE_RB_IFX_TC33X_40NM_CMPL → FEE_RB_IFX_TC33X_CMPL
- FEE_RB_IFX_TC33X_40NM_SE → FEE_RB_IFX_TC33X_SE
- MCU_RB_IFX_TC35X_A → FEE_RB_IFX_TC35X_CMPL
- FEE_RB_IFX_TC35X_40NM_SE → FEE_RB_IFX_TC35X_SE
- MCU_RB_IFX_UC1_DEV3_40NM_A → FEE_RB_IFX_TC37X_CMPL
- FEE_RB_IFX_TC37X_40NM_SE → FEE_RB_IFX_TC37X_SE
- MCU_RB_IFX_UC1_DEV4_40NM_A → FEE_RB_IFX_TC38X_CMPL
- FEE_RB_IFX_TC38X_40NM_SE → FEE_RB_IFX_TC38X_SE
- MCU_RB_IFX_UC1_DEV5_40NM_A → FEE_RB_IFX_TC39X_CMPL
- MCU_RB_IFX_UC1_DEV5_40NM_B → FEE_RB_IFX_TC39X_CMPL
- FEE_RB_IFX_TC39X_40NM_SE → FEE_RB_IFX_TC39X_SE
- FEE_RB_IFX_TC3EX_40NM_CMPL → FEE_RB_IFX_TC3EX_CMPL
- FEE_RB_IFX_TC3EX_40NM_SE → FEE_RB_IFX_TC3EX_SE
- FEE_RB_IFX_40NM_CMPL_HSM → FEE_RB_IFX_TC3XX_CMPL_HSM
- MCU_RB_IFX_TC35X_A_HSM → FEE_RB_IFX_TC3XX_CMPL_HSM
- MCU_RB_IFX_UC1_DEV3_40NM_A_HSM → FEE_RB_IFX_TC3XX_CMPL_HSM
- MCU_RB_IFX_UC1_DEV4_40NM_A_HSM → FEE_RB_IFX_TC3XX_CMPL_HSM
- MCU_RB_IFX_UC1_DEV5_40NM_A_HSM → FEE_RB_IFX_TC3XX_CMPL_HSM
- MCU_RB_IFX_UC1_DEV5_40NM_B_HSM → FEE_RB_IFX_TC3XX_CMPL_HSM
- FEE_RB_IFX_40NM_SE_HSM → FEE_RB_IFX_TC3XX_SE_HSM
- FEE_RB_IFX_TC49X → FEE_RB_IFX_TC49X
- MCU_RB_JDP_UC1_DEV1 → FEE_RB_JDP_MPC5726
- MCU_RB_JDP_UC1_DEV2_C2 → FEE_RB_JDP_MPC5744
- MCU_RB_JDP_UC1_DEV3_C2 → FEE_RB_JDP_MPC5746B
- MCU_RB_ST_UC1_DEV3PLUS_C1 → FEE_RB_JDP_MPC5746B
- MCU_RB_JDP_UC1_DEV3_C2_HSM → FEE_RB_JDP_MPC5746B_HSM
- MCU_RB_ST_UC1_DEV3PLUS_C1_HSM → FEE_RB_JDP_MPC5746B_HSM
- MCU_RB_ST_UC1_DEV3_40NM_C3_HSM → FEE_RB_JDP_MPC5746B_HSM
- MCU_RB_ST_UC1_DEV3_40NM_C3_2_HSM → FEE_RB_JDP_MPC5746B_HSM

- MCU_RB_JDP_UC1_DEV4_C2 → FEE_RB_JDP_MPC5777
- MCU_RB_JDP_UC1_DEV4_C2_HSM → FEE_RB_JDP_MPC5777_HSM
- MCU_RB_ST_UC1_DEV3_40NM_C3 → FEE_RB_ST_SPC58XNX
- MCU_RB_ST_UC1_DEV3_40NM_C3_2 → FEE_RB_ST_SPC58XNX
- FLS_RB_ST_STELLAR_16M_C1 → FEE_RB_ST_CENTAURI
- FLS_RB_ST_STELLAR_16M_C1_HSM → FEE_RB_ST_CENTAURI_HSM
- FLS_RB_ST_UCU_DEV5_SR6P6 → FEE_RB_ST_SR6P6
- FLS_RB_ST_UCU_DEV5_SR6P6_HSM → FEE_RB_ST_SR6P6_HSM
- FEE_RB_ST_CHORUS_1M_DF64K → FEE_RB_ST_SPC58XBX
- FEE_RB_ST_CHORUS_2M_DF64K → FEE_RB_ST_SPC58XBX
- FEE_RB_ST_CHORUS_4M_DF128K → FEE_RB_ST_SPC58XCX
- FEE_RB_ST_CHORUS_6M_DF256K → FEE_RB_ST_SPC58X_GH_X
- FEE_RB_ST_CHORUS_10M_DF256K → FEE_RB_ST_SPC58X_GH_X
- FEE_RB_ST_CHORUS_4M_HSM_DF32K → FEE_RB_ST_SPC58XXX_HSM
- FEE_RB_ST_CHORUS_6M_HSM_DF32K → FEE_RB_ST_SPC58XXX_HSM
- FEE_RB_ST_CHORUS_10M_HSM_DF32K → FEE_RB_ST_SPC58XXX_HSM
- MCU_RB_ST_CHORUS_4M_HSM → FEE_RB_ST_SPC58XXX_HSM
- FEE_RB_MPC5746C → FEE_RB_NXP_MPC5746C
- FEE_RB_MPC5746C_HSM → FEE_RB_NXP_MPC5746C_HSM
- MCU_RB_V850R1X_512K → FEE_RB_REN_RH850R1X_512KB_4S_8KB
- MCU_RB_V850R1X_1M → FEE_RB_REN_RH850R1X_1MB_4S_8KB
- MCU_RB_V850R1X_512K_REDUCED → FEE_RB_REN_RH850R1X_512K-B_4S_7_5KB
- MCU_RB_V850R1X_1M_REDUCED → FEE_RB_REN_RH850R1X_1M-B_4S_7_5KB
- FEE_RB_RH850_F1KM_S1_DF64K → FEE_RB_REN_RH850F1K-M_S1_4S_16KB
- FEE_RB_RH850_F1KM_S1_DF62_2K_REDUCED → FEE_R-B_REN_RH850F1KM_S1_4S_15_5KB
- FEE_RB_RH850_F1KM_S1_DF60_4K_REDUCED → FEE_R-B_REN_RH850F1KM_S1_4S_15KB
- FEE_RB_RH850_F1KM_S4_104K_Host → FEE_RB_REN_RH850F1K-M_S4_6S_17_375KB
- FEE_RB_RH850_F1KM_S4_23K_HSM → FEE_RB_REN_RH850F1K-M_S4_4S_5_9375KB_HSM
- FEE_RB_CYT2B7 → FEE_RB_CYP_CYT2B7X_4S_18K
- FEE_RB_CYT2B7_HSM → FEE_RB_CYP_CYT2B7X_4S_6K_HSM
- FEE_RB_CYT2B9 → FEE_RB_CYP_CYT2B9X_6S_16K

- FEE_RB_CYT2B9_HSM → FEE_RB_CYP_CYT2B9X_4S_8K_HSM
- MCU_RB_RH850_F1M_CF3M_DF64K → FEE_RB_REN_RH850F1M_4S_16KB
- MCU_RB_RH850_F1M_CF3M_DF62_2K_REDUCED → FEE_R-B_REN_RH850F1M_4S_15_5KB
- MCU_RB_RH850_F1M_CF3M_DF60_4K_REDUCED → FEE_R-B_REN_RH850F1M_4S_15KB
- FEE_RB_RCAR_S25FS256 → FEE_RB_EXT_CYP_S25FS256_IN-T_REN_RCAR
- FEE_RB_RCAR_S25FS512 → FEE_RB_EXT_CYP_S25FS512_INT_REN_RCAR
- FEE_RB_RCAR_S70FS01GS → FEE_RB_EXT_CYP_S70FS01GS_IN-T_REN_RCAR
- FEE_RB_RCAR_S70FS01GS_HSM → FEE_RB_EXT_CYP_S70FS01GS_IN-T_REN_RCAR_HSM
- FEE_RB_RCAR_S25HS01GT → FEE_RB_EXT_CYP_S25HS01GT_IN-T_REN_RCAR
- FEE_RB_RCAR_S25HS01GT_HSM → FEE_RB_EXT_CYP_S25HS01GT_IN-T_REN_RCAR_HSM
- FEE_RB_ZYNQ_W25Q128JV → FEE_RB_EXT_WIN_W25Q128JV_INT_X-IL_ZYNQ
- FEE_RB_ZYNQ_W25Q256JV → FEE_RB_EXT_WIN_W25Q256JV_INT_X-IL_ZYNQ
- FEE_RB_ZYNQ_S70FS01GS → FEE_RB_EXT_CYP_S70FS01GS_INT_X-IL_ZYNQ
- FEE_RB_ZYNQ_S25FS512S → FEE_RB_EXT_CYP_S25FS512S_INT_X-IL_ZYNQ
- FEE_RB_ZYNQ_MT25QU01G → FEE_RB_EXT_MIC_MT25QU01G_INT_X-IL_ZYNQ
- FEE_RB_ZYNQ_MT25QU02G → FEE_RB_EXT_MIC_MT25QU02G_INT_X-IL_ZYNQ
- FEE_RB_ZYNQ_MX66U1G → FEE_RB_EXT_MAC_MX66U1G_INT_XIL_ZYNQ
- FEE_RB_ZYNQ_MX66U2G → FEE_RB_EXT_MAC_MX66U2G_INT_X-IL_ZYNQ
- FEE_RB_S32_MT35XU512 → FEE_RB_EXT_MIC_MT35XU512_INT_NX-P_S32
- FEE_RB_RH850U2A16_4S_240KBDF_16KBM → FEE_R-B_REN_RH850U2A16_4S_60KB
- FEE_RB_RH850U2A16_HSM → FEE_RB_REN_RH850U2A16_4S_16KB_HSM
- FEE_RB_RH850U2A8_4S_16KBM → FEE_RB_REN_RH850U2A8_4S_60KB
- FEE_RB_RH850U2A8_HSM → FEE_RB_REN_RH850U2A8_4S_16KB_HSM
- MCU_RB_REN_RH850_P1LC_D1 → FEE_RB_REN_RH850P1L-

C_D1_4S_7_75KBFEE_RB_REN_RH850P1LC_D1_4S_7_75KB

- MCU_RB_REN_RH850_P1LC_D2 → FEE_RB_REN_RH850P1L-C_D2_4S_15_75KB
- FLS_RB_RH850P1M_1MB → FEE_RB_REN_RH850P1M_4S_8KB
- MCU_RB_RH850P1XC_DEV3 → FEE_RB_REN_RH850P1X-C_D3_4S_15_75KB
- MCU_RB_RH850P1XC_DEV3_6S → FEE_RB_REN_RH850P1X-C_D3_6S_10_5KB
- MCU_RB_RH850P1XC_DEV3_REDUCED → FEE_RB_REN_RH850P1X-C_D3_4S_14_75KB
- MCU_RB_RH850P1XC_DEV3_REDUCED_6S → FEE_RB_REN_RH850P1X-C_D3_6S_9_8125KB
- FLS_RB_RH850P1XC_DEV3_9K4S_24KC → FEE_RB_REN_RH850P1X-C_D3_4S_9_75KB
- MCU_RB_RH850P1XC_DEV3_HSM → FEE_RB_REN_RH850P1X-C_D3_2S_15_5KB_HSM
- MCU_RB_RH850P1XC_DEV4 → FEE_RB_REN_RH850P1X-C_D4_4S_31_75KB
- MCU_RB_RH850P1XC_DEV4_REDUCED → FEE_RB_REN_RH850P1X-C_D4_4S_30_75KB
- MCU_RB_RH850P1XC_DEV4_HSM → FEE_RB_REN_RH850P1X-C_D4_2S_15_5KB_HSM
- MCU_RB_RH850P1XC_DEV5 → FEE_RB_REN_RH850P1X-C_D5_8S_23_875KB
- MCU_RB_RH850P1XC_DEV5_REDUCED → FEE_RB_REN_RH850P1X-C_D5_8S_23_375KB
- MCU_RB_RH850P1XC_DEV5_HSM → FEE_RB_REN_RH850P1X-C_D5_2S_15_5KB_HSM
- FEE_RB_RH850P1XC_MX25L32 → FEE_RB_EXT_MAC_MX25L32_IN-T_REN_RH850P1XC
- MCU_RB_PC → FEE_RB_PC_6S_64KB
- MCU_RB_PC_16B → FEE_RB_PC_6S_64KB_16BP
- MCU_RB_PC_2SEC → FEE_RB_PC_2S_64KB
- FEE_RB_PC_4SEC_24KB → FEE_RB_PC_4S_24KB
- MCU_RB_PC_4SEC_64KB → FEE_RB_PC_4S_64KB
- MCU_RB_PC_4SEC_256KB → FEE_RB_PC_4S_256KB
- MCU_RB_PC_6SEC_128KB → FEE_RB_PC_6S_128KB
- MCU_RB_PC_8SEC_8KB → FEE_RB_PC_8S_8KB
- MCU_RB_PC_8SEC_16KB → FEE_RB_PC_8S_16KB
- MCU_RB_PC_8SEC_32KB → FEE_RB_PC_8S_32KB

- MCU_RB_PC_8SEC_64KB → FEE_RB_PC_8S_64KB
- FEE_RB_FULL_MANUAL_CFG → FEE_RB_FULL_MANUAL_CFG

To create a manual configuration, you must set *FeeRbMachineType* to **FEE_RB_-FULL_MANUAL_CFG** and follow these steps:

1. Create a new *rba_FeeRefDeviceCfg* module with:

- Name is *rba_FeeRefDeviceCfg*.
- AR Package Path is */ETAS_Project/EcucModuleConfigurationValuess*.
- File name is default or any name.

2. We have 4 filesystems: *rba_FeeRefFs1*, *rba_FeeRefFs1x*, *rba_FeeRefFs2*, and *rba_FeeRefFs3*. If you choose *rba_FeeRefFs1x*:

- You must set the value of parameter *rba_FeeRefFs1xDeviceCfg* to **FEE_RB_FULL_MANUAL_CFG**. After that, configure all mandatory parameters of *rba_FeeRefFs1x*.
- Configure dummy values for parameters *rba_FeeRefFs1CommonCfg*, *rba_FeeRefFs2CommonCfg*, and *rba_FeeRefFs3CommonCfg* of the other filesystems (*rba_FeeRefFs1*, *rba_FeeRefFs2*, and *rba_FeeRefFs3*).

You can extend the manual configuration (i.e. add more manual options for *FeeRbMachineType*) by following these steps:

- In file *ecu_config/paramdefs/Fee_MachineType_EcucParamDef.arxml_tpl*, remove the *_tpl* suffix to create the *Fee_MachineType_EcucParamDef.arxml filename*.
- Open *Fee_MachineType_EcucParamDef.arxml* and edit it. Follow **User Hint** at the bottom to create new manual configuration options:

1. Copy and paste or replace *ECUC-ENUMERATION-LITERAL-DEF* tags.
2. Update the tags *SHORT-NAME, LONG-NAME and ORIGIN*.
3. Rename file and provide in configuration.
4. Provide the *transtemplate* configuration in the *rba_FeeRefDeviceCfg* module for new machine name.

2.8.5 Configure Multiple Machines

1. Set the *Machine Type* parameter as "Multiple devices(FEE_RB_MPLDEV)".
2. Set the Fee Multiple Devices container (`FeeRbMplDevCfg`). Within this container, one sub-container should be configured for each device.
3. Reference to the Device configuration (`FeeRbMplDevRef`): Select the machine type from the Fee Reference Device Configuration.
4. Fee Blocks (`FeeRbBlockConfigurationRef`): Reference to the blocks from the

`FeeBlockConfiguration` for this device.

5. Select the machine type using the `Fee_Rb_SetCurMachine` API before Fee is initialized. The list of configured machine types is provided as an enum in the *Fee.h* file.

> **NOTE**
>
> Block operations on a Block that is not configured for the current selected device will cause a DET error.

6. The User has to provide the correct machine-type information to Fee.

This feature is limited to the *Fs1x* file system and machine types *RH850 U2A, RH850 U2B* and *RH850 U2C*.

Also, refer to the MemAcc section (Chapter 3) for details on how to configure the driver for this feature.

2.8.6 Soft Re-organization Threshold (SSR)

When memory gets filled, Fee will have to free it up for new block writes. In this *sector re-organization* process, the latest data of user blocks will be identified and moved to a new sector location if needed. This allows old used sectors to be erased and used for new block writes without losing data, which is still needed.

Sector re-organization will either be done automatically as part of a user block write (Hard Sector Reorganization - HSR) or during a Fee background operation (Soft Sector Re-organization - SSR). Soft Sector Re-organization will be started when no user job is pending and a certain filling level of memory is reached. This helps to avoid delays on user write jobs caused by Hard Sector Re-oganization and uses Fee idle time instead.

The filling-level threshold for a Soft Sector Re-organization (i.e. SSR threshold) will be determined by Fee during configuration. There are different options to adapt and override this automatic determination and adapt the SSR to suit needs, while HSR is fixed and cannot be changed.

File System:

1. **Fs1x**

Auto configuration:

- HSR is calculated as free bytes in DFLASH memory: Variable `FreeSpaceBeforeHSR_u32` in struct `rba_FeeFs1x_FlashProp_Cfg_cast` of file *rba_FeeFs1x_Prv_Cfg.c*.
- SSR is calculated as free bytes in DFLASH memory: Variable `FreeSpaceBeforeSSR_u32` in struct `rba_FeeFs1x_FlashProp_Cfg_cast` of file *rba_FeeFs1x_Prv_Cfg.c*.
- Automatic determination of HSR and SSR is based on sector layout, block configuration and activated features.

- The user cannot set custom values or factors for the automatic calculation in manual device configuration.

Manual configuration:

- Use parameter `FeeRbNrBytesBtwSSRAndHSR` to configure the SSR manually.
- Parameter meaning: Space between SSR and HSR.
- SSR Calculation: `FreeSpaceBeforeHSR_u32` - `FeeRbNrBytesBtwSSRAndHSR`.

2. **Fs2**

Auto configuration:

- HSR is calculated as free space: (size of one logical sector - parameter `rba_FeeRefFs2SafetySpace`).
- SSR is calculated based on a factor of HSR (SSR = `rba_FeeRefFs2SsrFactor` * HSR). `rba_FeeRefFs2SsrFactor` is typically 1.5 - 2.0.
- For full manual configuration, the recommendation is to use following values for normal usecases without special requirements: `rba_FeeRefFs2SafetySpace` = 512; `rba_FeeRefFs2SsrFactor` = 1.5.

Manual configuration:

- Use parameter `FeeRbNrBytesBtwSSRAndHSR` to configure the SSR manually.
- Parameter meaning: Space between SSR and HSR.
- SSR Calculation: (size of one logical sector - `rba_FeeRefFs2SafetySpace`) - `FeeRbNrBytesBtwSSRAndHSR`.

3. **Fs3**

Auto configuration:

- Fs3 does not support sector re-organization. It uses defragmentation instead, which cannot be configured by the user.

2.9 Integration Advice

NOTE

Fee does not implement any features specific to a Hardware Target. It is not in the scope of this module to support specific modifications for a Hardware Target or MCAL.

2.9.1 Init Functions

Fee_Init initializes the Fee module, and it must be called after the flash driver initialization routine (Fls_Init) and before any user initialization routines are executed (e.g. NvM_Init, NvM_ReadBlock).

Fee_Rb_EndInit must be called once per call to Fee_Init, and this should be done after the Fee_Init and immediately before it is possible to do a background erase and reorganization of blocks (e.g. right after a time-critical BSW initialization routine is finished). If a system shutdown function is using Fee_Init, Fee_Rb_EndInit must be called afterwards.

A user (e.g. NvM or Eep Adapter) should be able to read or write data to/from non-volatile memory immediately after the initialization of the ECU (and the initialization of the Fee module) has completed.

Fee needs to be able to detect sector states ("UNDEF", "ERASE_REQUEST", "ERASED", "FULL" or "USED") and the next address to which a write access is necessary (next empty page). Sectors should be prepared for a write access after Fee_Init ends, which means Fee must ensure that at least one sector is in the state of "ERASED" or "USED".

Since at least one sector is checked in its entirety in order to locate the next empty page for "FULL" and "USED" sectors, every single page is checked for values that do not match the flash's erase pattern and valid block headers. If only erase patterns are found, a check-for-blank is also performed. This allows the utilization of special HW features (if supported in the underlying hardware) for the detection of erased flash cells.

With the sector fully checked, the latest version of every available (valid) block header is automatically added to the internal cache. The sector is also marked as "checked" inside the cache status array. Access to any block that was added to the cache is speeded up because the address for reading (which is also necessary before writing) is already known. It would otherwise be necessary to check all sectors in the states "USED" or "FULL" prior to reading, writing or invalidating.

User orders should be avoided during the ECU's initialization phase because of timing restrictions. Every write or invalidate order might lead to a sector reorganization, which might take some additional time, especially if it triggers an erase procedure. In Soft Sector Reorganization mode, the oldest sector will be erased with the next main function call if no user order is available, and in Hard Sector Reorganization mode, the oldest sector will be erased immediately after the reorganization has finished.

2.9.2 DeInit Functions

No specific handling necessary.

2.9.3 Scheduled Functions

The FeeRbMainFunctionScheduling parameter is provided to configure the type of scheduling required for Fee_MainFunction. The following options are available:

- **RTE Timed Task**: The FeeMainFunctionPeriod parameter is used to generate

a timing event in the BSWMD file. Fee_MainFunction is provided as a schedulable entity, and a Timing Event is generated with the interval defined by FeeMainFunctionPeriod. This is the default if FeeRbMainFunction-Scheduling is not configured.

- **RTE Background Task**: Fee_MainFunction is provided as a schedulable entity with a background event. With this option, FeeMainFunctionPeriod should be configured as 0.0.
- **Manual Task**: Entities and Events are not generated with BSWMD. With this option, FeeMainFunctionPeriod should be configured as 0.0.

Fee_MainFunction should be executed cyclically, and it should run as fast as possible (e.g. as a 1ms task or as a background task). The task in which Fee_Main-Function is placed must be interruptible by higher priority tasks.

It is important, however, that there is a smooth transition while going to shut-down (from cyclic execution to linear execution), so Fee_MainFunction should not be pre-empted by another operation before control goes to shutdown.

The Fee_Rb_MainFunctionAndDependency function is useful during system initialization and shutdown scenarios. Instead of invoking multiple Mainfunctions, invocation of this service will handle the invocation of lower layer dependencies (typically Fls/MemAcc).

Scheduling for Fee is set by the parameter FeeRbMainFunctionScheduling = FEE_RB_MAINFUNC_SCHEDULING_MANUAL or FEE_RB_MAINFUNC_SCHEDUL-ING_RTE_TIMED or FEE_RB_MAINFUNC_SCHEDULING_BACKGROUND. Sched-uling mode RTE_TIMED will configure Fee_MainFunction to be periodically called by RTE with a time period set by the parameter FeeMainFunctionPeriod. This parameter must be set if timed scheduling is used. Scheduling mode BACK-GROUND will configure Fee_MainFunction to be called by RTE in the background task. Scheduling mode MANUAL will not configure any calling of Fee_MainFunc-tion with any RTE events or entries. Fee_MainFunction will have to be manually called by the integrator.

2.9.4 Block Configuration

Logical blocks that are written only rarely (less than once per driving cycle) should be configured in a way that the data is stored redundantly.

2.9.5 Fee Performance

For optimal performance, Fee variables have to be allocated in the core local RAM which executes the Fee functions. If variables are not in the core local RAM then the timing can become significantly slower.

To optimize code usage, the following API is visible only when the feature is enabled:

- Fee_Rb_GetMigrationResult

The availability of this API can be determined by checking the FEE_RB_-FIRST_READ_DATA_MIGRATION_API macro.

2.9.6 Multi-Instance Integration

How a second Fee Instance can be added to the Project:

- If Fee is used directly (and not via NvM), create a second Fee container: Set `FeeRbGeneralDeviceIndex` in `Fee_General` to a free device index (1…254, 0 is reserved for the first Instance). Give a nickname for the new Instance in the parameter `FeeRbGeneralDeviceNickName`. In order to assign the blocks to their intended Instance, configure them below the respective Fee container.
- One interface file is created per Instance with the naming convention *Fee_<FeeRbGeneralDeviceNickName>.h*. This interface file is the same as the *Fee.h* file for Device Index = 0. It contains the interfaces, block access macros and other macros. All macros (except block access macros) and interfaces in this file have the prefix *Fee_<FeeRbGeneralDeviceNickName>_<ServiceName/MacroName>*. Applications using the Fee instances != 0 should include this header file to access the APIs and macros.
- It is also possible to access the different instances using the `Fee_Rb_Idx` APIs with Device Name *Fee_Rb_DeviceName_<FeeRbGeneralDeviceNickName>*. The list of APIs supported and the macro names are generated in the *Fee_Rb_Idx.h* file. However, it is recommended to use the respective interfaces and not `Fee_Rb_Idx_`-based interfaces (as a feature might be active in the `Fee_Rb_Idx_` level because one of the other instances has activated the feature).
- Fee and Fls Main Functions for both the instances should be scheduled with respective device names.
- The Fee and Fls Init functions should be called during the initialization phase like single-instance APIs.

2.9.7 Header File structure

Fee provides the following public header files:

- **Fee_Rb_Types.h** (static under api folder): Fee data type definitions,
- **Fee.h** (generated during the execution of Code Generation): Interface file with function names (Provides AR and Fee_Rb* APIs) + generic macros + block access macros for device user 0.
- **Fee_<FeeRbGeneralDeviceNickName>.h** (generated during the execution of Code Generation): Interface file with function names (Provides AR and Fee_Rb* APIs) + generic macros + block access macros for device user ! = 0.
- **Fee_Cfg.h** (generated during the execution of Code Generation): This file has information that is common for all instances. Ideally, projects should not include this file but instead use the respective instance specific header files mentioned above.
- **Fee_Rb_Idx.h** (generated during the execution of Code Generation):

Provides Fee_Rb_Idx* APIs and macros, which could be used to place a job for a given instance using Device enumeration Fee_Rb_DeviceName_en.

In a single instance configuration, only Fee.h should be included (which includes the other Fee headers). In a multi-instance configuration, the respective interface file Fee_<FeeRbGeneralDeviceNickName>.h should be used to access the APIs and macros.

Note: In addition to above public header files, there are lots of other files generated during the execution of Code Generation which are used only for internal purposes. These files have the prefix _Prv_ in the file name and macro names. The contents of these files should be used in the application software outside Fee (e.g. Fee.c, Fee_Prv_Cfg.h, Fee_Prv_Cfg.c, rba_FeeFs1x_Prv_Cfg.h and rba_FeeFs1x_Prv_Cfg.c are generated when FeeFs1x is used).

2.9.8 OS Resources

The number of the active OS resources, and their usage, depends on the user configuration of the AUTOSAR System and Basic Software. It is the user's responsibility to ensure that the `ENTER` and `EXIT` resource API are implemented to disable/enable interrupts as required. Alternatively they must be configured in the OS with the correct task/ISR allocation.

The following is a list of all exclusive areas that this module might use regardless of the user configuration. It is the job of the integrator to ensure that the implementation of these exclusive areas is correctly defined. This implementation can be done using the RTE exclusive area configuration wizard or by manual implementation of the corresponding BSW API calls to ensure exclusivity.

The Fee module uses the following OS resources:

- Fee_Order

2.10 Hardware Requirements

Table 2.112.Fee Hardware Requirements

Support	Usage
FLASH	5680 bytes
RAM	1544 bytes

Usage calculated based on a sample configuration.

3 MemAcc

Module name	MemAcc
Package	RTA-COM
Module ID	N/A
AUTOSAR Version	23-11

3.1 Introduction

The Memory Access (MemAcc) module provides co-ordinated access to the different Mem drivers for programming, erasing, reading etc. of data to and from non-volatile memory (e.g. Flash, Eeprom).

MemAcc replaces Fls and extends its scope from a mere driver for the Fee to a multi-user driver.

3.1.1 Overview

MemAcc enables the partitioning of non-volatile memory into *Address Areas*. Each area typically has one owner, which handles jobs (e.g. read/write), which are asynchronuous. *Status Interfaces* are used to poll the job status/progress.

A maximum of one job can be requested for an Address Area at any given time.

Address Areas are not independent; they are grouped into *Device Instances*. Within one Device Instance, a job of up to one Address Area can be processed at any time. There is, therefore, some sequential processing, which is dependent upon the configurable priority of the Address Areas. For example, higher priority requests will automatically suspend lower priority ones. Suspended requests are resumed automatically, as soon as the priority is suitable again.

The Device Instance was introduced to allow parallel operation in different memory regions (if this is supported by the hardware). MemAcc can process Device Instance jobs in parallel without influencing other Device Instances. This allows for a system configuration which is distributed on cores and can handle different memory topologies.

In a further effort to reduce system integration, Device Instances can also be grouped into *Device Instance Groups*. This is done by assigning Address Areas to System Interfaces, which provide (among others) the `MemAcc_Init` and `MemAcc_MainFunction` functions to initialize and schedule the Device Instance Groups on different tasks/cores. In addition, a separate *MemMap .h* file and sections are available for each Device Instance Group/System Interface. MemAcc's internal variables are also assigned to the MemMap sections dependent upon this configuration.

MemAcc relationships:

	MemAcc	Has: Measurement Variables
contains multiple	Device Instance Group	Has: APIs like Init, MainFunction, own MemMap, event for RTE integration Needs: Os application/task assignment

contains multiple	Device Instance	Has: Capability to work in parallel with other Device Instances
contains multiple	Address Area	Has: APIs for job request, status handling, priority, virtual address space Needs: One owner
contains multiple	Sector Batch	Has: Physical start address, continuous physical memory space, mapping to virtual address space, assignment to Mem driver and driver instance, sector size, read/write page size, Burst parameters
contains multiple	Sector	Has: Physical start address

It is possible to combine Sector Batches from different Mem drivers or Hw instances in one Address Area. This means, however, that they then belong to the same Device Instance and thus can only be processed sequentially, and not in parallel.

Module Configuration

This module can be generated by the RTA-BSW Configuration Generator.

For any unsupported automatic configuration, the module can be manually configured by the user.

3.2 Supported Features

3.2.1 Fls Compatibility Mode

As MemAcc is replacing Fls, it provides some compatibility features to support the transition from an Fls-based system.

3.2.2 Parallel Device Support

The AUTOSAR standard only refers to one MemAcc instance. However, this does prevent the use of two different devices in parallel e.g. from different cores. In contrast, RTA-CAR's MemAcc offers the possibility of using several MemAcc instances in parallel, each with independent initialization and `MemAcc_MainFunction`, as well as other system-oriented functions.

3.2.3 Initialization

According to the AUTOSAR standard, the initialization function `MemAcc_Init` is a synchronuous (one-time) call that also initializes the Mem driver. This is not always possible/successful e.g. if the initialization happens directly after a running reset, since the latter does not stop an ongoing job. However, MemAcc has an asynchronuous initialization functionality.

3.2.4 Asynchronuous Job-Handling

All jobs like read/write/erase etc. are asynchronuous operations. They operate on Address Areas which are "owned" by a dedicated user. Each Address Area can receive only one job at a time.

Jobs are triggered by a dedicated MemAcc interface and processed within the

`MemAcc_MainFunction`. The job status can be checked via the API `MemAcc_GetJobStatus`, while the result is available either via a notification function or the API `MemAcc_GetJobResult`.

3.2.5 Timeout Supervision

MemAcc provides the timeout functionality from the Fls level as an extension to AUTOSAR. All asynchronuous operations passed to the Mem drivers are therefore monitored for a timeout. This ensures that a stuck Hw will be detected and reported as an error to the ASW (plus as a runtime error to the Det). The intention is to supervise an ongoing user request to ensure it completes (either with OK or ERROR).

For more details on each feature, please see Configuration Advice.

3.3 Extensions

3.3.1 Configuration Extensions

The following configuration parameters are available in version v12.7.0 in addition to those defined in AR v22-11

3.3.1.1 MemAcc/MemAccAddressAreaConfiguration/MemAccJobFlsEndNotificationName

Short Name	MemAccJobFlsEndNotificationName
Long Name	Job Fls End Notification
Description	This is only needed for Fls compatibility! Function parameters are different from MemAccJobEndNotificationName! If this pointer is set then in case of an error, this JobFlsEnd notification is called.
Introduction	This is a takeover from Fls. Normally only the JobEnd-Notification is necessary to be called.
Multiplicity	0..1
Default Value	-
Type	ECUC-FUNCTION-NAME-DEF

3.3.1.2 MemAcc/MemAccAddressAreaConfiguration/MemAccJobFlsErrorNotificationName

Short Name	MemAccJobFlsErrorNotificationName
Long Name	Job Fls Error Notification
Description	This is only needed for Fls compatibility! Function parameters are different from MemAccJobEndNotificationName! If this pointer is set then in case of an error, this JobFlsError notification is called.
Introduction	This is a takeover from Fls. Normally only the JobEnd-Notification is necessary to be called.

Multiplicity	0..1
Default Value	-
Type	ECUC-FUNCTION-NAME-DEF

3.3.1.3 MemAcc/MemAccAddressAreaConfiguration/MemAccRbConfig

Short Name	MemAccRbConfig
Long Name	Specific Configuration
Description	Specific configuration settings
Multiplicity	0..1
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

3.3.1.4 MemAcc/MemAccAddressAreaConfiguration/MemAccRbConfig/ MemAccFlsDriverIndex

Short Name	MemAccFlsDriverIndex
Long Name	Device Index (only for Fls)
Description	FlsDriverIndex taken from Fls
Multiplicity	0..1
Value Range	0..255
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

3.3.1.5 MemAcc/MemAccAddressAreaConfiguration/MemAccRbConfig/ MemAccGetJobResultApi

Short Name	MemAccGetJobResultApi
Long Name	Activation of GetJobResult API
Description	Switch to enable/disable the “GetJobResult” function. Please note that only 1 user for each memo-ry-driver-SubInstance is allowed to have this Api acti-vated. All other users of the same memo-ry-driver-SubInstance have to use Job-notification callback functions or the specific “GetJobResultAdr” interfaces.
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

3.3.1.6 MemAcc/MemAccAddressAreaConfiguration/MemAccRbConfig/

MemAccRbApiRef

Short Name	MemAccRbApiRef
Long Name	Reference to Application Interface Definition
Description	Reference to an Api which is used to address the Address Area. This is mandatory if MemAccRbFromFls=true
Multiplicity	0..1
Default Value	-
Type	ECUC-SYMBOLIC-NAME-REFERENCE-DEF

3.3.1.7 MemAcc/MemAccAddressAreaConfiguration/MemAccRbConfig/MemAccRbDeviceName

Short Name	MemAccRbDeviceName
Long Name	Device Nick name (only for Fls)
Description	Internal Device Nick name (optional). If not given, an automatic name is defined.
Multiplicity	0..1
Default Value	-
Type	ECUC-STRING-PARAM-DEF

3.3.1.8 MemAcc/MemAccAddressAreaConfiguration/MemAccRbConfig/MemAccRbExclusive

Short Name	MemAccRbExclusive
Long Name	Exclusive AddressArea
Description	Indicates that the address-space of the SubAddressArea are exclusively used by this AddressArea. Overlaps into this range result in a build abort.
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

3.3.1.9 MemAcc/MemAccAddressAreaConfiguration/MemAccRbConfig/MemAccRbFromFls

Short Name	MemAccRbFromFls
Long Name	Transfered from Fls
Description	Indication that Address Area is transfered from Fls (if true)
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

3.3.1.10 MemAcc/MemAccAddressAreaConfiguration/MemAccRbConfig/MemAccRbHsmMem4Host

Short Name	MemAccRbHsmMem4Host
Long Name	Activation of Hsm Memory Access
Description	Switch to allow the access to the HSM-Memory. This switch can only be used in non-Hsm mode.
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

3.3.1.11 MemAcc/MemAccAddressAreaConfiguration/MemAccRbConfig/MemAccRbMode

Short Name	MemAccRbMode
Long Name	Mem Transfer Mode Parameters
Description	Transfer mode parameters between MemAcc and MemDrv
Multiplicity	0..1
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

3.3.1.12 MemAcc/MemAccAddressAreaConfiguration/MemAccRbConfig/MemAccRbMode/MemAccDefaultMode

Short Name	MemAccDefaultMode
Long Name	Default mode
Description	Use fast mode default setting during initialization. false=Normal, true=Fast
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

3.3.1.13 MemAcc/MemAccAddressAreaConfiguration/MemAccRbConfig/MemAccRbMode/MemAccMaxCompareFastMode

Short Name	MemAccMaxCompareFastMode
Long Name	Compare bytes in fast mode
Description	The maximum number of bytes to be compared in one cycle of the Mem driver's job processing function in fast mode. If not given here, central value MemAccDefaultCompareJobSize is used instead
Multiplicity	0..1

Value Range	0..4294967295
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

3.3.1.14 MemAcc/MemAccAddressAreaConfiguration/MemAccRbConfig/MemAccRbMode/MemAccMaxCompareNormalMode

Short Name	MemAccMaxCompareNormalMode
Long Name	Compare bytes in normal mode
Description	The maximum number of bytes to be compared in one cycle of the Mem driver's job processing function in normal mode. If not given here, central value MemAccDefaultCompareJobSize is used instead
Multiplicity	0..1
Value Range	0..4294967295
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

3.3.1.15 MemAcc/MemAccAddressAreaConfiguration/MemAccRbConfig/MemAccRbMode/MemAccMaxReadFastMode

Short Name	MemAccMaxReadFastMode
Long Name	Read bytes in fast mode
Description	The maximum number of bytes to be read in one cycle of the Mem driver's job processing function in fast mode. If not given here, central value MemAccDefaultReadJobSize is used instead
Multiplicity	0..1
Value Range	0..4294967295
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

3.3.1.16 MemAcc/MemAccAddressAreaConfiguration/MemAccRbConfig/MemAccRbMode/MemAccMaxReadNormalMode

Short Name	MemAccMaxReadNormalMode
Long Name	Read bytes in normal mode
Description	The maximum number of bytes to be read in one cycle of the Mem driver's job processing function in normal mode. If not given here, central value MemAccDefaultReadJobSize is used instead

Multiplicity	0..1
Value Range	0..4294967295
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

3.3.1.17 MemAcc/MemAccAddressAreaConfiguration/MemAccRbConfig/MemAccRbSpiChannel

Short Name	MemAccRbSpiChannel
Long Name	QuadSpi Channel Assignment (not Standard-SPI)
Description	Quad spi channel number. Do not use for Standard Spi connected devices
Multiplicity	0..1
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

3.3.1.18 MemAcc/MemAccAddressAreaConfiguration/MemAccRbConfig/MemAccRbSync

Short Name	MemAccRbSync
Long Name	Sync Transfer Parameters
Description	For some external flash the address area can be configured to sync or async transfer.
Multiplicity	0..1
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

3.3.1.19 MemAcc/MemAccAddressAreaConfiguration/MemAccRbConfig/MemAccRbSync/MemAccSyncErase

Short Name	MemAccSyncErase
Long Name	Activation of Sync Erase Function
Description	This Function implements erase operation synchronously.
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

3.3.1.20 MemAcc/MemAccAddressAreaConfiguration/MemAccRbConfig/MemAccRbSync/MemAccSyncRead

Short Name	MemAccSyncRead

Long Name	Activation of Sync Read,Compare,BlankCheck Function
Description	This Function implements read ,blank check and compare operation synchronously.
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

3.3.1.21 MemAcc/MemAccAddressAreaConfiguration/MemAccRbConfig/MemAccRbSync/MemAccSyncWrite

Short Name	MemAccSyncWrite
Long Name	Activation of Sync Write Function
Description	Switch to enable /disable synchronous Mode for Write operation. Disabling this function will turn Write operation to Asynchronous Mode (Default). Note: taken over from FlsRbSyncWrite only with corrected logic (inverted)as a prt of a defect fix.
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

3.3.1.22 MemAcc/MemAccAddressAreaConfiguration/MemAccRbConfig/MemAccRbSysRef

Short Name	MemAccRbSysRef
Long Name	Reference to System Interface Definition
Description	Reference to an System-Api which is used to address the Address Area. Only 1 interface per device allowed
Multiplicity	0..1
Default Value	-
Type	ECUC-SYMBOLIC-NAME-REFERENCE-DEF

3.3.1.23 MemAcc/MemAccAddressAreaConfiguration/MemAccRbConfig/MemAccRbTime

Short Name	MemAccRbTime
Long Name	Timeout/Time-Measurement Parameters
Description	Specific timeout and time measurement configuration settings
Multiplicity	0..1
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

3.3.1.24 MemAcc/MemAccAddressAreaConfiguration/MemAccRbConfig/MemAccRbTime/MemAccTimeMeasurementEnabled

Short Name	MemAccTimeMeasurementEnabled
Long Name	Activation of Time measurement
Description	Switch to enable/disable the time measurement. The functionality is intended to verify the parameters set within the timeout-supervision.
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

3.3.1.25 MemAcc/MemAccAddressAreaConfiguration/MemAccRbConfig/MemAccRbTime/MemAccTimePageOffset

Short Name	MemAccTimePageOffset
Long Name	Timeout value offset (in s) for communication overhead
Description	Duration (in seconds) always added to the volume dependent duration of a single Mem-Driver job. Mainly compensating the effects of the extended duration of small transfers, scheduling, interaction effect Host/HSM or external communication channels.
Multiplicity	0..1
Value Range	0..100
Default Value	-
Type	ECUC-FLOAT-PARAM-DEF

3.3.1.26 MemAcc/MemAccAddressAreaConfiguration/MemAccRbConfig/MemAccRbTime/MemAccTimePagePerVolume

Short Name	MemAccTimePagePerVolume
Long Name	Time value (in s) for reference page write/transfer
Description	Duration (in sconds) for one (1) reference page transfer (read/write) operation (256 bytes) This parameter is needed to determine the volume dependent timeout part. This timeout duration is calculated as: MemTimePagePerVolume * bytes-to-be-programmed / 256 - Note: small size transfers are to be considered in the offset part. Please consider that MemAcc is transfering only pages/burst for write, and limited amout for reads in one Mem-Driver job.
Multiplicity	0..1

Value Range	0.0001..0.1
Default Value	-
Type	ECUC-FLOAT-PARAM-DEF

3.3.1.27 MemAcc/MemAccAddressAreaConfiguration/MemAccRbConfig/MemAccRbTime/MemAccTimeSectorOffset

Short Name	MemAccTimeSectorOffset
Long Name	Timeout value offset (in s) for communication overhead
Description	Duration (in seconds) always added to the volume dependent duration of a single Mem-Driver kob. Mainly compensating the effects of the extended duration on small sectors, scheduling, interaction effect Host/HSM or external communication channels.
Multiplicity	0..1
Value Range	0..100
Default Value	-
Type	ECUC-FLOAT-PARAM-DEF

3.3.1.28 MemAcc/MemAccAddressAreaConfiguration/MemAccRbConfig/MemAccRbTime/MemAccTimeSectorPerVolume

Short Name	MemAccTimeSectorPerVolume
Long Name	Time value (in s) for reference sector erase
Description	Duration (in seconds) for one (1) reference sector erase operation (32768 bytes). This parameter is needed to determine the volume dependent timeout part. This timeout duration is calculated as: MemTimeSectorPerVolume * bytes-in-Sectors / 32768 - Note: small size transfers are to be considered in the offset part. Please consider that MemAcc is transfering only single sectors or burst of sectors in one Mem-Driver job.
Multiplicity	0..1
Value Range	0.01..10
Default Value	-
Type	ECUC-FLOAT-PARAM-DEF

3.3.1.29 MemAcc/MemAccAddressAreaConfiguration/MemAccRbConfig/MemAccRbTime/MemAccTimeoutReqAcceptance

Short Name	MemAccTimeoutReqAcceptance
Long Name	Timeout value (in s) for request acceptance
Description	Timeout duration (in seconds) needed by the Mem-Driver to accept the request placed by the MemAcc. Note: until it is accepted MemAcc will retry (Note: only relevant in special cases)
Multiplicity	0..1
Value Range	0..100
Default Value	-
Type	ECUC-FLOAT-PARAM-DEF

3.3.1.30 MemAcc/MemAccAddressAreaConfiguration/MemAccRbConfig/MemAccRbTime/MemAccTimeoutSupervisionEnabled

Short Name	MemAccTimeoutSupervisionEnabled
Long Name	Activation of Timeout Supervision
Description	Switch to enable timeout supervision for the device. true: timeout supervision for read/erase/write/compare jobs enabled. false: timeout supervision is disabled.
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

3.3.1.31 MemAcc/MemAccAddressAreaConfiguration/MemAccRbConfig/MemAccRbTransient

Short Name	MemAccRbTransient
Long Name	Transient Error Parameters
Description	Specific activations for transient error handling
Multiplicity	0..1
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

3.3.1.32 MemAcc/MemAccAddressAreaConfiguration/MemAccRbConfig/MemAccRbTransient/MemAccTransientErrorCompare

Short Name	MemAccTransientErrorCompare
Long Name	Activation of Compare Transient Error Det reporting

Description	Switch to enable and disable the transient errors for Compare operation. When activated the related JobFailed incident is in addition reported to the Det. Note: in AR4.2 reporting happens as Runtime-Error
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

3.3.1.33 MemAcc/MemAccAddressAreaConfiguration/MemAccRbConfig/MemAccRbTransient/MemAccTransientErrorErase

Short Name	MemAccTransientErrorErase
Long Name	Activation of Erase Transient Error Det reporting
Description	Switch to enable and disable the transient errors for Erase operation. When activated the related JobFailed incident is in addition reported to the Det. Note: in AR4.2 reporting happens as Runtime-Error
Multiplicity	0..1
Default Value	true
Type	ECUC-BOOLEAN-PARAM-DEF

3.3.1.34 MemAcc/MemAccAddressAreaConfiguration/MemAccRbConfig/MemAccRbTransient/MemAccTransientErrorFlashId

Short Name	MemAccTransientErrorFlashId
Long Name	Activation of unexpected Flash ID Det reporting
Description	Switch to enable/disable the unexpected FlashId DET reporting. The FlashID is checked during initialization of external flash chips. If it fails the Fls driver remains in state UNINIT. By activating this function and addition Det reporting is performed.
Multiplicity	0..1
Default Value	true
Type	ECUC-BOOLEAN-PARAM-DEF

3.3.1.35 MemAcc/MemAccAddressAreaConfiguration/MemAccRbConfig/MemAccRbTransient/MemAccTransientErrorRead

Short Name	MemAccTransientErrorRead
Long Name	Activation of Read Transient Error Det reporting
Description	Switch to enable and disable the transient errors for Read operation. When activated the related JobFailed incident is in addition reported to the Det. Note: in AR4.2 reporting happens as Runtime-Error

Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

3.3.1.36 MemAcc/MemAccAddressAreaConfiguration/MemAccRbConfig/MemAccRbTransient/MemAccTransientErrorWrite

Short Name	MemAccTransientErrorWrite
Long Name	Activation of Write Transient Error Det reporting
Description	Switch to enable and disable the Fls transient errors for Write operation. When activated the related JobFailed incident is in addition reported to the Det. Note: in AR4.2 reporting happens as Runtime-Error
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

3.3.1.37 MemAcc/MemAccAddressAreaConfiguration/MemAccRbConfig/MemAccRbUseInterrupts

Short Name	MemAccRbUseInterrupts
Long Name	Use Interrupts
Description	Uses interrupts (only if the hardware supports it) whenever possible.
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

3.3.1.38 MemAcc/MemAccAddressAreaConfiguration/MemAccSubAddressAreaConfiguration/MemAccRbUseWriteBurstLimited

Short Name	MemAccRbUseWriteBurstLimited
Long Name	Use Limited Write Burst Length
Description	This parameter enables that the standard WriteBurst size is replaced by a special size which is suitable only in limited scope (e.g. production)
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

3.3.1.39 MemAcc/MemAccAddressAreaConfiguration/MemAccSubAddressAreaConfiguration/MemAccUseReadBurst

Short Name	MemAccUseReadBurst
Long Name	Enable Read Burst
Description	This parameter enables read bursting for the related sub address area
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

3.3.1.40 MemAcc/MemAccGeneral/MemAccRbMainFunctionScheduling

Short Name	MemAccRbMainFunctionScheduling
Long Name	MainFunction Scheduling Type
Description	Selection on type of scheduling required for the driver. This parameter shall be selected in consistency with FlsMainFunctionPeriod. Select MANUAL triggering if you manually control the calling of the MainFunction.
Multiplicity	0..1
Default Value	-
Enumeration Literals	RTE_BACKGROUND, RTE_TIMED, MANUAL
Type	ECUC-ENUMERATION-PARAM-DEF

3.3.1.41 MemAcc/MemAccRbGeneral

Short Name	MemAccRbGeneral
Long Name	General Parameters Extension
Description	Container for general configuration parameters of the MemAcc.
Multiplicity	0..1
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

3.3.1.42 MemAcc/MemAccRbGeneral/MemAccBlankCheckApi

Short Name	MemAccBlankCheckApi
Long Name	Activation of BlankCheck API
Description	Switch to enable / disable the API to check for the erase pattern within a certain address area.
Multiplicity	0..1
Default Value	-

Type	ECUC-BOOLEAN-PARAM-DEF

3.3.1.43 MemAcc/MemAccRbGeneral/MemAccCancelApi

Short Name	MemAccCancelApi
Long Name	Activation of Cancel API
Description	Switch to enable/disable the “Cancel” function.
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

3.3.1.44 MemAcc/MemAccRbGeneral/MemAccDdrModeApi

Short Name	MemAccDdrModeApi
Long Name	Activation of DDR-Handling API
Description	Switch to enable/disable DDR related functions.
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

3.3.1.45 MemAcc/MemAccRbGeneral/MemAccDeInitApi

Short Name	MemAccDeInitApi
Long Name	Activation of DeInit API
Description	Switch to enable/disable the DeInit functions.
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

3.3.1.46 MemAcc/MemAccRbGeneral/MemAccDefaultCompareJobSize

Short Name	MemAccDefaultCompareJobSize
Long Name	Default transfer size for compare jobs
Description	In case a default value for the Compare JobSize is not given for an individual Address Area, then this default value will be taken. If it is not defined, then 512 bytes are used.
Multiplicity	0..1
Value Range	8..4096
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

3.3.1.47 MemAcc/MemAccRbGeneral/MemAccDefaultReadJobSize

Short Name	MemAccDefaultReadJobSize
Long Name	Default transfer size for read jobs
Description	In case a default value for the Read JobSize is not given for an individual Address Area, then this default value will be taken. If it is not defined, then 512 bytes are used.
Multiplicity	0..1
Value Range	8..4096
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

3.3.1.48 MemAcc/MemAccRbGeneral/MemAccFota1Api

Short Name	MemAccFota1Api
Long Name	Activation of Special Fota1 API
Description	Switch to enable and disable the Extended mode API Mode switch and Memory swap API function. (interface is only usable for dedicated CPUs)
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

3.3.1.49 MemAcc/MemAccRbGeneral/MemAccGetDeviceInfoApi

Short Name	MemAccGetDeviceInfoApi
Long Name	Activation of Get Device Information API
Description	Switch to enable/disable the “GetDeviceInfo” function
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

3.3.1.50 MemAcc/MemAccRbGeneral/MemAccGetJobErrDetailsApi

Short Name	MemAccGetJobErrDetailsApi
Long Name	Activation of GetJobErrDetails API
Description	Switch to enable/disable the “GetJobErrDetails” function. Function can be used to get more insight in the reason why the last job has failed.
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

3.3.1.51 MemAcc/MemAccRbGeneral/MemAccGetJobInfoApi

Short Name	MemAccGetJobInfoApi
Long Name	Activation of GetJobInfo API
Description	Switch to enable/disable the “GetJobInfo” function. Function can be used to get more insight in the reason what the current job is doing.
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

3.3.1.52 MemAcc/MemAccRbGeneral/MemAccGetMediaAddrApi

Short Name	MemAccGetMediaAddrApi
Long Name	Activation of GetMediaAddress API
Description	Switch to enable/disable the interface to get/read the current media address of a finished operation.
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

3.3.1.53 MemAcc/MemAccRbGeneral/MemAccGetModeApi

Short Name	MemAccGetModeApi
Long Name	Activation of GetMode API
Description	Switch to enable/disable the “GetModus” function.
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

3.3.1.54 MemAcc/MemAccRbGeneral/MemAccGetSectorInfo

Short Name	MemAccGetSectorInfo
Long Name	Activation of GetSectorInfo API
Description	Switch to enable/disable the “GetSectorInfo” function.
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

3.3.1.55 MemAcc/MemAccRbGeneral/MemAccGetStatusApi

Short Name	MemAccGetStatusApi
Long Name	Activation of GetStatus API
Description	Switch to enable/disable the “GetStatus” function.
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

3.3.1.56 MemAcc/MemAccRbGeneral/MemAccHighSpeedModeApi

Short Name	MemAccHighSpeedModeApi
Long Name	Activation of the HighSpeedMode APIs
Description	The HighSpeedMode is a combination of SetMode and Suspend-APIs. Activating this function will activate: MemAccSetModeApi, MemAccPriorityApi,
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

3.3.1.57 MemAcc/MemAccRbGeneral/MemAccHsmEnable

Short Name	MemAccHsmEnable
Long Name	Activation of Hsm
Description	Switch to enable/disable the Hsm mode
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

3.3.1.58 MemAcc/MemAccRbGeneral/MemAccInclFlsPrjHeader

Short Name	MemAccInclFlsPrjHeader
Long Name	(only Fls) Activate optional project extension header include
Description	This switch can be used to activate the optional #include “Fls_Rb_Prj.h” (see template folder) to activate/modify internal functionality within the Fls-HLD and the Fls-LLD. Note: when activating this switch, you have to provide this header file in your project.
Multiplicity	0..1
Default Value	-

Type	ECUC-BOOLEAN-PARAM-DEF

3.3.1.59 MemAcc/MemAccRbGeneral/MemAccPowerModeApi

Short Name	MemAccPowerModeApi
Long Name	Activation of PowerMode API
Description	Switch to enable/disable the “Deep Power Mode” function.
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

3.3.1.60 MemAcc/MemAccRbGeneral/MemAccPriorityApi

Short Name	MemAccPriorityApi
Long Name	Activation of Raise/Lower Priority API
Description	Switch to enable/disable the priority raising/lowering function.
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

3.3.1.61 MemAcc/MemAccRbGeneral/MemAccReadApi

Short Name	MemAccReadApi
Long Name	Activation of Read API
Description	Switch to enable/disable the “Read” function (only in exceptional cases)
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

3.3.1.62 MemAcc/MemAccRbGeneral/MemAccReplaceSectorApi

Short Name	MemAccReplaceSectorApi
Long Name	Activation of ReplaceSector API
Description	Switch to enable/disable the”ReplaceSector” API function. (interface is only usable for dedicated CPUs)
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

3.3.1.63 MemAcc/MemAccRbGeneral/MemAccSectorLockApi

Short Name	MemAccSectorLockApi
Long Name	Activation of SetSectorLock/CheckSectorUnlocked API
Description	Switch to enable/disable the “SetSectorLock” and “CheckSectorUnlocked” function. (interface is only usable for dedicated CPUs)
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

3.3.1.64 MemAcc/MemAccRbGeneral/MemAccSetAuthentIdApi

Short Name	MemAccSetAuthentIdApi
Long Name	Activation of SetAuthentId API
Description	Switch to enable/disable the “SetAuthentId” function. This function is applicable only for some uC to pass the authentification Id for the different flash regions.
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

3.3.1.65 MemAcc/MemAccRbGeneral/MemAccSetModeApi

Short Name	MemAccSetModeApi
Long Name	Activation of SetMode API
Description	Switch to enable/disable “SetMode” function.
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

3.3.1.66 MemAcc/MemAccRbGeneral/MemAccTransferModeApi

Short Name	MemAccTransferModeApi
Long Name	Activation of TransferMode API
Description	Switch to enable/disable the Transfer-mode APIs needed for LLDs using SPI/QuadSpi communications. Only if this function is enabled, the LLD will perform asynchronuous tranfer operations.
Multiplicity	0..1
Default Value	-

Type	ECUC-BOOLEAN-PARAM-DEF

3.3.1.67 MemAcc/MemAccRbGeneral/MemAccUseUnalignedWrite

Short Name	MemAccUseUnalignedWrite
Long Name	Allow unaligned Write length
Description	Switch to enable/disable the usage of unaligned write commands. Per default: this switch is FALSE in a mere MemAcc-style configuration. It is TRUE in a mixed or mere Fls-style configuration.
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

3.3.1.68 MemAcc/MemAccRbGeneral/MemAccVersionInfoApi

Short Name	MemAccVersionInfoApi
Long Name	Activation of VersionInfo API
Description	Switch to enable / disable the API to read out the modules version information.
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

3.3.1.69 MemAcc/MemAccRbGeneral/MemAccWriteWithParallelActionApi

Short Name	MemAccWriteWithParallelActionApi
Long Name	Activation of WriteWithParallelAction API
Description	Switch to enable / disable the special API for write with a parallel operation (e.g. erase) dependent on the selected uC
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

3.3.1.70 MemAcc/MemAccRbGeneral/MemAccWriteWithoutTransientErrorApi

Short Name	MemAccWriteWithoutTransientErrorApi
Long Name	Activation of WriteWithoutTransientError API
Description	Switch to enable / disable the special API for write which prevents transient error entries. This API is needed in special cases and some controllers for invalidating data.

Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

3.3.1.71 MemAcc/MemAccRbListApi

Short Name	MemAccRbListApi
Long Name	Interfaces for Users/API
Description	Container/List of User-Api interfaces
Introduction	The User APIs are used by the diffrent users to realize their functionality. System interfaces are related to the scheduling/ initialization of the MemAcc devices are listed separately.
Multiplicity	0..1
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

3.3.1.72 MemAcc/MemAccRbListApi/MemAccRbApi

Short Name	MemAccRbApi
Long Name	Application Interface Definition
Description	Container for Api generation. ShortName is the name of the Api interface
Introduction	The shortname of a system interface can be the same as the one for an user interface
Multiplicity	0..255
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

3.3.1.73 MemAcc/MemAccRbListApi/MemAccRbApi/MemAccBaseAddress

Short Name	MemAccBaseAddress
Long Name	Base Address for relative Addressing
Description	(Optional) If this address is defined, then it is added to each address passed via this API interface. In normal cases, this shall be left empty.
Multiplicity	0..1
Value Range	0..4294967295
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

3.3.1.74 MemAcc/MemAccRbListApi/MemAccRbApi/MemAccBlankCheck-Api

Short Name	MemAccBlankCheckApi
Long Name	Activation of BlankCheck API
Description	Switch to enable / disable the API to check for the erase pattern within a certain address area.
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

3.3.1.75 MemAcc/MemAccRbListApi/MemAccRbApi/MemAccCancelApi

Short Name	MemAccCancelApi
Long Name	Activation of Cancel API
Description	Switch to enable/disable the “Cancel” function.
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

3.3.1.76 MemAcc/MemAccRbListApi/MemAccRbApi/MemAccDdrModeApi

Short Name	MemAccDdrModeApi
Long Name	Activation of DDR-Handling API
Description	Switch to enable/disable DDR related functions.
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

3.3.1.77 MemAcc/MemAccRbListApi/MemAccRbApi/MemAccDeInitApi

Short Name	MemAccDeInitApi
Long Name	Activation of DeInit API
Description	Switch to enable/disable the DeInit functions.
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

3.3.1.78 MemAcc/MemAccRbListApi/MemAccRbApi/MemAccFota1Api

Short Name	MemAccFota1Api
Long Name	Activation of Special Fota1 API
Description	Switch to enable and disable the Extended mode API Mode switch and Memory swap API function. (interface is only usable for dedicated CPUs)
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

3.3.1.79 MemAcc/MemAccRbListApi/MemAccRbApi/MemAccGetDeviceInfoApi

Short Name	MemAccGetDeviceInfoApi
Long Name	Activation of Get Device Information API
Description	Switch to enable/disable the “GetDeviceInfo” function
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

3.3.1.80 MemAcc/MemAccRbListApi/MemAccRbApi/MemAccGetJobErrDetailsApi

Short Name	MemAccGetJobErrDetailsApi
Long Name	Activation of GetJobErrDetails API
Description	Switch to enable/disable the “GetJobErrDetails” function. Function can be used to get more insight in the reason why the last job has failed.
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

3.3.1.81 MemAcc/MemAccRbListApi/MemAccRbApi/MemAccGetMediaAddrApi

Short Name	MemAccGetMediaAddrApi
Long Name	Activation of GetMediaAddress API
Description	Switch to enable/disable the interface to get/read the current media address of a finished operation.
Multiplicity	0..1
Default Value	-

Type	ECUC-BOOLEAN-PARAM-DEF

3.3.1.82 MemAcc/MemAccRbListApi/MemAccRbApi/MemAccGetModeApi

Short Name	MemAccGetModeApi
Long Name	Activation of GetMode API
Description	Switch to enable/disable the “GetModus” function.
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

3.3.1.83 MemAcc/MemAccRbListApi/MemAccRbApi/MemAccGetSectorInfo

Short Name	MemAccGetSectorInfo
Long Name	Activation of GetSectorInfo API
Description	Switch to enable/disable the “GetSectorInfo” function.
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

3.3.1.84 MemAcc/MemAccRbListApi/MemAccRbApi/MemAccGetStatusApi

Short Name	MemAccGetStatusApi
Long Name	Activation of GetStatus API
Description	Switch to enable/disable the “GetStatus” function.
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

3.3.1.85 MemAcc/MemAccRbListApi/MemAccRbApi/MemAccHighSpeed-ModeApi

Short Name	MemAccHighSpeedModeApi
Long Name	Activation of the HighSpeedMode APIs
Description	The HighSpeedMode is a combination of SetMode and Suspend-APIs. Activating this function will activate: MemAccSetModeApi, MemAccPriorityApi,
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

3.3.1.86 MemAcc/MemAccRbListApi/MemAccRbApi/MemAccHsmEnable

Short Name	MemAccHsmEnable
Long Name	Activation of Hsm Mode
Description	Switch to enable/disable the Hsm mode
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

3.3.1.87 MemAcc/MemAccRbListApi/MemAccRbApi/MemAccInclFlsPrj-Header

Short Name	MemAccInclFlsPrjHeader
Long Name	(only Fls) Activate optional project extension header include
Description	This switch can be used to activate the optional #include “Fls_Rb_Prj.h” (see template folder) to activate/modify internal functionality within the Fls-HLD and the Fls-LLD. Note: when activating this switch, you have to provide this header file in your project.
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

3.3.1.88 MemAcc/MemAccRbListApi/MemAccRbApi/MemAccInfraManualApi

Short Name	MemAccInfraManualApi
Long Name	Activation of the MainFunction APIs (special)
Description	Switch to enable the generation of <Device>_MainFunction API-functions for non-task manual calling. This is only needed in rare cases if system-interface is not available
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

3.3.1.89 MemAcc/MemAccRbListApi/MemAccRbApi/MemAccPowerModeApi

Short Name	MemAccPowerModeApi
Long Name	Activation of PowerMode API

Description	Switch to enable/disable the “Deep Power Mode” function.
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

3.3.1.90 MemAcc/MemAccRbListApi/MemAccRbApi/MemAccPriorityApi

Short Name	MemAccPriorityApi
Long Name	Activation of Raise/Lower Priority API
Description	Switch to enable/disable the priority raising/lowering function.
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

3.3.1.91 MemAcc/MemAccRbListApi/MemAccRbApi/MemAccRbFromFls

Short Name	MemAccRbFromFls
Long Name	Transfered from Fls
Description	Indication that Address Area is transfered from Fls (if true)
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

3.3.1.92 MemAcc/MemAccRbListApi/MemAccRbApi/MemAccReadApi

Short Name	MemAccReadApi
Long Name	Activation of Read API
Description	Switch to enable/disable the “Read” function (only in exceptional cases)
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

3.3.1.93 MemAcc/MemAccRbListApi/MemAccRbApi/MemAccReplaceSectorApi

Short Name	MemAccReplaceSectorApi
Long Name	Activation of ReplaceSector API
Description	Switch to enable/disable the”ReplaceSector” API function. (interface is only usable for dedicated CPUs)
Multiplicity	0..1

Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

3.3.1.94 MemAcc/MemAccRbListApi/MemAccRbApi/MemAccSectorLock-Api

Short Name	MemAccSectorLockApi
Long Name	Activation of SetSectorLock/CheckSectorUnlocked API
Description	Switch to enable/disable the “SetSectorLock” and “CheckSectorUnlocked” function. (interface is only usable for dedicated CPUs)
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

3.3.1.95 MemAcc/MemAccRbListApi/MemAccRbApi/MemAccSetAuthentI-dApi

Short Name	MemAccSetAuthentIdApi
Long Name	Activation of SetAuthentId API
Description	Switch to enable/disable the “SetAuthentId” function. This function is applicable only for some uC to pass the authentification Id for the different flash regions.
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

3.3.1.96 MemAcc/MemAccRbListApi/MemAccRbApi/MemAccSetHwDevi-ceIdApi

Short Name	MemAccSetHwDeviceIdApi
Long Name	Activation of SetHWDeviceId API
Description	Switch to enable/disable the “SetHWDeviceId” function. This function is applicable only for some uC to pass the used HW device identifier.
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

3.3.1.97 MemAcc/MemAccRbListApi/MemAccRbApi/MemAccSetModeApi

Short Name	MemAccSetModeApi
Long Name	Activation of SetMode API

Description	Switch to enable/disable “SetMode” function.
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

3.3.1.98 MemAcc/MemAccRbListApi/MemAccRbApi/MemAccTransferModeApi

Short Name	MemAccTransferModeApi
Long Name	Activation of TransferMode API
Description	Switch to enable/disable the Transfer-mode APIs needed for LLDs using SPI/QuadSpi communications. Only if this function is enabled, the LLD will perform asynchronuous tranfer operations.
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

3.3.1.99 MemAcc/MemAccRbListApi/MemAccRbApi/MemAccVersionInfoApi

Short Name	MemAccVersionInfoApi
Long Name	Activation of VersionInfo API
Description	Switch to enable / disable the API to read out the modules version information.
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

3.3.1.100 MemAcc/MemAccRbListApi/MemAccRbApi/MemAccWriteWithParallelActionApi

Short Name	MemAccWriteWithParallelActionApi
Long Name	Activation of WriteWithParallelAction API
Description	Switch to enable / disable the special API for write with a parallel operation (e.g. erase) dependent on the selected uC
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

3.3.1.101 MemAcc/MemAccRbListApi/MemAccRbApi/MemAccWriteWithout-TransientErrorApi

Short Name	MemAccWriteWithoutTransientErrorApi
Long Name	Activation of WriteWithoutTransientError API
Description	Switch to enable / disable the special API for write which prevents transient error entries. This API is needed in special cases and some controllers for invalidating data.
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

3.3.1.102 MemAcc/MemAccRbListSys

Short Name	MemAccRbListSys
Long Name	Interfaces for System-Integration
Description	Container/List of System interfaces / MemAcc Devices
Introduction	A MemAcc devices can work in parallel. Therefore they need separate interface descriptions. Whether or not an API is actually generated depends on the configuration. If no system interface is used then automatically a default “MemAcc” Sys API will be generated.
Multiplicity	0..1
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

3.3.1.103 MemAcc/MemAccRbListSys/MemAccRbSys

Short Name	MemAccRbSys
Long Name	System Interface Definition
Description	Container for System interfaces ShortName is the name of the System Api interface.
Introduction	The shortname of a system interface can be the same as the one for an user interface
Multiplicity	0..255
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

3.3.1.104 MemAcc/MemAccRbListSys/MemAccRbSys/MemAccRbMainFunctionScheduling

Short Name	MemAccRbMainFunctionScheduling
Long Name	MainFunction Scheduling Type
Description	Selection on type of scheduling required for the driver. This parameter shall be selected in consistency with FlsMainFunctionPeriod. Select MANUAL triggering if you manually control the calling of the MainFunction.
Multiplicity	0..1
Default Value	-
Enumeration Literals	RTE_BACKGROUND, RTE_TIMED, MANUAL
Type	ECUC-ENUMERATION-PARAM-DEF

3.3.1.105 MemAcc/MemAccRbListSys/MemAccRbSys/MemAccUseFlsSysStyle

Short Name	MemAccUseFlsSysStyle
Long Name	Style of System-Interface
Description	Defines the style in which the System-APIs are generated. FALSE/Default = MemAcc, TRUE=Fls
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

3.3.1.106 MemAcc/MemAccRbNvMData

Short Name	MemAccRbNvMData
Long Name	Storage to NvM
Description	Container for NvM data generation.
Multiplicity	0..1
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

3.3.1.107 MemAcc/MemAccRbNvMData/MemAccPersistTimeMeasData

Short Name	MemAccPersistTimeMeasData
Long Name	Activation of Time measurement Data Storage
Description	Switch to enable time measurement data storage in NvM.
Multiplicity	0..1

Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

3.3.1.108 MemAcc/MemAccRbNvMData/MemAccPersistTimeNrElements

Short Name	MemAccPersistTimeNrElements
Long Name	Number of Data Storage Elements
Description	Number of measurement tuples stored to the NvM. Recommended: to set this to the number of devices expected in a later configuration step. If left empty or 0, then 2 is assumed
Multiplicity	0..1
Value Range	1..6
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

3.3.2 Additional Services

The following API are available in version v12.7.0 in addition to those defined in AR v22-11

3.3.2.1 MemAcc_HwSpecificServiceSync

Syntax	`MemAcc_HwSpecificServiceSync`
Parameters	`idAdrArea_o` Address Area Id. `idHw` Addressed Hw (0=automatic selection). `idHwService` Service Id. `data_pu8` Result/exchange data storage (caller has to ensure sufficient memory). `nrBytes_puo` Pointer to length information.
Returns	`E_OK` Order was accepted. `E_NOT_OK` User parameters are incorrect. `E_MEM_SERVICE_NOT_AVAIL` Service is not supported.
Description	Generates an Hw service request (bi-directional) for synchronous operation.

3.3.2.2 MemAcc_Rb_IsMainFunctionLocked

Syntax	`MemAcc_Rb_IsMainFunctionLocked`
Returns	`TRUE` MainFunction currently locked. `FALSE` MainFunction not currently locked.
Description	Checks whether the MainFunction of a Device Instance Group is currently locked for execution.

3.3.2.3 MemAcc_IsStatusInitialized

Syntax	`MemAcc_IsStatusInitialized`
Returns	`TRUE` Device Instance Group is initialized. `FALSE` Device Instance Group is not initialized.
Description	Reports whether the Device Instance Group is initialized.

3.3.2.4 MemAcc_IsStatusOperational

Syntax	`MemAcc_IsStatusOperational`
Returns	`TRUE` Device Instance Group is operational. `FALSE` Device Instance Group is not operational.
Description	Reports whether the Device Instance Group is operational.

3.3.2.5 MemAcc_IsStatusPowerDown

Syntax	`MemAcc_IsStatusPowerDown`
Returns	`TRUE` Device Instance Group is powered down. `FALSE` Device Instance Group is not powered down.
Description	Reports whether the Device Instance Group is initialized but in power-down state.

3.3.2.6 MemAcc_MainFunctionFromIsr

Syntax	`MemAcc_MainFunctionFromIsr`
Description	Performs processing of jobs within Isr context for a Device Instance Group.

3.3.2.7 MemAcc_MainFunctionAndDependency

Syntax	`MemAcc_MainFunctionAndDependency`
Description	Performs the cyclic processing of jobs of a Device Instance Group and invocation of lower-layer dependencies.

3.3.2.8 MemAcc_Rb_Cancel

Syntax	`MemAcc_Rb_Cancel`
Parameters	`idAdrArea_o` Address Area Id.
Description	Immediately cancels a pending job.

3.3.2.9 MemAcc_Rb_SetCalibParameter

Syntax	`MemAcc_Rb_SetCalibParameter`
Parameters	`idHw_o` Hardware ID. `adrCalib_u32` Address of calibration data. `data_u32` Reference data value.
Returns	`E_OK` Calibration parameters updated. `E_NOT_OK` Setting calibration parameters failed.
Description	Sets the DDR calibration parameter.

3.3.2.10 MemAcc_Rb_SetDdrMode

Syntax	`MemAcc_Rb_SetDdrMode`
Parameters	`idDevice_en` Device Instance Id. `TargetAddress` Any address within the user area. `isDdrMode_b` Desired mode of operation (TRUE=DDR).
Returns	`E_OK` Mode switch request accepted. `E_NOT_OK` Mode switch request failed.
Description	Sets selected user area into the defined Mode.

3.3.2.11 MemAcc_Rb_SetAuthentId

Syntax	`MemAcc_Rb_SetAuthentId`
Parameters	`idHw_o` Hardware Id. `id_pcu32` Pointer to Id storage. `nrEntry_u8` Number of entries in this storage. `stSelMsk_u16` Selection mask of the Id.
Returns	`E_OK` Parameters updated. `E_NOT_OK` Parameter update failed.
Description	Sets the authentification parameter.

3.3.2.12 MemAcc_Rb_RaisePriority

Syntax	`MemAcc_Rb_RaisePriority`
Parameters	`idAdrArea_o` Address Area Id.
Returns	`E_OK` Command accepted. `E_NOT_OK` Command rejected.
Description	Sets an elevated priority to the job.

3.3.2.13 MemAcc_Rb_LowerPriority

Syntax	`MemAcc_Rb_LowerPriority`
Parameters	`idAdrArea_o` Address Area Id.
Returns	`E_OK` Command accepted. `E_NOT_OK` Command rejected.
Description	Lowers the elevated priority of the job to the standard one.

3.3.2.14 MemAcc_Rb_AllowJobsDuringRaisedPrio

Syntax	`MemAcc_Rb_AllowJobsDuringRaisedPrio`
Parameters	`idAdrArea_o` Address Area Id. `flgJob_b` Flag to select the mode.
Returns	`E_OK` Command accepted. `E_NOT_OK` Command rejected.
Description	Differentiates between the standard mode and special mode.

3.3.2.15 MemAcc_Rb_IsPrioRaised

Syntax	`MemAcc_Rb_IsPrioRaised`
Parameters	`idAdrArea_o` Address Area Id.
Description	Reports whether the Device Instance assigned to the Address Area has a raised priority.

3.3.2.16 MemAcc_Rb_MainFunctionDependencyPerAdrArea

Syntax	`MemAcc_Rb_MainFunctionDependencyPerAdrArea`
Parameters	`idAdrArea_o` Address Area Id.
Description	Calls MainFunction and dependency for a dedicated user.

3.3.2.17 MemAcc_Rb_Init

Syntax	`MemAcc_Rb_Init`
Parameters	`idDevice_en` Device Instance Id. `flgInitHw_b` Initialize the Hw (normally TRUE only if multi-instance needed).
Description	Init function of the MemAcc device.

3.3.2.18 MemAcc_Rb_DeInit

Syntax	`MemAcc_Rb_DeInit`
Parameters	`idDevice_en` Device Instance Id.
Description	De-initializes the MemAcc device.

3.3.2.19 MemAcc_Rb_MainFunction

Syntax	`MemAcc_Rb_MainFunction`
Parameters	`idDevice_en` Device Instance Id. `isDependencyRequired_b` Specifies if cyclic dependency is required.
Description	Cyclic-called entrance function of MemAcc.

3.3.2.20 MemAcc_Rb_MainFunctionFromIsr

Syntax	`MemAcc_Rb_MainFunctionFromIsr`
Parameters	`idDevice_en` Device Instance Id.
Description	MainFunction handling from Isr.

3.3.2.21 MemAcc_Rb_IsChunkModeFast

Syntax	`MemAcc_Rb_IsChunkModeFast`
Parameters	`idDevice_en` Device Instance Id.
Returns	`boolean` TRUE=Fast, FALSE=Normal.
Description	Checks whether the chunk mode is NORMAL(FALSE) or FAST(TRUE).

3.3.2.22 MemAcc_Rb_SetChunkModeFast

Syntax	`MemAcc_Rb_SetChunkModeFast`
Parameters	`idDevice_en` Device Instance Id. `flgChunkMode_b` ChunkMode TRUE=Fast, FALSE=Normal.
Description	Set the chunk mode of a device to NORMAL(FALSE) or FAST(TRUE).

3.3.2.23 MemAcc_Rb_ReqTransferMode

Syntax	`MemAcc_Rb_ReqTransferMode`

Parameters	`idDevice_en` Device Instance Id. `flgReq_b` Passed user request (TRUE=Standard, FALSE=restricted/synchronous)
Description	Sets the request to change to standard transfer mode.

3.3.2.24 MemAcc_Rb_IsTransferModeStdDisabled

Syntax	`MemAcc_Rb_IsTransferModeStdDisabled`
Parameters	`idDevice_en` Device Instance Id.
Returns	`TRUE` Synchronous mode. `FALSE` Standard mode/transition mode.
Description	Checks whether the standard transfer mode is disabled and a restricted transfer.

3.3.2.25 MemAcc_Rb_ReqDeepPowerDown

Syntax	`MemAcc_Rb_ReqDeepPowerDown`
Parameters	`idDevice_en` Device Instance Id. `flgReq_b` Passed user request (TRUE=DeepPowerDown, FALSE=Normal)
Returns	`E_OK` If idDevice is correct. `E_NOT_OK` If idDevice is incorrect.
Description	Sets the request to change to deep power-down mode.

3.3.2.26 MemAcc_IsDeviceInitialized

Syntax	`MemAcc_IsDeviceInitialized`
Parameters	`idDevice_en` Device Instance Id.
Returns	`boolean` TRUE or FALSE.
Description	Is the device initialized?

3.3.2.27 MemAcc_IsDeviceOperational

Syntax	`MemAcc_IsDeviceOperational`
Parameters	`idDevice_en` Device Instance id.
Returns	`boolean` TRUE or FALSE.
Description	Is the device in operation?

3.3.2.28 MemAcc_IsDevicePowerDown

Syntax	`MemAcc_IsDevicePowerDown`
Parameters	`idDevice_en` Device Instance Id.
Returns	`boolean` TRUE or FALSE.
Description	Is the device in power-down state?

3.3.2.29 MemAcc_Rb_RequestAdr

Syntax	`MemAcc_Rb_RequestAdr`
Parameters	`idDevice_en` Device Instance Id. `RequestType_pst` Pointer for request type (not modified). `idxJobType_en` Index of the job type.
Returns	`E_OK` Order was accepted. `E_NOT_OK` User parameters are incorrect or MemAcc is currently busy.
Description	Accepts the request order from the top layer/application.

3.3.2.30 MemAcc_Rb_RequestId

Syntax	`MemAcc_Rb_RequestId`
Parameters	`idAdrArea_o` Address Area identifier. `RequestType_pst` Pointer for request type (not modified). `idxJobType_en` Index of the job type.
Returns	`E_OK` Order was accepted. `E_NOT_OK` User parameters are incorrect or MemAcc is currently busy.
Description	Accepts the request order from the top layer/application.

3.3.2.31 MemAcc_Rb_GetHwDeviceInfo

Syntax	`MemAcc_Rb_GetHwDeviceInfo`
Parameters	`idHw_o` Hw device identifier. `DeviceInfo_pst` Device information structure containing manufacturer ID, size etc.
Description	Transfers the internally available Hw device information of the first HwId assigned.

3.3.2.32 MemAcc_Rb_SetHwDeviceInfo

Syntax	`MemAcc_Rb_SetHwDeviceInfo`
Parameters	`idHw_o` Hw device identifier. `DeviceInfo_pst` Device information structure containing manufacturer ID, size etc.
Description	Transfers the Hw device information to the internal memory.

3.3.2.33 MemAcc_Rb_AllowJobsDuringRaisedPrioDev

Syntax	`MemAcc_Rb_AllowJobsDuringRaisedPrioDev`
Parameters	`idDevice_en` Device Instance Id. `flgJob_b` Flag to select the mode.
Returns	`E_OK` Command accepted. `E_NOT_OK` Command rejected.
Description	Differentiates between standard mode (FALSE, only jobs for the elevated Address Area will be started) and special mode (TRUE, other jobs can be started if no job for the elevated Address Area is requested).

3.3.2.34 MemAcc_Rb_IsPrioRaisedDev

Syntax	`MemAcc_Rb_IsPrioRaisedDev`
Parameters	`idDevice_en` Device Instance Id.
Description	Reports whether the Device Instance has a raised priority.

3.3.2.35 MemAcc_Rb_GetHwIdFromDevice

Syntax	`MemAcc_Rb_GetHwIdFromDevice`
Parameters	`idDevice_en` Device Instance Id. `idxHw_u8` Index in idHwList of the device.
Returns	`MemAcc_HwIdType` Hw Id or a value >= MemAccHwId_nrEntries for not found.
Description	Returns the HwId of the Hw assigned to a device with reference index idxHw.

3.3.2.36 MemAcc_Rb_QueryInfo

Syntax	`MemAcc_Rb_QueryInfo`

Parameters	`idAdrArea_o` Address area to be checked. `adrSource_uo` Logical start address / or 0 to get the first one. `nrBytes_uo` Length of logical range / or 0 to get remaining length. `QueryInfo_pst` Pointer to the result structure.
Returns	`E_OK` If a result is available. `E_NOT_OK` If a result is unavailable.
Description	Performs a translation of logical information to memory information.

3.3.2.37 MemAcc_Rb_FlsWrite

Syntax	`MemAcc_Rb_FlsWrite`
Parameters	`idxJob_en` Can be used to set a special handling procedure.
Description	Generates a write request.

3.3.2.38 MemAcc_Rb_FlsErase

Syntax	`MemAcc_Rb_FlsErase`
Parameters	`idxJob_en` Can be used to set a special handling procedure.
Description	Generates an erase request.

3.3.2.39 MemAcc_Rb_FlsRead

Syntax	`MemAcc_Rb_FlsRead`
Description	Generates a read request.

3.3.2.40 MemAcc_Rb_FlsCompare

Syntax	`MemAcc_Rb_FlsCompare`
Description	Triggers a job to compare the passed data to the memory content of the provided Address Area.

3.3.2.41 MemAcc_Rb_FlsBlankCheck

Syntax	`MemAcc_Rb_FlsBlankCheck`

3.3.2.42 MemAcc_Rb_FlsExtendedOper

Syntax	`MemAcc_Rb_FlsExtendedOper`
Parameters	`idxJob_en` Can be used to set a special handling procedure.

Description	Generates an extended operation request.

3.3.2.43 MemAcc_FlsCancelAdr

Syntax	`MemAcc_FlsCancelAdr`
Parameters	`idxJob_en` Device Instance Id. `adr_uo` Target address.
Description	Initiates the cancellation of a pending request.

3.3.2.44 MemAcc_FlsCancelUser

Syntax	`MemAcc_FlsCancelUser`
Parameters	`idAdrArea_o` Address Area Id.
Description	Initiates the cancellation of a pending request.

3.3.2.45 MemAcc_FlsGetStatus

Syntax	`MemAcc_FlsGetStatus`
Parameters	`idDevice_en` Device Instance Id.
Returns	`MEMIF_UNINIT` Device is uninitialized. `MEMIF_IDLE` Device is currently idle. `MEMIF_BUSY` Device is currently busy in executing order.
Description	Returns module status.

3.3.2.46 MemAcc_FlsGetJobResultAdr

Syntax	`MemAcc_FlsGetJobResultAdr`
Parameters	`idDevice_en` Device Instance Id.
Returns	`MEMIF_UNINIT` Device is uninitialized. `MEMIF_IDLE` Device is currently idle. `MEMIF_BUSY` Device is currently busy in executing order.
Description	Returns the job result.

3.3.2.47 MemAcc_FlsGetJobResultId

Syntax	`MemAcc_FlsGetJobResultId`
Description	Returns the current job result.

3.3.2.48 MemAcc_FlsGetJobErrDetails

Syntax	`MemAcc_FlsGetJobErrDetails`
Description	Returns the current job result details.

3.3.2.49 MemAcc_FlsGetMediaAddress

Syntax	`MemAcc_FlsGetMediaAddress`
Parameters	`idDevice_en` Device Instance Id. `TargetAddress` Target address to identify the user. `Address_puo` Returned current/last working address passed as pointer.
Returns	`E_OK` Device/address info valid. `E_NOT_OK` Device/address info not valid.
Description	Gets the current media address processed.

3.3.2.50 MemAcc_FlsGetSectorInfo

Syntax	`MemAcc_FlsGetSectorInfo`
Parameters	`idDevice_en` Device Instance Id. `adr_uo` Target address. `SectorInfo_pst` Pointer to Fls_Rb_SectorInfo_tst structure.
Returns	`E_OK` Sector info found. `E_NOT_OK` Sector info not found.
Description	Get the sector information.

3.3.2.51 MemAcc_FlsGetMode

Syntax	`MemAcc_FlsGetMode`
Parameters	`idDevice_en` Device Instance Id.
Returns	`MEMIF_Mode_SLOW`. `MEMIF_Mode_FAST` (also in error case).
Description	Gets the FLS internal mode.

3.3.2.52 MemAcc_FlsSetMode

Syntax	`MemAcc_FlsSetMode`

Parameters	`idDevice_en` Device Instance Id. `MEMIF_Mode_SLOW`. `MEMIF_Mode_FAST`.
Description	Sets the FLS internal mode.

3.3.2.53 MemAcc_Rb_FlsSuspend

Syntax	`MemAcc_Rb_FlsSuspend`
Parameters	`idDevice_en` Device Instance Id. `adr_uo` Target address.
Returns	`E_OK` Command accepted. `E_NOT_OK` Not accepted.
Description	Initiates a suspend operation.

3.3.2.54 MemAcc_Rb_FlsResume

Syntax	`MemAcc_Rb_FlsResume`
Parameters	`idDevice_en` Device Instance Id. `adr_uo` Target address.
Returns	`E_OK` Command accepted. `E_NOT_OK` Not accepted.
Description	Initiates a resume operation.

3.3.2.55 MemAcc_FlsGetDeviceInfo

Syntax	`MemAcc_FlsGetDeviceInfo`
Parameters	`idDevice_en` Device Instance Id. `DeviceInfo_pst` Device information structure containing manufacturer ID, Size etc.
Description	Transfers the internally available device information.

3.3.2.56 MemAcc_FlsSetCalibParameter

Syntax	`MemAcc_FlsSetCalibParameter`
Parameters	`idDevice_en` Device Instance Id. `adrCalib_u32` Address of calibration data. `data_u32` Reference data value.
Returns	`E_OK` Calibration parameters updated. `E_NOT_OK` Setting calibration parameters failed.

Description	Sets the DDR calibration parameter.

3.3.2.57 MemAcc_FlsSetDdrMode

Syntax	`MemAcc_FlsSetDdrMode`
Parameters	`idDevice_en` Device Instance Id. `TargetAddress` Any address within the user area. `isDdrMode_b` Desired mode of operation (TRUE=DDR).
Returns	`E_OK` Mode switch request accepted. `E_NOT_OK` Mode switch request failed.
Description	Sets selected user area into the defined mode.

3.3.2.58 MemAcc_FlsSetAuthentId

Syntax	`MemAcc_FlsSetAuthentId`
Parameters	`idDevice_en` Device Instance Id. `id_pcu32` Pointer to Id storage. `nrEntry_u8` Number of entries in this storage. `stSelMsk_u16` Selection mask of the Id.
Description	Set the authentification information.

3.3.2.59 MemAcc_Rb_FlsEnterHighSpeedModeAdr

Syntax	`MemAcc_Rb_FlsEnterHighSpeedModeAdr`
Parameters	`idDevice_en` Device Instance Id. `adr_uo` Target address.
Returns	`E_OK` Command accepted. `E_NOT_OK` Command not accepted.
Description	Initiates a suspend operation.

3.3.2.60 MemAcc_Rb_FlsExitHighSpeedModeAdr

Syntax	`MemAcc_Rb_FlsExitHighSpeedModeAdr`
Parameters	`idDevice_en` Device Instance Id. `adr_uo` Target address.
Returns	`E_OK` Command accepted. `E_NOT_OK` Command not accepted.
Description	Initiates a resume operation.

3.3.2.61 MemAcc_FlsGetDeviceIdFromDrvIndex

Syntax	`MemAcc_FlsGetDeviceIdFromDrvIndex`
Parameters	`idxFlsDriver` Fls driver index as configured.
Returns	`idDevice_en` Device Id corresponding to the requested address or Fls_Rb_nrFlsDevice (signal none-existing idxFlsDriver).
Description	Get the Id device name from the configured FlsDriverIndex.

3.3.2.62 MemAcc_FlsGetVersionInfo

Syntax	`MemAcc_FlsGetVersionInfo`
Parameters	`VersionInfo_pst` Pointer to the version information of this module.
Description	Returns general and software version information, with Fls module Id.

3.3.2.63 MemAcc_Rb_SetHwDeviceId

Syntax	`MemAcc_Rb_SetHwDeviceId`
Parameters	`idHw_o` Hardware Id. `idHwDevice_u32` Information of HW device used.
Returns	`E_OK` Request accepted. `E_NOT_OK` Request not accepted.
Description	Sets the HW device info and may be called before MemAcc_Init. Also calls Lld without MainFunction.

3.4 Limitations

Version v12.7.0 has following limitations:

- Burst configuration parameters are optional (not mandatory). If not configured explicitely, Burst is enabled and MemAcc automatically calculates the optimum Burst size. De-activating Burst does not give any advantage.

Affected SWS items: SWS_MemAcc_00087, SWS_MemAcc_00102

- Only the “indirect, static” invocation mode is provided (parameter `MemAccMemInvocation`).

Affected SWS items: SWS_MemAcc_00085, ECUC_MemAcc_00026 Affected SWS items: SWS_MemAcc_10033, SWS_MemAcc_00088, SWS_MemAcc_00089, SWS_MemAcc_00121 Affected SWS items: SWS_MemAcc_10034, SWS_MemAcc_00090, SWS_MemAcc_00122, SWS_MemAcc_00123

3.4.1 Temporary

- `MemAcc_RequestLock` and `MemAcc_ReleaseLock` are not supported.

Affected SWS items: SWS_MemAcc_10030, SWS_MemAcc_00071, SWS_MemAcc_00072, SWS_MemAcc_00073, SWS_MemAcc_00099, SWS_MemAcc_00115, SWS_MemAcc_00116 Affected SWS items: SWS_MemAcc_10031, SWS_MemAcc_00076, SWS_MemAcc_00077, SWS_MemAcc_00093 Affected SWS items: SWS_MemAcc_10032

3.5 Exclusions

`MemAcc_RequestLock` `MemAcc_ReleaseLock`

The following configuration parameters from AR v22-11 are not supported in version v12.7.0.

- MemAcc/MemAccAddressAreaConfiguration/MemAccAddressAreaPriority
- MemAcc/MemAccAddressAreaConfiguration/MemAccBufferAlignmentValue
- MemAcc/MemAccAddressAreaConfiguration/MemAccJobEndNotificationName
- MemAcc/MemAccAddressAreaConfiguration/MemAccSubAddressAreaConfiguration/MemAccLogicalStartAddress
- MemAcc/MemAccAddressAreaConfiguration/MemAccSubAddressAreaConfiguration/MemAccMemInvocation
- MemAcc/MemAccAddressAreaConfiguration/MemAccSubAddressAreaConfiguration/MemAccMemNamePrefix
- MemAcc/MemAccAddressAreaConfiguration/MemAccSubAddressAreaConfiguration/MemAccNumberOfEraseRetries
- MemAcc/MemAccAddressAreaConfiguration/MemAccSubAddressAreaConfiguration/MemAccNumberOfSectors
- MemAcc/MemAccAddressAreaConfiguration/MemAccSubAddressAreaConfiguration/MemAccNumberOfWriteRetries
- MemAcc/MemAccAddressAreaConfiguration/MemAccSubAddressAreaConfiguration/MemAccSectorBatchRef
- MemAcc/MemAccAddressAreaConfiguration/MemAccSubAddressAreaConfiguration/MemAccSectorOffset
- MemAcc/MemAccAddressAreaConfiguration/MemAccSubAddressAreaConfiguration/MemAccUseEraseBurst
- MemAcc/MemAccAddressAreaConfiguration/MemAccSubAddressAreaConfiguration/MemAccUseWriteBurst
- MemAcc/MemAccGeneral/MemAcc64BitSupport
- MemAcc/MemAccGeneral/MemAccCompareApi
- MemAcc/MemAccGeneral/MemAccDevErrorDetect

- MemAcc/MemAccGeneral/MemAccMainFunctionPeriod
- MemAcc/MemAccGeneral/MemAccPublicCddHeaderFile
- MemAcc/MemAccGeneral/MemAccUseMemFuncPtrTable
- MemAcc/MemAccRbGeneral/MemAccCompareApi
- MemAcc/MemAccRbListApi/MemAccRbApi/MemAccCompareApi
- MemAcc/MemAccRbListSys/MemAccRbSys/MemAccMainFunctionPeriod

3.6 Deviations

Configuration parameters which differ from the AR v22-11 specification are listed below.

3.6.1 MemAcc

[RTA-BSW] Description	Configuration of the Memory Access Layer
[AUTOSAR] Description	The MemAcc (Memory Access module) coordinates the memory access by multiple users in order to avoid conflicts with this shared memory resource.

3.6.2 MemAcc/MemAccAddressAreaConfiguration

[RTA-BSW] Description	This container includes the configuration of AddressArea specific parameters for the MemAcc module (limited to 255 to ensure efficient implementation). At least 1 area needs to be configured for a proper MemAcc operation.
[AUTOSAR] Description	This container includes the configuration of AddressArea specific parameters for the MemAcc module.
[RTA-BSW] Multiplicity	0..255
[AUTOSAR] Multiplicity	1..65535

3.6.3 MemAcc/MemAccAddressAreaConfiguration/MemAccAddressAreaId

[RTA-BSW] Description	This value specifies a unique identifier which is used to reference to an AddressArea. Naming: MemAccConf_MemAccAddressAreaConfiguration_<shortname-MemAccAddressAreaConfiguration>
[AUTOSAR] Description	This value specifies a unique identifier which is used to reference to an AddressArea.
[RTA-BSW] Lower Multiplicity	0

[AUTOSAR] Lower Multiplicity	1
[REMOVED] Default Value	0

3.6.4 MemAcc/MemAccAddressAreaConfiguration/MemAccSubAddressAreaConfiguration

[RTA-BSW] Description	Configuration and assignments to Mem SectorBatches
[AUTOSAR] Description	This container includes the configuration parameters for a physically continuous area of memory.
[RTA-BSW] Multiplicity	0..65535
[AUTOSAR] Multiplicity	1..65536

3.6.5 MemAcc/MemAccGeneral

[RTA-BSW] Description	Container for general configuration parameters of the MemAcc.
[AUTOSAR] Description	General configuration parameters of the MemAcc.
[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1

3.7 API Definition

List of supported API functions

Table 3.179.MemAcc Supported API functions

`MemAcc_Init`	Initializes all variables and sets the module state to initialized.
`MemAcc_DeInit`	Terminates pending access jobs and sets the module state to uninitialized. MemAcc must be re-initialized before accepting new job requests.
`MemAcc_GetVersionInfo`	Returns the version information of the MemAcc module.
`MemAcc_GetJobResult`	Returns the consolidated job result of the Address Area referenced by `addressAreaId`.
`MemAcc_GetJobStatus`	Returns the status of the MemAcc job referenced by `addressAreaId`.
`MemAcc_GetMemoryInfo`	Retrieves the physical memory device information of a specific Address Area.
`MemAcc_GetProcessedLength`	Returns the accumulated number of bytes that have already been processed in the current job.
`MemAcc_GetJobInfo`	Returns detailed information of the current memory job e.g. memory device ID, job type, job processing state or job result, Address Area as well as address and length.

`MemAcc_ActivateMem`	Dynamic activation and initialization of a Mem driver referenced by `hwId` and `headerAddress`.
`MemAcc_DeactivateMem`	Dynamic deactivation of a Mem driver referenced by `hwId` and `headerAddress`.
`MemAcc_Cancel`	Triggers a cancel operation of the pending job for the Address Area referenced by the Address Area Id.
`MemAcc_Read`	Triggers a read job to copy data from the source address into the referenced destination data buffer.
`MemAcc_Write`	Triggers a write job to store the passed data to the provided Address Area with given address and length.
`MemAcc_Erase`	Triggers an erase job of the given area (defined by `targetAddress` and length).
`MemAcc_Compare`	Triggers a job to compare the passed data to the memory content of the provided Address Area.
`MemAcc_BlankCheck`	Checks if the passed address space is blank, i.e. erased and writeable.
`MemAcc_HwSpecificService`	Triggers a hardware-specific job request referenced by `hwServiceId`.
`MemAcc_MainFunction`	Service to handle the requested jobs and the internal management operations. Depending on the configuration, MemAcc will call the Mem driver Main Functions.

For further information on these functions, please refer to AUTOSAR_SWS_MemoryAccess.pdf

3.8 Configuration Advice

3.8.1 General

The configuration parameters of MemAcc are grouped as follows:

`MemAccAddressAreaConfiguration` **- Address Area Configurations**

This is where the properties of an Address Area are defined. An Address Area provides a logical memory with a start address and length. Ideally, it provides a continuous logical memory. However, it is also possible to define a fragmented logical memory. The link to the physical memory is made by references to the Sector Batches provided by the Mem driver. Several physical parts of the same or different Mem drivers can be combined under one Address Area.

`MemAccGeneral` **- General Parameters**

Scheduling aspects and Det usage.

`MemAccRbGeneral` **- General Parameters Extension**

The default selection, whether APIs are available or not. This gives the possibility of reducing the code-size footprint of both MemAcc and the underlying driver. Additional features of MemAcc can also be activated.

`MemAccRbListSys` **- Interfaces for System Integration**

Defines separate System Interfaces (`MemAcc_Init`, `MemAcc_DeInit`, `MemAcc_MainFunction`), which is needed when using different Device Instance Groups. Different settings (similar to `MemAccGeneral`) can be defined for each Device Instance Group. If no additional Device Instance Groups are available, then it is not necessary to configure this container.

`MemAccRbListApi` **- Interfaces for Users/API**

Separate API headers can be defined and referenced from the Address Area, making it possible to provide a dedicated interface to the users.

Hint This might be useful in the transition from Fls to MemAcc, as it is possible to generate an API with the "old" Fls interfaces.

`MemAccRbNvMData` **- Storage to NvM**

This container is needed only when using the time measurement and storing the result within the NvM.

3.8.2 Fls Compatibility Mode

This requires steps which draw upon modules in the Mem stack:

- *rba_MemLib* provides the possibility to select either a standard Fls or an Fls emulated by MemAcc.
- *Mem_11_Cmn* converts the available definition of the Fls low-level drivers into a Mem driver.
- *rba_MemAccFlsAdpr* converts the Fls user configuration into a MemAcc configuration (provided that the Mem driver configuration above is available).
- MemAcc itself will generate Fls interfaces and function calls, instead of MemAcc ones.

A mixed Fls/MemAcc style is possible e.g. changes from Fls-style interfaces to MemAcc ones can be made user-by-user, rather than all at once.

It is also possible to reuse the Fls low-level drivers together with MemAcc, thus providing greater migration flexibility from Fls LLDs to Mem drivers.

Scenario: MemAcc using Mem driver

This is the future target scenario.

Scenario: MemAcc using an Fls LLD

Transition scenario with the restriction that Hw-specific services are not available.

Scenario: Emulated Fls using Fls LLD

Early transition scenario that allows standard Fls interfaces with exactly the same functionality but realized with MemAcc component. This scenario might be useful in combination with others.

Performance of MemAcc is comparable with Fls (depending upon Burst configu-

ration).configuration).

Scenario: Emulated Fls using Mem Driver

Special transition scenario limited to standard Fls functionality.

3.8.3 Parallel Device Support

A device is a MemAcc instance which is independently operated from a physical point of view i.e. it does not share any Device Instance or Address Area with another device. So, for example, a `MemAcc_Init`, `MemAcc_DeInit` sequence at device level has no impact on other devices, and the device can operate in parallel with others.

MemAcc also allows several devices to combine into a Device Instance Group. A Device Instance Group provides separate System Interfaces, such as `MemAcc_Init`, `MemAcc_DeInit`, `MemAcc_MainFunction` etc., with a dedicated name (defined by the container short name `MemAccRbSys`) replacing the term "MemAcc". Therefore, only Device Instance Groups allow a separation on different cores, or different Os tasks. The relevant scheduling parameters are also available in the container `MemAccRbSys`.

Multi-instance drivers, in this context, are Mem drivers which support several channels (e.g. external flash circuits connected via SPI lines). In order to use them in parallel, it is necesary to provide different `MemAcc_Init`, `MemAcc_DeInit` and `MemAcc_MainFunction` interfaces, which have additional `idInstance` parameters (an extension of the AUTOSAR standard). Without this parameter, the complete driver (with all the Device Instances) has to be assigned to one Device Instance Group.

The Mem driver indicates such interfaces by enabling the parameter `MemRbUseSysInstanceId`. In addition, the names of the related software interfaces have to be provided within the container `MemRbFuncMem`.

3.8.4 Initialization

Initialization is started by a `MemAcc_Init` call and completed within the following `MemAcc_MainFunction` calls.

Until the Hw initialization (by the Mem driver) is completed, the assigned MemAcc device is in a state in which job requests are accepted but unprocessed. Only after the Hw initialization are the jobs routed to the Mem drivers.

In such a configuration, it is strongly recommended to use the initialization timeout functionality to limit the delay (setting `MemDrvCfgTimeoutInit` to "0" will prevent the retry of the initialization).

After the initialization timeout has occured, the device performs a `MemAcc_DeInit` sequence and thus all pending jobs are aborted (callbacks are called). The related device will enter the UNINIT state.

3.8.5 Asynchronuous Job-Handling

Each Address Area owns a constant priority (parameter `MemAccAddressAreaPriority`). Within each `MemAcc_MainFunction` cycle, MemAcc selects the Address Area having the highest priority (A). It is possible that the job

of another Address Area (B) with a lower priority is already in progress: MemAcc will suspend the operation of (B) as soon as possible and start the operation on (A). Once (A) is completed, then the operation on (B) is resumed.

Normally, the Address Areas define different physical memory locations; however, they can also overlap. Such an overlap typically requires some additional co-ordination or mode-handling outside of MemAcc.

To process a job, MemAcc calls the Mem driver to perform sub-jobs (called *MemJobs*). Typically, a job is split into several MemJob chunks with sector/-page/Burst page granularity. The optimum chunk size is defined by the Mem driver configuration. Only in read cases can the size be configured for an Address Area to define the compromise between the processing speed and execution duration of one `MemAcc_MainFunction`. In the erase and write cases, the software processing time is negligible compared to the Hw duration.

When a MemJob finishes, MemAcc is checking for the next job with the highest priority. If all the MemJobs are finished, the job is finished.

3.8.6 Timeout Supervision

WARNING

This feature is not intended or suitable to verify the timings given within the controller's data sheet. Therefore the timings selected shall be robust and not too tight, so as to avoid false problem reporting.

The timeout parameters are configurable by the user (but slower than the lower limit defined by the Mem driver).

NOTE

Reference sizes for value calculation: 256 byte page-size and 32 kByte sector-size.

The parameter `MemAccTimeoutSupervionEnabled` is used to enable/disable the timeout supervision feature. Timeout supervision itself is done in two stages:

1. Mem driver request acceptance (`MemAccTimoutReqAcceptance`);
2. Mem driver request processing (which is calculated depending upon the request size). Page or sector-oriented commands are differentiated:

constant part: `MemAccTimePageOffset` - variable part: `MemAccTimePagePerVolume` (256 bytes)

constant part: `MemAccTimeSectorOffset` - variable part: `MemAccTimeSectorPerVolume` (32 kB)

MemAcc also provides a time measurement feature (enabled via the parameter `MemAccTimeMeasurementEnabled`) which tracks the maximum value of each category and stores the measured result as maximum in the NvM.

After a timeout has occured, MemAcc automatically starts a `MemAcc_DeInit`, `MemAcc_Init` sequence of the related Mem driver instance. However, if the Hw is stuck, it is most likely that the initialization will also fail and the related device will end up in the UNINIT state.

3.8.7 RTE Interface

3.8.7.1 Trigger Interfaces

Each Device Instance (belonging to a Device Instance Group<Instance>) has its own schedulable event. This event is based on the parameter `MemAccRbMainFunctionScheduling`.

- If the literal *RTE_BACKGROUND* is selected, then a background event is created:

/AUTOSAR_MemAcc/ModuleDescriptions/<Instance>/BswInternalBehavior/BE_MainFunction

- If the literal *RTE_TIMED* is selected, then a timing event with period `MemAccMainFunctionPeriod` is created:

/AUTOSAR_MemAcc/ModuleDescriptions/<Instance>/BswInternalBehavior/TE_MainFunction

- If the literal *MANUAL* is selected, then no RTE event is generated.

The later configuration might be useful, if the MemIf component already calls Fee and MemAcc.

Hint Scheduling a MemAcc Device Instance in the same scheduling scope as its users is recommended, typically in a background loop or task.

3.8.7.2 Scheduling Interface

Each Device Instance/Device Instance Group<MemAcc> has its own `<MemAcc>_MainFunction` and thus needs separate scheduling. The RTE integration is described in:

/AUTOSAR_MemAcc/BswModuleDescriptions/MemAcc/BswDocInterfaces/BswDocInterfaceRte/Bsw

In addition, it is possible to activate the interrupt mode (`MemAccRbUseInterrupts` for an Address Area) and call the `<MemAcc>_MainFunctionFromIsr` from a CAT-2 ISR.

3.8.8 Memory Mapping

MemMaps are typically generated from the definition within the generated *BSWMD.arxml* files.

However, it is possible to prevent the MemMap generation centrally within the rba_MemLib (the parameter `rba_MemLibEnableMemMapGeneration` needs to be set to “FALSE”).

The following sections have to be defined for a Device Instance Group<MemAcc>:

Filename: <MemAcc>_MemMap.h

Code	Code executed from ROM
VAR_CLEARED_UNSPECIFIED	Internal structures.
VAR_SAVED_ZONE_UNSPECIFIED	Measurement result stored in NvM (only if feature is activated).
VAR_SECURED_CLEARED_UNSPECIFIED	Intermediate buffers which are read protected. Note: Might not be needed in all configurations.
VAR_SECURED_CACHED_RAM_ALIGNED_CLEARED_UNSPECIFIED	Intermediate buffers which are read protected AND 64-byte aligned. Note: Might not be needed in all configurations.
CONST_UNSPECIFIED	Tables etc.

Thus, for each Device Instance Group, a separate memory allocation is possible. This eases the use of core-specific protections, using MPU etc.

3.8.9 Dependencies

In Fls compatibility mode, MemAcc needs the following components:

- rba_MemLib - Central utilities/configurations of the Mem stack, timer-handling.
- MemIf - For Fls-used type definitions.
- Fls
- rba_MemAccFlsAdpr - Conversion of Fls configuration to MemAcc configuration.
- Mem_11_Cmn - Common definitions/ParamDefs of the Mem driver. Contains a conversion from Fls LLD definition to Mem driver definition.
- Suitable Fls low-level drivers for the project (in a fitting version).

In MemAcc Mode, MemAcc needs the following components:

- rba_MemLib - Central utilities/configurations of the Mem stack, timer handling.
- Mem_11_Cmn - Common definitions/ParamDefs of the Mem driver. Contains a conversion from Fls LLD definition to Mem driver definition.
- Suitable Mem drivers for the project (in a fitting version).

Some more components are needed for the infrastructure, but they are not dependent on MemAcc:

- CUCEL package - For central header, type definitions and MemMap generation.

> **NOTE**
>
> Before configuring this module, please note that some of the default values used by ConfGen to produce the default ECU Configuration can be changed, and this is done by either adding a Conf-Gen Enhancer Mapping (CEM) or by adding a rule to an *algo.properties* file. For more information, see the "Configuration Generation" section of the RTA-CAR User Guide.

3.9 Integration Advice

3.9.1 Enable MemAcc Build/Usage

This step is needed only during the transition from Fls to MemAcc i.e. MemAcc is being configured to replace Fls. In order to do so, set the parameter `rba_MemLibEnableMemAccBuild` in rba_MemLib to "TRUE".

To use MemAcc interfaces instead of Fls interfaces in Fee, Ea, MemIf, rba_MemBckp etc. set the parameter `rba_MemLibEnableMemAccUsage` to "TRUE".

> **Hint** Currently, `rba_MemLibEnableMemAccUsage` should remain "FALSE" as Ea and rba_MemBckp do not support MemAcc use. Fee, however, supports MemAcc use.

MemAcc selection possibility:

...EnableMemAccBuild	...EnableMemAccUsage	Build Impact	Fee/Ea etc. Usage
FALSE	FALSE	Fls build, MemAcc inactive	Fls interfaces used
TRUE	FALSE	MemAcc build (Fls Mode), Fls inactive	Fls interfaces used
TRUE	TRUE	MemAcc build, Fls inactive	MemAcc interfaces used

After switching the configuration, it is necessary to perform a clean build.

If scheduling is not done via MemIf then, in addition, the RTE scheduling configuration needs to be changed.

The following components have to be in the project to realize the Fls migration mode:

- rba_MemLib - Central utilities/configurations of the Mem stack.
- MemAcc
- Fls
- rba_MemAccFlsAdpr - Conversion of Fls configuration to MemAcc configuration.
- Mem_11_Cmn - Common definitions/ParamDefs of the Mem driver. Contains a conversion from the Fls LLD definition to the Mem driver definition.

- Suitable Fls low-level drivers for the project (in a fitting version).

3.9.2 Mem Driver Integration

3.9.2.1 Mem Driver Burst-Setting Extension

The AUTOSAR standard only knows Burst settings based on the alignment of the start address. In many cases, however, it is suitable to have a more fine-grained option to configure the Mem driver features and consider them in MemAcc. The configuration options are as follows:

Read Burst Extension

The read operation involves the memory copy operation from the media or the transfer buffer to the user result buffer. Because of this, the runtime of the `MemAcc_MainFunctionunction` is dependent upon the number of transfers.

The user can define the maximum number, either with the default parameter `MemAccDefaultReadJobSize` (in number of bytes) in the container `MemAccRbGeneral`, or individually for each Address Area via the parameter `MemAccMaxReadNormalMode` (respectively `MemAccMaxReadFastMode`) in the container `MemAccRbMode`.

Typical use cases and configuration selection:

Use Case	Configuration Needs
Internal and external flash	No specific parameter to be set in addition

> **NOTE**
> The maximum configuration of the read chunk size needs to be defined in any case.

Compare Burst Extension

The compare operation involves the memory compare operation of the media or the transfer buffer against the user result buffer. Because of this, the runtime of the `MemAcc_MainFunction` is dependent upon the number of compares.

The user can define the maximum number, either with the default parameter `MemAccDefaultCompareJobSize` (in number of bytes), or individually for each Address Area via the parameter `MemAccMaxCompareNormalMode` (respectively `MemAccMaxCompareFastMode`).

In most cases, the compare Burst can be set to the same value as the read Burst. However, if the compare functionality is realized in MemAcc, this size also defines the buffer size (RAM) in MemAcc.

Blank-Check Burst Extension

The Blank-Check Burst size calculation can either be based upon the one of the compare operations (*BURST_COMPARE*), or limited in size and without any alignment limitation (*BURST_NO_ALIGN*). The selection is made via the parameter `MemRbBurstModeBlank` in the container `MemBurstSettings`.

In *BURST_COMPARE* mode, the Burst size is the same as for the compare operation (see above).

In *BURST_NO_ALIGN* mode, the Burst size is limited to the parameter `MemRbBlankBurstSize` of the container `MemBurstSettings`. This mode is useful if the Blank-Check operation in the controller is either very slow or very fast, and does not involve memory reads. In the generated C file, the settings of the Burst size are available in the element `nrBlankLim_u32` of `MemAccConf_BurstDescr_cast[]`.

Erase Burst Extension

For Erase Bursts, either an unaligned Burst (*BURST_NO_ALIGN*), or an erase start address-aligned Burst (*BURST_START*-Modes), or an erase end address-aligned Burst (*BURST_END*-Modes) is configurable. The erase Burst mode is selectable via the parameter `MemRbBurstModeErase` in the container `MemBurstSettings`.

The Burst size is given by the parameter `MemEraseBurstSize` in the container `MemBurstSettings` and must be a multiple of power of 2 (e.g. 4kB, 8kB, 16kB,...256kB,...) if an address alignment mode is selected.

In the generated C file, the Burst mode settings are available in the element `stBurstMode_au8` of `MemAccConf_BurstDescr_cast[]` and the size (if given) in the element `nrBytes_au32` of `MemAccConf_BurstDescr_cast[]`.

Erase Burst modes:

MemRbBurstModeErase	Description
BURST_NONE	No erase Burst possible.
BURST_NO_ALIGN	Burst is not limited to a sector start address alignment, but multiple sectors can be erased from any start address (maximum size defined by `MemEraseBurstSize`).
BURST_START	`MemEraseBurstSize` defines the alignment which MemAcc is trying to achieve by applying single-sector erases. If alignment is achieved, then the Burst is applied (standard AUTOSAR Mode).
BURST_START_EXT1	`MemEraseBurstSize` is defining the maximum start alignment in a similar way to *BURST_START*. However, smaller alignments can also be selected by applying a selection mask. This is determined during initialization by the Mem driver and is stored in the RAM element `stBurstInfo_u8` of `MemHwData DeviceInfo_st`. This is bitcoded in Bit 0 (32kB erase supported); Bit 1 (64kB erase supported); Bit 2 (128kB erase supported); Bit 3 (256kB erase supported).
BURST_END	`MemEraseBurstSize` defines the alignment which ends the Burst. Multiple sectors can be erased until this threshold, but the alignment threshold is not crossed.

Write Burst Extension

For write Bursts, either an unaligned Burst (*BURST_NO_ALIGN*), or a write start address-aligned Burst (*BURST_START Modes*), or a write end address-aligned Burst (*BURST_END Modes*) is configurable. The write Burst mode is selectable via

the parameter `MemRbBurstModeWrite` in the container `MemBurstSettings`.

The Burst size is given by the parameter `MemWriteBurstSize` in the container `MemBurstSettings` and must be a multiple of power of 2 (e.g. 8B, 16B, 32B, 64B, 128B, 256B ...) if an address alignment mode is selected.

In the generated C file, the Burst mode settings are available in the element `stBurstMode_au8` of `MemAccConf_BurstDescr_cast[]`, and the size (if given) in the element `nrBytes_au32` of `MemAccConf_BurstDescr_cast[]`.

Write Burst modes:

MemRbBurstModeWrite	Description
BURST_NONE	No write Burst possible.
BURST_NO_ALIGN	Burst is not limited to a start address alignment, but multiple pages can be written from any start address (maximum size defined by `MemWriteBurstSize`).
BURST_START	`MemWriteBurstSize` defines the alignment which MemAcc tries to achieve by applying single page writes. If the alignment is achieved, then the Burst is applied (standard AUTOSAR Mode).
BURST_START_FAST	Similar to *BURST_START* but the Burst mode is only applied if the *FAST ChunkMode* is active. Otherwise, on page writes are applied.
BURST_START_BIN	`MemWriteBurstSize` defines the maximum possible alignment. Smaller Burst alignments can also be applied. These smaller alignments are defined as 256B, 128B, 64B, 32B, 16B, 8B. Of course, the lower limit (which is configured write page size) is considered.
BURST_START_SEL1	`MemWriteBurstSize` defines the maximum possible alignment. Smaller Burst alignments can also be applied. These smaller alignments are defined as - 128B, 32B. Of course, the lower limit (which is configured write page size) is considered.
BURST_END	`MemWriteBurstSize` defines the alignment which ends the Burst. Multiple pages can be written until this threshold, but the alignment threshold is not crossed.
BURST_END_FLEX	Similar to *BURST_END*, but the configured value `MemWriteBurstSize` is internally doubled if Bit7 of the device RAM element `stBurstInfo_u8` is set (part of `MemHwData DeviceInfo_st`, which can be set during `MemAcc_Init` by the Mem driver).

3.9.3 Enable Measurement Definition Generation

MemAcc can generate the measurement type and variable description for the RTE if the feature is enabled via rba_MemLib. The RTE then generates a raw definition of all the measurement variables in *Rte_McSupport.xml*, which is to be post-processed by follow-up tools like *ice_ara2l*. At the end of the processing change, an A2L file is generated. As this is a general, component-independent

procedure, it is not described here.

Component dependent, however, is that the RTE has to be informed that measurement information should be collected.

Within */AUTOSAR_Rte/EcucModuleDefs/Rte*, it is necessary to add following container:

```
<ECUC-CONTAINER-VALUE><SHORT-NAME>MemAcc</SHORT-NAME><DEFINITION-REF
DEST="ECUC-PARAM-CONF-CONTAINER-DEF">/AUTOSAR_Rte/EcucModuleDefs/Rte/RteBswModuleInstance</DEFINITION-REF><REFERENCE-VALUES><ECUC
DEST="ECUC-FOREIGN-REFERENCE-DEF">/AUTOSAR_Rte/EcucModuleDefs/Rte/RteBswModuleInstance/RteBswImplementationRef</DEFINITION-REF><V
DEST="BSW-IMPLEMENTATION">/AUTOSAR_MemAcc/BswImplementations/MemAcc</VALUE-REF></ECUC-REFERENCE-VALUE></REFERENCE-VALUES></ECUC-C
```

3.9.4 Scheduling Aspects

MemAcc System Integration/Device Instance Groups shall be scheduled close to their users i.e. in the same cores and, ideally, in the same task. For more details, please consider:

/AUTOSAR_MemAcc/BswModuleDescriptions/MemAcc/BswDocInterfaces/BswDocInterfaceSched

3.9.5 OS Resources

The number of the active OS resources, and their usage, depends on the user configuration of the AUTOSAR System and Basic Software. It is the user's responsibility to ensure that the `ENTER` and `EXIT` resource API are implemented to disable/enable interrupts as required. Alternatively they must be configured in the OS with the correct task/ISR allocation.

The following is a list of all exclusive areas that this module might use regardless of the user configuration. It is the job of the integrator to ensure that the implementation of these exclusive areas is correctly defined. This implementation can be done using the RTE exclusive area configuration wizard or by manual implementation of the corresponding BSW API calls to ensure exclusivity.

The MemAcc module uses the following OS resources:

- MemAcc_Exclusive
- MemAcc_MainFunction

3.10 Hardware Requirements

Information relating to the hardware resources required by this module is not currently available.

4 Memory Abstraction Interface

Module name	MemIf
Package	RTA-MEM
AUTOSAR Module ID	22
AUTOSAR Version	23-11

4.1 Introduction

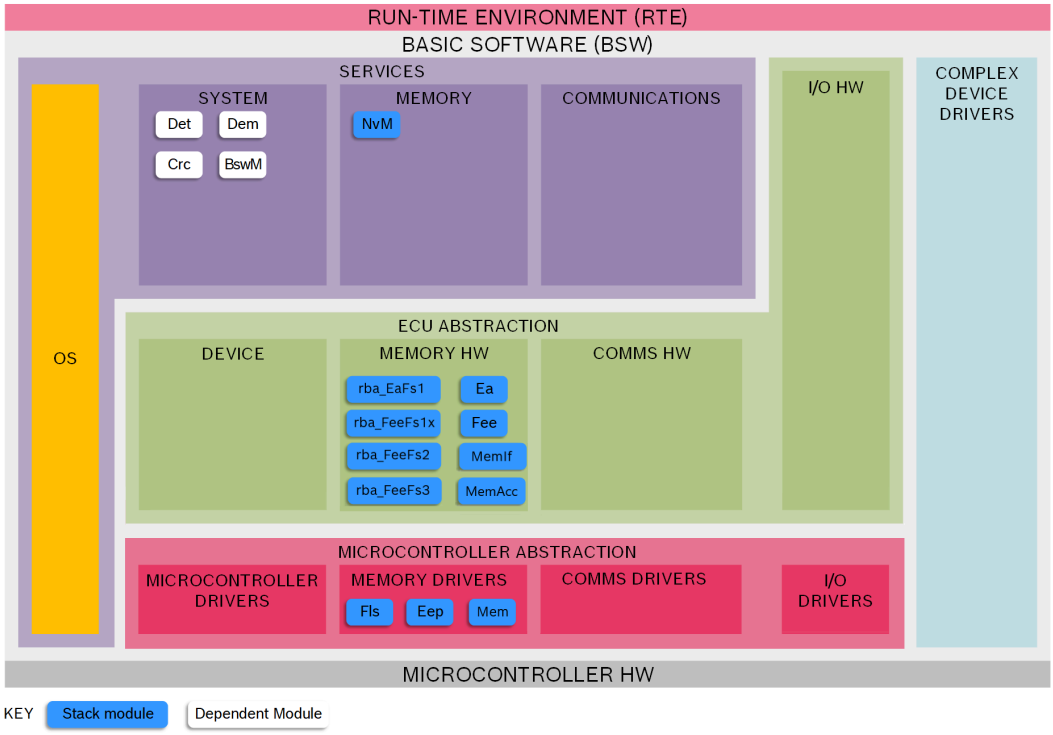


Figure 4.1.Memory Stack

The Memory Abstraction Interface module (MemIf) is a basic software module in the Memory Hardware Abstraction layer, and it forms part of the RTA-MEM stack.

MemIf provides a common interface for NvM to access the underlying memory hardware (e.g. external EEPROM, internal EEPROM or flash emulation), and it has interactions with the EEPROM Abstraction (Ea) and Flash EEPROM Emulation (Fee) memory abstraction modules. The EEPROM Driver (Eep) is only scheduled from MemIf if it is configured.

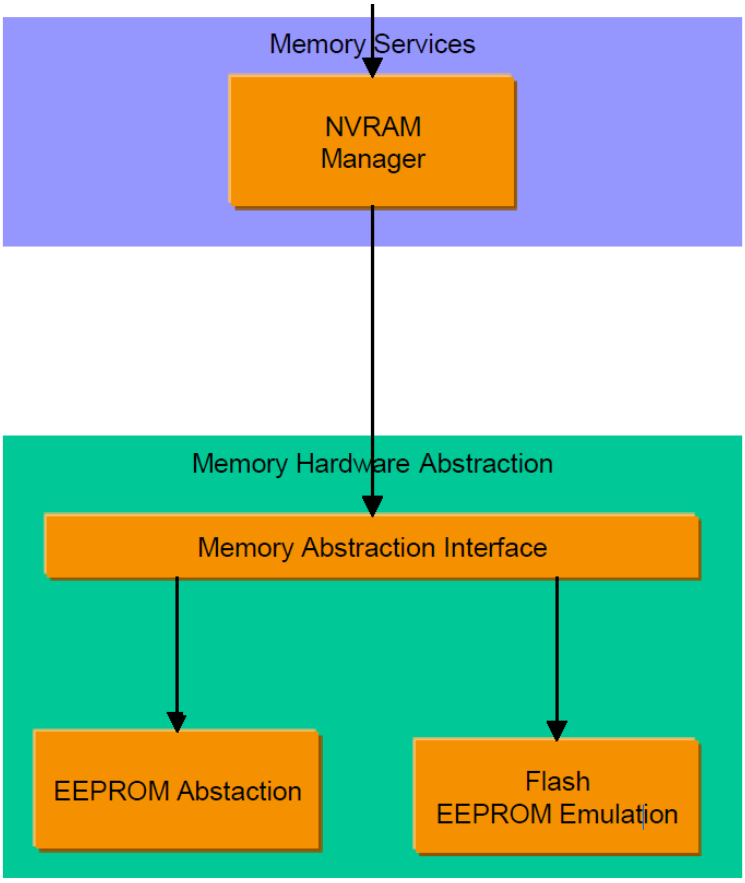


Figure 4.2.Memory Hardware Abstraction

MemIf allows NvM to act independently from the underlying memory hardware. A device index is configured for each NvM block, and with this index, MemIf determines which underlying module is affected (Fee or EA).

- If the device index is set to zero in NvM's block configuration, calls to MemIf are forwarded to Fee.
- If the device index is set to greater than zero in NvM's block configuration, calls to MemIf are forwarded to Ea.

MemIf abstracts from the number of underlying Ea and Fee memory modules by presenting the upper layer NvM with a contiguous address space, which is divided into virtual segments according to the underlying storage hardware.

Module Configuration

This module can be generated by the RTA-BSW Configuration Generator.

For any unsupported automatic configuration, the module can be manually configured by the user.

4.2 Supported Features

The MemIf module itself has no functionality, it provides only the functionality of

the underlying modules (Fee, EA) as an API.

4.3 Extensions

4.3.1 Configuration Extensions

The following configuration parameters are available in version v12.7.0 in addition to those defined in AR v22-11

4.3.1.1 MemIf/MemIfGeneral/MemIfMainFunctionCycleTime

Short Name	MemIfMainFunctionCycleTime
Description	Defines the cycle time in seconds of the periodic calling of MemIf main function. Parameter is just meaningful if the parameter MemIfRbMainFunction-Scheduling which defines the type of scheduling is set to MEMIF_RB_MAINFUNC_SCHEDULING_RTE_TIMED.
Multiplicity	0..1
Value Range	0..1
Default Value	-
Type	ECUC-FLOAT-PARAM-DEF

4.3.1.2 MemIf/MemIfGeneral/MemIfRbDisableForwardingScheduleInfo

Short Name	MemIfRbDisableForwardingScheduleInfo
Long Name	MemIf Disable forwarding of scheduling Info
Description	Defines if the scheduling info shall be prepared from NvM or not. If configured as false or unconfgured, the information from NvM is used to prepare MemIf scheduling info. If configured as true, MemIf will not read info from NvM, user has to manually configure scheduling info for MemIf, Fee/Ea and Fls/Eep.
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

4.3.1.3 MemIf/MemIfGeneral/MemIfRbFeeUsed

Short Name	MemIfRbFeeUsed
Description	Status if Fee is configured
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

4.3.1.4 MemIf/MemIfGeneral/MemIfRbMainFunctionScheduling

Short Name	MemIfRbMainFunctionScheduling
Long Name	Scheduling of MemIf_Rb_MainFunction
Description	Defines if MemIf_Rb_MainFunction() is being called in the background-task via RTE, in a scheduled task via RTE or if it is not called via RTE and therefore has to be scheduled manually. In case it is scheduled manually no RTE events or entities are generated. For the case that the parameter is set to MEMIF_RB_MAINFUNC_SCHEDULING_RTE_TIMED the corresponding period/cycle time of the timed task is defined via the parameter MemIfMainFunctionCycleTime. ..... Instead of MEMIF_RB_MAINFUNC... also MEMIF_RB_MAINFUNC_AND_DEP... can be selected to call MemIf_Rb_MainFunctionAndDependency() instead. (Default setting is taking over the NvM setting or if no NvM is available use MEMIF_RB_MAINFUNC_SCHEDULING_MANUAL). Te forwarding can be disabled by MemIfRbDisableForwardingScheduleInfo
Multiplicity	0..1
Default Value	-
Enumeration Literals	MEMIF_RB_MAINFUNC_SCHEDULING_MANUAL, MEMIF_RB_MAINFUNC_SCHEDULING_RTE_BACKGROUND, MEMIF_RB_MAINFUNC_SCHEDULING_RTE_TIMED, MEMIF_RB_MAINFUNC_AND_DEP_SCHEDULING_RTE_BACKGROUND, MEMIF_RB_MAINFUNC_AND_DEP_SCHEDULING_RTE_TIMED
Type	ECUC-ENUMERATION-PARAM-DEF

4.3.1.5 MemIf/MemIfGeneral/MemIfRbNumberOfEADevices

Short Name	MemIfRbNumberOfEADevices
Description	Number of configured EA/Eep Devices
Multiplicity	0..1
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

4.3.2 Additional Services

The following API are available in version v12.7.0 in addition to those defined in AR v22-11

4.3.2.1 MemIf_CheckDeviceIndex

Syntax	`MemIf_CheckDeviceIndex(uint8 DeviceIndex)`
Parameter	`DeviceIndex` Index of requested device
Description	This function checks the device index.

4.3.2.2 MemIf_Rb_GetStatus

Syntax	`MemIf_StatusType MemIf_Rb_GetStatus(void)`
Parameters	
Description	The function returns the status of underlying FEE and of EA module, depending if Fee and EA are configured.

4.3.2.3 MemIf_Rb_MainFunction

Syntax	`void MemIf_Rb_MainFunction(void)`
Parameters	
Description	The function calls the required underlying main functions, depending on which modules are configured.

4.3.2.4 MemIf_Rb_BlockMaintenance

Syntax	`Std_ReturnType MemIf_Rb_BlockMaintenance(uint8 DeviceIndex, uint16 BlockNumber)`
Parameters	`DeviceIndex` Index of requested device `BlockNumber` Nvm index
Description	The function calls the required underlying main functions, depending on which modules are configured.

4.3.2.5 MemIf_Rb_GetMigrationResult

Syntax	`MemIf_Rb_MigrationResult_ten MemIf_Rb_GetMigrationResult(uint8 DeviceIndex, uint16 BlockNumber)`
Parameters	`DeviceIndex` Index of requested device `BlockNumber` Nvm index
Description	The function calls the required underlying main functions, depending on which modules are configured.

4.3.2.6 MemIf_Rb_GetDetailedBlkInfo

Syntax	`MemIf_Rb_DetailedBlkInfo_ten MemIf_Rb_GetDetailedBlkInfo(uint8 DeviceIndex, uint16 BlockNumber)`
Parameter	`DeviceIndex` Index of requested device. `BlockNumber` Corresponding Fee or Ea block number.
Returns	MEMIF_RB_DETAILED_BLK_INFO_NOT_AVAILABLE_E - Job not supported by Fee/Ea. MEMIF_RB_DETAILED_BLK_INFO_INIT_E - Block was never read. MEMIF_RB_DETAILED_BLK_INFO_INCONSISTENT_MISSING_E - Valid block (header) was not found on the medium. MEMIF_RB_DETAILED_BLK_INFO_INCONSISTENT_DATA_E - Data CRC of block is wrong. MEMIF_RB_DETAILED_BLK_INFO_INCONSISTENT_LENGTH_E - Block length is not consistent with config.
Description	Returns detailed block info regarding valid block header, data CRC and block length. Calls the underlaying module API's depending on configured device.

4.3.2.7 MemIf_Rb_MainFunctionAndDependency

Syntax	`void MemIf_Rb_MainFunctionAndDependency(void)`
Description	Calls underlaying Mainfunctions and dependencies of all instances.

4.3.2.8 MemIf_Rb_EnterHighSpeedMode

Syntax	`Std_ReturnType  MemIf_Rb_EnterHighSpeedMode(uint8 DeviceIndex)`
Parameter	`DeviceIndex` Index of requested device.
Returns	`E_OK` Order was successfully placed. `E_NOT_OK` Order was rejected. `E_MEMIF_SERVICE_NOT_AVAIL` Feature not supported for the passed device.
Description	Calls the corresponding low level enter high speed mode function if available.

4.3.2.9 MemIf_Rb_ExitHighSpeedMode

Syntax	`Std_ReturnType MemIf_Rb_ExitHighSpeedMode(uint8 DeviceIndex)`
Parameter	`DeviceIndex` Index of requested device.
Returns	`E_OK` Order was successfully placed. `E_NOT_OK` Order was rejected. `E_MEMIF_SERVICE_NOT_AVAIL` Feature not supported for the passed device
Description	Calls the corresponding low level enter high speed mode function if available.

4.4 Limitations

There are currently no known limitations in this version of the MemIf module.

4.5 Exclusions

The following features from AR v22-11 are not supported in version v12.7.0.

API functions not currently implemented:

- `MemIf_SetMode`

The following configuration parameters from AR v22-11 are not supported in version v12.7.0.

- There are no known exclusions in this version.

4.6 Deviations

4.6.1 Configuration Deviations

Configuration parameters which differ from the AR v22-11 specification are listed below.

4.6.1.1 MemIf

[RTA-BSW] Description	Configuration of the MemIf component
[AUTOSAR] Description	Configuration of the MemIf (Memory Abstraction Interface) module.

4.6.1.2 MemIf/MemIfGeneral

[RTA-BSW] Description	Configuration of the memory abstraction interface (Memif) module
[AUTOSAR] Description	Configuration of the memory abstraction interface (Memif) module.

4.6.1.3 MemIf/MemIfGeneral/MemIfDevErrorDetect

[RTA-BSW] Description	Switch for DevErrorDetect
[AUTOSAR] Description	Switches the development error detection and notification on or off.
[REMOVED] Introduction	* true: detection and notification is enabled. * false: detection and notification is disabled.
[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1
[REMOVED] Default Value	false

4.6.1.4 MemIf/MemIfGeneral/MemIfNumberOfDevices

[RTA-BSW] Description	Number of devices for MemIf
[AUTOSAR] Description	Concrete number of underlying memory abstraction modules.
[REMOVED] Introduction	Calculation Formula: Count number of configured EA and FEE modules.
[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1
[REMOVED] Value Range	1..2

4.6.1.5 MemIf/MemIfGeneral/MemIfVersionInfoApi

[RTA-BSW] Description	Switch for VersionInfoApi
[AUTOSAR] Description	Pre-processor switch to enable / disable the API to read out the modules version information.
[REMOVED] Introduction	true: Version info API enabled. false: Version info API disabled.
[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1
[REMOVED] Default Value	false

4.7 API Definition

`MemIf_GetVersionInfo`	Return version information.
`MemIf_Read`	This function places a read job to the FEE or EA.
`MemIf_Write`	This function places a write job to the FEE or EA.
`MemIf_InvalidateBlock`	This function places an invalidate block job to the FEE or EA.
`MemIf_GetStatus`	This function returns the status of a a device.
`MemIf_GetJobResult`	This function returns the status of a a job.
`MemIf_Cancel`	This function cancels an order.
`MemIf_EraseImmediateBlock`	This function places an erase block job to the FEE or EA.

For further information please refer to AUTOSAR_SWS_MemoryAbstractionInterface.pdf

4.8 Configuration Advice

> **NOTE**
>
> Before configuring this module, please note that some of the default values used by ConfGen to produce the default ECU Configuration can be changed, and this is done by either adding a Conf-Gen Enhancer Mapping (CEM) or by adding a rule to an *algo.properties* file. For more information, see the "Configuration Generation" section of the RTA-CAR User Guide.

4.8.1 General

Only the MemIfNumberOfDevices configuration parameter is mandatory - all other parameters (including all extension parameters) are optional.

4.8.2 MemIfRbMainFunctionScheduling

In addition to the existing options to schedule MemIf in a background task or in a timed task (via the MemIfMainFunctionCycleTime parameter), it is also now possible to have MemIf called manually, and this is achieved via the MemIfRbMainFunctionScheduling extension parameter.

4.8.3 MemIfRbDisableForwardingScheduleInfo

By default, MemIf will read the NvM configuration to prepare the values for MemIfMainFunctionCycleTime and MemIfRbMainFunctionScheduling. This operation is controlled via the MemIfRbDisableForwardingScheduleInfo extension parameter. If it is configured as FALSE (or left unconfigured), MemIf will forward MANUAL scheduling to the default instance of lower layers Fee, Fls and Ea). Also, if MemIfRbDisableForwardingScheduleInfo is disabled, MemIf will read the configuration on scheduling aspect from NvM.

(Note: This feature is only relevant if your project does not use ConfGen. If you follow the standard workflow with ConfGen, the setting of MemIfRbDisableForwardingScheduleInfo has no effect).

4.9 Integration Advice

4.9.1 Init Functions

No specific handling necessary.

4.9.2 DeInit Functions

No specific handling is necessary, but it is important that there is smooth transition while going to shutdown (i.e. from cyclic execution to linear execution). MemIf_Rb_MainFunction should not be pre-empted allowing another RTA-MEM stack operation to be requested.

4.9.3 Scheduled Functions

`MemIf_Rb_MainFunction` has to be called periodically, and it should be scheduled in the same task or container as the *NvM Main Function*.

MemIf inherits scheduling information from NvM (configured in `NvMMainFunctionPeriod`) and it generates (into *MemIf_Cfg_BSWMD.arxml*) that same scheduling information for use by the RTE. This ensures that the NvM and MemIf main functions are scheduled within the same task (either background or time triggered), or both have to be called manually without RTE, in which case no further configuration is necessary.

Since all the primary activities of lower layers (e.g. Fee and Ea) are carried out as part of `MemIf_Rb_MainFunction`, RTA-MEM stack performance will directly correlate to its invocation frequency.

If different instances are needed, then also call `MemIf_Rb_MainFunctionAndDependency`. Alternatively, schedule the lower level components directly and do not use `MemIf_Rb_MainFunction`.

Mem Stack scheduling can be determined flexibly. By default, it is handled wholly by MemIf and NvM and only has to be set for NvM. NvM scheduling parameters will be automatically forwarded to MemIf during code generation. To disable this, set `MemIfRbDisableForwardingScheduleInfo` to TRUE. If this parameter is set, then scheduling for each component, except MemIf, must be set in the `<component>RbMainFunctionScheduling` parameters.

For most use cases, however, it is recommended to retain the default scheduling i.e. only set the NvM scheduling parameters.

Forwarding from MemIf down to the underlying module is only performed to the existing and "used" modules. "Used" means that there must be at least one NvM block assigned to the module. This is also the condition when a module's MainFunction is called from MemIf.

e.g. If you do not forward any NvM block to an Ea component, then `Ea_MainFunction` will not be called from MemIf and thus its scheduling requires configuration outside of MemIf.

Without Os scheduling, it is highly recommended to call `MemIf_Rb_MainFunction` within `NvM_Rb_MainFunctionAndDependency`. Then, no more integration steps are needed. Do not call `MemIf_Rb_MainFunction` outside the Os scheduling context.

RTE Usage/Configuration

The scheduling information used within RTE has to be configured with the MemIfMainFunctionCycleTime and MemIfRbMainFunctionScheduling parameters. MemIf uses this information and generates the scheduling information for MemIf usage by the RTE into MemIf_Cfg_BSWMD.arxml. As previously stated, the NvM and MemIf main functions must be scheduled within the same task (either background or time triggered) or both have to be called manually without RTE.

4.9.3.1 Preventing reentrant execution of MemIf_Rb_MainFunction

MemIf_Rb_MainFunction is non-reentrant, so the MEMIF_RB_USE_PROTECTION

switch (located in the file MemIf_Cfg_SchM.h) can be used to prevent its multiple execution.

- MEMIF_RB_USE_PROTECTION = TRUE: Protection against multiple execution enabled (Default).
- MEMIF_RB_USE_PROTECTION = FALSE: Protection against multiple execution is disabled. This is useful in single-core/task configurations. The detection of re-entrant invocation will still be performed, but the reentrant execution will not be prevented.

The following consequences may result if the protection is enabled in a project where reentrant invocation of the MemIf_Rb_MainFunction is not handled completely,

- Additional consumption of runtime.
- Priority reversal (i.e. if an interrupted event waits for completion of a certain operation upon calling MemIf_Rb_MainFunction, and the interrupt occurred in the middle of MemIf_Rb_Mainfunction execution).
- Jumping out in the middle of MemIf_Rb_MainFunction to shutdown is forbidden (i.e. if MemIf_Rb_MainFunction is expected to perform some activity in the shutdown sequence, it will not work because the interrupted function will hold the lock indefinitely).

4.9.4 Only one Memory Device

If there is only one memory device being used (Ea or Fee), and because MemIf is only a routing layer, all of the MemIf functions can be generated as macros.

4.9.5 OS Resources

The number of the active OS resources, and their usage, depends on the user configuration of the AUTOSAR System and Basic Software. It is the user's responsibility to ensure that the `ENTER` and `EXIT` resource API are implemented to disable/enable interrupts as required. Alternatively they must be configured in the OS with the correct task/ISR allocation.

The following is a list of all exclusive areas that this module might use regardless of the user configuration. It is the job of the integrator to ensure that the implementation of these exclusive areas is correctly defined. This implementation can be done using the RTE exclusive area configuration wizard or by manual implementation of the corresponding BSW API calls to ensure exclusivity.

The MemIf module uses the following OS resources:

- MemIf_Main

4.10 Hardware Requirements

Table 4.22.MemIf Hardware Requirements

Support	Usage
FLASH	88 bytes
RAM	2 bytes

Usage calculated based on a sample configuration.

5 NVRAM Manager

Module name	NvM
Package	RTA-MEM
AUTOSAR Module ID	20
AUTOSAR Version	23-11

5.1 Introduction

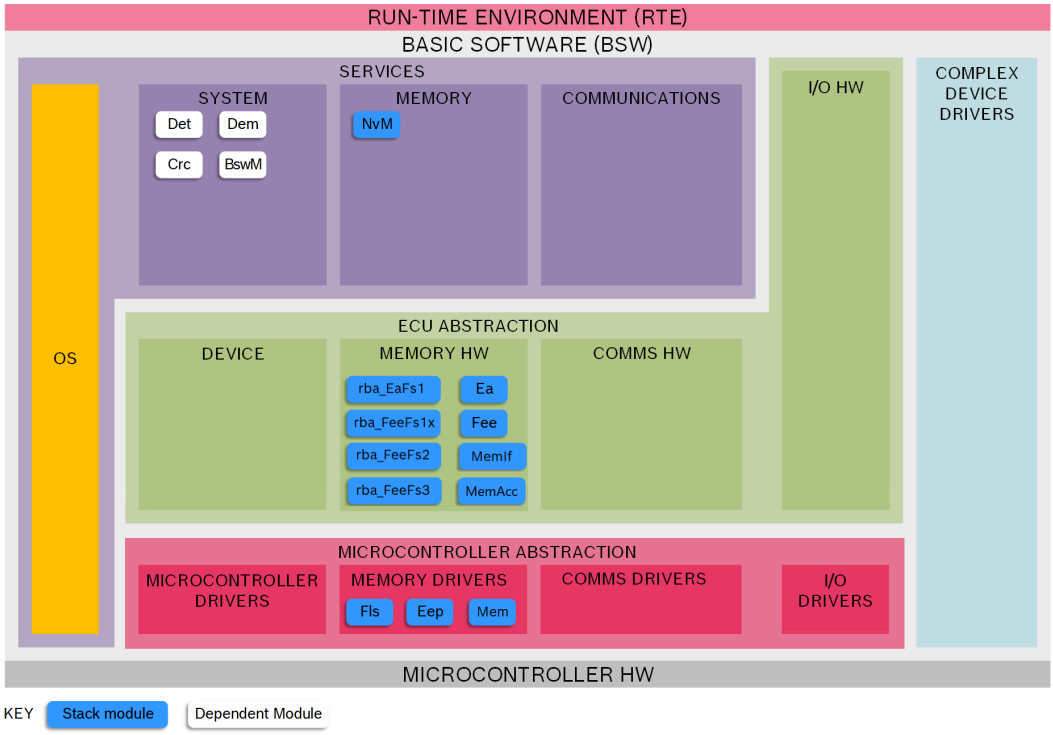


Figure 5.1.Memory Stack

The NVRAM Manager (NvM) is a basic software module in the Memory Services abstraction layer, and it forms part of the RTA-MEM stack.

NvM organizes an ECU's persistent data storage facility in separately manageable units called "NvM blocks". It administers these blocks independently from their actual storage location (e.g. serial EEPROM, or EEPROM emulation in flash memory devices) and offers services to read, write, restore, invalidate and erase these blocks.

All calls related to the actual memory media are made by NvM through MemIf, which in turn dispatches these calls to either Ea or Fee. MemIf allows NvM to act independently from the underlying memory hardware.

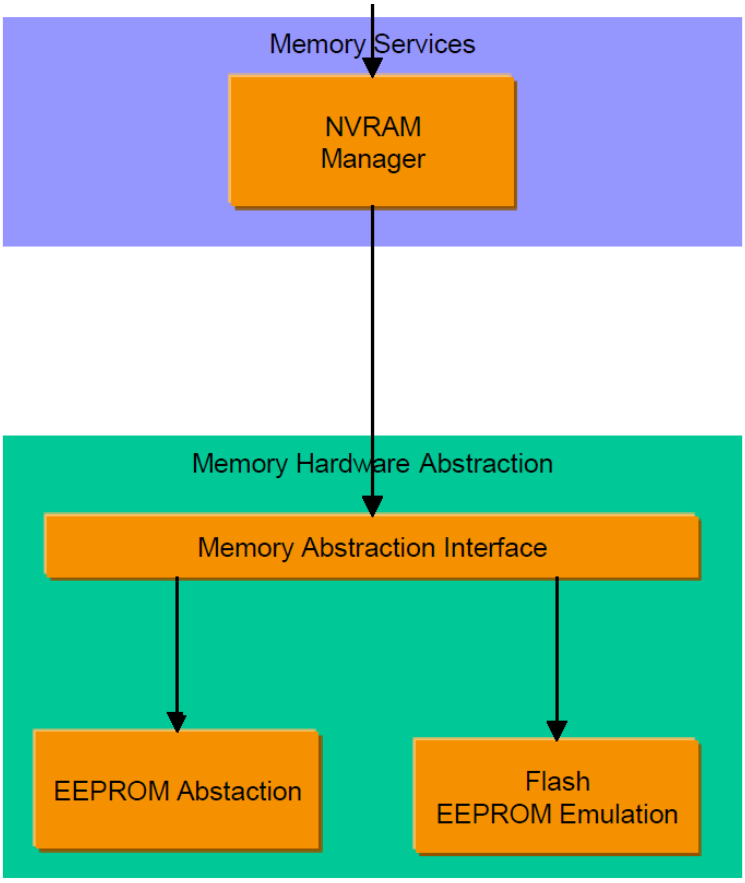


Figure 5.2.Memory Hardware Abstraction

NvM provides the following services related to NvM blocks:

- Read a block from persistent storage to a permanent or temporary RAM block.
- Write a block from a permanent or temporary RAM block to persistent storage.
- Perform a multi-block read to read multiple blocks from the permanent storage.
- Perform a multi-block write to write multiple blocks to the permanent storage.
- Restore a permanent or temporary RAM block with default data.
- Invalidate a block in persistent storage (i.e. intentionally mark it as not being readable).
- Erase a block in persistent storage (i.e. completely remove it from the persistent storage media).

A permanent RAM block is a block in the volatile memory that, if changed, has the changes mirrored back to the permanent memory. A temporary RAM block is one

that does not mirror changes back to the permanent memory.

All of the above jobs are triggered by invoking an NvM service, and they are carried out asynchronously - the NvM service returns immediately after queueing the job request. In general, only one job can be requested at a time for a given block.

Module Configuration

This module can be generated by the RTA-BSW Configuration Generator.

For any unsupported automatic configuration, the module can be manually configured by the user.

5.2 Supported Features

5.2.1 NvM Queues

The NvM_MainFunction is sub-structured in four parts/stages:

- The "arbitration" part determines the next NvM job to be executed if no job processing is currently active. This is done by checking all queues for pending jobs to be processed. The prioritization of jobs is carried out in this part of NvM's main function.
- Once the arbitration and prioritization part has determined which job is to be processed next, control is handed over to the "job start" part. Here, either the corresponding MemIf job is started (for read, write, invalidate and erase jobs) or if it is a restore job which is based on ROM block contents, this is carried out by NvM itself. If the NvM block is configured with single-block start callbacks and/or Explicit Synchronization and a write service is requested, then before starting the MemIf job's Explicit Synchronization, these callbacks are called (i.e. the single-block start callback is called before the Explicit Synchronization).
- If a MemIf job has been successfully started by the previous step, NvM is put in a state which polls for the job result.
- Once the MemIf job (or an NvM restore job) is finished, its result is evaluated and stored internally. Potential response actions might then be triggered, such as Implicit Recovery, Explicit Synchronization read callback and single-block callbacks. If Implicit Recovery is needed, the NvM activity changes back to the "job start" part in order to restore default data from ROM. Otherwise, control is handed back over to the "arbitration" part.

5.2.2 Single-block Operations

Each of the jobs mentioned in the Introduction section may be triggered at any time for single NvM blocks. As shown in the table below, BSW users of NvM and complex drivers will use NvM C service APIs for this, while ASW users will access the NvM via RTE using the indicated client/server operations:

Action	NvM C service API	RTE ASW API
Read NV → RAM block	NvM_ReadBlock	ReadBlock

Write RAM → NV block	NvM_WriteBlock	WriteBlock
Restore RAM block	NvM_RestoreBlockDefaults	RestoreBlockDefaults
Invalidate NV block	NvM_InvalidateNvBlock	InvalidateNvBlock
Erase NV block	NvM_EraseNvBlock	EraseBlock
Set NV Block Status	NvM_SetRamBlockStatus	SetRamBlockStatus
Set NV block Protection	NvM_SetRamBlockProtection	SetRamBlockProtection

Before invoking a single-block service for blocks of type NVM_BLOCK_DATASET, the NvM user has to select the dataset on which the operation has to be performed (i.e. an NV block for read, write, invalidate and erase operations, or a ROM block for read and restore operations). This is done through the NvM_SetDataIndex service. Both the dataset, and the data index of the block to be operated upon, must be specified.

Operations that modify the NV block can only be carried out for a limited number of times during the life time of an ECU. This is due to hardware limitations and cannot be entirely concealed by software. The absolute maximum number of block changes depends on the storage medium being used, and on the block layout properties.

In contrast to multi-block write operations (see below), single-block write operations are always carried out unconditionally. This is the case regardless of whether the particular block has been marked as changed or not, and independently from its validity status.

If a block is configured with block length null, the requests of single-block operations are blocked (i.e. they are neither queued nor operated).

If NvMRbNoSingleBlockBeforeReadAll is set to TRUE, any single block access will not be processed before the completion of a “ReadAll” service, and will instead be rejected with E_NOT_OK.

5.2.3 Multi-block Operations

Blocks which have a permanent RAM block assigned in the ECU configuration (via the NvMRamBlockDataAddress parameter), or which are configured with the Explicit Synchronization feature, may not only be operated with the single-block NvM services described above, but may also be part of bulk operations covering several blocks at once. NvM offers “multi-block” read, write, and first initialization services.

NvM also offers a special feature to automatically remove particular blocks from the media after a multi-block read operation has finished and a block layout change has been detected.

A block can only participate in multi-block read (NvM_ReadAll), write (NvM_WriteAll), or first initialization (NvM_Rb_FirstInitAll) operations if this has been explicitly enabled in ECU configuration via the block-specific parameters NvMSelectBlockForReadAll, NvMSelectBlockForWriteAll, and NvMRbSelectBlockForFirstInitAll.

Action	NvM C service API	Configuration Parameter
Read NV → RAM blocks	NvM_ReadAll	NvMSelectBlockForReadAll
Write RAM → NV blocks	NvM_WriteAll	NvMSelectBlockForWriteAll
Init NV blocks	NvM_Rb_FirstInitAll	NvMRbSelectBlockForFirstInitAll

Multi-block read and write services may only be started by system control, not by any other ASW or BSW component.

A multi-block operation will never interrupt an ongoing single-block operation, and vice versa. Multi-block operations will also never be interrupted by other multi-block operations.

All single-block callbacks have an argument called OperationType of type NvM_Rb_OperationType. This argument can either have value NVM_RB_OPERATIONTYPE_MULTI (if the ongoing action is part of a multi-block operation), or NVM_RB_OPERATIONTYPE_SINGLE (if the ongoing action is a single-block operation).

5.2.4 Maintenance of redundant blocks

This is an internal maintenance operation that ensures greater robustness against data loss when using redundant blocks. It is part of the execution of the WriteAll job, and it is performed for all redundant blocks just after the blocks have been written to the medium.

The maintenance operation involves restoring data from the uncorrupted NV block to a possibly corrupted NV block within a redundant NvM block. Only NvM can trigger this maintenance operation, and the actual restoration of data is performed by the lower layers (Ea or Fee).

- Due to the additional processing time required to perform this maintenance operation, the WriteAll job's execution time is extended.
- If the block layout contains redundant blocks, this maintenance operation is still performed even if no blocks have been previously marked to be written by the WriteAll job.
- If the NvM_Rb_GetActiveService API is called during the execution of this operation, NVM_SERVICE_ID_WRITE_ALL will be indicated as the currently active service.
- The WriteAll job finishes when the writing of the suitable blocks, and the maintenance operation on the redundant blocks, is finished. After that, a configured multi-block callback is called to provide the WriteAll operation result (not the result of the maintenance operation).
- Due to its internal nature, the maintenance operation has no effect on the request results of the maintained blocks or on their RAM states.
- A single-block service request can be called/queued for a redundant block whose only pending operation is a maintenance operation. (See Limitations). If the queued request is an immediate request, its job is executed once the processing of the currently active block in the

multi-block operation finishes. If it is a standard priority request, it is processed by the NvM main function once the WriteAll job finishes.

- A configured single-block callback for a redundant block, which is invoked during the execution of the WriteAll job, indicates only the end of the execution of the write job for this block. This callback delivers the result of the write job.

5.2.5 Immediate requests

Immediate requests are single-block write requests for NvM blocks that have been configured with a priority of "immediate". See the Configuration Advice section for details of this configuration.

The aim of this feature is to write user data as soon as possible to the storage media. Immediate requests have a higher priority than standard single and multi-block requests. They are queued separately, in an immediate queue, and they are fetched before single-block requests from the standard queue.

Priority-based job processing is supported by the NvM itself. But canceling a pending job immediately in a destructive manner is only possible with support from the lower layer (Fee, Ea). If it does not provide support, the NvM can cancel a pending job only if it is currently not processed by the lower layer i.e. if a job is processed by the lower layer, the NvM will wait and only after lower-layer processing has finished will the NvM cancel it in a destructive manner.

More information on how Fee and Ea support such canceling can be found in their respective reference documents.

If an immediate request is queued while a job is currently being processed by NvM, then processing of the current job is canceled immediately if the lower layer supports job canceling. Otherwise, the NvM waits until the lower layer has finished processing a job before canceling it.

The user has to consider this canceling delay if using a lower layer without canceling support.

By setting the parameter `NvMBlockJobPriority`, the User can define the execution priority of user requests for individual blocks. Blocks with lower priority will be executed first.

For blocks configured with immediate priority (`NvMBlockJobPriority` = 0) the User has the possibility to define the sub-priority level using the parameter `NvMRbBlockJobImmSubPriority`. NvM executes user requests for blocks with a lower sub-priority first.

Non-write requests for blocks that have been configured with a priority "immediate" are handled like standard requests (i.e. they are queued in the standard queue and are processed in the regular first-come-first-served manner, together with other standard requests).

An immediate write request can temporarily suspend the execution of multi-block operations, but cannot suspend the execution of the job currently being processed. If a block is being processed during a multi-block operation, then the processing of this block will continue, and only once it has completed will the

multi-block operation be suspended. The immediate request will then be processed and, when it finishes, the multi-block operation is resumed.

If an `NvM_Rb_FirstInitAll` multi-block operation is active when an immediate write request is queued, and if the NvM block that is subject to the immediate write request is the one which is currently being processed by the `NvM_Rb_FirstInitAll` multi-block operation, then the job of the queued immediate write request will wait until the invalidation of all NV blocks of an NvM block of type *NVM_BLOCK_DATASET* has finished.

This is also the case when an NvM block of type *NVM_BLOCK_DATASET* is currently being processed by a multi-block operation that is removing all non-resistant NvM blocks (enabled via the configuration parameter `NvMRbRemoveNonResistantBlocks`). An immediate write request for a block which is non-resistant to SW change might nevertheless be rejected until the removal of this block is finished.

5.2.6 Implicit Recovery

Implicit Recovery is a mechanism to automatically put default data in the RAM block after an unsuccessful read from the corresponding NV block. Implicit Recovery is activated when the data of the corresponding NV block could not be read due to corruption, or the data not existing on the memory media, or due to a hardware error. Implicit Recovery does not change anything on the persistent memory media.

See the Configuration Advice for details on the sources of the default data that is used by Implicit Recovery.

Implicit Recovery can only occur:

- For NvM blocks having a configured ROM block or a configured initialization callback.
- During single-block or multi-block read requests.

Implicit Recovery does NOT occur if:

- The NV block is invalidated.
- The Explicit Synchronization read callback fails (if the NvM block is configured with Explicit Synchronization).
- The NvM block is of type NVM_BLOCK_DATASET.
- The request result of an NvM block is set to NVM_REQ_RESTORED_FROM_ROM if during a read job an implicit recovery took place. Despite the name of this request result, this is also true if the only data source for implicit recovery of this block is the corresponding initialization callback.

5.2.7 Write Protection

NvM blocks can be write protected to minimize the risk of inadvertent write accesses. The initial write protection is enabled via the NvMBlockWriteProt

configuration parameter, which can then be changed programmatically by the NvM_SetBlockProtection API or the corresponding SetBlockProtection client/server operation.

5.2.7.1 Write Once

For certain cases, e.g. blocks containing unique part numbers, it is possible to configure a block as “write once” rather than write protected. This is enabled via the NvMWriteBlockOnce configuration parameter.

To reliably use this feature, the first operation on the related block after NvM initialization should always be a read operation (preferably a multi-block read operation). Writing is then only possible if this previous read operation failed because no valid data for this block could be found on the memory media. As soon as NvM has successfully read or written a block which has the write once feature activated, the write protection remains in place (and cannot be disabled) until the next full NvM initialization.

5.2.8 NvM Status - query and notification

The current status of NvM can be queried either by polling or by configuring callback notifications, as summarized in the following table:

Notification method	NvM C service API	RTE ASW API
Polling: Get overall NvM status	NvM_Rb_GetStatus	n/a
Polling: Get block-specific status	NvM_GetErrorStatus	GetErrorStatus
Callback: User notification of a finished multi-block operation	(name configured with NvMMultiBlockCallback ECUC parameter)	(name configured with NvMMultiBlockCallback ECUC parameter, must begin with “Rte_Call_”)
Callback: User notification of a finished single-block job	(name configured with NvMSingleBlockCallback ECUC parameter)	(name configured with NvMSingleBlockCallback ECUC parameter, must begin with “Rte_Call_”)
Callback: User notification of a started multi-block job	(name configured with NvMRbMultiBlockStartCallback ECUC parameter)	(name configured with NvMRbMultiBlockStartCallback ECUC parameter, must begin with “Rte_Call_”)
Callback: User notification of a started single-block job	(name configured with NvMRbSingleBlockStartCallback ECUC parameter)	(name configured with NvMRbSingleBlockStartCallback ECUC parameter), name must begin with “Rte_Call_”

Callback: Observer notification of a finished single-block job	(name configured with NvMRbObserverCallback ECUC parameter)	(name configured with NvMRbObserverCallback ECUC parameter, must begin with "Rte_Call_")

The observer callback is configured via the NvMRbObserverCallback configuration parameter. Unlike multi-block callback, the observer callback is invoked for any configured block, including the configuration ID block (when dynamic configuration is enabled) and excluding block #0. Similar to the single-block request, it provides the block specific request result when the processing of the block finishes.

NvM also notifies BswM about the state change of multi-block operations by invoking the BswM service BswM_NvM_CurrentJobMode. This notification, which is optional, is configured through the NvMBswMMultiBlockJobStatusInformation parameter. Unlike multi-block operation callbacks, which are only invoked when the multi-block operation is finished, BswM_NvM_CurrentJobMode is invoked when multi-block operations are started, finished or cancelled.

If the BswM notification related to a multi-block operation state change is enabled, then the configured multi-block operation callback (configurable via NvMMultiBlockCallback), and the configured multi-block start callback (configurable via NvMRbMultiBlockStartCallback), are not invoked.

The status of a multi-block operation can be obtained using the NvM_GetErrorStatus function and passing in a block ID of 0.

5.2.9 Explicit Synchronization

Explicit Synchronization is an alternative to permanent RAM blocks, and it is configured at the NvM block level (see Configuration Advice for more information).

Explicit Synchronization allows NvM to notify when a new block of data is available (during read and restore) or if a block of data is needed by NvM during a write operation. It also introduces an intermediate NvM-internal buffer (called the "NvM mirror") for the data to be copied from/to NvM.

It is possible to configure either Explicit Synchronization or a permanent RAM block for a particular NvM block, but not both. If Explicit Synchronization is enabled, it is performed for all operations which would otherwise involve a permanent RAM block (NvM_ReadAll, NvM_WriteAll, NvM_Rb_FirstInitAll, NvM_ReadBlock, NvM_WriteBlock and NvM_RestoreBlockDefaults).

Explicit Synchronization enhances data consistency due to a double buffering scheme:

- After calling a READ service, NvM requests Fee or Ea to perform the actual read job. Once the lower layer module reports that the read job was successful (or Implicit Recovery took place), the read or restored data is stored in the NvM mirror. NvM then requests the service caller to copy the data from the NvM mirror. This is done by invoking the configured Explicit Synchronization read callback (NvMReadRam-

BlockFromNvCallback).Restore jobs work according to the same principle, using the same callback function.

- After calling a WRITE service, NvM requests the caller to copy the desired data to the NvM mirror by invoking the configured Explicit Synchronization write callback (NvMWriteRamBlockToNvCallback), and then requests Fee or Ea to perform the actual write job, providing the NvM mirror as source for the data to be written.
- The copy operation described above, from RAM to NvM mirror or vice-versa, can be re-tried a configurable number of times if the Explicit Synchronization callback routine is not successful.

As a result of this double buffering, the time during which the NvM user's RAM block data should not be touched is reduced down to the execution time of the Explicit Synchronization callbacks.

- During a READ operation, an Implicit Recovery using the configured initialization callback is performed if the Explicit Synchronization read callback fails. The request result is set to NVM_REQ_RESTORED_FROM_ROM.
- During a RESTORE operation that has been requested by calling the NvM_RestoreBlockDefaults service, a restore using the configured initialization callback is performed if the Explicit Synchronization read callback fails. The request result is set to NVM_REQ_OK.
- The Explicit Synchronization callback is not invoked for a restore job if no ROM block has been configured for the block.
- If an NvM block has Explicit Synchronization enabled, and if the caller provides a valid temporary RAM address to a single-block read/restore/write request, then the job will be performed with this temporary RAM block - the NvM mirror is not used, and the Explicit Synchronization callback is not invoked.

5.2.10 Identification of blocks

Many of NvM's APIs, and all of its client/server operations, are related to a dedicated NvM block. For this reason, you not only need to define one or more blocks via ECU configuration (giving each one a unique "short name"), but you also have to be able to address these blocks in APIs and client/server operation calls.

For APIs, each block gets its ID published by NvM as a #define (accessible from other components via NvM.h). The name of the #define is derived from the unique short name of the block. Note that the ID values can change whenever the NvM configuration changes, so rather than rely on a particular ID value, the #define should always be used. In cases where the use of potentially changing #define values is not feasible (e.g. in library code) it is possible to set fixed ID values for NvM blocks by manually filling in the NvMNvramBlockIdentifier configuration parameter. NvM relies on the fact, however, that the IDs of all blocks are in ascending order without any gaps, and if this rule is violated when manually defining block IDs, configuration processing will produce an error. Note also that

NvM reserves block IDs 0 and 1.

For the client/server interface of NvM, blocks are identified by the name of the corresponding port(s). For example, for a block named "MyModule_MyBlock", the NvM would generate the following block-specific ports by setting parameter NvMBlockUsePort=true. The following items are then generated:

- If a single block start callback is configured for this block, NvM will additionally generate a RPortPrototype "MyModule_MyBlock_Started" according to the ClientServerInterface /AUTOSAR_NvM/PortInterfaces/NvMRbNotifyJobStarted.
- If a single-block start callback is configured for this block, NvM will additionally generate a RPortPrototype "MyModule_MyBlock_Started".
- If an empty container for initialization callback is configured for this block, NvM will also generate a RPortPrototype "PNIB_MyModule_MyBlock".
- If an empty container for single-block callback is configured for this block, NvM will additionally generate a RPortPrototype "PNJF_MyModule_MyBlock".
- If explicit synchronization is activated for this block and Write RAM copy callback/Read RAM copy callback are not configured, NvM will additionally generate a RPortPrototype "PM_MyModule_MyBlock".
- If a single block start callback is configured for this block, NvM will additionally generate a RPortPrototype "MyModule_MyBlock_Started" according to the ClientServerInterface/AUTOSAR_NvM/PortInterfaces/NvMRbNotifyJobStarted, provided by the same ServiceSwComponentType/AUTOSAR_NvM/SwComponentTypes/NvM.

The following applies to AR version > AR42:

- NvM would additionally generate a PPortPrototype "PS_MyModule_MyBlock" according to the ClientServerInterface/AUTOSAR_NvM/PortInterfaces/NvMService, provided by the ServiceSwComponentType/AUTOSAR_NvM/SwComponentTypes/NvM.
- NvM would additionally generate a PPortPrototype "PAdmin_MyModule_MyBlock" according to the ClientServerInterface/AUTOSAR_NvM/PortInterfaces/NvMAdmin, provided by the same ServiceSwComponentType/AUTOSAR_NvM/SwComponentTypes/NvM.
- If an empty container for an initialization callback is configured for this block, NvM will additionally generate a RPortPrototype "PNIB_MyModule_MyBlock" according to the ClientServerInterface/AUTOSAR_NvM/PortInterfaces/NvMNotifyInitBlock, provided by the same ServiceSwComponentType/AUTOSAR_NvM/SwComponentTypes/NvM.
- If an empty container for a single block callback is configured for this block, NvM will additionally generate a RPortPrototype "PNJF_MyModule_MyBlock" according to the ClientServerInterface/AUTOSAR_NvM/PortInterfaces/NvMNotifyJobFinished, provided by the same ServiceSwComponentType/AUTOSAR_NvM/SwComponentTypes/NvM.
- If explicit synchronization is activated for this block and either the Write RAM

copy callback/Read RAM copy callback are not configured, NvM will additionally generate a RPortPrototype "PM_MyModule_MyBlock" according to the ClientServerInterface/AUTOSAR_NvM/PortInterfaces/NvMMirror, provided by the same ServiceSwComponentType/AUTOSAR_NvM/SwComponent-Types/NvM.

5.2.11 Dealing with Block Layout Changes

The following features allow you to configure the handling of block layout changes.

5.2.11.1 Persistent ID

On the persistent storage media, each Block is uniquely identified with a *Persistent ID*. This ID should not be confused with the Block ID described above, which is used in NvM's API functions. The Block ID may vary with every change in NvM's overall Block layout, while the Persistent ID only changes if the Block's name, size or the content incompatibly changes.

The Persistent ID can either be manually configured via the `NvMRbBlockPersistentId` parameter, or it can be automatically derived by NvM. For blocks with a configured Block length, the ID is calculated from the Block's name and size via a hashing algorithm. For blocks with automatic Block length calculation, the ID is calculated with the same algorithm but only the Block name is used as input.

- The Persistent ID of a Block is the key information for finding Block data on the persistent media. If the Persistent ID found on the media is equal to the Persistent ID defined in the Block configuration, it is assumed to be compatible and hence will be loaded to RAM upon any read request for this Block.
- If the Block size or the Block content incompatibly changes, a new Persistent ID has to be provided.
- Blocks with a configured Block length: If the Persistent ID is not manually configured (i.e. automatically generated), NvM derives a new Persistent ID if the name or the size of the Block changes.
- Blocks with an automatic Block length calculation: If the persistent ID is not manually configured (i.e. automatically generated), NvM derives a new Persistent ID if the name of the Block changes.

For dataset NvM blocks (type NVM_BLOCK_DATASET) the Persistent IDs of the individual NV blocks belonging to that NvM Block are in a contiguous range of ascending integer values. The first NV Block (which has the data-index 0) has the same Persistent ID as the NvM Block itself. The second NV Block has the Persistent ID of the first NV Block incremented by 1, and so on. If the NvM Block is configured manually, the Persistent ID of the NV blocks are generated automatically and cannot be configured manually. For example, for a dataset NvM Block whose manually-configured Persistent ID is 5555, the first NV Block gets the Persistent ID 5555, the next one is 5556, and so on.

5.2.11.2 Dynamic Configuration

To deal with layout changes, AUTOSAR defines an additional mechanism called "dynamic configuration", which can be enabled by setting the configuration parameter NvMDynamicConfiguration to TRUE.

When Dynamic Configuration is enabled, a special block named "NvM_ConfigId" will be read as the first block during NvM_ReadAll. If the number stored inside this special block is found to be different from the number configured in the NvMCompiledConfigId configuration parameter (i.e. there's a "configuration ID mismatch"), and if a memory layout change is detected ("configuration ID mismatch"), then NvM reads only resistant blocks (i.e. blocks with NvMResistantToChangedSw set to TRUE). Handling of non-resistant blocks depends on the configuration of the NvMRbDynConfigLoadDefaultsForNonResistant parameter:

- **TRUE**: NvM loads default value if available, i.e. RAM block address or initialization callback is configured.
- **FALSE**: NvM skips reading of non-resistant blocks and returns NVM_REQ_BLOCK_SKIPPED as result of NvM_ReadAll for this particular block

Dynamic Configuration, as defined by AUTOSAR, has the following functional shortcomings:

- Blocks which are marked as not being resistant to changed software are skipped during multi-block read when a configuration ID mismatch occurs, but they are still accessible via single-block read operations.
- According to the AUTOSAR specification, the block holding the configuration ID shall be written as the last block during multi-block writes. If an NvM user writes a block which is not resistant to changed software before that point under the interpretation of the new layout, however, an inconsistent state persists on the media until the configuration ID is finally written to it. This is not robust against several interruption scenarios, including resets triggered by severe errors.

It is highly recommended, therefore, not to trigger any single-block operation on the block holding the configuration ID (i.e. block # 1). If the Enhanced Dynamic Configuration feature is enabled (see below), the incompatible layout changes need to be sanitized by ReadAll. Therefore all of the single block operations which operate on the NV memory will be blocked before the NvM_ReadAll is initiated, if the current block is chosen to ReadAll and is not resistant to changed software.

Enhanced Dynamic Configuration

To address the above shortcomings, the Enhanced Dynamic Configuration functionality is available by enabling the NvMRbRemoveNonResistantBlocks configuration parameter. This parameter should be enabled whenever NvMDynamicConfiguration is enabled.

Enhanced Dynamic Configuration modifies the standard dynamic configuration behaviour as follows:

- When there's a configuration ID mismatch, any NV block which is not config-

uredto be resistant to changed software is automatically removed from the persistent media directly after a multi-block read operation has finished. In the current implementation of NvM, this removal is achieved through NV block invalidation.

- Just like any other job involving persistent storage, block removal might also fail under special circumstances. This is reported by setting the status of the affected block to NVM_REQ_NOT_OK and the block might still be accessible in subsequent read jobs.
- Immediately after all such blocks have been removed from the persistent media, the updated configuration ID is written to the persistent media (i.e. without waiting until a multi-block write operation is triggered for this).
- The entire phase of removing non-resistant blocks from the persistent media and writing back the updated configuration ID is directly and automatically appended after every multi-block read operation. It is internally handled just like another multi-block operation. As such, it also updates the states of all affected single blocks as well as the multi-block state (which is reported by retrieving the state of block #0). It also triggers multi-block callback and single block callbacks if such callbacks have been configured.
- No other standard single-block or multi-block requests are processed by NvM while the multi-block read operation (including the potential subsequent removal phase) is ongoing.

In the scenario where Dynamic Configuration is enabled but Enhanced Dynamic Configuration is disabled and the initialization at layout change is disabled (NvMRbInitAtLayoutChange set to FALSE), the request result of the configuration ID block (block #1) is set by NvM_ReadAll as follows:

- NVM_REQ_OK: The configuration ID block is successfully read from the medium during ReadAll and the configured config ID matches the read config ID (read from the medium).
- NVM_REQ_NOT_OK: The configuration ID block is successfully read from the medium during ReadAll and the configured config ID does not match the read config ID (read from the medium).
- NVM_REQ_NV_INVALIDATED: The configuration ID was never written before, or the configuration ID is corrupted on the medium, or the configuration ID was previously invalidated.
- NVM_REQ_INTEGRITY_FAILED: The configuration ID block couldn't be read from the medium during ReadAll due to an error that occurred in one of the lower layers.

If Dynamic Configuration is enabled, and if the configuration ID block was not marked to be written in the initialization phase, then this block has the request result as specified above. If the new compiled ID is written in the initialization phase following ReadAll, the request result of the configuration ID block is set to NVM_REQ_OK or NVM_REQ_NOT_OK, depending on whether the write operation was successful or not.

The new compiled configuration ID is also written to the medium if the reading of the configuration ID block during ReadAll fails.

5.2.11.3 Resistance to layout removal

This feature, which is enabled via the NvMRbResistantToLayoutRemoval configuration parameter, ensures that the related NV block is not removed from persistent storage even if it is no longer part of the block layout. This provides the flexibility to change the layout without losing already saved NvM blocks on the non-volatile memory. If removed from the block layout, the block is no longer accessible from NvM, but once reconfigured again, access to this block from NvM is re-enabled.

5.2.12 Initialization of blocks at a layout change

This feature, which is enabled via the NvMRbInitAtLayoutChange configuration parameter, allows the initialization of an NvM block which has been newly (re-)-configured and which does not exist on the persistent storage medium, or is corrupted. The initialization consists of writing the initialization data of this block just after the execution of ReadAll.

5.2.12.1 Initialization steps

- ReadAll starts its execution by reading the configuration ID block.
- The saved ID (i.e. the ID read from the medium) is compared to the newly compiled ID (i.e. the ID configured using the NvMCompiledConfigId configuration parameter). A mismatch is signalled if the saved ID is different from the compiled ID. If a mismatch is found (and in some other cases too, see “signalling initialization” further down), the configuration ID block is marked to be written to the medium.
- ReadAll moves to the next blocks. If the reading of a user block which has been configured with “init at layout change” is not successful (i.e. MemIf returns INCONSISTENT or FAILED), an Implicit Recovery is performed. The source for the recovery is the ROM block or the initialization callback.
- The block is then marked to be written to the medium.
- Once ReadAll finishes the reading of the blocks, an initialization phase begins, during which the previously marked user blocks are written to the medium.

5.2.12.2 Block configuration for initialization

- See the NvMRbInitBlockAtLayoutChange configuration parameter.

5.2.12.3 Configuration of ID block handling

Compiled configuration ID

The compiled ID is compared to the saved ID to detect a layout change. See the NvMCompiledConfigId configuration parameter for more details.

Configuration ID writing

The configuration ID block can be potentially marked to be written to the medium if Dynamic Configuration is enabled, or if NvMRblnitBlockAtLayoutChange is enabled.

If either of these features is enabled, then the configuration ID block will be marked (during ReadAll) to be written to the medium if a mismatch is detected, or if the reading of the configuration ID block was not successful (i.e. MemIf returns FAILED or INCONSISTENT).

When a configuration ID block is marked to be written, the newly compiled ID is written to the medium after ReadAll at the end of the following initialization phase (if Dynamic Configuration is disabled or if Enhanced Dynamic Configuration is enabled) or at the end of the WriteAll (if Dynamic Configuration is enabled and Enhanced Dynamic Configuration is disabled).

5.2.12.4 Signalling initialization

If the reading of the configuration ID block is not successful, the initialization of the blocks, as well as the writing of the newly compiled configuration ID, take place.

If the configuration ID block is found invalidated, the initialization of the blocks takes place but the newly compiled configuration ID is not written to the medium.

If the reading of the configuration ID block was successful and no mismatch was detected, the initialization and the writing of the compiled ID do not take place.

5.2.12.5 Additional Hints & Tips

- A block which is found to be invalidated during ReadAll is not initialized.
- The initialization of a block at a layout change is performed even if the block has the Write Protection feature enabled.
- An NvM block configured with the “Write Once” option of Write Protection can be initialized at a layout change. If the block is initialized (i.e. its recovery data is written to the medium after ReadAll), attempting any subsequent write/invalidate/erase operation on this block will fail.
- The RAM status of the config ID block is set to VALID_CHANGED if, during ReadAll, the block is signalled to be written in WriteAll. If, after ReadAll, a single block request is called for this block which modifies its RAM block status, the block will not be written in WriteAll.

5.2.12.6 Initialization at layout change (additional details)

The entire phase of initialization, including the writing of the compiled configuration ID, is directly and automatically appended after every multi-block read operation. It is internally handled just like another multi-block operation. As such, it also updates the states of all affected single blocks as well as the multi-block state (which is reported by retrieving the state of block #0).

- No other multi-block operation can be queued before the end of the ReadAll

and the subsequent initialization phase.

- If a standard single-block request is queued during ReadAll or during the subsequent initialization phase, the execution of this single-block request starts after the end of the execution of the initialization phase.
- The initialization phase that follows a ReadAll will trigger single-block callbacks (start and end) if such callbacks have been configured.
- If the multi-block callback is configured, it is invoked when all blocks are read during ReadAll. It is invoked again at the end of the execution of the initialization phase. The same thing applies to the BswM notification.
- The initialization phase can be executed after the system startup because the writing of the blocks to the medium could extend beyond the duration of the startup. A configured multi-block callback (or BswM notification) is called (or triggered) once the reading of the blocks during ReadAll has finished. This call or notification marks the point in time between the end of reading the blocks during readAll and the start of writing the blocks in the subsequent initialization phase.

5.2.13 Dealing with saved zone changes

NvM RAM buffers should be located in the dedicated RAM section called “saved zone” (see Configuration Advice), and you can choose whether NvM_Init should check the integrity of the saved zone.

If this check is enabled, and if the integrity of the saved zone is found to be intact, then the status bits of the RAM blocks are not modified in NvM_Init. If the integrity of the saved zone is not intact, these status bits are set to INVALID.

When NvM_ReadAll is invoked, an NvM block that has a RAM status bit of VALID will not be filled with NV data from the persistent storage medium. This ensures that ReadAll does not overwrite valid and up-to-date data in the RAM blocks with older data from the persistent storage. This also has the advantage of saving the amount of time that would otherwise be needed to reload NvM blocks from the storage media.

Sometimes it is necessary to ensure that ReadAll reads NV data from the persistent storage into the RAM blocks even if the integrity of the saved zone is found to be intact. In these cases, you should enable the NvMRbRamInitUnconditional parameter for the corresponding block.

5.2.14 Auxiliary Services

For special use cases, such as concurrent tester access, a separate set of NvM “auxiliary” services (with a prefix of NvM_Rb_Aux*) is provided:

- NvM_Rb_AuxInvalidateNvBlock
- NvM_Rb_AuxReadBlock
- NvM_Rb_AuxWriteBlock
- NvM_Rb_AuxGetErrorInfoDetail
- NvM_Rb_AuxGetErrorStatus

- NvM_Rb_AuxGetDataIndex
- NvM_Rb_AuxSetDataIndex
- NvM_Rb_AuxGetMigrationResult

These auxiliary services operate just like their counterparts, but are mostly invisible to the regular owner of an NvM block. Auxiliary services do NOT do any of the following:

- Alter the permanent RAM block configured for an NvM block.
- Trigger explicit synchronization callbacks configured for an NvM block.
- Modify the error status or dataset index.
- Trigger initialization or single-block (start) callbacks configured for an NvM block.

The invocation or execution of an auxiliary service will never block regular services on the same NvM block, and in contrast to the regular NvM interface, reading or writing a block using an auxiliary interface always requires a temporary RAM block upon invocation.

Note that if an auxiliary service is used that alters the contents of the NV block on the media (e.g. NvM_Rb_AuxInvalidateNvBlock, NvM_Rb_AuxWriteBlock), this will also be visible to the owner of the block with the next operation that reads the block.

In a multi-partition environment, the auxiliary services shall only be called from the Master partition.

5.2.15 Queue Monitoring

NvM queues have limited yet configurable sizes. Configuring queue sizes too large will require an unnecessarily high amount of RAM, and configuring them too small will result in service denial on asynchronous single-block service requests due to a queue overflow.

Two public monitoring variables, one for the standard queue (NvM_Rb_minNrFreeStdQueueEntries_u8) and one for the immediate queue (NvM_Rb_minNrFreeImmQueueEntries_u8) are provided in order to help you adequately estimate and define the appropriate queue sizes.

These variables save the minimum number of empty entries reached in the queue during the runtime of the ECU. Using this information, the previously configured queue length can be manually increased or decreased as required.

5.2.16 API Overloading

The API Overloading feature allows you to call additional user code as part of the following NvM services:

Single-block services

- EraseNvBlock
- GetErrorStatus

- InvalidateNvBlock
- Rb_SetWriteAllTrigger
- ReadBlock
- RestoreBlockDefaults
- SetBlockLockStatus
- SetBlockProtection
- SetDataIndex
- SetRamBlockStatus
- WriteBlock
- GetErrorStatus

Multi-block services

- CancelWriteAll
- Rb_FirstInitAll
- ReadAll
- ValidateAll
- WriteAll

Auxilliary Services

- Rb_AuxGetErrorStatus
- Rb_AuxInvalidateNvBlock
- Rb_AuxReadBlock
- Rb_AuxSetDataIndex
- Rb_AuxWriteBlock

API Overloading can be enabled separately for each NvM service. See the Configuration Advice and Integration Advice sections for more details.

5.2.17 Write Request Monitoring

NvM allows the monitoring of all write requests, this includes all write jobs done by single write requests, and also all those within multi-block write requests.

With the write request monitoring active, NvM provides following data via public variables:

Data provided in a non-persistent way (i.e. data is cleared after power-on)

- NvM_Rb_cntrWrites_u16 = number of write requests since last power-on
- NvM_Rb_nrBytesWritten_u32 = number of bytes written since last power-on
- NvM_Rb_idLastWrittenBlock_uo = NvM ID of the last written block

Data provided in a persistent way (i.e. data will be stored on non-volatile memory during NvM_WriteAll)

- NvM_Rb_WriteMon_st.cntrWritesAbs_u32 = accumulated number of all write requests
- NvM_Rb_WriteMon_st.nrBytesWrittenMax_u32 = maximum number of write requests since last power-on
- NvM_Rb_WriteMon_st.cntrWritesMax_u16 = maximum number of bytes written since last power-on

See the Configuration Advice section for details on how to enable and configure this feature.

5.2.18 Write Counter

NvM includes a Write Counter feature, which allows the monitoring of write requests for individual NVRAM blocks, regardless of whether the write job is part of a multi-block or a single-block write request. A default value can be provided so that you can configure Write Counters for several blocks at once.

NvM stores the Write Counter in the non-volatile memory as administrative data, alongside the user data. This means that the size of data stored in non-volatile memory will increase (blocksize = user-data-size + write-counter-size). As part of the administrative block, the Write Counter is cleared during a reset.

If a NVRAM block is invalidated or erased, and it is not written before the next power-on reset, NvM will lose the block's Write Counter after the reset (NvM will restart the counter).

To get the current Write Counter value, NvM must read the NVRAM block from the non-volatile memory, and AUTOSAR dictates that NvM can only read a NVRAM block if there is a user request. So, if the user requests a write job without a preceding read request, NvM sets the write counter value to 0 and writes it to the non-volatile memory. You can always determine which NVRAM blocks for which a write job has been requested without a preceding read request by analyzing the memory dump.

You can get the current Write Counter for a NVRAM block via the NvM_Rb_GetWriteCounter extension API.

See the Configuration Advice section for details on how to enable and configure this feature.

5.2.19 Block Length Check

The Block Length Check feature checks the size of the data defined by NvMRamBlockDataAddress or NvMRomBlockDataAddress against the configured NvMNvBlockLength for each block. If the configured NvM block length does not match the size of the data, a compile time error is created.

5.2.20 RAM block CRC

To monitor the integrity of data in RAM, you can enable CRC recalculation for

permanent RAM blocks, or blocks which are configured to use the Explicit Synchronization mechanism. If the CRC recalculation feature is enabled, NvM will recalculate the CRC for user data in a RAM block every time it has been changed (by a read, write or restore operation).

During NvM_ReadAll, if the recalculated RAM CRC matches the stored RAM CRC, NvM treats the data as valid and omits reading from NV memory. This avoids the overwriting of RAM data, which might still be valid, with outdated NV data. If a block processed by NvM_ReadAll is configured for Explicit Synchronization, the block's owner must provide the related RAM data for the comparison (i.e. NvM will call the Explicit Synchronization Write callback to get RAM data for the comparison).

If a block is marked as changed and valid (NvM_SetRamBlockStatus and NvM_-ValidateAll), NvM will recalculate the RAM CRC in the background. If NvM is idle (i.e. no user requests are pending), then it recalculates the RAM block CRC for changed blocks. NvM_SetRamBlockStatus should be called frequently for NVRAM blocks which are configured to be protected by RAM CRC. Otherwise this function only needs to be called once to mark a block as "changed" and to be processed during NvM_WriteAll.

NOTE

Since the re-calculation of the RAM CRC happens in the background, it usually takes multiple calls of NvM_MainFunction. Hence calling NvM_SetRamBlock-Status too often - e.g. once every 10ms - can be too fast for the NvM to catch up with the RAM CRC calculation and, in this case, there will never be a valid RAM CRC value matching the data. When using the RAM CRC, therefore, a consideration should be to change the data less frequently to enable timely re-calculation of the RAM CRC by the NvM.

The RAM Block CRC feature can also be used to skip write operations to NV memory if the RAM block has not been updated since the last read or write operation. In this case, if the RAM block CRC calculated during a write job is equal to the CRC calculated during the last successful read or write job, NvM will skip the writing and consider the job as successfully finished.

5.2.21 CRC in Nonvolatile Memory (Nv CRC)

A CRC checksum can be optionally appended to user data stored on the storage media in order to check its integrity. Note the following points regarding the use of Nv CRC:

- Data integrity is also assured via CRC checksums on lower software layers (Fee/Ea), so projects that include these lower layer modules may not require the Nv CRC feature.
- This Nv CRC feature, and the RAM CRC feature (described earlier), can be configured independently of each other.

5.2.22 Cryptographic services

To further increase the security of data stored in the non-volatile memory, you can enable the usage of cryptographic services for NVRAM blocks, which NvM will automatically execute while reading or writing the NVRAM block. You can use all the standard NvM APIs (e.g. NvM_ReadBlock or NvM_WriteBlock) and NvM will execute the configured cryptographic service(s) in the background.

The following cryptographic services are supported:

- Random Number: NvM will generate a random number in one of two ways: Per default, meaning the NvM will generate a random number during ReadAll as the last operation; and the API `NvM_Rb_SetCryptoInitVector`, which can initiate the generation at any time.

Before encrypting/authenticating user data, NvM will increase the generated random number and use it as the key element InitializationVector, allowing it to get different results by encrypting/authenticating the same user data. If key element InitializationVector is configured for encryption/authentication, NvM writes it together with the user data into the non-volatile memory so that it can be used for decryption/authentication after reading.

- Encryption/Authentication: Before writing user data, NvM will encrypt/authenticate it and only encrypted/authenticated user data will be written into the non-volatile memory. After reading encrypted/authenticated data from the non-volatile memory, NVM will decrypt/authenticate it, and only if decryption/authentication has succeeded will it write plain user data into the RAM block accessible by the user.
- Signature: Before writing user data, NvM will generate a signature (which can be also encrypted/authenticated), then it will write the user data and generated signature into the non-volatile memory. After reading data from the non-volatile memory, NvM will verify the read signature and only if signature verification has succeeded will it write the user data into the RAM block accessible by the user.
- MAC: Before writing user data, NvM will generate a MAC (Message Authentication Code) over the data (which can be also be encrypted/authenticated and signed beforehand). The NvM will then write the data, the generated signature(if configured) and the generated MAC into the non-volatile memory. The NvM will verify the read MAC after reading the user data from the non-volatile memory; only if this succeeds will it write the data into the RAM block accessible by the user.

NvM uses jobs provided by the Crypto Service Manager (Csm) to execute these services. These Csm jobs are executed in the operation mode for single calls (CRYPTO_OPERATIONMODE_SINGLECALL), which means that they are executed asynchronously. After starting a Csm job, NvM waits until Csm provides the result via NvM_Csm_Callback.

5.2.23 High-Speed Mode

To support exclusive processing of user requests with immediate priority (see the parameter NvMJobPrioritization), the NvM provides a special high-speed mode.

To enter/exit high-speed mode, the user shall use the APIs NvM_Rb_EnterHigh-SpeedMode and NvM_Rb_ExitHighSpeedMode.

The NvM enters high-speed mode only if following conditions are true:

- NvM is initialized.
- NvM has entered high-speed mode for all configured memory devices (see Configuration Advice for more details).

> **NOTE**
>
> Currently, not all memory devices (Fee, Ea) support high-speed mode; the user can only configure devices which support it. For more details, please refer to the corresponding memory device's documentation.
> If NvM cannot enter high-speed mode for all the configured memory devices, then the NvM exits high-speed mode for all the memory devices already in it.

In high-speed mode, the NvM handles only user requests with immediate priority. The NvM suspends all other user requests as long as high-speed mode is active.

While entering high-speed-mode, the NvM suspends all ongoing jobs (do not confuse pending and ongoing jobs) and resumes them only after exiting high-speed mode.

While entering high-speed-mode, the NvM also suspends requests for blocks with immediate priority, and re-starts processing them after high-speed mode has been entered.

While in high-speed mode, the NvM accepts all non-immediate user requests as long as the corresponding request queue is not full. However, processing of all these accepted user requests is suspended until the user has exited high-speed mode.

5.2.24 Multi-Partitioning

The primary purpose of a multi-partition/multi-core/multi-ASIL setup is to protect the memory assigned to one (Ecuc)Partition from the memory of another.

5.2.24.1 IOC/CallTrustedFunction

In general, cross-partition memory access can be handled by the RTE using an Os protection mechanism such as *IOC* or *CallTrustedFunction*:

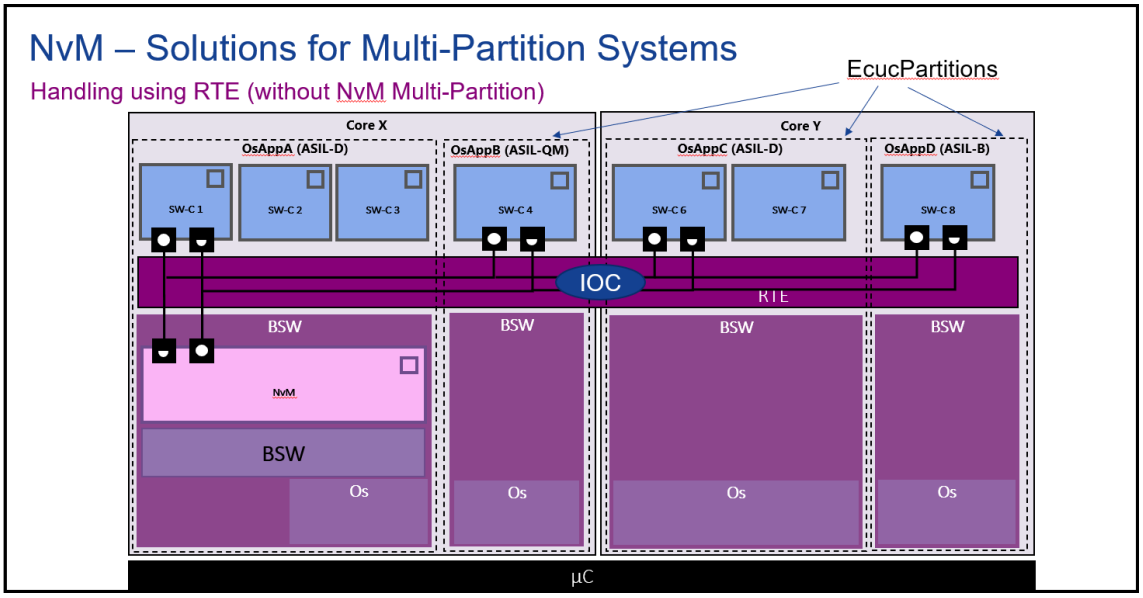


Figure 5.3.Multi-partitioning handled via RTE

5.2.24.2 NvBlockSwComponentType

More specifically, multi-partitioning can be handled so that NvM is accessed through the RTE's `NvBlockSwComponentType`.

Using this solution:

- The NvBlock of the `NvBlockSwComponentType`'s `NvBlockDescriptor` can be accessed from *any* Partition.
- The (RTE) `NvBlockDescriptor` and (BSW) `NvMBlockDescriptor` are handled in the *same* Partition.
- IOC is, again, involved. However, many IOC invocations will be expensive in terms of memory consumption.

Please see section 4.3.4.4 of *AUTOSAR_CP_SWS_RTE.pdf* and *RTA-RTE Reference Manual.pdf* for more details.

5.2.24.3 Master-Satellite

The RTE multi-partition solutions outlined above can lead to an inefficient use of resources e.g.:

- The RTE must make *copies* of NvM data when SWCs in separate partitions request access.
- IOC/CallTrustedFunction handling for cross-core communication is expensive.

Such problems can be mitigated at the BSW level with the use of a *Master-Satellite* system, in which one NvBlock is configured as the *Master*, with write access

to NvBlocks in other partitions, which are designated *Satellites*. This means that, while the SWC can access a Satellite NvBlock in the same Partition, it can also (via the Master) access Satellite NvBlocks in *other* Partitions.

Client-Server communication is therefore performed within a Core/Partition:

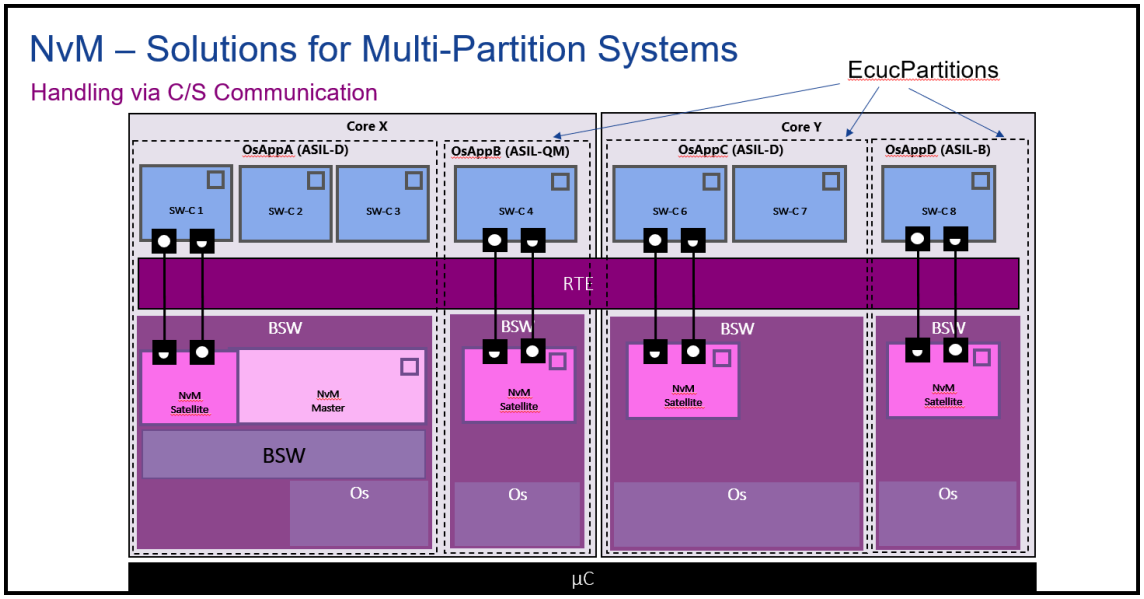


Figure 5.4.Multi-partitioning of NvM using Master-Satellite

However, *an efficient cross-Core/Partition communication is also established between Master and Satellite at the NvM level.*

In general, NvM multi-partitioning using Master-Satellite means that:

- NvM User APIs (both asynchronous and synchronous) must only be called from a configured Partition.
- NvM system functions (e.g. scheduling, *ReadAll, WriteAll*) must only be called from the configured NvM Master.
- Every component in the Mem stack below NvM (e.g. MemIf, Fee, Fls,..) must only be called by the configured NvM Master.

The behavior of asynchronous and synchronous APIs using Master-Satellite multi-partitioning:

Asynchronous API Calls

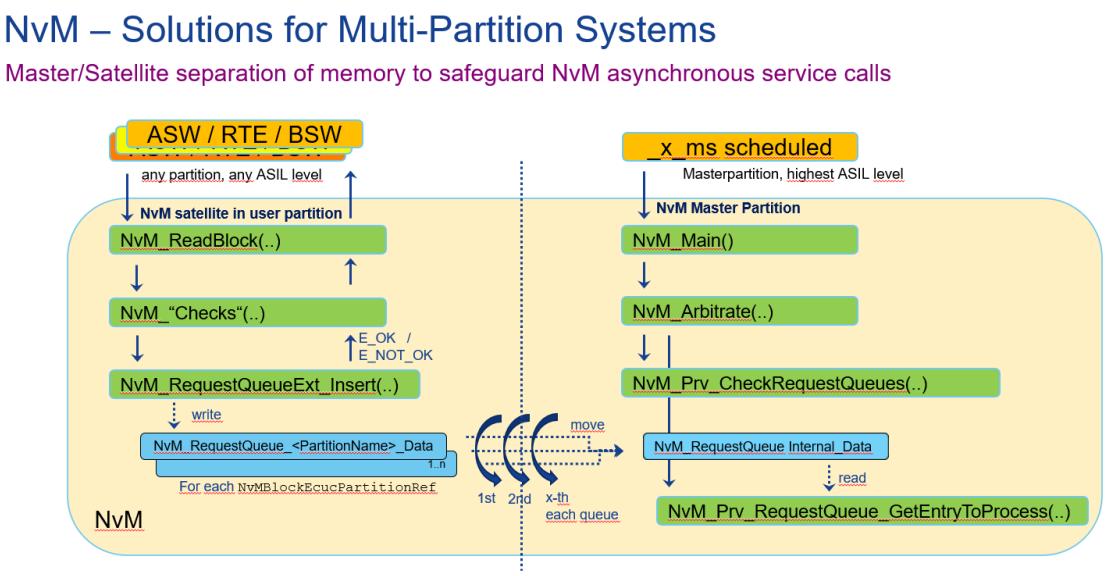


Figure 5.5.NvM asynchronous calls safeguarded by Master-Satellite memory separation

Asynchronous calls are placed in the request queue in the NvM Satellite Partition (on the left). To ensure *freedom from interference*, each Partition configured for an NvM Block has its own external request queue. When NvM is scheduled, it moves the asynchronous requests from the external queue to the internal queue, which is located in the NvM Master Partition (on the right).

If an asynchronous request has callbacks from NvM to the NvM User configured, an NvM internal mailbox system is used to trigger the NvM Satellite to call the NvM User within its Partition.

Synchronous API Calls

Some synchronous function calls (e.g. `NvM_SetRamBlockStatus`) write to NvM internal/private data. With multi-partitioning, all such data *remains separated* in partitions.

System calls

NvM calls that are *not* to a specific NvBlock can only be made from an ASIL level >= than that of the NvM Master Partition. (e.g. `NvM_MainFunction`, multi-block requests such as `NvM_ReadAll`, `NvM_WriteAll`, `NvM_ValidateAll`).

Measures in NvM for a safe data exchange between NvM and its User

Implicit synchronization - A User with a low ASIL rating passes a pointer when calling `NvM_ReadBlock`. The NvM running in its Master Partition would write to this pointer, *even when it is referencing the wrong Partition*. To prevent this, NvM performs a Pointer Access Check on the rights of the User.

> **NOTE**
>
> The above also applies to Permanent RAM Block access, but with a lower risk of occurrence as the pointer is set in the configuration.

Explicit synchronization - The callback process involves the User writing to the NvM buffer. To ensure freedom from interference, therefore, the buffers are separated from the NvM into User Partitions.

Memory Protection Unit (MPU) - Any potential Partition interference (e.g. the wrong Block Id) is checked, either by the Pointer Access Check or the MPU.

Please see Configuration Advice (Section 5.8.26) for more details on both the Pointer Access Check and MPU.

5.2.25 Auto Block Length

The Auto Block Length feature automatically determines the size of an NvM block based on the size of a data structure (e.g., structure, union, array) in the application code during compile time, eliminating the need to manually configure the length of an NvM block.

When enabled, the configured block length (`NvMNvBlockLength`) is ignored, and the size is determined by NvM during compile time using the *sizeof()* operator in C.

This feature affects the whole Memstack e.g. Fee will only get the block length at compile time. As a result, the layout checks performed during script processing are complemented by additional checks during compile time. The final result is, therefore, only available after compilation.

The block length is calculated using the following parameters:

Parameter	Description
`NvMRbNvBlockLengthString`	If configured then this string is directly taken as the size. The *sizeof()* operator is not added automatically i.e. if needed, the user must add it manually.
`NvMRamBlockDataAddress`	If configured AND the above parameter is not, then sizeof(`NvMRamBlockDataAddress`) will be taken as the size.
`NvMRomBlockDataAddress`	If configured AND the above ones are not, then sizeof(`NvMRomBlockDataAddress`) will be taken as the size.

In the current version v12.7.0 the feature is *not* compatible with any of the following (block-specific) features:

- Encryption of the NvM block;
- Backup of the NvM Block;
- Block Split (aka Block Compression) of the NvM block.

For more details on Auto Block Length, please see Configuration Advice (Section 5.8.27).

5.3 Extensions

5.3.1 Configuration Extensions

The following configuration parameters are available in version v12.7.0 in addition to those defined in AR v22-11

5.3.1.1 NvM/NvMBlockCiphering/NvMRbAddAuthentication

Short Name	NvMRbAddAuthentication
Long Name	Add block authentication
Description	[BSW_SWCS_AR_NVRAMManager_Ext-3356] Defines whether user data in NVRAM block shall be authenticated in NV memory. The NvM authenticates user data via the Csm job for AEAD encryption. IMPORTANT: If user data authentication is configured then the user has also to define NvMRbTagLength. Parameters NvMRbAssociatedDataLength and NvMRbAssociatedDataPtr shall be defined only if individual associated data is required.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

5.3.1.2 NvM/NvMBlockCiphering/NvMRbAddEncryption

Short Name	NvMRbAddEncryption
Long Name	Add block encryption
Description	[BSW_SWCS_AR_NVRAMManager_Ext-3249] Defines whether user data in NVRAM block shall be encrypted in NV memory.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

5.3.1.3 NvM/NvMBlockCiphering/NvMRbAddMac

Short Name	NvMRbAddMac
Long Name	Add message authentication code (MAC)
Description	[BSW_SWCS_AR_NVRAMManager_Ext-3415] Defines whether user data in NVRAM block shall be authenticated by a MAC (message authentication code) in NV memory.
Multiplicity	0..1
Default Value	false

Type	ECUC-BOOLEAN-PARAM-DEF

5.3.1.4 NvM/NvMBlockCiphering/NvMRbAddSignature

Short Name	NvMRbAddSignature
Long Name	Add signature
Description	[BSW_SWCS_AR_NVRAMManager_Ext-3251] Defines whether user data in NVRAM block shall be signed in NV memory.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

5.3.1.5 NvM/NvMBlockCiphering/NvMRbAssociatedDataLength

Short Name	NvMRbAssociatedDataLength
Long Name	Associated data length
Description	[BSW_SWCS_AR_NVRAMManager_Ext-3267] Defines the length of associated data in bytes used for AEAD encryption. If this parameter is defined then NvM will use Csm job for AEAD encryption.
Multiplicity	0..1
Value Range	0..65535
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

5.3.1.6 NvM/NvMBlockCiphering/NvMRbAssociatedDataPtr

Short Name	NvMRbAssociatedDataPtr
Long Name	Pointer to the associated data
Description	[BSW_SWCS_AR_NVRAMManager_Ext-3268] Defines the pointer to the associated data used for AEAD encryption. You may either give an address in hexadecimal notation here (e.g. 0x1234ABCD) or the name of a linker symbol (e.g. MyAssociatedData) or the address (e.g. &MyAssociatedData). It is also possible to specify no associated data by entering NULL_PTR. If this parameter is defined then NvM will use an AEAD encryption Csm job.
Multiplicity	0..1
Default Value	-
Type	ECUC-STRING-PARAM-DEF

5.3.1.7 NvM/NvMBlockCiphering/NvMRbCsmJobSignatureGenerateRef

Short Name	NvMRbCsmJobSignatureGenerateRef
Long Name	Csm job to generate a signature
Description	[BSW_SWCS_AR_NVRAMManager_Ext-3258] Reference to the Csm job to generate a signature for user data in NV memory.
Multiplicity	0..1
Default Value	-
Type	ECUC-SYMBOLIC-NAME-REFERENCE-DEF

5.3.1.8 NvM/NvMBlockCiphering/NvMRbCsmJobSignatureVerifyRef

Short Name	NvMRbCsmJobSignatureVerifyRef
Long Name	Csm job to verify a signature
Description	[BSW_SWCS_AR_NVRAMManager_Ext-3259] Reference to the Csm job to verify a signature from NV memory.
Multiplicity	0..1
Default Value	-
Type	ECUC-SYMBOLIC-NAME-REFERENCE-DEF

5.3.1.9 NvM/NvMBlockCiphering/NvMRbCsmMacGenerateJobReference

Short Name	NvMRbCsmMacGenerateJobReference
Long Name	Csm job to generate a message authentication code (MAC)
Description	[BSW_SWCS_AR_NVRAMManager_Ext-3417] Reference to the Csm job to generate a message authentication code (MAC) for user data in NV memory.
Multiplicity	0..1
Default Value	-
Type	ECUC-SYMBOLIC-NAME-REFERENCE-DEF

5.3.1.10 NvM/NvMBlockCiphering/NvMRbCsmMacVerifyJobReference

Short Name	NvMRbCsmMacVerifyJobReference
Long Name	Csm job to verify a message authentication code (MAC)
Description	[BSW_SWCS_AR_NVRAMManager_Ext-3418] Reference to the Csm job to verify a message authentication code (MAC) from NV memory.
Multiplicity	0..1

Default Value	-
Type	ECUC-SYMBOLIC-NAME-REFERENCE-DEF

5.3.1.11 NvM/NvMBlockCiphering/NvMRbInitVectorLength

Short Name	NvMRbInitVectorLength
Long Name	Initialization vector length
Description	[BSW_SWCS_AR_NVRAMManager_Ext-3357] Defines the length of the initialization vector in bytes used for user data encryption / authentication.
Multiplicity	0..1
Value Range	0..65535
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

5.3.1.12 NvM/NvMBlockCiphering/NvMRbMacLength

Short Name	NvMRbMacLength
Long Name	Message authentication code (MAC) length
Description	[BSW_SWCS_AR_NVRAMManager_Ext-3416] Defines the length of the message authentication code (MAC) in bytes used for user data authentication.
Multiplicity	0..1
Value Range	0..65535
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

5.3.1.13 NvM/NvMBlockCiphering/NvMRbSetInitVector

Short Name	NvMRbSetInitVector
Long Name	Set random number as key element InitVector
Description	[BSW_SWCS_AR_NVRAMManager_Ext-3358] Defines whether a random number shall be used as key element InitVector for user data encryption / authentication.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

5.3.1.14 NvM/NvMBlockCiphering/NvMRbSignatureLength

Short Name	NvMRbSignatureLength

Long Name	Signature length
Description	[BSW_SWCS_AR_NVRAMManager_Ext-3254] Defines the length of signature in bytes used to sign user data in NV memory.
Multiplicity	0..1
Value Range	0..65535
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

5.3.1.15 NvM/NvMBlockCiphering/NvMRbTagLength

Short Name	NvMRbTagLength
Long Name	Tag length
Description	[BSW_SWCS_AR_NVRAMManager_Ext-3269] Defines the length of the tag in bytes used for AEAD encryption. If this parameter is defined then NvM will use Csm job for AEAD encryption.
Multiplicity	0..1
Value Range	0..65535
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

5.3.1.16 NvM/NvMBlockCiphering/NvMRbUseIndividualAssociatedData

Short Name	NvMRbUseIndividualAssociatedData
Long Name	Use individual associated data
Description	[BSW_SWCS_AR_NVRAMManager_Ext-3359] Defines whether individual associated data shall be used to authenticate user data. - true = for user data authetication NvM uses associated data provided via parameters NvMRbAssociatedDataPtr and NvMRbAssociatedDataLength - false = for user data authetication NvM uses the block persistent ID defined in parameter NvMRbBlockPersistentId
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

5.3.1.17 NvM/NvMBlockDescriptor/NvMRbAutoBlockLength

Short Name	NvMRbAutoBlockLength
Long Name	Automatic block length

Description	Enables the automatic calculation of the block length during compile time. For the block size to be calculated automatically during compile time, at least one of the following parameters must be configured. The order in which these parameters are evaluated is: NvMRbNvBlockLengthString, NvMRamBlockDataAddress, NvMRomBlockDataAddress. TRUE: The configured block length (NvMNvBlockLength) is ignored, and the actual block length is determined during compilation. FALSE: The block length is determined by the configured block length (NvMNvBlockLength).
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

5.3.1.18 NvM/NvMBlockDescriptor/NvMRbBackupBlock

Short Name	NvMRbBackupBlock
Long Name	Include block in Mem backup
Description	Defines that the block should be included in the block backup using rba_MemBckp component. Note: Feature is only available for Fee blocks and is not supporting the combination with variable length blocks (NvMNvBlockLength = 0). TRUE: Block is included in the backup. FALSE: Block is not included in the backup.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

5.3.1.19 NvM/NvMBlockDescriptor/NvMRbBlockJobImmSubPriority

Short Name	NvMRbBlockJobImmSubPriority
Long Name	Job sub-priority
Description	[BSW_SWCS_AR_NVRAMManager_Ext-3480] Defines the sub-priority for immediate job priority for a NVRAM block. Sub-priority for Immediate job priority can be enabled only if priority jobs handling is enabled via the NvMJobPrioritization configuration parameter.
Multiplicity	0..1
Value Range	1..255
Default Value	255

Type	ECUC-INTEGER-PARAM-DEF

5.3.1.20 NvM/NvMBlockDescriptor/NvMRbBlockPersistentId

Short Name	NvMRbBlockPersistentId
Long Name	Persistent ID
Multiplicity	0..1
Value Range	1..65534
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

Description: [BSW_SWCS_AR_NVRAMManager_Ext-2749] Defines a unique ID for this block representing its block-specific properties. It shall be changed if and only if these block-specific properties are modified in a non-compatible way. This ID is stored on the memory media and is the basis for block identification in the memory drivers (Fee, Eep). In contrast to the IDs which are taken for block identification at the C API, this ID is (for a given block) identical in all modules of the memory stack. Its value typically remains the same even if other blocks are added to or removed from the block layout. It is not allowed that any two blocks have the same persistent ID. Additionally, the persistent IDs 0 and 65535 are reserved for internal use. If you don't explicitly specify a persistent ID for a block, such an identifier is automatically assigned by NvM based on the block name and block length for fixed size blocks. For blocks that have automatic block length calculation enabled the identifier is based on the block name only.

5.3.1.21 NvM/NvMBlockDescriptor/NvMRbBlockVersion

Short Name	NvMRbBlockVersion
Long Name	Block version
Description	[BSW_SWCS_AR_NVRAMManager_Ext-3343] Defines the block version. If the version is configured for this block, it will be stored into the non-volatile memory with the user data. The user can enable this feature by setting this parameter to a non zero value, the value zero means the block has no block version. If the user did a major change in the internal structure of the block then the block version should be updated. If the version is configured for a block, during a read block request, if the configured version matches the read version then the read request is executed normally if not the default values will be used and if no default values are available then NvM provides no values.
Multiplicity	0..1
Value Range	0..255

Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

5.3.1.22 NvM/NvMBlockDescriptor/NvMRbFirstReadDataMigration

Short Name	NvMRbFirstReadDataMigration
Long Name	First Read Data Migration
Description	[BSW_SWCS_AR_NVRAMManager_Ext-3027] Enables/disables First Read Data Migration feature for this block. When First Read Data Migration is enabled a read job is also performed when a missmatch between the configured NV block length and the block length on medium (EEPROM) occurs. In case the NV block length is smaller then the block on medium the remaining data are discarded. In case the NV block length is bigger then the block on medium the remaining data are filled with zero. If the feature is disabled and a length missmatch occurs the read job is not performed.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

5.3.1.23 NvM/NvMBlockDescriptor/NvMRbInitBlockAtLayoutChange

Short Name	NvMRbInitBlockAtLayoutChange
Long Name	Initialize block in case of a layout change
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

Description: [BSW_SWCS_AR_NVRAMManager_Ext-2881] Defines whether the NvM block shall be initialized if a block layout change is detected. In such case, the block is firstly read during ReadAll. If the block couldn't be retrieved from the medium, initialization data (from the ROM block or via the initialization callback) is written to the medium after the execution of ReadAll.
 This configuration parameter is allowed to be set to "true" under the following conditions:
 (1) The common configuration parameter NvMRbInitAtLayoutChange is set to "true".
 (2) The block is native or redundant
 (3) The block has a standard priority
 (4) The block has a ROM block or an initialization callback
 (5) The block has a RAM block or is configured with explicit synchronization
 (6) The block is configured for ReadAll
 (7) Additionally, if dynamic configuration is enabled, then this block has to be resistant to sw change.
 Hint: The block is initialized (i.e. written to the medium after ReadAll) if a configuration ID mismatch is detected. Therefore, when configuring a block with this feature, a

new compiled configuration ID should be configured via the NvMCompiledConfigId parameter (otherwise the block wouldn't be initialized).

5.3.1.24 NvM/NvMBlockDescriptor/NvMRbNoFallback

Short Name	NvMRbNoFallback
Long Name	No fallback to the old value
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

Description: [BSW_SWCS_AR_NVRAMManager_Ext-2925] This tag activates or deactivates a fallback mechanism to older copies of the same block. It is only evaluated for blocks located on Fee and ignored for blocks located on Ea. By default, this feature is set to false which is the standard behavior for Fee blocks (i.e. older values of the same block are preferred over delivering no block data at all). In Ea, older values are never available by principle and therefore blocks located on Ea will behave as if this tag had been set to true, regardless of its actual setting. If this tag is set to true and the corresponding block is located on Fee, older values will not be returned if the latest block instance is completely corrupted. In this case, inconsistent state is reported instead. Caution: When enabling this feature for the first time, already existing instances of the corresponding block on the media might not be readable anymore. Additionally, some Fee configurations might not support this feature at all. Please refer to the Fee documentation for details.

5.3.1.25 NvM/NvMBlockDescriptor/NvMRbNumberOfWriteCycles

Short Name	NvMRbNumberOfWriteCycles
Long Name	Number of block writes over ECU lifetime

Description	[BSW_SWCS_AR_NVRAMManager_Ext-3603] Expected number of write cycles required for block (during the complete ECU lifetime). This parameter is evaluated over all blocks of a device to estimate the overall required write cycles on that (Fee) device and an approximate number of driving cycles for the device can be estimated. This parameter is not used in NvM, value is forwarded to Fee where the calculation is done. Hint: 1) For blocks configured as dataset, the write cycles configured is used for each dataset. Eg. Number of write cycles = 100, number of datasets = 4. In this case, for each dataset value of 100 is used for calculations. Total 100 x 4 = 400 write cycles for this block. In the project context, if always only 1 dataset is written then this parameter can be adjusted to 25, so the number of write cylces considered is 25 x 4 = 100. 2) If the Block split is used for the block, number of write cycles for each part is considered equal to the configured value.
Multiplicity	0..1
Value Range	0..2147483647
Default Value	1
Type	ECUC-INTEGER-PARAM-DEF

5.3.1.26 NvM/NvMBlockDescriptor/NvMRbNvBlockLengthString

Short Name	NvMRbNvBlockLengthString
Long Name	Block length (C expression)
Description	[BSW_SWCS_AR_NVRAMManager_Ext-2996] The parameter is used for the automatic calculation of block length at compile time when NvMRbAutoBlockLength is set to TRUE. It defines the block length as an r-value (as in C programming), with examples including “sizeof(MyType)” or “MY_DEFINE”. Standalone numerical literals are not supported. All r-values must be declared in a header file, which the user must include in NvM by specifying the header name in the NvMBlockHeaderInclude parameter. This parameter is required when automatic calculation is enabled (NvMRbAutoBlockLength == TRUE) and both NvMRamBlockDataAddress and NvMRomBlockDataAddress are not configured.
Multiplicity	0..1
Default Value	-
Type	ECUC-STRING-PARAM-DEF

5.3.1.27 NvM/NvMBlockDescriptor/NvMRbRamInitAddSavedZoneNameRef

Short Name	NvMRbRamInitAddSavedZoneNameRef
Long Name	RAM Init - select additional savedzone name extension
Description	[BSW_SWCS_AR_NVRAMManager_Ext-3440] Only used if NvMRbRamInitCheck = TRUE and NvMRbRamInitUnconditional = FALSE and a PRAM Block is configured. If empty the standard SAVEDZONE pragma section is used for the reset marker, so that the block data has the init behaviour of the normal SAVEDZONE RAM. If the PRAM is located in RAM area with a different reset behaviour, the NvM init behaviour need to be in sync with it. Thatfore an additional reset marker must be placed in the same section as the PRAM is located. For the additional reset marker the pragma NVM_{START/STOP}_SEC_VAR_SAVED_ZONE_<NvMRbRamInitAddSavedZoneName>_32 is used. (This parameter NvMRbRamInitAddSavedZoneName is the {refinement} as defined for the SEC_VAR_SAVED_ZONE in the AUTOSAR_SWS_MemoryMapping) INTEGRATION hint: This section must be added in the NvM_MemMap.h (without MemMapGen) or in the MemMap-Pragma-configuration xpt file (if MemMapGen is used).
Multiplicity	0..1
Default Value	-
Type	ECUC-SYMBOLIC-NAME-REFERENCE-DEF

5.3.1.28 NvM/NvMBlockDescriptor/NvMRbRamInitUnconditional

Short Name	NvMRbRamInitUnconditional
Long Name	RAM init unconditional
Description	[BSW_SWCS_AR_NVRAMManager_Ext-3175] This parameter defines whether a permanent RAM block will be initialized unconditionally. - true: permanent RAM block will be initialized even if it has remained unchanged after last reset (common parameter NvMRbRamInitCheck = NVM_RAM_INIT_CHECK_QUICK) - false: permanent RAM block will be initialized dependent on the state of the parameter NvMRbRamInitCheck
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

5.3.1.29 NvM/NvMBlockDescriptor/NvMRbResistantToLayoutRemoval

Short Name	NvMRbResistantToLayoutRemoval
Long Name	Resistant to layout removal
Description	[BSW_SWCS_AR_NVRAMManager_Ext-2783] Defines whether the NvM block is resistant to layout removal. Once configured and written, lower layers keep this block saved on the persistent media even when this block is not part of the layout anymore (i.e. when the configuration changes and this block is removed from the layout). In this case, this block is not accessible from NvM. If the configuration changes again and this block is reconfigured, access to this block is reenabled again. If dynamic configuration is active (via NvMDynamicConfiguration), then resistant-to-layout-removal blocks must be resistant-to-changed-SW (configurable via NvMResistantToChangedSw parameter).
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

5.3.1.30 NvM/NvMBlockDescriptor/NvMRbSelectBlockForFirstInitAll

Short Name	NvMRbSelectBlockForFirstInitAll
Long Name	Initialize block during FirstInitAll
Description	[BSW_SWCS_AR_NVRAMManager_Ext-2828] Defines whether a NvM block shall be processed during NvM_Rb_FirstInitAll or not. This configuration parameter is allowed to be set in the following cases: (i) The NvM block (is native or redundant) and (has a ROM block or an InitblockCallback) and (has a permanent RAM block or is configured to use explicit synchronization mechanism) (ii) The NvM block (is native or redundant) and (has no ROM block and no InitBlockCallback). In this case the NvM block is simply invalidated by the NvM_Rb_FirstInitAll job. (iii) The NvM block is of type NVM_BLOCK_DATASET. In this case, all the NV blocks of the NvM block are invalidated by the NvM_Rb_FirstInitAll job.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

5.3.1.31 NvM/NvMBlockDescriptor/NvMRbSingleBlockStartCallback

Short Name	NvMRbSingleBlockStartCallback
Long Name	Single block start callback (Rb)
Description	[BSW_SWCS_AR_NVRAMManager_Ext_3062] Name of the block-specific callback routine which shall be invoked on start of each access to the actual memory media. If you explicitly want to specify that no such callback routine shall be called, enter NULL_PTR here. The signature of the callback-function has to be: For NvMRbCallbackSignatureAR45=false: Std_ReturnType CallbackName(uint8 ServiceId); For NvMRbCallbackSignatureAR45=true: Std_ReturnType CallbackName(NvM_BlockRequestType BlockRequest). It is not allowed to call any NvM or DEM functions from within this callback function! Runtime shall be minimized. Note: If parameter NvMBlockHeaderInclude is defined, then NvM does not create prototype declarations but expects that the prototype definition is within that header.
Multiplicity	0..1
Default Value	-
Type	ECUC-FUNCTION-NAME-DEF

5.3.1.32 NvM/NvMBlockDescriptor/NvMRbSplitBlock

Short Name	NvMRbSplitBlock
Long Name	Block split container
Description	[BSW_SWCS_AR_NVRAMManager_Ext-3445] Container for definition of split block
Multiplicity	0..1
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

5.3.1.33 NvM/NvMBlockDescriptor/NvMRbSplitBlock/NvMRbSubBlock

Short Name	NvMRbSubBlock
Long Name	Sub-block container
Description	[BSW_SWCS_AR_NVRAMManager_Ext-3449] Container for definition of sub-blocks within a split block
Multiplicity	1..127
Default Value	-

Type	ECUC-PARAM-CONF-CONTAINER-DEF

5.3.1.34 NvM/NvMBlockDescriptor/NvMRbSplitBlock/NvMRbSubBlock/NvMRbSubBlockLength

Short Name	NvMRbSubBlockLength
Long Name	Sub-Block length
Description	[BSW_SWCS_AR_NVRAMManager_Ext-3450] Defines the sub-block length in bytes.
Multiplicity	1..1
Value Range	1..65407
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

5.3.1.35 NvM/NvMBlockDescriptor/NvMRbSplitBlock/NvMRbSubBlock/NvMRbSubBlockNum

Short Name	NvMRbSubBlockNum
Long Name	Number of sub-blocks
Description	[BSW_SWCS_AR_NVRAMManager_Ext-3451] Defines number of contiguous sub-blocks with same length within a split block.
Multiplicity	1..1
Value Range	1..127
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

5.3.1.36 NvM/NvMBlockDescriptor/NvMRbWriteCounter

Short Name	NvMRbWriteCounter
Long Name	Write Counter
Description	[BSW_SWCS_AR_NVRAMManager_Ext-3296] This parameter defines whether the write counter is enabled/disabled for a NVRAM block. Attention: When write counter is enabled the size of data stored in non-volatile memory changes (block-size = user-data-size + write-counter-size) with all its implications on backwards-compatibility!
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

5.3.1.37 NvM/NvMCommon/NvMDrvModeSwitch

Short Name	NvMDrvModeSwitch
Long Name	Driver mode switch
Description	[ECUC_NvM_00496] Enables/disables switching memory drivers to fast mode during performing NvM_ReadAll and NvM_WriteAll. Fast node is not and will not be supported by layers underneath NvM. Therefore, NvM does not support this feature, so this parameter has currently always to be set to false.
Multiplicity	1..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

5.3.1.38 NvM/NvMCommon/NvMRbApiOverloading

Short Name	NvMRbApiOverloading
Long Name	API overloading
Description	[BSW_SWCS_AR_NVRAMManager_Ext-2914] Container for API overloading options. The parameters within this container should only be used with extreme care after thoroughly consulting the NvM documentation. Using the API overloading feature the wrong way can corrupt the entire Mem stack's functionality.
Multiplicity	0..1
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

5.3.1.39 NvM/NvMCommon/NvMRbApiOverloading/NvMRbAuxGetErrorStatus

Short Name	NvMRbAuxGetErrorStatus
Long Name	Rb_AuxGetErrorStatus function name
Description	[BSW_SWS_AR_NVRAMManager_Ext-2914] Name of the function to be called in the body of NvM_Rb_AuxGetErrorStatus instead of NvM's own implementation (which is available as NvM_Rb_AuxGetErrorStatusBase if this parameter is set).
Multiplicity	0..1
Default Value	-
Type	ECUC-FUNCTION-NAME-DEF

5.3.1.40 NvM/NvMCommon/NvMRbApiOverloading/NvMRbAuxInvalidateNvBlock

Short Name	NvMRbAuxInvalidateNvBlock
Long Name	Rb_AuxInvalidateNvBlock function name
Description	[BSW_SWCS_AR_NVRAMManager_Ext-2914] Name of the function to be called in the body of NvM_Rb_AuxInvalidateNvBlock instead of NvM's own implementation (which is available as NvM_Rb_AuxInvalidateNvBlockBase if this parameter is set).
Multiplicity	0..1
Default Value	-
Type	ECUC-FUNCTION-NAME-DEF

5.3.1.41 NvM/NvMCommon/NvMRbApiOverloading/NvMRbAuxReadBlock

Short Name	NvMRbAuxReadBlock
Long Name	Rb_AuxReadBlock function name
Description	[BSW_SWCS_AR_NVRAMManager_Ext-2914] Name of the function to be called in the body of NvM_Rb_AuxReadBlock instead of NvM's own implementation (which is available as NvM_Rb_AuxReadBlockBase if this parameter is set).
Multiplicity	0..1
Default Value	-
Type	ECUC-FUNCTION-NAME-DEF

5.3.1.42 NvM/NvMCommon/NvMRbApiOverloading/NvMRbAuxSetDataIndex

Short Name	NvMRbAuxSetDataIndex
Long Name	Rb_AuxSetDataIndex function name
Description	[BSW_SWCS_AR_NVRAMManager_Ext-2914] Name of the function to be called in the body of NvM_Rb_AuxSetDataIndex instead of NvM's own implementation (which is available as NvM_Rb_AuxSetDataIndexBase if this parameter is set).
Multiplicity	0..1
Default Value	-
Type	ECUC-FUNCTION-NAME-DEF

5.3.1.43 NvM/NvMCommon/NvMRbApiOverloading/NvMRbAuxWriteBlock

Short Name	NvMRbAuxWriteBlock
Long Name	Rb_AuxWriteBlock function name
Description	[BSW_SWCS_AR_NVRAMManager_Ext-2914] Name of the function to be called in the body of NvM_Rb_AuxWriteBlock instead of NvM's own implementation (which is available as NvM_Rb_AuxWriteBlockBase if this parameter is set).
Multiplicity	0..1
Default Value	-
Type	ECUC-FUNCTION-NAME-DEF

5.3.1.44 NvM/NvMCommon/NvMRbApiOverloading/NvMRbCancelWriteAll

Short Name	NvMRbCancelWriteAll
Long Name	CancelWriteAll function name
Description	[BSW_SWCS_AR_NVRAMManager_Ext-2914] Name of the function to be called in the body of NvM_CancelWriteAll instead of NvM's own implementation (which is available as NvM_Rb_CancelWriteAllBase if this parameter is set).
Multiplicity	0..1
Default Value	-
Type	ECUC-FUNCTION-NAME-DEF

5.3.1.45 NvM/NvMCommon/NvMRbApiOverloading/NvMRbEraseNvBlock

Short Name	NvMRbEraseNvBlock
Long Name	EraseNvBlock function name
Description	[BSW_SWCS_AR_NVRAMManager_Ext-2914] Name of the function to be called in the body of NvM_EraseNvBlock instead of NvM's own implementation (which is available as NvM_Rb_EraseNvBlockBase if this parameter is set).
Multiplicity	0..1
Default Value	-
Type	ECUC-FUNCTION-NAME-DEF

5.3.1.46 NvM/NvMCommon/NvMRbApiOverloading/NvMRbFirstInitAll

Short Name	NvMRbFirstInitAll
Long Name	Rb_FirstInitAll function name
Description	[BSW_SWCS_AR_NVRAMManager_Ext-2914] Name of the function to be called in the body of NvM_Rb_FirstInitAll instead of NvM's own implementation (which is available as NvM_Rb_FirstInitAllBase if this parameter is set).
Multiplicity	0..1
Default Value	-
Type	ECUC-FUNCTION-NAME-DEF

5.3.1.47 NvM/NvMCommon/NvMRbApiOverloading/NvMRbGetErrorStatus

Short Name	NvMRbGetErrorStatus
Long Name	GetErrorStatus function name
Description	[BSW_SWS_AR_NVRAMManager_Ext-2914] Name of the function to be called in the body of NvM_GetErrorStatus instead of NvM's own implementation (which is available as NvM_Rb_GetErrorStatusBase if this parameter is set).
Multiplicity	0..1
Default Value	-
Type	ECUC-FUNCTION-NAME-DEF

5.3.1.48 NvM/NvMCommon/NvMRbApiOverloading/NvMRbInvalidateNvBlock

Short Name	NvMRbInvalidateNvBlock
Long Name	InvalidateNvBlock function name
Description	[BSW_SWCS_AR_NVRAMManager_Ext-2914] Name of the function to be called in the body of NvM_InvalidateNvBlock instead of NvM's own implementation (which is available as NvM_Rb_InvalidateNvBlockBase if this parameter is set).
Multiplicity	0..1
Default Value	-
Type	ECUC-FUNCTION-NAME-DEF

5.3.1.49 NvM/NvMCommon/NvMRbApiOverloading/NvMRbReadAll

Short Name	NvMRbReadAll
Long Name	ReadAll function name
Description	[BSW_SWCS_AR_NVRAMManager_Ext-2914] Name of the function to be called in the body of NvM_ReadAll instead of NvM's own implementation (which is available as NvM_Rb_ReadAllBase if this parameter is set).
Multiplicity	0..1
Default Value	-
Type	ECUC-FUNCTION-NAME-DEF

5.3.1.50 NvM/NvMCommon/NvMRbApiOverloading/NvMRbReadBlock

Short Name	NvMRbReadBlock
Long Name	ReadBlock function name
Description	[BSW_SWCS_AR_NVRAMManager_Ext-2914] Name of the function to be called in the body of NvM_ReadBlock instead of NvM's own implementation (which is available as NvM_Rb_ReadBlockBase if this parameter is set).
Multiplicity	0..1
Default Value	-
Type	ECUC-FUNCTION-NAME-DEF

5.3.1.51 NvM/NvMCommon/NvMRbApiOverloading/NvMRbReadPRamBlock

Short Name	NvMRbReadPRamBlock
Long Name	ReadPRamBlock function name
Description	[BSW_SWCS_AR_NVRAMManager_Ext-2914] Name of the function to be called in the body of NvM_Read-PRAMBlock instead of NvM's own implementation (which is available as NvM_Rb_ReadPRAMBlockBase if this parameter is set).
Multiplicity	0..1
Default Value	-
Type	ECUC-FUNCTION-NAME-DEF

5.3.1.52 NvM/NvMCommon/NvMRbApiOverloading/NvMRbRestoreBlockDefaults

Short Name	NvMRbRestoreBlockDefaults

Long Name	RestoreBlockDefaults function name
Description	[BSW_SWCS_AR_NVRAMManager_Ext-2914] Name of the function to be called in the body of NvM_RestoreBlockDefaults instead of NvM's own implementation (which is available as NvM_Rb_RestoreBlockDefaultsBase if this parameter is set).
Multiplicity	0..1
Default Value	-
Type	ECUC-FUNCTION-NAME-DEF

5.3.1.53 NvM/NvMCommon/NvMRbApiOverloading/NvMRbRestorePRamBlockDefaults

Short Name	NvMRbRestorePRamBlockDefaults
Long Name	RestorePRamBlockDefaults function name
Description	[BSW_SWCS_AR_NVRAMManager_Ext-2914] Name of the function to be called in the body of NvM_RestorePRAMBlockDefaults instead of NvM's own implementation (which is available as NvM_Rb_RestorePRAMBlockDefaultsBase if this parameter is set).
Multiplicity	0..1
Default Value	-
Type	ECUC-FUNCTION-NAME-DEF

5.3.1.54 NvM/NvMCommon/NvMRbApiOverloading/NvMRbSetBlockLockStatus

Short Name	NvMRbSetBlockLockStatus
Long Name	SetBlockLockStatus function name
Description	[BSW_SWCS_AR_NVRAMManager_Ext-2914] Name of the function to be called in the body of NvM_SetBlockLockStatus instead of NvM's own implementation (which is available as NvM_Rb_SetBlockLockStatusBase if this parameter is set).
Multiplicity	0..1
Default Value	-
Type	ECUC-FUNCTION-NAME-DEF

5.3.1.55 NvM/NvMCommon/NvMRbApiOverloading/NvMRbSetBlockProtection

Short Name	NvMRbSetBlockProtection

Long Name	SetBlockProtection function name
Description	[BSW_SWCS_AR_NVRAMManager_Ext-2914] Name of the function to be called in the body of NvM_SetBlockProtection instead of NvM's own implementation (which is available as NvM_Rb_SetBlockProtectionBase if this parameter is set).
Multiplicity	0..1
Default Value	-
Type	ECUC-FUNCTION-NAME-DEF

5.3.1.56 NvM/NvMCommon/NvMRbApiOverloading/NvMRbSetDataIndex

Short Name	NvMRbSetDataIndex
Long Name	SetDataIndex function name
Description	[BSW_SWCS_AR_NVRAMManager_Ext-2914] Name of the function to be called in the body of NvM_SetDataIndex instead of NvM's own implementation (which is available as NvM_Rb_SetDataIndexBase if this parameter is set).
Multiplicity	0..1
Default Value	-
Type	ECUC-FUNCTION-NAME-DEF

5.3.1.57 NvM/NvMCommon/NvMRbApiOverloading/NvMRbSetRamBlockStatus

Short Name	NvMRbSetRamBlockStatus
Long Name	SetRamBlockStatus function name
Description	[BSW_SWCS_AR_NVRAMManager_Ext-2914] Name of the function to be called in the body of NvM_SetRamBlockStatus instead of NvM's own implementation (which is available as NvM_Rb_SetRamBlockStatusBase if this parameter is set).
Multiplicity	0..1
Default Value	-
Type	ECUC-FUNCTION-NAME-DEF

5.3.1.58 NvM/NvMCommon/NvMRbApiOverloading/NvMRbSetWriteAllTrigger

Short Name	NvMRbSetWriteAllTrigger
Long Name	Rb_SetWriteAllTrigger function name

Description	[BSW_SWCS_AR_NVRAMManager_Ext-2914] Name of the function to be called in the body of NvM_Rb_SetWriteAllTrigger instead of NvM's own implementation (which is available as NvM_Rb_SetWriteAllTriggerBase if this parameter is set).
Multiplicity	0..1
Default Value	-
Type	ECUC-FUNCTION-NAME-DEF

5.3.1.59 NvM/NvMCommon/NvMRbApiOverloading/NvMRbValidateAll

Short Name	NvMRbValidateAll
Long Name	ValidateAll function name
Description	[BSW_SWCS_AR_NVRAMManager_Ext-2914] Name of the function to be called in the body of NvM_Rb_ValidateAll instead of NvM's own implementation (which is available as NvM_Rb_ValidateAllBase if this parameter is set).
Multiplicity	0..1
Default Value	-
Type	ECUC-FUNCTION-NAME-DEF

5.3.1.60 NvM/NvMCommon/NvMRbApiOverloading/NvMRbWriteAll

Short Name	NvMRbWriteAll
Long Name	WriteAll function name
Description	[BSW_SWCS_AR_NVRAMManager_Ext-2914] Name of the function to be called in the body of NvM_WriteAll instead of NvM's own implementation (which is available as NvM_Rb_WriteAllBase if this parameter is set).
Multiplicity	0..1
Default Value	-
Type	ECUC-FUNCTION-NAME-DEF

5.3.1.61 NvM/NvMCommon/NvMRbApiOverloading/NvMRbWriteBlock

Short Name	NvMRbWriteBlock
Long Name	WriteBlock function name
Description	[BSW_SWCS_AR_NVRAMManager_Ext-2914] Name of the function to be called in the body of NvM_WriteBlock instead of NvM's own implementation (which is available as NvM_Rb_WriteBlockBase if this parameter is set).
Multiplicity	0..1

Default Value	-
Type	ECUC-FUNCTION-NAME-DEF

5.3.1.62 NvM/NvMCommon/NvMRbApiOverloading/NvMRbWritePRamBlock

Short Name	NvMRbWritePRamBlock
Long Name	WritePRamBlock function name
Description	[BSW_SWCS_AR_NVRAMManager_Ext-2914] Name of the function to be called in the body of NvM_WritePRAMBlock instead of NvM's own implementation (which is available as NvM_Rb_WritePRAMBlockBase if this parameter is set).
Multiplicity	0..1
Default Value	-
Type	ECUC-FUNCTION-NAME-DEF

5.3.1.63 NvM/NvMCommon/NvMRbAuxInterface

Short Name	NvMRbAuxInterface
Long Name	Auxiliary interface
Description	[BSW_SWCS_AR_NVRAMManager_Ext-2945] Enables/disables the auxiliary service interface. This interface allows a special "auxiliary" user to trigger NvM single-block jobs for arbitrary blocks which do not interfere with jobs or states of "regular" users.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

5.3.1.64 NvM/NvMCommon/NvMRbBlockHeader

Short Name	NvMRbBlockHeader
Long Name	Block header
Multiplicity	0..1
Value Range	0..2
Default Value	0
Type	ECUC-INTEGER-PARAM-DEF

Description: [BSW_SWCS_AR_NVRAMManager_Ext-3406] Defines the storage of the block header in non-volatile memory together with user data. Block header is always stored before user data, i.e block data in non-volatile memory starts with block header.
 Possible values:
 - 0 = block header disabled
 -

1 = block header with layout version 1 stored:
 Byte I Name I Usage
 0-1 I CrcHdr I CRC16 over header bytes 2-14, LSB first
 2 I LngHdr I Header length in bytes [1 .. 255]
 3 I VerHdr I Header Version [1 .. 255]
 4-5 I LngPlainData I Length of plain user data in bytes [1 .. 65407], LSB first
 6-7 I LngEncryptData I Length of encrypted user data in bytes [0, 1 .. 65407], LSB first, 0 = user data not encrypted
 8 I CrcPlainData I Length of the CRC over plain user data stored with encrypted user data [0, 1, 2, 4], 0 = no CRC, 1 = CRC8, 2 = CRC16, 4 = CRC32
 9 I LngIV I Length of the Init Vector used for encryption in multiples of 4 bytes [0, 1 .. 255], 0 = no Init Vector, 1 = 4 bytes, 2 = 8 bytes, 3 = 12 bytes, etc.
 10 I LngTag I Length of the tag stored in case of AEAD encryption in multiples of 4 bytes [0, 1 .. 255], 0 = no tag, 1 = 4 bytes, 2 = 8 bytes, 3 = 12 bytes, etc.
 11 I LngSgn I Length of the signature in multiples of 4 bytes [0, 1 .. 255], 0 = no signature, 1 = 4 bytes, 2 = 8 bytes, 3 = 12 bytes, etc.
 12 I LngWrtCntr I Length of the block write counter [0, 4], 0 = no write counter used, 4 = 4 bytes stored
 13 I LngBlkVer I Length of the block version [0, 1], 0 = no block version used, 1 = byte stored
 14 I LngCrcStoredData I Length of the CRC over all data stored in non-volatile memory [0, 1, 2, 4], 0 = no CRC stored, 1 = CRC8 stored, 2 = CRC16 stored, 4 = CRC32 stored
 - 2 = block header with layout version 2 stored:
 Byte I Name I Usage
 0-1 I CrcHdr I CRC16 over header bytes 2-15, LSB first
 2 I LngHdr I Header length in bytes [1 .. 255]
 3 I VerHdr I Header Version [1 .. 255]
 4-5 I LngPlainData I Length of plain user data in bytes [1 .. 65407], LSB first
 6-7 I LngEncryptData I Length of encrypted user data in bytes [0, 1 .. 65407], LSB first, 0 = user data not encrypted
 8 I CrcPlainData I Length of the CRC over plain user data stored with encrypted user data [0, 1, 2, 4], 0 = no CRC, 1 = CRC8, 2 = CRC16, 4 = CRC32
 9 I LngIV I Length of the Init Vector used for encryption in multiples of 4 bytes [0, 1 .. 255], 0 = no Init Vector, 1 = 4 bytes, 2 = 8 bytes, 3 = 12 bytes, etc.
 10 I LngTag I Length of the tag stored in case of AEAD encryption in multiples of 4 bytes [0, 1 .. 255], 0 = no tag, 1 = 4 bytes, 2 = 8 bytes, 3 = 12 bytes, etc.
 11 I LngSgn I Length of the signature in multiples of 4 bytes [0, 1 .. 255], 0 = no signature, 1 = 4 bytes, 2 = 8 bytes, 3 = 12 bytes, etc.
 12 I LngMac I Length of the message authentication code (MAC) in multiples of 4 bytes [0, 1 .. 255], 0 = no signature, 1 = 4 bytes, 2 = 8 bytes, 3 = 12 bytes, etc.
 13 I LngWrtCntr I Length of the block write counter [0, 4], 0 = no write counter used, 4 = 4 bytes stored
 14 I LngBlkVer I Length of the block version [0, 1], 0 = no block version used, 1 = byte stored
 15 I LngCrcStoredData I Length of the CRC over all data stored in non-volatile memory [0, 1, 2, 4], 0 = no CRC stored, 1 = CRC8 stored, 2 = CRC16 stored, 4 = CRC32 stored

5.3.1.65 NvM/NvMCommon/NvMRbCallbackSignatureAR45

Short Name	NvMRbCallbackSignatureAR45
Long Name	Callback function parameter type according to AR45

Description	[BSW_SWCS_AR_NVRAMManager_Ext-3436] Changes the parameter type of the callback functions. TRUE: NvM_<Multi;Init>BlockRequestType according to AR44 and after ; FALSE: ServiceId according to AR43 and before; Details of the parameter of the function calls see the callback parameters. If NvMRbCallbackSignatureAR45 = TRUE the function NvM_Rb_GetActiveService is not available, because it is not needed anymore and it has disadvantages when usedwith the MultiFee and/or crytographic service feature. See NvM Doku NvM_EN_detailed_CDG.pdf
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

5.3.1.66 NvM/NvMCommon/NvMRbCommonCalcRamBlockCrc

Short Name	NvMRbCommonCalcRamBlockCrc
Long Name	Common CRC (re)calculation
Description	[BSW_SWCS_AR_NVRAMManager_Ext-3186] Defines the default value for parameters NvMCalcRamBlockCrc and NvMBlockUseCRCCompMechanism. If this parameter is set to true then default value of parameters NvMCalcRamBlockCrc and NvMBlockUseCRCCompMechanism is true for all NVRAM blocks for which these parameters can be true. I.e. for all NVRAM blocks with permanent RAM blocks or which are configured to use explicit synchronization mechanism. For NVRAM blocks which have not configured a CRC type the NvM will use the default CRC type of 16 bit (NVM_CRC16).
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

5.3.1.67 NvM/NvMCommon/NvMRbCommonWriteCounter

Short Name	NvMRbCommonWriteCounter
Long Name	Common write counter

Description	[BSW_SWCS_AR_NVRAMManager_Ext-3295] Defines the default value for the block specific parameter NvMRbWriteCounter. - true: default value of the parameter NvMRbWriteCounter is true for all NVRAM blocks. I.e. the write counter is enabled for all NVRAM blocks and disabled only for NVRAM blocks where the parameter NvMRbWriteCounter is explicitely set to false. - false: default value of the parameter NvMRbWriteCounter is false for all NVRAM blocks. I.e. the write counter is disabled for all NVRAM blocks and enabled only for NVRAM blocks where the parameter NvMRbWriteCounter is explicitely set to true.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

5.3.1.68 NvM/NvMCommon/NvMRbCsmJobRandomNrRef

Short Name	NvMRbCsmJobRandomNrRef
Long Name	Csm job to get a random number used for encryption
Description	[BSW_SWCS_AR_NVRAMManager_Ext-3257] Reference to the Csm job to get a random number used for cryptographic services.
Multiplicity	0..1
Default Value	-
Type	ECUC-SYMBOLIC-NAME-REFERENCE-DEF

5.3.1.69 NvM/NvMCommon/NvMRbDeviceProperty

Short Name	NvMRbDeviceProperty
Long Name	Memory devices
Description	[BSW_SWCS_AR_NVRAMManager_Ext-3463] Container for a management structure to configure a memory device.
Multiplicity	0..254
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

5.3.1.70 NvM/NvMCommon/NvMRbDeviceProperty/NvMRbEnableHigh-SpeedMode

Short Name	NvMRbEnableHighSpeedMode
Long Name	High-speed-mode

Description	[BSW_SWCS_AR_NVRAMManager_Ext-3464] Defines whether the corresponding NVRAM device shall store NVRAM blocks with immediate priority in high-speed-mode.
Multiplicity	1..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

5.3.1.71 NvM/NvMCommon/NvMRbDeviceProperty/NvMRbNvramDeviceId

Short Name	NvMRbNvramDeviceId
Long Name	Memory device ID
Description	[BSW_SWCS_AR_NVRAMManager_Ext-3465] Defines the NVRAM device ID for which properties in this container are defined.
Multiplicity	1..1
Value Range	0..254
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

5.3.1.72 NvM/NvMCommon/NvMRbDynConfigLoadDefaultsForNonResistant

Short Name	NvMRbDynConfigLoadDefaultsForNonResistant
Long Name	Load defaults for non-resistant blocks
Description	[BSW_SWCS_AR_NVRAMManager_Ext-3307] Defines whether NvM loads default values or skips reading for non-resistant blocks in case of layout change for dynamic configuration defined by Autosar (extended run-time preparation) TRUE = NvM loads default values for non-resistant blocks in case of layout change FALSE = NvM skips reading of non-resistant blocks in case of layout change ATTENTION: this parameter affects only dynamic configuration as defined by AUTOSAR. If enhanced dynamic configuration is enabled (NvMRbRemoveNonResistantBlocks = TRUE) then this parameter has no effect.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

5.3.1.73 NvM/NvMCommon/NvMRbEaDevices

Short Name	NvMRbEaDevices
Long Name	Ea Devices

Description	[BSW_SWCS_AR_NVRAMManager_Ext-3316] Container for device IDs to address Ea devices.
Multiplicity	0..1
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

5.3.1.74 NvM/NvMCommon/NvMRbEaDevices/NvMRbEaDeviceId

Short Name	NvMRbEaDeviceId
Long Name	Device ID to address Ea device
Description	[BSW_SWCS_AR_NVRAMManager_Ext-3314] Identification of the device to address Ea. Attention: If a NVRAM block shall be located on the Ea device identified by this parameter then the parameter NvMNvramDeviceId of this NVRAM block shall be equal to the value defined here. Attention: Device ID 0 is fixed to address Fee. So device ID 0 cannot be used for Ea devices.
Multiplicity	0..1
Value Range	1..254
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

5.3.1.75 NvM/NvMCommon/NvMRbErrorInfoDetail

Short Name	NvMRbErrorInfoDetail
Long Name	Error Info Detail
Description	[BSW_SWCS_AR_NVRAMManager_Ext-3405] Defines whether the API NvM_Rb_GetErrorInfoDetail is available. It can be called after each block oriented job to fetch additional error details. - true: NvM_Rb_GetErrorInfoDetail function is available. - false: NvM_Rb_GetErrorInfoDetail function is not available.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

5.3.1.76 NvM/NvMCommon/NvMRbExplcitSyncReadWithConst

Short Name	NvMRbExplcitSyncReadWithConst
Long Name	Enable const qualifier for read callback

Description	[BSW_SWCS_AR_NVRAMManager_Ext-3333] Enables const qualifier in signature of the read callback parameter NvMBuffer. Read callback can be defined using block parameter NvMReadRamBlock FromNvCallback. TRUE = Read callback parameter has a const qualifier. FALSE = Read callback parameter has no const qualifier.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

5.3.1.77 NvM/NvMCommon/NvMRbInitAtLayoutChange

Short Name	NvMRbInitAtLayoutChange
Long Name	Initialize blocks at a layout change
Description	[BSW_SWCS_AR_NVRAMManager_Ext-2893] Enables/disables the initialization of NvM blocks having the configuration parameter NvMRbInitBlock AtLayoutChange set to “true”. This initialization is perfomed in case a layout change is detected.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

5.3.1.78 NvM/NvMCommon/NvMRbInitBufferBeforeSyncWrite

Short Name	NvMRbInitBufferBeforeSyncWrite
Long Name	Initialize buffer before sync write callback
Description	[BSW_SWCS_AR_NVRAMManager_Ext-3298] Enables/disables the initialization of the NvM internal buffer before invoking the write callback with explicit synchronization.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

5.3.1.79 NvM/NvMCommon/NvMRbMainFunctionScheduling

Short Name	NvMRbMainFunctionScheduling
Long Name	Scheduling for NvM_MainFunction

Description	[BSW_SWCS_AR_NVRAMManager_Ext-3183] Defines that NvM_MainFunction() shall be called in the 1) background-task via RTE, in a 2) scheduled task via RTE or that 3) the integrator schedules it within RTE ("manually"). In case it is scheduled manually, no RTE events or entities are generated. Adjust the setting in parameter NvMMainFunctionPeriod accordingly.
Multiplicity	0..1
Default Value	NVM_RB_MAINFUNC_SCHEDULING_RTE_TIMED
Enumeration Literals	NVM_RB_MAINFUNC_SCHEDULING_RTE_BACKGROUND, NVM_RB_MAINFUNC_SCHEDULING_RTE_TIMED, NVM_RB_MAINFUNC_SCHEDULING_MANUAL
Type	ECUC-ENUMERATION-PARAM-DEF

5.3.1.80 NvM/NvMCommon/NvMRbMultiBlockStartCallback

Short Name	NvMRbMultiBlockStartCallback
Long Name	Multiblock start callback
Description	[BSW_SWCS_AR_NVRAMManager_Ext-2762] Entry address of the common callback routine which shall be invoked whenever an asynchronous multi block request is about to start. If you explicitly want to specify that no such callback routine shall be called, enter NULL_PTR here. The signature of the callback-function has to be: For NvMRbCallbackSignatureAR45=false: Std_ReturnType CallbackName(uint8 ServiceId) For NvMRbCallbackSignatureAR45=true: Std_ReturnType CallbackName(NvM_MultiBlockRequestType MultiBlockRequest). It is not allowed to call any NvM or DEM functions from within this callback function! Runtime shall be minimized.
Multiplicity	0..1
Default Value	-
Type	ECUC-FUNCTION-NAME-DEF

5.3.1.81 NvM/NvMCommon/NvMRbNoInitCallbackRomBlockConfigured

Short Name	NvMRbNoInitCallbackRomBlockConfigured
Long Name	No Initialization callback if a ROM block is configured

Description	[BSW_SWCS_AR_NVRAMManager_Ext-3414] Enables/disables no initialization callback is called if a block has ROM block configured feature, if the parameter NvMRbNoInitCallbackRomBlockConfigured is set to “true”, the initialization callback shall be called only if a ROM block is not configured. For AR46 and higher, this feature shall be enabled.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

5.3.1.82 NvM/NvMCommon/NvMRbNoSingleBlockBeforeReadAll

Short Name	NvMRbNoSingleBlockBeforeReadAll
Long Name	No single block request before ReadAll
Description	[BSW_SWCS_AR_NVRAMManager_Ext-3330] Enables/disables no single block request before ReadAll feature, if the parameter NvMRbNoSingle BlockBeforeReadAll is set to “true”, any single block access shall not be processed before completion of “ReadAll” service - instead they shall be rejected with E_NOT_OK.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

5.3.1.83 NvM/NvMCommon/NvMRbNvCrcEnable

Short Name	NvMRbNvCrcEnable
Long Name	Enable CRC useage in NV memory

Description	[BSW_SWCS_AR_NVRAMManager_Ext-3187] Globally enables/disables CRC useage in NV memory for NVRAM blocks which have the block-specific parameter NvMBlockUseCrc set to TRUE. If NvMRbNvCrcEnable is TRUE the block-specific parameter NvMBlockUseCrc defines if a CRC on NVM level is used for the block. If NvMRbNvCrcEnable is set to FALSE no CRCs on NVM level will be used (even though the block-specific parameter NvMBlockUseCrc might be set to TRUE). Hint: For projects which do not use the complete MemStack, the parameter NvMRbNvCrcEnable should be set to TRUE to enable CRC useage. For projects which use the complete Memstack it is adviced to keep the Nv CRC on NvM level turned off (NvMRbNvCrcEnable= FALSE) because the lower software layers already apply CRC integrity checks. Attention: When CRC useage is activated the block-size of the NVM block changes compared to the case that CRC is deactivated (block-size = user-data-size + crc-size) with all its implications on backwards-compatibility!
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

5.3.1.84 NvM/NvMCommon/NvMRbObserverCallback

Short Name	NvMRbObserverCallback
Long Name	Observer callback
Description	[BSW_SWCS_AR_NVRAMManager_Ext-2843] Name of the callback routine which is invoked on the termination of the processing of a job for a block within an asynchronous request. Unlike multiblock callback, if configured, this callback is invoked for any configured block including the configuration ID block (when dynamic configuration is enabled) and excluding block #0. Similar to the single block request, it provides the block specific request result when the processing of the block finishes. If you explicitly want to specify that no such callback routine shall be called, enter NULL_PTR here. It is not allowed to call any NvM or DEM functions from within this callback function! Runtime shall be minimized.
Multiplicity	0..1
Default Value	-

Type	ECUC-FUNCTION-NAME-DEF

5.3.1.85 NvM/NvMCommon/NvMRbRamInitCheck

Short Name	NvMRbRamInitCheck
Long Name	RAM init check
Description	[BSW_SWCS_AR_NVRAMManager_Ext-2786] This parameter allows to choose the behavior of the NvM module when a reset occurs. NVM_RB_RAM_INIT_CHECK_NONE: After a reset, NvM_ReadAll() reads all blocks suitable for ReadAll. NVM_RB_RAM_INIT_CHECK_QUICK: After a reset, it is checked whether the saved zone is lost. If not lost, only blocks whose RAM status bits are INVALID are filled with data from Nv blocks during NvM_ReadAll(). If the RAM status bit of a block is found VALID, then the block is skipped by NvM_ReadAll().
Multiplicity	0..1
Default Value	NVM_RB_RAM_INIT_CHECK_NONE
Enumeration Literals	NVM_RB_RAM_INIT_CHECK_NONE, NVM_RB_RAM_INIT_CHECK_QUICK
Type	ECUC-ENUMERATION-PARAM-DEF

5.3.1.86 NvM/NvMCommon/NvMRbRamInitCheckQuickSkippedToOk

Short Name	NvMRbRamInitCheckQuickSkippedToOk
Long Name	RAM init check result
Description	[BSW_SWCS_AR_NVRAMManager_Ext-3601] This parameter defines the request result evaluated by NvM in case of quick initialization for user blocks which are configured not to participate in NvM_ReadAll: - true = NvM evaluates request result as NVM_REQ_OK - false = NvM evaluates request result as NVM_REQ_BLOCK_SKIPPED
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

5.3.1.87 NvM/NvMCommon/NvMRbRandomNrLength

Short Name	NvMRbRandomNrLength
Long Name	Random number length

Description	[BSW_SWCS_AR_NVRAMManager_Ext-3253] Defines the length of random number in bytes used for user data encryption / authentication.
Multiplicity	0..1
Value Range	0..65535
Default Value	16
Type	ECUC-INTEGER-PARAM-DEF

5.3.1.88 NvM/NvMCommon/NvMRbRedundantBlockArEnable

Short Name	NvMRbRedundantBlockArEnable
Long Name	Redundant block handling AR
Description	Enables the redundant block handling according to AUTOSAR. Parameter is not yet active and cannot be configured.
Multiplicity	0..0
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

5.3.1.89 NvM/NvMCommon/NvMRbRemoveNonResistantBlocks

Short Name	NvMRbRemoveNonResistantBlocks
Long Name	Remove non-resistant blocks
Description	[BSW_SWCS_AR_NVRAMManager_Ext-2787] Enables/disables enhanced dynamic configuration. If enabled and if the configured configuration ID is different from the configuration ID stored on the memory media, NV blocks which are configured to be non-resistant against SW changes (parameter NvMResistantToChangedSw set to false) are removed from the media directly after NvM_ReadAll has finished. Afterwards, the changed configuration ID is immediately written to the memory media before serving any other NvM request. This feature is only available if dynamic configuration is enabled (see parameter NvMDynamicConfiguration). Caution: if the enhanced dynamic configuration is enabled, all of the single block operations will be neither accepted nor executed before the ReadAll is called, if the block is as ReadAll block configured and is not resistant to changed software.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

5.3.1.90 NvM/NvMCommon/NvMRbReqMonDevId

Short Name	NvMRbReqMonDevId
Long Name	Device ID for request monitoring
Description	[BSW_SWCS_AR_NVRAMManager_Ext-3141] This parameter defines the device where the NVRAM block for request monitoring will be located.
Multiplicity	0..1
Value Range	0..254
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

5.3.1.91 NvM/NvMCommon/NvMRbReqMonitor

Short Name	NvMRbReqMonitor
Long Name	Request monitoring
Description	[BSW_SWCS_AR_NVRAMManager_Ext-3124] This parameter defines the request monitoring function for the NvM. NVM_RB_REQ_MON_DISABLED [Default]: Request monitoring disabled. NVM_RB_REQ_MON_WRITE: Monitoring of write requests enabled. In this case following counters are available: - NvM_Rb_cntrWrites_u16 = Number of write requests since last power-on, - NvM_Rb_nrBytesWritten_u32 = Number of bytes written since last power-on, - NvM_Rb_idLastWrittenBlock_uo = ID of the last written block, - NvM_Rb_WriteMon_st.nrBytesWrittenMax_u32 = maximum number of bytes written since last power-on, - NvM_Rb_WriteMon_st.cntrWritesAbs_u32 = accumulated number of all write requests, - NvM_Rb_WriteMon_st.cntrWritesMax_u16 = maximum number of write requests since last power-on
Multiplicity	0..1
Default Value	NVM_RB_REQ_MON_DISABLED
Enumeration Literals	NVM_RB_REQ_MON_DISABLED, NVM_RB_REQ_MON_WRITE
Type	ECUC-ENUMERATION-PARAM-DEF

5.3.1.92 NvM/NvMCommon/NvMRbSetWriteAllTriggerApi

Short Name	NvMRbSetWriteAllTriggerApi
Long Name	SetWriteAllTrigger API

Description	[BSW_SWCS_AR_NVRAMManager_Ext-2982] Enables/disables NvM_Rb_SetWriteAllTrigger API. This API allows to trigger WriteAll participation independent of RAM block states or whether a block is currently busy or not. If Dem is used, "NvMRbSetWriteAllTriggerApi" must be set to TRUE.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

5.3.1.93 NvM/NvMCommon/NvMRbSwcdToEcucValuesEnable

Short Name	NvMRbSwcdToEcucValuesEnable
Long Name	Enable SWCD to EcucValues translation
Description	On SWCD level it is possible to configure NvBlockNeeds. For such NvBlockNeeds it is required to configure a corresponding NvM block on EcucValues side. On enabling this switch the available information in SWCD is automatically translated to EcucValues. SWCD does support only a subset of pure AUTOSAR parameters. Additional information has to be provided on EcucValues level. Currently it is not possible to automatically calculate the mandatory parameters block length and the device ID from the SWCD configuration. The 2 parameters NvMNvBlockLength and NvMNvramDeviceId have to be provided additionally on EcucValues level.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

5.3.1.94 NvM/NvMCommon/NvMRbTimeoutCallback

Short Name	NvMRbTimeoutCallback
Long Name	Timeout Duration for Callback (normal)
Description	Timeout in 1 ms resolution for all callbacks in the system without considering repetitions (NvMRepeatMirrorOperations)
Multiplicity	0..1
Value Range	0..1000
Default Value	100
Type	ECUC-INTEGER-PARAM-DEF

5.3.1.95 NvM/NvMCommon/NvMRbTimeoutCallbackRepeat

Short Name	NvMRbTimeoutCallbackRepeat
Long Name	Timeout Duration for Callback (repetition)
Description	Timeout in 1 ms resolution for all callbacks in the system allowing repetitions for mirror-operations (NvMRepeatMirrorOperations). If no value is given then it is internally determined as NvMRbTimeoutCallback.+ NvMRepeatMirrorOperations * 20
Multiplicity	0..1
Value Range	0..1000
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

5.3.1.96 NvM/NvMDemEventParameterRefs

Short Name	NvMDemEventParameterRefs
Long Name	Dem event parameter references
Description	[ECUC_NvM_00541] Container for the references to DemEventParameter elements which are invoked using the Dem_ReportErrorStatus API in case the corresponding error occurs.
Multiplicity	0..1
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

5.3.1.97 NvM/NvMRbBlockDefault

Short Name	NvMRbBlockDefault
Long Name	Block Default Handling
Description	Handling structure for default parameters of BlockDescriptors (see NvMRbBlockDefaultValues) for a dedicated use-case scenario (see NvMRbBlockDefaultScenario)
Multiplicity	0..255
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

5.3.1.98 NvM/NvMRbBlockDefault/NvMRbBlockDefaultCfg

Short Name	NvMRbBlockDefaultCfg

Long Name	Block Default Cfg
Description	Configuration values of this scenario. The parameters here are not directly applicable for the NvMBlockDecriptor.
Multiplicity	1..1
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

5.3.1.99 NvM/NvMRbBlockDefault/NvMRbBlockDefaultCfg/NvMJobPrioritizationHigh

Short Name	NvMJobPrioritizationHigh
Long Name	Job sub-priority (HIGH)
Description	(only for SwC conversion) Definition of NvMJobPrioritization value if SwC selects HIGH. Note: The lower the value, the higher the priority. The value order is HIGH < =MED < = LOW.
Multiplicity	0..1
Value Range	1..255
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

5.3.1.100 NvM/NvMRbBlockDefault/NvMRbBlockDefaultCfg/NvMJobPrioritizationLow

Short Name	NvMJobPrioritizationLow
Long Name	Job sub-priority (LOW)
Description	(only for SwC conversion) Definition of NvMJobPrioritization value if SwC selects LOW. Note: The lower the value, the higher the priority. The value order is HIGH < =MED < = LOW.
Multiplicity	0..1
Value Range	1..255
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

5.3.1.101 NvM/NvMRbBlockDefault/NvMRbBlockDefaultCfg/NvMJobPrioritizationMedium

Short Name	NvMJobPrioritizationMedium
Long Name	Job sub-priority (MEDIUM)

Description	(only for SwC conversion) Definition of NvMJobPrioritization value if SwC selects MEDIUM. Note: The lower the value, the higher the priority. The value order is HIGH < =MED < = LOW.
Multiplicity	0..1
Value Range	1..255
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

5.3.1.102 NvM/NvMRbBlockDefault/NvMRbBlockDefaultScenario

Short Name	NvMRbBlockDefaultScenario
Long Name	Block Default Scenario
Description	Scenario defines the use-case for which the block default handling is applied. The scenarios or applied in following general order of the Execution classes (ExecClass) 1) BLOCK (not AIII) 2) SERVICE (not AII), 3) SERVICE (AII), 4) BLOCK (AII)
Multiplicity	1..1
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

5.3.1.103 NvM/NvMRbBlockDefault/NvMRbBlockDefaultScenario/NvMRb-BlockDefaultAII

Short Name	NvMRbBlockDefaultAII
Long Name	Applicable for all blocks
Description	Is this scenario applicable for all blocks/services in a ExecClass? Default: false. If set to true, then NvMRbBlockDefaultRegex has to be empty.
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

5.3.1.104 NvM/NvMRbBlockDefault/NvMRbBlockDefaultScenario/NvMRb-BlockDefaultExecClass

Short Name	NvMRbBlockDefaultExecClass
Long Name	Scenario Execution Class
Description	Defines the source to which is checked within this scenario. It can be either a BLOCK(name) or a SERVICE(dependency name). Not all selections are possible in all use-cases.
Multiplicity	1..1

Default Value	-
Enumeration Literals	BLOCK, SERVICE
Type	ECUC-ENUMERATION-PARAM-DEF

5.3.1.105 NvM/NvMRbBlockDefault/NvMRbBlockDefaultScenario/NvMRb-BlockDefaultExecPos

Short Name	NvMRbBlockDefaultExecPos
Long Name	Scenario Execution Position
Description	Relative execution position (lower value is applied first) within ExecClass. For same values the alphabtic order of the scenario is applicable
Multiplicity	0..1
Value Range	0..999
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

5.3.1.106 NvM/NvMRbBlockDefault/NvMRbBlockDefaultScenario/NvMRb-BlockDefaultRegex

Short Name	NvMRbBlockDefaultRegex
Long Name	Regex for name (Block/service)
Description	List of different regular expression (RegEx) which are applied in this scenario. Example: Use .* for any character. Details about regular expressions (RegEx) can be found in the Internet e.g. https://pubs.open-group.org/onlinepubs/9699919799/basedefs/V1_chap09.html
Multiplicity	0..65535
Default Value	-
Type	ECUC-STRING-PARAM-DEF

5.3.1.107 NvM/NvMRbBlockDefault/NvMRbBlockDefaultScenario/NvMRb-BlockDefaultUseCase

Short Name	NvMRbBlockDefaultUseCase
Long Name	Scenario Use-Case
Description	Defines the generation use-case when the default scenario shall be applied. SWC shall be used if applicable when converting SwC definition into EcucValues.
Multiplicity	1..1
Default Value	-

Enumeration Literals	SWC
Type	ECUC-ENUMERATION-PARAM-DEF

5.3.1.108 NvM/NvMRbBlockDefault/NvMRbBlockDefaultValues

Short Name	NvMRbBlockDefaultValues
Long Name	Block Default Values
Description	Collection of all NvMBlockDescriptor parameters for which a default value is defined
Multiplicity	1..1
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

5.3.1.109 NvM/NvMRbBlockDefault/NvMRbBlockDefaultValues/NvMRbAutoBlockLength

Short Name	NvMRbAutoBlockLength
Long Name	Automatic block length
Description	Enables the automatic calculation of the block length during compile time. For the block size to be calculated automatically during compile time, at least one of the following parameters must be configured. The order in which these parameters are evaluated is: NvMRbNvBlockLengthString, NvMRamBlockDataAddress, NvMRomBlockDataAddress. TRUE: The configured block length (NvMNvBlockLength) is ignored, and the actual block length is determined during compilation. FALSE: The block length is determined by the configured block length (NvMNvBlockLength).
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

5.3.1.110 NvM/NvMRbBlockDefault/NvMRbBlockDefaultValues/NvMRbBackupBlock

Short Name	NvMRbBackupBlock
Long Name	Include block in Mem backup

Description	Defines that the block should be included in the block backup using rba_MemBckp component. Note: Feature is only available for Fee blocks and is not supporting the combination with variable length blocks (NvMNvBlockLength = 0). TRUE: Block is included in the backup. FALSE: Block is not included in the backup.
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

5.3.1.111 NvM/NvMRbBlockDefault/NvMRbBlockDefaultValues/NvMRbBlock-Version

Short Name	NvMRbBlockVersion
Long Name	Block version
Description	[BSW_SWCS_AR_NVRAMManager_Ext-3343] Defines the block version. If the version is configured for this block, it will be stored into the non-volatile memory with the user data. The user can enable this feature by setting this parameter to a non zero value, the value zero means the block has no block version. If the user did a major change in the internal structure of the block then the block version should be updated. If the version is configured for a block, during a read block request, if the configured version matches the read version then the read request is executed normally if not the default values will be used and if no default values are available then NvM provides no values.
Multiplicity	0..1
Value Range	0..255
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

5.3.1.112 NvM/NvMRbBlockDefault/NvMRbBlockDefaultValues/NvMRb-FirstReadDataMigration

Short Name	NvMRbFirstReadDataMigration
Long Name	First Read Data Migration

Description	[BSW_SWCS_AR_NVRAMManager_Ext-3027] Enables/disables First Read Data Migration feature for this block. When First Read Data Migration is enabled a read job is also performed when a missmatch between the configured NV block length and the block length on medium (EEPROM) occurs. In case the NV block length is smaller then the block on medium the remaining data are discarded. In case the NV block length is bigger then the block on medium the remaining data are filled with zero. If the feature is disabled and a length missmatch occurs the read job is not performed.
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

5.3.1.113 NvM/NvMRbBlockDefault/NvMRbBlockDefaultValues/NvMRbInit-BlockAtLayoutChange

Short Name	NvMRbInitBlockAtLayoutChange
Long Name	Initialize block in case of a layout change
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

Description: [BSW_SWCS_AR_NVRAMManager_Ext-2881] Defines whether the NvM block shall be initialized if a block layout change is detected. In such case, the block is firstly read during ReadAll. If the block couldn't be retrieved from the medium, initialization data (from the ROM block or via the initialization callback) is written to the medium after the execution of ReadAll.
 This configuration parameter is allowed to be set to "true" under the following conditions:
 (1) The common configuration parameter NvMRbInitAtLayoutChange is set to "true".
 (2) The block is native or redundant
 (3) The block has a standard priority
 (4) The block has a ROM block or an initialization callback
 (5) The block has a RAM block or is configured with explicit synchronization
 (6) The block is configured for ReadAll
 (7) Additionally, if dynamic configuration is enabled, then this block has to be resistant to sw change.
 Hint: The block is initialized (i.e. written to the medium after ReadAll) if a configuration ID mismatch is detected. Therefore, when configuring a block with this feature, a new compiled configuration ID should be configured via the NvMCompiledConfigId parameter (otherwise the block wouldn't be initialized).

5.3.1.114 NvM/NvMRbBlockDefault/NvMRbBlockDefaultValues/NvMRbNo-Fallback

Short Name	NvMRbNoFallback

Long Name	No fallback to the old value
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

Description: [BSW_SWCS_AR_NVRAMManager_Ext-2925] This tag activates or deactivates a fallback mechanism to older copies of the same block. It is only evaluated for blocks located on Fee and ignored for blocks located on Ea. By default, this feature is set to false which is the standard behavior for Fee blocks (i.e. older values of the same block are preferred over delivering no block data at all). In Ea, older values are never available by principle and therefore blocks located on Ea will behave as if this tag had been set to true, regardless of its actual setting. If this tag is set to true and the corresponding block is located on Fee, older values will not be returned if the latest block instance is completely corrupted. In this case, inconsistent state is reported instead. Caution: When enabling this feature for the first time, already existing instances of the corresponding block on the media might not be readable anymore. Additionally, some Fee configurations might not support this feature at all. Please refer to the Fee documentation for details.

5.3.1.115 NvM/NvMRbBlockDefault/NvMRbBlockDefaultValues/NvMRbRamInitUnconditional

Short Name	NvMRbRamInitUnconditional
Long Name	RAM init unconditional
Description	[BSW_SWCS_AR_NVRAMManager_Ext-3175] This parameter defines whether a permanent RAM block will be initialized unconditionally. - true: permanent RAM block will be initialized even if it has remained unchanged after last reset (common parameter NvMRbRamInitCheck = NVM_RAM_INIT_CHECK_QUICK) - false: permanent RAM block will be initialized dependent on the state of the parameter NvMRbRamInitCheck
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

5.3.1.116 NvM/NvMRbBlockDefault/NvMRbBlockDefaultValues/NvMRbResistantToLayoutRemoval

Short Name	NvMRbResistantToLayoutRemoval
Long Name	Resistant to layout removal

Description	[BSW_SWCS_AR_NVRAMManager_Ext-2783] Defines whether the NvM block is resistant to layout removal. Once configured and written, lower layers keep this block saved on the persistent media even when this block is not part of the layout anymore (i.e. when the configuration changes and this block is removed from the layout). In this case, this block is not accessible from NvM. If the configuration changes again and this block is reconfigured, access to this block is reenabled again. If dynamic configuration is active (via NvMDynamicConfiguration), then resistant-to-layout-removal blocks must be resistant-to-changed-SW (configurable via NvMResistantToChangedSw parameter).
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

5.3.1.117 NvM/NvMRbBlockDefault/NvMRbBlockDefaultValues/NvMRbSelectBlockForFirstInitAll

Short Name	NvMRbSelectBlockForFirstInitAll
Long Name	Initialize block during FirstInitAll
Description	[BSW_SWCS_AR_NVRAMManager_Ext-2828] Defines whether a NvM block shall be processed during NvM_Rb_FirstInitAll or not. This configuration parameter is allowed to be set in the following cases: (i) The NvM block (is native or redundant) and (has a ROM block or an InitblockCallback) and (has a permanent RAM block or is configured to use explicit synchronization mechanism) (ii) The NvM block (is native or redundant) and (has no ROM block and no InitBlockCallback). In this case the NvM block is simply invalidated by the NvM_Rb_FirstInitAll job. (iii) The NvM block is of type NVM_BLOCK_DATASET. In this case, all the NV blocks of the NvM block are invalidated by the NvM_Rb_FirstInitAll job.
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

5.3.1.118 NvM/NvMRbBlockDefault/NvMRbBlockDefaultValues/NvMRbWriteCounter

Short Name	NvMRbWriteCounter

Long Name	Write Counter
Description	[BSW_SWCS_AR_NVRAMManager_Ext-3296] This parameter defines whether the write counter is enabled/disabled for a NVRAM block. Attention: When write counter is enabled the size of data stored in non-volatile memory changes (block-size = user-data-size + write-counter-size) with all its implications on backwards-compatibility!
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

5.3.1.119 NvM/NvMRbRamInitAddSavedZone

Short Name	NvMRbRamInitAddSavedZone
Long Name	RAM Init - Additional SavedZone Names
Description	[BSW_SWCS_AR_NVRAMManager_Ext-3441] Container for the reference from NvMRbRamInitAddSavedZone-NameRef. For each additional SAVEDZONE a container can be added. (see also NvMRbRamInitAddSavedZone-NameRef)
Multiplicity	0..65535
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

5.3.1.120 NvM/NvMRbRamInitAddSavedZone/NvMRbRamInitAddSavedZone-Name

Short Name	NvMRbRamInitAddSavedZoneName
Long Name	RAM Init - additional savedzone name extension
Description	[BSW_SWCS_AR_NVRAMManager_Ext-3443] For the additional reset marker the pragma NVM_{START/STOP}_SEC_VAR_SAVED_ZONE_<NvMRbRamInitAddSavedZoneName>_32 is used. This parameter NvMRbRamInitAddSavedZoneName is used as {refinement} as defined for the SEC_VAR_SAVED_ZONE in the AUTOSAR_SWS_MemoryMapping. (see also NvMRbRamInitAddSavedZone-NameRef)
Multiplicity	1..1
Default Value	-
Type	ECUC-STRING-PARAM-DEF

5.3.1.121 NvM/NvMRbSecurityEventParameterRefs

Short Name	NvMRbSecurityEventParameterRefs
Long Name	Security event parameter references
Description	Container for the references to IdsMEventParameter elements representing the security events that the NvM module shall report to the IdsM in case the coresponding security related event occurs.
Multiplicity	0..1
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

5.3.1.122 NvM/NvMRbSecurityEventParameterRefs/NvMRbSecurityEventIntegrityFailed

Short Name	NvMRbSecurityEventIntegrityFailed
Long Name	Security event “integrity failed”
Description	Reference to the IdsMEventParameter which is invoked in case an inconsistent block is encountered on the memory media.
Multiplicity	0..1
Default Value	-
Type	ECUC-SYMBOLIC-NAME-REFERENCE-DEF

5.3.1.123 NvM/NvMRbSecurityEventParameterRefs/NvMRbSecurityEventLossOfRedundancy

Short Name	NvMRbSecurityEventLossOfRedundancy
Long Name	Security event “loss of redundancy”
Description	Reference to the IdsMEventParameter which is invoked in case a loss of redundancy is detected.
Multiplicity	0..1
Default Value	-
Type	ECUC-SYMBOLIC-NAME-REFERENCE-DEF

5.3.1.124 NvM/NvMRbSecurityEventParameterRefs/NvMRbSecurityEventQueueOverflow

Short Name	NvMRbSecurityEventQueueOverflow
Long Name	Security event “queue overflow”

Description	Reference to the IdsMEventParameter which is invoked in case a queue overflow occurs.
Multiplicity	0..1
Default Value	-
Type	ECUC-SYMBOLIC-NAME-REFERENCE-DEF

5.3.1.125 NvM/NvMRbSecurityEventParameterRefs/NvMRbSecurityEventRequestFailed

Short Name	NvMRbSecurityEventRequestFailed
Long Name	Security event “request failed”
Description	Reference to the IdsM EventParameter which is invoked in case a request fails.
Multiplicity	0..1
Default Value	-
Type	ECUC-SYMBOLIC-NAME-REFERENCE-DEF

5.3.1.126 NvM/NvMRbSecurityEventParameterRefs/NvMRbSecurityEventVerifyFailed

Short Name	NvMRbSecurityEventVerifyFailed
Long Name	Security event “verify failed”
Description	Reference to the IdsM EventParameter which is invoked in case a verify operation fails.
Multiplicity	0..1
Default Value	-
Type	ECUC-SYMBOLIC-NAME-REFERENCE-DEF

5.3.1.127 NvM/NvMRbSecurityEventParameterRefs/NvMRbSecurityEventWriteProtected

Short Name	NvMRbSecurityEventWriteProtected
Long Name	Security event “write protected”
Description	Reference to the IdsMEventParameter which is invoked in case a write operation to a write protected block is encountered.
Multiplicity	0..1
Default Value	-
Type	ECUC-SYMBOLIC-NAME-REFERENCE-DEF

5.3.1.128 NvM/NvMRbSecurityEventParameterRefs/NvMRbSecurityEventWrongBlockId

Short Name	NvMRbSecurityEventWrongBlockId
Long Name	Security event “wrong block ID”
Description	Reference to the IdsMEventParameter which is invoked in case a static block ID check fails
Multiplicity	0..1
Default Value	-
Type	ECUC-SYMBOLIC-NAME-REFERENCE-DEF

5.3.2 Additional Services

The following API are available in version v12.7.0 in addition to those defined in AR v22-11

5.3.2.1 NvM_Rb_GetBlockId

Syntax	`Std_ReturnType NvM_Rb_GetBlockId(uint16 PersistentId, NvM_BlockIdType *BlockIdPtr)`
Parameters	`PersistentId` The persistent ID of a block. `*BlockIdPtr` A pointer to return the block ID of the NvM block.
Description	Service to get the NvM block ID using the persistent ID input parameter.

5.3.2.2 NvM_Rb_GetActiveService

Syntax	`NvM_Rb_GetActiveService(uint8 *ServiceIdPtr)`
Parameters	`*ServiceIdPtr` A pointer to return the active service.
Description	Service to read the ID of the currently active service of the NVRAM manager.

5.3.2.3 NvM_Rb_GetMigrationResult

Syntax	`Std_ReturnType NvM_Rb_GetMigrjationResult(NvM_BlockIdType BlockId, NvM_Rb_MigrationResult_ten *MigrationResultPtr)`
Parameters	`BlockId` The ID of an NvM block. `*MigrationResultPtr` A pointer to return the result of the migration.
Description	Service to read the result of migration operation.

5.3.2.4 NvM_Rb_GetNvBlockLength

Syntax	`Std_ReturnType NvM_Rb_GetNvBlockLength(NvM_BlockIdType BlockId, uint16 *NvBlockLengthPtr)`

Parameters	`BlockId` The ID of an NvM block. `*NvBlockLengthPtr` A pointer to return the length in bytes of the NvM block.
Description	Service to get the NvM block ID using the persistent ID input parameter.

5.3.2.5 NvM_Rb_GetStatus

Syntax	`Std_ReturnType NvM_Rb_GetStatus(NvM_Rb_StatusType *StatusPtr)`
Parameters	`*StatusPtr` A pointer to return the status of the NVRAM Manager.
Description	Service to read the overall status of the NVRAM manager.

5.3.2.6 NvM_Rb_AuxGetDataIndex

Syntax	`NvM_Rb_AuxGetDataIndex(uint8 *DataIndexPtr)`
Parameters	`*DataIndexPtr` A pointer to return the data index.
Description	Service for getting the currently set DataIndex of a dataset NVRAM block

5.3.2.7 NvM_Rb_AuxGetErrorStatus

Syntax	`Std_ReturnType NvM_Rb_AuxGetErrorStatus(NvM_RequestResultType *RequestResultPtr)`
Parameters	`*RequestResultPtr` A pointer to return the error/status of a block.
Description	Service to read the block dependent error/status information

5.3.2.8 NvM_Rb_AuxSetDataIndex

Syntax	`Std_ReturnType NvM_Rb_AuxSetDataIndex(uint8 DataIndex)`
Parameters	`DataIndex` The data index to set.
Description	Service for setting the DataIndex of a dataset NVRAM block

5.3.2.9 NvM_Rb_AuxGetMigrationResult

Syntax	`Std_ReturnType NvM_Rb_AuxGetMigrationResult(NvM_Rb_MigrationResult_ten *MigrationResultPtr)`
Parameters	`*MigrationResultPtr` A pointer to return the result of a migration.
Description	Service to read the result of AUX migration operation

5.3.2.10 NvM_Rb_GetBlockIdCausingLastDetError

Syntax	`NvM_BlockIdType NvM_Rb_GetBlockIdCausingLastDetError(void)`
Parameters	None

Description	Returns the block ID used in NvM service which has caused the last DET error.

5.3.2.11 NvM_Rb_ClrWriteMonCntrs

Syntax	`NvM_Rb_ClrWriteMonCntrs`
Parameters Out	`NvM_Rb_WriteMon_st` Structure to store write request counters in persistent way. `NvM_Rb_idLastWrittenBlock_uo` ID of the last written block. `NvM_Rb_nrBytesWritten_u32` Number of bytes written since last power-on. `NvM_Rb_cntrWrites_u16` Number of write requests since last power-on.
Description	This function clears all write monitoring counters.

5.3.2.12 NvM_Rb_GetBlockType

Syntax	`SNvM_Rb_GetBlockType(NvM_BlockIdType BlockId)`
Parameters	`BlockId` Block ID for which the type of NVRAM storage shall be returned.
Description	This function returns the type of NVRAM storage for a given block.

5.3.2.13 NvM_Rb_IsInitAtLayoutChangeEnqueued

Syntax	`NvM_Rb_IsInitAtLayoutChangeEnqueued`
Returns	`TRUE` Initialization at layout change is enqueued and NOT in progress. `FALSE` Initialization at layout change is NOT enqueued or is already in progress.
Description	This function returns whether the initialization at layout change is enqueued into the queue multi-block requests but is not yet in progress.

5.3.2.14 NvM_Rb_IsBlockSelectedForInitAtLayoutChange

Syntax	`NvM_Rb_IsBlockSelectedForInitAtLayoutChange(NvM_BlockIdType BlockId)`
Parameters	`BlockId` Block ID for which the selection status for initialization at layout change shall be returned.
Returns	`TRUE` Given block is selected for initialization at layout change. `FALSE` Given block is NOT selected for initialization at layout change.
Description	This function returns whether a block is selected for initialization at layout change.

5.3.2.15 NvM_Rb_IsBlockSelectedForFirstInitAll

Syntax	`NvM_Rb_IsBlockSelectedForFirstInitAll(NvM_BlockIdType BlockId)`
Parameters	`BlockId` Block ID for which the selection status for first initialization shall be returned.

Returns	`TRUE` Given block is selected for first initialization. `FALSE` Given block is NOT selected for first initialization.
Description	This function returns whether a block is selected for first initialization.

5.3.2.16 NvM_Rb_ReadBlockBase

Syntax	`Std_ReturnType NvM_Rb_ReadBlockBase(NvM_BlockIdType BlockId, void *NvM_DstPtr)`
Parameters	`BlockId` The ID of an NvM block. `*NvM_DstPtr` A pointer in RAM to read the NvM block to.
Description	Service to support the overloading of `NvM_ReadBlock()`

5.3.2.17 NvM_Rb_WriteBlockBase

Syntax	`Std_ReturnType NvM_WriteBlock(NvM_BlockIdType BlockId, const void *NvM_SrcPtr)`
Parameters	`BlockId` The ID of an NvM block. `*BlockIdPtr` A pointer in RAM to write the NvM block from.
Description	Service to support the overloading of `NvM_WriteBlock()`

5.3.2.18 NvM_Rb_RestoreBlockDefaultsBase

Syntax	`Std_ReturnType NvM_Rb_RestoreBlockDefaultsBase(NvM_BlockIdType BlockId, void *NvM_DstPtr)`
Parameters	`BlockId` The ID of an NvM block. `*NvM_DstPtr` A pointer to RAM for the location to restore the data to.
Description	Service to support the overloading of `NvM_RestoreBlockDefaults()`

5.3.2.19 NvM_Rb_EraseNvBlockBase

Syntax	`Std_ReturnType NvM_Rb_EraseNvBlockBase(NvM_BlockIdType BlockId)`
Parameters	`BlockId` The ID of an NvM block.
Description	Service to support the overloading of `NvM_EraseNvBlock()`

5.3.2.20 NvM_Rb_InvalidateNvBlockBase

Syntax	`Std_ReturnType NvM_Rb_InvalidateNvBlockBase(NvM_BlockIdType BlockId)`
Parameters	`BlockId` The ID of an NvM block.
Description	Service to support the overloading of `NvM_InvalidateNvBlock()`

5.3.2.21 NvM_Rb_AuxInvalidateNvBlock

Syntax	`Std_ReturnType NvM_Rb_AuxInvalidateNvBlock(NvM_BlockIdType BlockId)`
Parameters	`BlockId` The ID of an NvM block.
Description	Service to invalidate a NV block

5.3.2.22 NvM_Rb_AuxInvalidateNvBlockBase

Syntax	`Std_ReturnType NvM_Rb_AuxInvalidateNvBlockBase(NvM_BlockIdType BlockId)`
Parameters	`BlockId` The ID of an NvM block.
Description	Service to invalidate a NV block

5.3.2.23 NvM_Rb_AuxReadBlock

Syntax	`Std_ReturnType NvM_Rb_AuxReadBlock(NvM_BlockIdType BlockId, void *NvM_DstPtr)`
Parameters	`BlockId` The ID of an NvM block. `*NvM_DstPtr` A pointer to a location in RAM to read the NV block to.
Description	Service to copy the data of the NV block to its corresponding RAM block

5.3.2.24 NvM_Rb_AuxReadBlockBase

Syntax	`Std_ReturnType NvM_Rb_AuxReadBlockBase(NvM_BlockIdType BlockId, void *NvM_DstPtr)`
Parameters	`BlockId` The ID of an NvM block. `*NvM_DstPtr` A pointer to a location in RAM to read the NV block to.
Description	Service to copy the data of the NV block to its corresponding RAM block

5.3.2.25 NvM_Rb_AuxWriteBlock

Syntax	`Std_ReturnType NvM_Rb_AuxWriteBlock(NvM_BlockIdType BlockId, const void *NvM_SrcPtr)`
Parameters	`BlockId` The ID of an NvM block. `*NvM_SrcPtr` A pointer to a location in the non-volatile memory to write the data to.
Description	Service to copy the data of the RAM block to its corresponding NV block

5.3.2.26 NvM_Rb_AuxWriteBlockBase

Syntax	`Std_ReturnType NvM_Rb_AuxWriteBlockBase(NvM_BlockIdType BlockId, const void *NvM_SrcPtr)`

Parameters	`BlockId` The ID of an NvM block. `*NvM_SrcPtr` A pointer to a location in the non-volatile memory to write the data to.
Description	Service to copy the data of the RAM block to its corresponding NV block

5.3.2.27 NvM_Rb_ReadAllBase

Syntax	`void NvM_Rb_ReadAllBase(void)`
Parameters	None
Description	Service to support the overloading of `NvM_ReadAll()`

5.3.2.28 NvM_Rb_WriteAllBase

Syntax	`void NvM_Rb_WriteAllBase(void)`
Parameters	None
Description	Service to support the overloading of `NvM_WriteAll()`

5.3.2.29 NvM_Rb_FirstInitAll

Syntax	`void NvM_Rb_FirstInitAll(void)`
Parameters	None
Description	Service to trigger a multi block (re-) initialization operation

5.3.2.30 NvM_Rb_FirstInitAllBase

Syntax	`void NvM_Rb_FirstInitAllBase(void)`
Parameters	None
Description	Service to trigger a multi block (re-) initialization operation

5.3.2.31 NvM_Rb_ValidateAllBase

Syntax	`void NvM_Rb_ValidateAllBase(void)`
Parameters	None
Description	Service to initiate a multiblock validation request

5.3.2.32 NvM_Rb_ReadPRAMBlockBase

Syntax	`Std_ReturnType NvM_Rb_ReadPRAMBlockBase(NvM_BlockIdType BlockId)`
Parameters	`BlockId` ID of the block to be read from the corresponding RAM block.

Description	Invokes the private function NvM_Prv_ReadBlock to read a NVRAM-block into the configured permanent RAM block from the non-volatile memory within the overloaded API for NvM_ReadBlock. Requires the configuration of an overloaded API for the request to read the configured permanent RAM block from the non-volatile memory. You are responsible for invoking this function within the overloaded API.

5.3.2.33 NvM_Rb_WritePRAMBlockBase

Syntax	`Std_ReturnType NvM_Rb_WritePRAMBlockBase(NvM_BlockIdType BlockId)`
Parameters	`BlockId` ID of the block to be read from the corresponding RAM block.
Description	Invokes the private function NvM_Prv_WriteBlock to write a NVRAM-block from the configured permanent RAM block into the non-volatile memory within the overloaded API for NvM_WritePRAMBlock. Requires the configuration of an overloaded API for the request to write the configured permanent RAM block into the non-volatile memory. You are responsible for invoking this function within the overloaded API.

5.3.2.34 NvM_Rb_RestorePRAMBlockDefaultsBase

Syntax	`Std_ReturnType NvM_Rb_RestorePRAMBlockDefaultsBase(NvM_BlockIdType BlockId)`
Parameters	`BlockId` ID of the block to be restored into the corresponding RAM block.
Description	Invokes the private function NvM_Prv_RestoreBlockDefaults to restore default data into the permanent RAM block within the overloaded API for NvM_RestorePRAMBlockDefaults. Requires the configuration of an overloaded API for the request to restore default data into the permanent RAM block. You are responsible for invoking this function within the overloaded API.

5.3.2.35 NvM_Csm_Callback

Syntax	`void NvM_Csm_Callback(uint32 jobId, Std_ReturnType result)`
Parameters In	`jobId` Csm job ID for which the Csm has invoked this callback function. `result` Csm job result, possible values depend on the Csm job type.
Description	Defines the callback for Csm jobs and sets corresponding results.

Notes	This function stores the Csm job result and the Csm job ID for later evaluation in a static variable (s. NvM_Prv_Csm_JobResult_st). Typically a Csm job invokes a callback function if it finishes and this function enables the Csm job callback to set the result and execute all required actions. To ensure the result's consistency, this function writes Csm job results with an active scheduler lock.

5.3.2.36 NvM_Rb_GetWriteCounter

Syntax	`Std_ReturnType NvM_Rb_GetWriteCounter(NvM_BlockIdType idBlock_uo, uint32* cntrWrite_puo)`
Parameters	`BlockId` ID of the block for which the write counter shall be returned. `cntrWrite_puo` Pointer to the buffer for write counter.
Description	Copies the write counter for the given NVRAM block into the provided buffer.

5.3.2.37 NvM_Rb_MainFunctionAndDependency

Syntax	`void NvM_Rb_MainFunctionAndDependency(void)`
Description	Calls Nvm MainFunction and dependencies.

5.3.2.38 NvM_Rb_AuxGetErrorInfoDetail

Syntax	`Std_ReturnType NvM_Rb_AuxGetErrorInfoDetail(NvM_Rb_ErrorInfoDetail_ten *result_pen)`
Description	Reads error information details after a failing read of auxiliary user.

5.3.2.39 NvM_Rb_GetErrorInfoDetail

Syntax	`Std_ReturnType NvM_Rb_GetErrorInfoDetail(NvM_BlockIdType BlockId, NvM_Rb_ErrorInfoDetail_ten *result_pen)`
Description	Reads error information details after a failing read.

5.3.2.40 NvM_Rb_AuxSetDataIndexBase

Syntax	`Std_ReturnType NvM_Rb_AuxSetDataIndexBase(uint8 DataIndex)`

5.3.2.41 NvM_Rb_GetErrorStatusBase

Syntax	`Std_ReturnType NvM_Rb_GetErrorStatusBase(NvM_BlockIdType BlockId, NvM_RequestResultType *RequestResultPtr)`

5.3.2.42 NvM_Rb_SetDataIndexBase

Syntax	`Std_ReturnType NvM_Rb_SetDataIndexBase(NvM_BlockIdType BlockId, uint8 DataIndex)`
Parameters In	`BlockId` ID of the block for which dataset index will be set. `DataIndex` Required dataset index.
Description	Allows you to set dataset index within the overloaded API for NvM_SetDataIndex.

5.3.2.43 NvM_Rb_SetBlockProtectionBase

Syntax	`Std_ReturnType NvM_Rb_SetBlockProtectionBase(NvM_BlockIdType BlockId, boolean ProtectionEnabled)`

5.3.2.44 NvM_Rb_SetRamBlockStatusBase

Syntax	`Std_ReturnType NvM_Rb_SetRamBlockStatusBase(NvM_BlockIdType BlockId, boolean BlockChanged)`

5.3.2.45 NvM_Rb_SetWriteAllTrigger

Syntax	`Std_ReturnType NvM_Rb_SetWriteAllTrigger(NvM_BlockIdType BlockId, boolean WriteAllTrigger)`
Parameters In	`BlockId` ID of the block for which participation in multi-block write will be triggered. `WriteAllTrigger` Required state for WriteAll-Trigger (TRUE/FALSE).
Description	Initiates a single-block request to trigger participation in multi-block write for a NVRAM-block.

5.3.2.46 NvM_Rb_SetWriteAllTriggerBase

Syntax	`Std_ReturnType NvM_Rb_SetWriteAllTriggerBase(NvM_BlockIdType BlockId, boolean WriteAllTrigger)`

5.3.2.47 NvM_Rb_SetBlockLockStatusBase

Syntax	`void NvM_Rb_SetBlockLockStatusBase(NvM_BlockIdType BlockId, boolean BlockLocked)`
Parameters In	`BlockId` ID of the block for which lock status will be set. `BlockLocked` Required state for lock status.
Description	Provided for the overloading of NvM_SetBlockLockStatus.

5.3.2.48 NvM_Rb_AuxGetErrorStatusBase

Syntax	`Std_ReturnType NvM_Rb_AuxGetErrorStatusBase(NvM_RequestResultType *RequestResultPtr)`
Parameters	`*RequestResultPtr` A pointer to return the error/status of a block.
Returns	`E_OK` `E_NOT_OK`
Description	Supports API overloading for NvM_Rb_AuxGetErrorStatus.

5.3.2.49 NvM_Rb_GetBlockErrors

Syntax	`Std_ReturnType NvM_Rb_GetBlockErrors(NvM_BlockIdType idBlock_uo, NvM_Rb_BlockErrors_tuo* BlockErrors_puo )`
Parameters	`idBlock_uo` ID of the block for which production error is reported. `BlockErrors_puo` Variable saving blocks' errors.
Returns	`E_OK` Read request into the configured permanent RAM block has been initiated successfully. `E_NOT_OK` Initiation of the read request into the configured permanent RAM block has failed.
Description	This function returns the block errors.

5.3.2.50 NvM_Rb_EnterHighSpeedMode

Syntax	`Std_ReturnType NvM_Rb_EnterHighSpeedMode(void)`
Returns	`E_OK` NvM and all configured memory devices are in high-speed-mode. `E_NOT_OK` NvM and all configured memory devices are NOT in high-speed-mode.
Description	A public function that enters the high-speed-mode if possible.

5.3.2.51 NvM_Rb_ExitHighSpeedMode

Syntax	`Std_ReturnType NvM_Rb_ExitHighSpeedMode(void)`
Returns	`E_OK` NvM and all configured memory devices are NOT in high-speed-mode anymore. `E_NOT_OK` It is not possible to exit high-speed-mode because it was not active before.
Description	A public function that exits the high-speed-mode.

5.3.2.52 NvM_Rb_CancelWriteAllBase

Syntax	`void NvM_Rb_CancelWriteAllBase(void)`
Description	Support API overloading for NvM_CancelWriteAll.

5.3.2.53 NvM_Rb_GetBlockIdCausingLastDetErrorInstance

Syntax	`NvM_BlockIdType NvM_Rb_GetBlockIdCausingLastDetErrorInstance(uint8 InstanceId)`
Parameters	`InstanceId` As reported to Det.
Returns	Block identifier used in the NvM service which caused the last DET error.
Description	Returns the block ID used in the NvM service which caused the last DET error.

5.3.2.54 NvM_Rb_SetCryptoInitVector

Syntax	`Std_ReturnType NvM_Rb_SetCryptoInitVector(void)`
Returns	`E_OK` Initialization-vector generation has been initiated successfully. `E_NOT_OK` Initiation of the initialization-vector generation has failed.
Description	Initiates the initialization-vector generation to be used for cryptographic services.

5.3.2.55 NvM_Rb_GetStatusCryptoInitVector

Syntax	`Std_ReturnType NvM_Rb_GetStatusCryptoInitVector(NvM_RequestResultType* stCryptoInitVector_puo)`
Parameters	`NVM_REQ_OK` Initialization-vector generation initiated and completed successfully.
In/Out	`NVM_REQ_NOT_OK` Initialization-vector generation initiated but failed. `NVM_REQ_PENDING` Initialization-vector generation pending while initiated by NvM_ReadAll via NvM_Rb_SetCryptoInitVector. `NVM_RB_REQ_NOT_INITIALIZED` Initialization-vector generation not yet initiated.
Description	Returns the status of the initialization-vector to be used for cryptographic services.

5.4 Limitations

The following features are not supported for version v12.7.0

- NvM does not support the AUTOSAR configuration parameter NvMBuffer-AlignmentValue. Relevant buffers can be allocated using the MemMap section NVM_START_SEC_VAR_SECURED_CACHED_RAM_ALIGNED_-CLEARED_UNSPECIFIED. Affected SWS items: ECUC_NvM_00573, SWS_N-vM_00985.
- The erase job does not currently erase data, but instead internally performs the same actions as the invalidate job. Affected SWS items: NVM415.
- NvM does not itself currently support write verification. However, the current implementations of both Ea/Eep and Fee can do a write verification on lower layers. It has yet to be determined whether or not this lower layer

support is sufficient to meet the overall write verification goals. Affected SWS items: BSW_SWS_AR4_0_R2_NVRAMManager-4415, NVM527, NVM528, NVM529, NVM530, NVM594, NVM534_Conf, NVM538_Conf, NVM545_Conf.

- NvM does not currently retry failed read/write requests automatically, so if this is desired for particular NvM blocks, the owners of these blocks need to re-issue these requests themselves. Affected SWS items: write retries: NVM168, NVM213, NVM659, NVM216, NVM296, NVM683, NVM684, NVM499_Conf, read retries: NVM526, NVM581, NVM582, NVM583, NVM533_Conf.
- Static ID checking is currently not supported. It has yet to be determined whether or not this feature will improve safety levels when using NvM together with the current Ea/Eep and Fee implementations. Affected SWS items: NVM125, NVM126, NVM522, NVM523, NVM524, NVM525, NVM580, BSW_SWS_AR4_0_R2_NVRAMManager-4405, NVM593, NVM532_Conf, NVM544_Conf.
- NvM currently only supports 32 bit processors. Affected SWS items: BSW006.
- NvM currently does not support the reduction of the NvM code size based on the amount of required/configured features. Consequently, NvM might use more ECU resources than would be necessary to provide only the NvM features actually required by a particular project. Affected SWS items: NVM149, NVM561, NVM562, NVM365, NVM491_Conf, NVM734, NVM150, NVM344, NVM345, NVM408, NVM734.
- NvM currently does not support block locking via the NvM_SetBlockLock-Status service. It only provides a stub of this function without any functionality. This service will mainly be used by Dcm to keep other NvM users from overwriting block data changed by test equipment. Affected SWS items: NVM548, NVM732.
- Interaction of NvM with the lower layer drivers is currently done by cyclic polling in NvM's main function. AUTOSAR specifies a callback interface between NvM and low level drivers, but this has not been implemented as there is some doubt as to whether switching to a callback interface would yield a significant performance boost. Affected SWS items: NVM548, NVM732.
- NvM currently does not support the Dynamic Configuration feature to its fullest extent. The configuration ID (on which this feature is based) is not yet automatically generated. Affected SWS items: NVM249, NVM308, NVM309, NVM034, NVM310.
- The AUTOSAR specification states that NvM shall give a summary of used memory resources based on the current configuration, but this feature is not currently implemented. Affected SWS items: NVM052.
- The current implementation of NvM does not yet support Redundant Block Handling and Read/Write retries at the NvM level. Although blocks can be configured as type "redundant" NvM will not save two copies. This limitation

is only relevant for projects that do not use the full RTA-MEM stack - if the full stack is used, Redundant Block Handling and double storage is handled by the lower level software. (Ea/Fee).

- The NVM_E_WRITE_PROTECTED development error has the value 0x20 in the current implementation of NvM, but in older versions NVM_E_WRITE_PROTECTED has been used as an error bit mask. This usage of NVM_E_WRITE_PROTECTED as an error bit mask has now been deprecated and NVM_E_RB_WRITE_PROTECTED is used instead. For the time being, NvM will use NVM_E_WRITE_PROTECTED as a development error as well as an error bit mask, and to avoid backward compatibility issues, the value of NVM_E_WRITE_PROTECTED remains as 0x20. A future implementation of NvM will use NVM_E_WRITE_PROTECTED only as a development error (with value 0x1B), as defined in AR v19-11. Affected SWS items: SWS_NvM_00961.

- The identification concept of blocks throughout the Mem stack is different from what has been defined by the NvM specification. This is an interface-neutral deviation which results in a more resource efficient implementation and simplified configuration. Affected SWS items: NVM122, NVM123, NVM343, NVM494_Conf, NVM478_Conf, NVM481_Conf.
- The identification concept of devices throughout the Mem stack is slightly different from what has been defined by the NVM specification. Device ID 0 always addresses Fee, while device IDs from 1 onwards address EEPROMs. This is done to simplify the configuration and provide a leaner implementation without sacrificing functionality. Affected SWS items: NVM035_Conf.
- The identifier of an NvM block is not reflected back into the configuration if it is automatically generated (i.e. instead of being maintained manually). Affected SWS items: ecuc_sws_2107.
- NvM automatically generates Ea/Fee block configuration based on NvM block configuration, so references from NvM block configurations to Ea/Fee block configurations are superfluous and have been removed from NvM's ECU parameter definition file (by setting the upper multiplicity to zero). Affected SWS items: NVM486_Conf, NVM487_Conf, NVM488_Conf, NVM489_Conf, NVM490_Conf.
- A single-block service is allowed to be called/queued for a redundant block whose only pending operation is a maintenance operation. No PENDING error is reported to DET. This is an exception to NVM385, which prohibits the queueing of a single-block request for a block which is already queued or currently in progress. Affected SWS items: NVM385, NVM587, NVM598, NVM607, NVM615, NVM620, NVM626, NVM632, NVM639, NVM644, NVM729.
- The activation of BswM notification for state changes of multi-block operations (via the NvMBswMMultiBlockJobStatusInformation configuration parameter) is optional instead of mandatory. This is to ensure a backward compatibility to the already delivered NvM software which is based on the AR 4.0.2 specification. The default value for this parameter is "false" instead of "true". Affected SWS items (from Autosar revision 4.0.3): NVM550_Conf.

- Triggering Dem events for production errors is not implemented because block-independent information about such errors is not generally of any use for production error tracking. Instead, it is the responsibility of NvM users to trigger appropriate application-specific Dem events wherever applicable. For debugging purposes, NvM provides block-specific flags indicating comparable error conditions, as well as block-independent summary information on these errors.
- The AUTOSAR specification for NvM suggests unique SWCD interfaces per block. The RTA-BSW implementation of NvM generates a single "superset" NvM Service interface rather than unique ones for each port. This is the most compatible approach because a single well-defined interface name can be used (e.g. "NvMService") and "smaller" interfaces on the R-Port side are always compatible to this "superset" interface as long as the operations themselves are compatible. Affected SWS items: BSW_SWS_AR_NVRAM-Manager-8855, BSW_SWS_AR_NVRAMManager-8878, BSW_SWS_AR_N-VRAMManager-8882.
- The AUTOSAR specification suggests that P-Ports and R-Ports shall only be defined in the generated SWCD if there is a counterpart in the project. This cannot be automatically determined, however, so the creation of P-Ports and R-Ports are controlled instead by configuration parameters. Affected SWS items: BSW_SWS_AR4_0_R2_NVRAMManager-8965, BSW_SWS_AR_NVRAMManager-8971, BSW_SWS_AR_NVRAMMan-ager-8977.
- The CRC type in RAM can be different from the CRC type on the persistent storage media. Affected SWS items: NVM127, NVM476_Conf.
- The AUTOSAR specification suggests that a write operation of a redundant block shall be successful if only one NV block is successfully written, but in this RTA-BSW implementation of NvM the write operation of an NVRAM block of type NVM_BLOCK_REDUNDANT is successful when both NV blocks were written successfully. Affected SWS items: NVM284, NVM337, NVM760, NVM761.
- The NvMRepeatMirrorOperations parameter, which specifies the maximum number of retries for copying data from/to NvM mirror in case of explicit synchronization, has a lower multiplicity of 0. If none of the blocks is configured to use explicit synchronization, then this parameter doesn't have to be defined. Affected SWS items: NVM518_Conf.
- The AUTOSAR specification describes switching Eep/Ea or Fee to fast mode while multi-block operations are ongoing. The performance improvement provided by this feature is heavily dependent on hardware properties and the configuration of the low level drivers (Eep and Fls). In many situations, this improvement is quite limited, so NvM does not switch to fast mode during multi-block operations. Affected SWS items: NVM426, NVM427, NVM430, NVM431, NVM681, NVM496_Conf, NVM501_Conf.
- Only PreCompile time configuration is supported, not LinkTime configuration. Affected SWS items: NVM447, NVM727, NVM880, NVM881.
- The lower multiplicity of the NvM module is '1' instead of '0'.

- The job of the function NvM_ReadAll does not set each proceeding block specific request result in advance. Affected SWS items: BSW_SWS_AR_NVRAMManager-5582, NVM304.
- To ease the transition to port usage, the NvMBlockUsePort parameter is optional rather than mandatory, and has a default of FALSE. This parameter should nevertheless be set according to the specific project needs, and should not be left undefined. Affected SWS items: ECUC_NVM_00559.
- NvM currently implements block ciphering as defined an AR v20-11 with the following deviations:
  - If data encryption has succeeded (CRYPTO_E_OK returned) but the length returned by the Csm job is different to the NvMNvBlockNVRAMDataLength, then NvM aborts the write job and reports the development error NVM_E_BLOCK_CIPHER_LENGTH_MISMATCH. Affected SWS items: SWS_NvM_00979.
  - If data decryption fails, NvM sets the read request result to NVM_REQ_INTEGRITY_FAILED. Affected SWS items: SWS_NvM_00981, SWS_NvM_00982. If data decryption has succeeded (CRYPTO_E_OK returned), NvM does not check the returned length. Affected SWS items: SWS_NvM_00983.
  - The lower multiplicity for all parameters in container NvMBlockCiphering is set to 0. Affected SWS items: ECUC_NvM_00569, ECUC_NvM_00571, ECUC_NvM_00570.
- NvM is reporting a Runtime-Error when a queue is full. In standard configuration, this error will be reported even if development errors are deactivated. However, in the multi-partition environment, the runtime-error reporting only happens if the development error reporting is activated. This is because the Det is not yet prepared for a multi-partition environment for series. Affected SWS items: SWS_NvM_00948.

5.5 Exclusions

The following features from AR v22-11 are not supported in version v12.7.0.

API functions not currently implemented:

- `NvM_CancelJobs`

The following configuration parameters from AR v22-11 are not supported in version v12.7.0.

- NvM/NvMBlockDescriptor/NvMBswMBlockStatusInformation
- NvM/NvMBlockDescriptor/NvMInitBlockCallback/NvMInitBlockCallbackFnc
- NvM/NvMBlockDescriptor/NvMNvBlockBaseNumber
- NvM/NvMBlockDescriptor/NvMSelectBlockForFirstInitAll
- NvM/NvMBlockDescriptor/NvMTargetBlockReference
- NvM/NvMBlockDescriptor/NvMTargetBlockReference/NvMEaRef

- NvM/NvMBlockDescriptor/NvMTargetBlockReference/NvMEaRef/NvMName-OfEaBlock
- NvM/NvMBlockDescriptor/NvMTargetBlockReference/NvMFeeRef
- NvM/NvMBlockDescriptor/NvMTargetBlockReference/NvMFeeRef/NvMNameOfFeeBlock
- NvM/NvMCommon/NvMBswMMultiBlockJobStatusInformation
- NvM/NvMCommon/NvMBufferAlignmentValue
- NvM/NvMCommon/NvMDatasetSelectionBits
- NvM/NvMDemEventParameterRefs/NVM_E_INTEGRITY_FAILED
- NvM/NvMDemEventParameterRefs/NVM_E_LOSS_OF_REDUNDANCY
- NvM/NvMDemEventParameterRefs/NVM_E_REQ_FAILED
- NvM/NvMDemEventParameterRefs/NVM_E_VERIFY_FAILED
- NvM/NvMDemEventParameterRefs/NVM_E_WRITE_PROTECTED
- NvM/NvMDemEventParameterRefs/NVM_E_WRONG_BLOCK_ID
- NvM/NvmDemEventParameterRefs
- NvM/NvmDemEventParameterRefs/NVM_E_HARDWARE

5.6 Deviations

Configuration parameters which differ from the AR v22-11 specification are listed below.

5.6.1 NvM

[RTA-BSW] Description	[ECUC_NVM_00539] Configuration of the NvM (Non volatile RAM Manager) module.
[AUTOSAR] Description	Configuration of the NvM (NvRam Manager) module.
[RTA-BSW] Lower Multiplicity	1
[AUTOSAR] Lower Multiplicity	0

5.6.2 NvM/NvMBlockCiphering

[RTA-BSW] Description	[ECUC_NvM_00568] Container for a ciphering of the block.
[AUTOSAR] Description	Container for a chiphering of the Block. Tags: atp.Status=draft

5.6.3 NvM/NvMBlockCiphering/NvMCsmDecryptionJobReference

[RTA-BSW] Description	[ECUC_NvM_00571] This parameter references a CSM decrypt job used to decrypt ciphered data after reading it from NV RAM. If the Csm job reference uses AEAD algorithm then NvMRbTagLength shall be defined. Parameters NvMRbAssociatedDataLength and NvMRbAssociatedDataPtr shall be defined only if individual associated data is required.
[AUTOSAR] Description	This parameter references a CSM decrypt job used to decrypt ciphered data after reading it from NV RAM. Tags: atp.Status=draft
[RTA-BSW] Lower Multi-plicity	0
[AUTOSAR] Lower Multi-plicity	1

5.6.4 NvM/NvMBlockCiphering/NvMCsmEncryptionJobReference

[RTA-BSW] Description	[ECUC_NvM_00570] This parameter references a CSM encrypt job used to encrypt plain data before writing it to the NV RAM. If the Csm job reference uses AEAD algorithm then NvMRbTagLength shall be defined. Parameters NvMRbAssociatedDataLength and NvMRbAssociatedDataPtr shall be defined only if individual associated data is required.
[AUTOSAR] Description	This parameter references a CSM encrypt job used to encrypt plain data before writing it to the NV RAM. Tags: atp.Status=draft
[RTA-BSW] Lower Multi-plicity	0
[AUTOSAR] Lower Multi-plicity	1

5.6.5 NvM/NvMBlockCiphering/NvMNvBlockNVRAMDataLength

[RTA-BSW] Description	[ECUC_NvM_00569] This value specified the block length in case ciphered data is stored in NV RAM - in that case the plain data length may not match the ciphered data length. This value needs be filled out, if the ciphering, encryption and decryption, of the data is enabled for this block. It will be used instead of the NvMNvBlockLength to access the NV RAM. Hint: This value can be equal or bigger than 'NvMNvBlockLength' depending on the characteristics of the referenced CSM Job inside 'NvMNameOfEncryptionJob' (e.g. CSM job adds padding information).
[AUTOSAR] Description	This value specified the block length in case ciphered data is stored in NV RAM - in that case the plain data length may not match the ciphered data length. Tags: atp.Status=draft
[REMOVED] Introduction	This value needs be filled out, if the ciphering, encryption and decryption, of the data is enabled for this block. It will be used instead of the NvMNvBlockLength to access the NV RAM. Hint: This value can be equal or bigger than 'NvMNvBlockLength' depending on the characteristics of the referenced CSM Job inside 'NvMNameOfEncryptionJob' (e.g. CSM job adds padding information).
[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1
[REMOVED] Default Value	16

5.6.6 NvM/NvMBlockDescriptor

[RTA-BSW] Description	[ECUC_NVM_00061] Container for all block-specific configuration parameters. For each NvM block, one instance of this container shall be specified.
[AUTOSAR] Description	Container for a management structure to configure the composition of a given NVRAM Block Management Type. Its multiplicity describes the number of configured NVRAM blocks, one block is required to be configured. The NVRAM block descriptors are condensed in the NVRAM block descriptor table.
[RTA-BSW] Multiplicity	2..65535
[AUTOSAR] Multiplicity	1..65536

5.6.7 NvM/NvMBlockDescriptor/NvMBlockCipheringRef

[RTA-BSW] Description	[ECUC_NvM_00567] Reference to ciphering container. If configured, NvM encrypt the data before storage and decrypt the data after restoring. If empty, the NvM stores and restore the original user data.
[AUTOSAR] Description	Reference to ciphering container. Tags: atp.Status=draft
[REMOVED] Introduction	If configured, NvM encrypt the data before storage and decrypt the data after restoring. If empty, the NvM stores and restore the original user data.

5.6.8 NvM/NvMBlockDescriptor/NvMBlockCrcType

[RTA-BSW] Description	[ECUC_NVM_00476] Defines the CRC data width for the NVRAM block. Due to Autosar this parameter shall define the CRC data width for data in NV memory and for data in permanent RAM block. Currently this parameter defines the CRC data width only for data in permanent RAM block (see parameter NvMCalcRamBlockCrc). For data in NV memory the CRC will be always calculated by lower layers independently from this parameter.
[AUTOSAR] Description	Defines CRC data width for the NVRAM block. Default: NVM_CRC16, i.e. CRC16 will be used if NVM_BLOCK_USE_CRC==true
[RTA-BSW] Enumeration Literals	NVM_CRC8, NVM_CRC16, NVM_CRC32
[AUTOSAR] Enumeration Literals	NVM_CRC16, NVM_CRC32, NVM_CRC8
[ADDED] Default Value	NVM_CRC16

5.6.9 NvM/NvMBlockDescriptor/NvMBlockEcucPartitionRef

[RTA-BSW] Description	[ECUC_NvM_00564] Maps the NV block to zero or one ECUC partition to limit the access to this NV block. The ECUC partition referenced is within the subset of the ECUC partitions where the NvM is mapped to.
[AUTOSAR] Description	Maps the NV block to zero or one ECUC partition to limit the access to this NV block. The ECUC partition referenced is within the subset of the ECUC partitions where the NvM is mapped to.

5.6.10 NvM/NvMBlockDescriptor/NvMBlockHeaderInclude

[RTA-BSW] Description: [BSW_SWCS_AR_NVRAMManager_Ext-2997] Defines a

header file that is included. Only the header file name without the extension has to be entered here. For example “MyHeader” instead of “MyHeader.h”. If a header file is defined the declarations of the RAM data block (NvMRamBlockDataAddress) and the ROM data block (NvMRomBlockDataAddress) and all used callback-functions have to be privided within this header-file by the user. The inclusion of the header file is needed for the two mutually exclusive features: 1. Compile time check of RAM and ROM block length. 2. Auto Block Length for “non RTE blocks”. (“RTE blocks” do not need the include as Rte_Type.h is always inlcuded.) Compile Time Check: If a header file is defined for a specific NvM block, a compile time check is carried out to check if the configured NvM block length (NvMNvBlockLength) matches the size of the data given in in the RAM data block (NvMRamBlockDataAddress) and the ROM data block (NvMRomBlockDataAddress). This check is done using the sizeof operator and causes a compile time error in case of a mismatch. Note that the compile time check just can be carried out if for the NvM block the following two conditions are fulfilled: 1. The values entered in the RAM data block (NvMRamBlockDataAddress) and the ROM data block (NvMRomBlockDataAddress) are names of a linker symbol, pointers or subelements of data structures declared in the header (e.g. MyVar, MyStruct.MyMember, MyArray[3], &MyVar, …) but are not addresses in hexadecimal notation (e.g. 0x8000A000). 2. Auto Block Length is not active for this block.

[AUTOSAR] Description: Defines the header file where the owner of the NVRAM block has the declarations of the permanent RAM data block, ROM data block (if configured) and the callback function prototype for each configured callback. If no permanent RAM block, ROM block or callback functions are configured then this configuration parameter shall be ignored.

5.6.11 NvM/NvMBlockDescriptor/NvMBlockJobPriority

[RTA-BSW] Description: [NVM477_Conf] Defines the job priority for a NVRAM block. The current version of NvM supports two job priorities:
 0 : Immediate priority,
 1-255 : Standard priority. (higher value leads to lower prio)
 If no explicit standard priority is required then the priority 255 can be used, so that other blocks can define higher priority. If priority jobs handling is disabled then all blocks are handled with the same priority, i.e. in FCFS order.
 Immediate priority can be enabled only if priority jobs handling is enabled via the NvMJobPrioritization configuration paramter.
 Immediate priority blocks shall participate neither in multi-block-read (see also NvMSelectBlockForReadAll) nor in multi-block-write (see also NvMSelectBlockForWriteAll) operations. If an immediate block is selected for ReadAll or WriteAll, then it is not possible to queue an immediate write request for this block while the multiblock operation is being executed (as the state of this block would be PENDING (NVM385)).

[AUTOSAR] Description: Defines the job priority for a NVRAM block (0 = Immediate priority).

5.6.12 NvM/NvMBlockDescriptor/NvMBlockManagementType

[RTA-BSW] Description	[ECUC_NVM_00062] Defines the block management type for the NVRAM block. An NvM block can be a native, a redundant or a dataset block.
[AUTOSAR] Description	Defines the block management type for the NVRAM block-.[SWS_NvM_00137]
[RTA-BSW] Enumeration Literals	NVM_BLOCK_NATIVE, NVM_BLOCK_REDUNDANT, NVM_BLOCK_DATASET
[AUTOSAR] Enumeration Literals	NVM_BLOCK_DATASET, NVM_BLOCK_NATIVE, NVM_BLOCK_REDUNDANT

5.6.13 NvM/NvMBlockDescriptor/NvMBlockUseAutoValidation

[RTA-BSW] Description	[ECUC_NvM_00557] Defines whether a NVRAM block shall be processed during NvM_ValidateAll or not. This configuration parameter is only allowed to be set for NVRAM blocks which are configured to have a permanent RAM block or which are configured to use explicit synchronization mechanism.
[AUTOSAR] Description	Defines whether the RAM Block shall be auto validated during shutdown phase.
[REMOVED] Introduction	true: if auto validation mechanism is used, false: otherwise
[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1

5.6.14 NvM/NvMBlockDescriptor/NvMBlockUseCRCCompMechanism

[RTA-BSW] Description	[ECUC_NvM_00556] Defines whether the CRC of the permanent RAM Block shall be compared during a write job with the CRC which was calculated during the last successful read or write job. This parameter can be set to true only if parameter NvMCalcRamBlockCrc is also true.
[AUTOSAR] Description	Defines whether the CRC of the RAM Block shall be compared during a write job with the CRC which was calculated during the last successful read or write job.
[REMOVED] Introduction	true: if compare mechanism is used, false: otherwise
[RTA-BSW] Lower Multiplicity	0

[AUTOSAR] Lower Multiplicity	1

5.6.15 NvM/NvMBlockDescriptor/NvMBlockUseCompression

[RTA-BSW] Description	[ECUC_NvM_00563] Defines whether the data is compressed before written. RTA-CAR NvM provides currently block split as compression type. The user has also to define the size of block splits if using compression, see also container NvMRbSplitBlock. - true: data compression activated (takes more time to read and write) - false: no compression
[AUTOSAR] Description	Defines whether the data is compressed before written. Tags: atp.Status=draft
[REMOVED] Introduction	true: data compression activated (takes more time to read and write) false: no compression
[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1

5.6.16 NvM/NvMBlockDescriptor/NvMBlockUseCrc

[REMOVED] Introduction	true: CRC will be used for this NVRAM block. false: CRC will not be used for this NVRAM block. Note: Configuring CRC for a block with immediate priority is not recommended, since the CRC calculation may extend over more than one NvM main function and this could increase the time of writing the immediate data significantly, thus defeating the purpose of immediate priority.

[RTA-BSW] Description: [ECUC_NVM_00036] Enables/disables CRC usage for the NVRAM block in NV memory. If enabled a CRC is calculated before a write operation which is then stored in the NV memory along with the block data. During a read operation this CRC is then checked again to ensure data integrity. Independant of parameter NvMBlockUseCrc the MemStack always uses CRCs to ensure data integrity, but this is done on lower software layers (not NVM level) and it is not possible to deactive it. Due to this reason the parameter NvMBlockUseCrc has to be globally enabled by setting the common parameter NvMRbNvCrcEnable to TRUE if a CRC also should be used at NVM level. Just if both parameters NvMBlockUseCrc and NvMRbNvCrcEnable are set to true the CRC will be used for this block on NVM level. Hint: For the case that the complete MemStack this does not affect any CRC useage on lower layers. Hint: If not the complete MemStack is used then the common parameter NvMRbNvCrcEnable should be set to TRUE to ensure that CRCs are used at NVM level.

[AUTOSAR] Description: Defines CRC usage for the NVRAM block, i.e. memory space for CRC is reserved in RAM and NV memory.

5.6.17 NvM/NvMBlockDescriptor/NvMBlockUsePort

[RTA-BSW] Description	[ECUC_NvM_00559] Defines whether NvMMirror, NvMNotifyInitBlock, NvMNotifyJobFinished, NvMAdmin ports shall be generated for this block. In cases where callbacks have been configured for this block, this configuration parameter also controls the creation of the corresponding R-Ports. For blocks which are accessed via RTE ports this parameter shall be set to true. In case you provide the include header via parameter NvMBlockHeaderInclude, then it is assumed that the callback prototypes are also available in this header. NvM does no longer generate the related prototypes.
[AUTOSAR] Description	If this parameter is true it defines whether:
[REMOVED] Introduction	* the port with interface 'NvMMirror' for synchronization mechanism callbacks are generated if the parameter NvMBlockUseSyncMechanism is configured TRUE; * the port with interface 'NvMNotifyInitBlock' for initialization block callback is generated if NvMInitBlockCallback parameter is configured (independent of the content); * the port with interface 'NvMNotifyJobFinished' for single block callback is generated if NvMSingleBlockCallback parameter is configured (independent of the content); * the port with interface 'NvMAdmin' for SetBlockProtection operation is generated.
[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1
[ADDED] Default Value	false

5.6.18 NvM/NvMBlockDescriptor/NvMBlockUseSetRamBlockStatus

[RTA-BSW] Description	[ECUC_NvM_00552] Defines if NvMSetRamBlockStatusApi shall be used for this block or not. Note: If NvMSetRamBlockStatusApi is disabled this configuration parameter shall be ignored. true: calling of NvMSetRamBlockStatus for this RAM block shall set the status of the RAM block. false: calling of NvMSetRamBlockStatus for this RAM block shall be ignored.
[AUTOSAR] Description	Defines if NvMSetRamBlockStatusApi shall be used for this block or not.

[REMOVED] Introduction	Note: If NvMSetRamBlockStatusApi is disabled this configuration parameter shall be ignored. true: calling of NvMSetRamBlockStatus for this RAM block shall set the status of the RAM block. false: calling of NvMSetRamBlock-Status for this RAM block shall be ignored.
[RTA-BSW] Lower Multi-plicity	0
[AUTOSAR] Lower Multi-plicity	1
[ADDED] Default Value	true

5.6.19 NvM/NvMBlockDescriptor/NvMBlockUseSyncMechanism

[RTA-BSW] Description	[ECUC_NVM_00519] Enables/disables the explicit synchronization mechanism with a RAM mirror and callback routines for transferring data to and from NvM module's RAM mirror.
[AUTOSAR] Description	Defines whether an explicit synchronization mechanism with a RAM mirror and callback routines for transferring data to and from NvM module's RAM mirror is used for NV block. true if synchronization mech-anism is used, false otherwise.

5.6.20 NvM/NvMBlockDescriptor/NvMBlockWriteProt

[RTA-BSW] Description	[ECUC_NVM_00033] Enables/disables an initial write protection of the NV block.
[AUTOSAR] Description	Defines an initial write protection of the NV block
[REMOVED] Introduction	true: Initial block write protection is enabled. false: Initial block write protection is disabled.

5.6.21 NvM/NvMBlockDescriptor/NvMCalcRamBlockCrc

[RTA-BSW] Descrip-tion	[ECUC_NVM_00119] Enables/disables the CRC (re)calculation for the permanent RAM block or NVRAM blocks which are configured to use explicit synchronization mechanism.
[AUTOSAR] Descrip-tion	Defines CRC (re)calculation for the permanent RAM block or NVRAM blocks which are configured to use explicit synchronization mechanism.
[REMOVED] Introduc-tion	true: CRC will be (re)calculated for this permanent RAM block. false: CRC will not be (re)calculated for this permanent RAM block.
[ADDED] Default Value	false

5.6.22 NvM/NvMBlockDescriptor/NvMInitBlockCallback

[RTA-BSW] Description	[ECUC_NvM_00556] Container for callback function
[AUTOSAR] Description	The presence of this container indicates, that a block specific callback routine is called if no
[REMOVED] Introduction	ROM data is available for initialization of the NVRAM block. If the container is not present, no callback routine is called for initialization of the NVRAM block with default data. In case the container has a NvMInit-BlockCallbackFnc, the NvM will call this function. In case there is no NvMInitBlockCallbackFnc, the NvM will have an port PNIB_{Block}.

5.6.23 NvM/NvMBlockDescriptor/NvMMaxNumOfReadRetries

[RTA-BSW] Description	[ECUC_NVM_00533] Defines the maximum number of read retries. Read retries are implemented for MemIf failure and NvCrc failure. Not for Static Block ID check. If NvM is used with RTA-CAR Fee and Ea than this value should be 0. The RTA-CAR Fee and Ea perform already retries. The failure reaction can be more sophisticated on Fee/Ea level, because the retries can there depend on the failure type.
[AUTOSAR] Description	Defines the maximum number of read retries.

5.6.24 NvM/NvMBlockDescriptor/NvMMaxNumOfWriteRetries

[RTA-BSW] Description	[ECUC_NVM_00499] Defines the maximum number of write retries. Write retries a implemented for MemIf failure. Not for Write Verification (Read Back). If NvM is used with RTA-CAR Fee and Ea than this value should be 0. The RTA-CAR Fee and Ea perform already retries. The failure reaction can be more sophisticated on Fee/Ea level, because the retries can there depend on the failure type.
[AUTOSAR] Description	Defines the maximum number of write retries for a NVRAM block with [ECUC_NvM_00061]. Regardless of configuration a consistency check (and maybe write retries) are always forced for each block which is processed by the request NvM_WriteAll and NvM_WriteBlock.

5.6.25 NvM/NvMBlockDescriptor/NvMNvBlockLength

[RTA-BSW] Description	[ECUC_NVM_00479] Defines the NV block data length in bytes. (Hint: NvM keeps an internal reserve of 128 bytes, so the maximum block length is 65535-128=65407 bytes.)
[AUTOSAR] Description	Defines the NV block data length in bytes.

[REMOVED] Introduction	Note: The implementer can add the attribute 'withAuto' to the parameter definition which indicates that the length can be calculated by the generator automatically (e.g. by using a parser that searches and analyzes the data structure corresponding to the block). When 'withAuto' is set to 'true' for this parameter definition the 'isAutoValue' can be set to 'true'. If 'isAutoValue' is set to 'true' the actual value will not be considered during ECU Configuration but will be (re-)calculated by the code generator and stored in the value attribute afterwards.
[RTA-BSW] Maximum Value	65407
[AUTOSAR] Maximum Value	65535

5.6.26 NvM/NvMBlockDescriptor/NvMNvBlockNum

[RTA-BSW] Description	[ECUC_NVM_00480] Defines the number of multiple NV blocks in a contiguous area according to the given block management type. The allowed number of NV blocks is 1 for a native block, 2 for a redundant block and 1..255 for a dataset block. Additionally, the following condition has to be satisfied: NvMNvBlockNum + NvMRomBlockNum <= 255 [NVM444]
[AUTOSAR] Description	Defines the number of multiple NV blocks in a contiguous area according to the given block management type.
[REMOVED] Introduction	1-255 For NVRAM blocks to be configured of block management type NVM_BLOCK_DATASET. The actual range is limited according to SWS_NvM_00444. 1 For NVRAM blocks to be configured of block management type NVM_BLOCK_NATIVE 2 For NVRAM blocks to be configured of block management type NVM_BLOCK_REDUNDANT

5.6.27 NvM/NvMBlockDescriptor/NvMNvramBlockIdentifier

[RTA-BSW] Description	[ECUC_NVM_00481] Identification of a NVRAM block via a unique block identifier which is used at the NvM API functions (not towards the memory hardware abstraction and also not on the media). The value 0 is reserved for multi block related operations and their results. The value 1 is reserved for the block containing the configuration ID used as part of the dynamic configuration feature. The identifiers of all blocks must be contiguous, i.e. there must not be any gaps in the sequence of block identifiers. All multi block operations operate in an ascending ID sequence, which means that blocks with lower IDs are read and written earlier than blocks with higher IDs. If you don't explicitly specify an identifier for a block, such an identifier is automatically assigned by NvM.
[AUTOSAR] Description	Identification of a NVRAM block via a unique block identifier.
[REMOVED] Introduction	Implementation Type: NvM_BlockIdType. min = 2 max = 2^(16- NVM_DATASET_SELECTION_BITS)-1 Reserved NVRAM block IDs: 0 → to derive multi block request results via NvM_GetErrorStatus 1 → redundant NVRAM block which holds the configuration ID (generation tool should check that this block is correctly configured from type,CRC and size point of view)
[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1
[RTA-BSW] Maximum Value	65534
[AUTOSAR] Maximum Value	65535

5.6.28 NvM/NvMBlockDescriptor/NvMNvramDeviceId

[RTA-BSW] Description	[ECUC_NVM_00035] Identification of the device where the NVRAM block is located. In contrast to AUTOSAR's NvM specification, this is not a parameter derived from the block configuration on the Ea/Fee module configuration, but the device identifier is directly specified here (and the Ea/Fee block configuration is completely auto-generated). Attention: If the NVRAM block shall be located on a Ea device then user shall use the Ea device ID defined in the container NvMRbEaDevices. If ID used here does not correspond to an ID defined in the container NvMRbEaDevices then the NVRAM block will not be located on a Ea device. Attention: Device ID 0 is fixed to address Fee. If Fee is used then all corresponding blocks shall use device ID 0. If there are more than 1 Fee then first Fee (typically Fee for internal flash) shall use device ID 0 and all other Fees can use other device IDs.
[AUTOSAR] Description	Defines the NVRAM device ID where the NVRAM block is located.
[REMOVED] Introduction	Calculation Formula: value = TargetBlockReference.[Ea/Fee]DeviceIndex
[RTA-BSW] Maximum Value	254
[AUTOSAR] Maximum Value	1

5.6.29 NvM/NvMBlockDescriptor/NvMRamBlockDataAddress

[REMOVED] Introduction	If this is not configured, no permanent RAM data block is available for the selected block management type.

[RTA-BSW] Description: [ECUC_NVM_00482] Defines the start address of permanent RAM block data. You may either give an address in hexadecimal notation here (e.g. 0x1234ABCD) or the name of a linker symbol (e.g. MyTestRamBlock) or the address (e.g. &MyTestRamBlock). If you explicitly want to specify that no permanent RAM block shall be used for this block, enter NULL_PTR here.
 The NvM users shall allocate each NvM RAM block in a RAM section called "saved zone". When using AUTOSAR mechanisms for memory allocation, the memory class <MODULE>_SEC_VAR_SAVED_ZONE_<SIZE> shall be used.
 Caution: Configuring the same permanent RAM for more than one NvM block is possible but highly not recommended. In fact, an owner of an NvM block sharing the same permanent RAM block with other NvM blocks has to be aware that the owners of those other blocks could also modify the content of the shared RAM block. Hence, a certain synchronization on read/write accesses, to the common RAM block, needs to be established between different NvM blocks'

owners sharing a common permanent RAM block.

[AUTOSAR] Description: Defines the start address of the RAM block data.

5.6.30 NvM/NvMBlockDescriptor/NvMReadRamBlockFromNvCallback

[REMOVED] Introduction	Implementation type: Std_ReturnType E_OK: copy was successful E_NOT_OK: copy was not successful, callback routine to be called again

[RTA-BSW] Description: [ECUC_NVM_00521] Name of the block-specific callback routine which shall be invoked in order to let the application copy data from the NvM module's mirror to RAM block (see also the NvMBlockUseSyncMechanism parameter). If you explicitly want to specify that no such callback routine shall be called, enter NULL_PTR here.
 For a user located above RTE and if NvMRbGenRteServicePort is set to TRUE, this callback has to be left empty in order to generate the callback automatically. The signature of the callback-function has to be: Std_ReturnType CallbackName(void* NvMBuffer). It is not allowed to call any NvM or DEM functions from within this callback function! Runtime shall be minimized. Note: If parameter NvMBlockHeaderInclude is defined, then NvM does not create prototype declarations but expects that the prototype definition is within that header.

[AUTOSAR] Description: Entry address of a block specific callback routine which shall be called in order to let the application copy data from the NvM module's mirror to RAM block.

5.6.31 NvM/NvMBlockDescriptor/NvMResistantToChangedSw

[RTA-BSW] Description	[ECUC_NVM_00483] Defines whether a NVRAM block shall be treated as resistant to configuration changes or not (see also the NvMCompiledConfigId and NvMDynamicConfiguration parameters for a description of this mechanism).
[AUTOSAR] Description	Defines whether a NVRAM block shall be treated resistant to configuration changes or not. If there is no default data available at configuration time then the application shall be responsible for providing the default initialization data. In this case the application has to use NvM_GetErrorStatus()to be able to distinguish between first initialization and corrupted data.
[REMOVED] Introduction	true: NVRAM block is resistant to changed software. false: NVRAM block is not resistant to changed software.

5.6.32 NvM/NvMBlockDescriptor/NvMRomBlockDataAddress

[RTA-BSW] Description	[ECUC_NVM_00484] Defines the start address of ROM block data. You may either give an address in hexadecimal notation here (e.g. 0x2345BCDE) or the name of a linker symbol (e.g. MyTestRomBlock) or the address (e.g. &MyTestRomBlock). If you explicitly want to specify that no ROM block shall be available for this block, enter NULL_PTR here. The NvM users shall allocate each NvM ROM block in the so called "saved recovery zone". When using AUTOSAR mechanisms for memory allocation, the memory class <MODULE>_SEC_CONST_SAVED_RECOVERY_ZONE_<SIZE> shall be used.
[AUTOSAR] Description	Defines the start address of the ROM block data.
[REMOVED] Introduction	If not configured, no ROM block is available for the selected block management type.

5.6.33 NvM/NvMBlockDescriptor/NvMRomBlockNum

[RTA-BSW] Description	[ECUC_NVM_00485] Defines the number of multiple ROM blocks in a contiguous area according to the given block management type. The allowed number of ROM blocks is 0..1 for a native or a redundant block and 0..254 for a dataset block. Additionally, the following condition has to be satisfied: NvMNvBlockNum + NvMRomBlockNum <= 255 [NVM444]
[AUTOSAR] Description	Defines the number of multiple ROM blocks in a contiguous area according to the given block management type.
[REMOVED] Introduction	0-254 For NVRAM blocks to be configured of block management type NVM_BLOCK_DATASET. The actual range is limited according to SWS_NvM_00444. 0-1 For NVRAM blocks to be configured of block management type NVM_BLOCK_NATIVE 0-1 For NVRAM blocks to be configured of block management type NVM_BLOCK_REDUNDANT

5.6.34 NvM/NvMBlockDescriptor/NvMSelectBlockForReadAll

[RTA-BSW] Description	[ECUC_NVM_00117] Defines whether a NVRAM block shall be processed during NvM_ReadAll or not. This configuration parameter is only allowed to be set for NVRAM blocks which are configured to have a permanent RAM block or which are configured to use explicit synchronization mechanism. Additionally, dataset blocks (configurable via NvMBlockManagementType parameter) as well as immediate priority blocks (configurable via NvMBlockJobPriority parameter) are not allowed to be selected for NvM_ ReadAll.
[AUTOSAR] Description	Defines whether a NVRAM block shall be processed during NvM_ReadAll or not. This configuration parameter has only influence on those NVRAM blocks which are configured to have a permanent RAM block or which are configured to use explicit synchronization mechanism.
[REMOVED] Introduction	true: NVRAM block shall be processed by NvM_ ReadAll false: NVRAM block shall not be processed by NvM_ReadAll
[ADDED] Default Value	false

5.6.35 NvM/NvMBlockDescriptor/NvMSelectBlockForWriteAll

[RTA-BSW] Description	[ECUC_NVM_00549] Defines whether a NVRAM block shall be processed during NvM_WriteAll or not. This configuration parameter is only allowed to be set for NVRAM blocks which are configured to have a permanent RAM block or which are configured to use explicit synchronization mechanism.
[AUTOSAR] Description	Defines whether a NVRAM block shall be processed during NvM_ WriteAll or not. This configuration parameter has only influence on those NVRAM blocks which are configured to have a permanent RAM block or which are configured to use explicit synchronization mechanism.
[REMOVED] Introduction	true: NVRAM block shall be processed by NvM_WriteAll false: NVRAM block shall not be processed by NvM_WriteAll
[ADDED] Default Value	false

5.6.36 NvM/NvMBlockDescriptor/NvMSingleBlockCallback

[RTA-BSW] Description	[ECUC_NvM_00562] Container for callback function

[AUTOSAR] Description	The presence of this container indicates, that the block specific callback routine which shall be invoked
[REMOVED] Introduction	on termination of each asynchronous single block request [SWS_NvM_00113] If the container is not present, no callback routine is called.. In case the container has a NvMSingleBlockCallbackFnc, the NvM will call this function. In case there is no NvMSingleBlockCallbackFnc, the NvM will have an port PNJF_{Block}.

5.6.37 NvM/NvMBlockDescriptor/NvMSingleBlockCallback/NvMSingleBlockCallbackFnc

[RTA-BSW] Description: [ECUC_NvM_00506] Name of the block-specific callback routine which shall be invoked on termination of each access to the actual memory media. If you explicitly want to specify that no such callback routine shall be called, enter NULL_PTR here.
 For a user located above RTE and if NvMRbGenRteServicePort is set to TRUE, this callback has to be left empty in order to generate the callback automatically. The signature of the callback-function has to be: For NvMRbCallbackSignatureAR45=false: Std_ReturnType CallbackName(uint8 ServiceId, NvM_RequestResultType JobResult). For NvMRbCallbackSignatureAR45=true: Std_ReturnType CallbackName(NvM_BlockRequestType BlockRequest, NvM_RequestResultType JobResult). It is not allowed to call any NvM or DEM functions from within this callback function! Runtime shall be minimized. Note: If parameter NvMBlockHeaderInclude is defined, then NvM does not create prototype declarations but expects that the prototype definition is within that header.

[AUTOSAR] Description: Entry address of the block specific callback routine which shall be invoked on termination of each asynchronous single block request [SWS_NvM_00113].

5.6.38 NvM/NvMBlockDescriptor/NvMStaticBlockIDCheck

[RTA-BSW] Description	[ECUC_NVM_00532] Enables/disables the static block ID check for this block. The current version of NvM does not support this feature, so this parameter has currently always to be set to false.
[AUTOSAR] Description	Defines if the Static Block ID check is enabled.
[REMOVED] Introduction	false: Static Block ID check is disabled. true: Static Block ID check is enabled.
[REMOVED] Default Value	false

5.6.39 NvM/NvMBlockDescriptor/NvMWriteBlockOnce

[REMOVED] Introduction	true: Defines write protection after first write is enabled. false: Defines write protection after first write is disabled.

[RTA-BSW] Description: [ECUC_NVM_00072] Enables/disables automatic write

protection of this block once it has been successfully read from or written to the memory media. This protects the block data to be overwritten once it has been successfully written. Please note that the fact that the block data has been successfully written is lost after NvM_Init; you typically recover this information by trying to read the block via NvM_ReadAll or NvM_ReadBlock before attempting any write/invalidate/erase operation on blocks configured to be written only once. An NvM block configured with the "Write Once" is allowed to be initialized at layout change (please see NvMRbInitBlockAtLayoutChange configuration parameter). If the block is initialized (i.e. its recovery data is written to the medium after ReadAll), attempting any write/invalidate/erase operation on this block will fail.

[AUTOSAR] Description: Defines write protection after first write. The NVRAM manager sets the write protection bit either after the NV block was written the first time or if the block was already written and it is detected as valid and consistent during a read for it.

5.6.40 NvM/NvMBlockDescriptor/NvMWriteRamBlockToNvCallback

[REMOVED] Introduction	Implementation type: Std_ReturnType E_OK: copy was successful E_NOT_OK: copy was not successful, callback routine to be called again

[RTA-BSW] Description: [ECUC_NVM_00520] Name of the block-specific callback routine which shall be invoked in order to let the application copy data from RAM block to the NvM module's mirror (see also the NvMBlockUseSyncMechanism parameter). If you explicitly want to specify that no such callback routine shall be called, enter NULL_PTR here.
 For a user located above RTE and if NvMRbGenRteServicePort is set to TRUE, this callback has to be left empty in order to generate the callback automatically. The signature of the callback-function has to be: Std_ReturnType CallbackName(void* NvMBuffer). It is not allowed to call any NvM or DEM functions from within this callback function! Runtime shall be minimized. Note: If parameter NvMBlockHeaderInclude is defined, then NvM does not create prototype declarations but expects that the prototype definition is within that header.

[AUTOSAR] Description: Entry address of a block specific callback routine which shall be called in order to let the application copy data from RAM block to NvM module's mirror.

5.6.41 NvM/NvMBlockDescriptor/NvMWriteVerification

[RTA-BSW] Description	[ECUC_NVM_00534] Enables/disables write verification. The current version of NvM does not support this feature, so this parameter has currently always to be set to false.
[AUTOSAR] Description	Defines if Write Verification is enabled.
[REMOVED] Introduction	false: Write verification is disabled. true: Write Verification is enabled.
[REMOVED] Default Value	false

5.6.42 NvM/NvMBlockDescriptor/NvMWriteVerificationDataSize

[RTA-BSW] Description	[ECUC_NVM_00538] Defines the number of bytes to compare in each step when comparing the content of a RAM Block and a block read back during write verification. This parameter is ignored if the write verification feature is disabled for this block.
[AUTOSAR] Description	Defines the number of bytes to compare in each step when comparing the content of a RAM Block and a block read back.
[RTA-BSW] Minimum Value	0
[AUTOSAR] Minimum Value	1

5.6.43 NvM/NvMCommon

[RTA-BSW] Description	[ECUC_NvM_00028] Container for common (i.e. not block-specific) configuration options.
[AUTOSAR] Description	Container for common configuration options.

5.6.44 NvM/NvMCommon/NvMApiConfigClass

[RTA-BSW] Description	[ECUC_NvM_00491] Allows to configure the feature set supported by the NvM service. Higher API classes mean more features at the expense of higher resource usage. Lower API classes mean less features but a smaller memory footprint. Please relate to the NvM documentation for a detailed description of which API class contains which set of features.
[AUTOSAR] Description	Preprocessor switch to enable some API calls which are related to NVM API configuration classes.
[RTA-BSW] Enumeration Literals	NVM_API_CONFIG_CLASS_3
[AUTOSAR] Enumeration Literals	NVM_API_CONFIG_CLASS_1, NVM_API_CONFIG_CLASS_2, NVM_API_CONFIG_CLASS_3

5.6.45 NvM/NvMCommon/NvMCompiledConfigId

[RTA-BSW] Description	[ECUC_NvM_00492] Configuration ID uniquely identifying the NV memory layout version. This ID is used for compatibility decisions during NvM_ReadAll if dynamic configuration feature of NvM is enabled (see also the NvMDynamicConfiguration parameter) or if the initialization of blocks at a layout change is enabled (see also the NvMRblnitAtLayoutChange parameter). This parameter is also used for the "initialization at layout change feature" (see also NvMRblnitBlockAtLayoutChange parameter). The configuration of the compiled ID is currently mandatory.
[AUTOSAR] Description	Configuration ID regarding the NV memory layout. This configuration ID shall be published as e.g. a SW-C shall have the possibility to write it to NV memory.

5.6.46 NvM/NvMCommon/NvMCrcNumOfBytes

[RTA-BSW] Description	[ECUC_NvM_00493] If CRC (re-) calculation is configured for at least one permanent RAM block, this parameter defines the maximum number of bytes which shall be processed within one cycle of job processing.
[AUTOSAR] Description	If CRC is configured for at least one NVRAM block, this parameter defines the maximum number of bytes which shall be processed within one cycle of job processing.

5.6.47 NvM/NvMCommon/NvMCsmRetryCounter

[RTA-BSW] Description	[ECUC_NvM_00572] This value specifies the number of CSM encryption/decryption job retry attempts. CSM jobs for block reading and writing may fail (e.g. module busy, queue full, …). To not directly abort the read/write with an error status, the NvM will retry the CSM job for the configured NvMCsmRetryCounter times. Configuring 0 means: no retry behavior; job will be aborted directly.
[AUTOSAR] Description	This value specifies the number of CSM encryption/decryption job retry attempts. Tags: atp.Status=draft
[REMOVED] Introduction	CSM jobs for block reading and writing may fail (e.g. module busy, queue full, …). To not directly abort the read/write with an error status, the NvM will retry the CSM job for the configured NvMCsmRetryCounter times. Configuring 0 means: no retry behavior; job will be aborted directly.

5.6.48 NvM/NvMCommon/NvMDevErrorDetect

[RTA-BSW] Description	[ECUC_NvM_00495] Enables/disables the detection and notification of development errors. If this parameter is enabled, then extended error checking is performed during runtime; any errors detected by these checks are reported to the Det module. This feature should be switched off in series software.
[AUTOSAR] Description	Switches the development error detection and notification on or off.
[REMOVED] Introduction	* true: detection and notification is enabled. * false: detection and notification is disabled.
[REMOVED] Default Value	false

5.6.49 NvM/NvMCommon/NvMDynamicConfiguration

[RTA-BSW] Description	[ECUC_NvM_00497] Enables/disables the dynamic configuration management feature. If enabled, the currently configured compiled configuration ID (see NvMCompiledConfigId parameter) is compared against the configuration ID stored on the memory media during NvM_ReadAll. If these ID's don't match, only the blocks marked as being resistant to SW changes (see also the NvMResistantToChangedSw parameter) are read from the memory media. If this parameter is disabled, NvM_ReadAll is processed without taking care of any possibly stored configuration ID.
[AUTOSAR] Description	Preprocessor switch to enable the dynamic configuration management handling by the NvM_ReadAll request.
[REMOVED] Introduction	true: Dynamic configuration management handling enabled. false: Dynamic configuration management handling disabled. This parameter affects all NvM processing related to Block with ID 1 and all processing related to Resistant to Changed Software. If the Dynamic Configuration is disabled, Block 1 cannot be used by NvM.

5.6.50 NvM/NvMCommon/NvMEcucPartitionRef

[RTA-BSW] Description	[ECUC_NvM_00565] Maps the NvM to one or multiple ECUC partitions to make its C-APIs available in the according partition.

[AUTOSAR] Description	Maps the NvM to one or multiple ECUC partitions to make its C-APIs available in the according partition.
[RTA-BSW] Multiplicity	0..65535
[AUTOSAR] Multiplicity	1..*

5.6.51 NvM/NvMCommon/NvMJobPrioritization

[RTA-BSW] Description	[ECUC_NvM_00498] Enables/disables job prioritization handling.
[AUTOSAR] Description	Preprocessor switch to enable job prioritization handling
[REMOVED] Introduction	true: Job prioritization handling enabled. false: Job prioritization handling disabled.

5.6.52 NvM/NvMCommon/NvMMainFunctionPeriod

[RTA-BSW] Description	[ECUC_NvM_00555] Defines the period between successive calls to the NvM main function in seconds. In case NvM main function is scheduled via RTE in a timed task the minimum value is 1.0e-4 [s] = 100us. In case the background scheduling is selected, set this value to 0. Note: The value given is rounded to 100us granularity. For details about the scheduling options, please refer to parameter NvMRbMainFunction-Scheduling.
[AUTOSAR] Description	The period between successive calls to the main function in seconds.
[REMOVED] Maximum Value	INF

5.6.53 NvM/NvMCommon/NvMMasterEcucPartitionRef

[RTA-BSW] Description	[ECUC_NvM_00566] Maps the NvM master to zero or one ECUC partition to assign the master functionality to a certain core. The ECUC partition referenced is a subset of the ECUC partitions where the NvM is mapped to.
[AUTOSAR] Description	Maps the NvM master to zero or one ECUC partition to assign the master functionality to a certain core. The ECUC partition referenced is a subset of the ECUC partitions where the NvM is mapped to.

5.6.54 NvM/NvMCommon/NvMMultiBlockCallback

[RTA-BSW] Description	[ECUC_NvM_00500] Name of the common callback routine which shall be invoked on termination of each asynchronous multi block request (NvM_Rb_FirstInitAll, NvM_ReadAll, removing of non-resistant to changed SW blocks or NvM_WriteAll). If you explicitly want to specify that no such callback routine shall be called, enter NULL_PTR here. The signature of the callback-function has to be: For NvMRbCallbackSignatureAR45=false: Std_ReturnType CallbackName(uint8 ServiceId, NvM_RequestResultType JobResult) For NvMRbCallbackSignatureAR45=true: Std_ReturnType CallbackName(NvM_MultiBlockRequestType MultiBlockRequest, NvM_RequestResultType JobResult).M114 It is not allowed to call any NvM or DEM functions from within this callback function!
[AUTOSAR] Description	Entry address of the common callback routine which shall be invoked on termination of each asynchronous multi block request

5.6.55 NvM/NvMCommon/NvMPollingMode

[RTA-BSW] Description	[ECUC_NvM_00501] Defines the way how NvM communicates to the memory media drivers. If set to true, the NvM_MainFunction polls the status from these drivers if some operation is ongoing. If set to false, the media drivers asynchronously report any changes in the operation status via dedicated callback functions provided by NvM. NvM does not support asynchronous notifications from the memory media drivers, so this parameter has currently always to be set to true.
[AUTOSAR] Description	Preprocessor switch to enable/disable the polling mode in the NVRAM Manager and at the same time disable/enable the callback functions useable by lower layers
[REMOVED] Introduction	true: Polling mode enabled, callback function usage disabled. false: Polling mode disabled, callback function usage enabled.

5.6.56 NvM/NvMCommon/NvMRepeatMirrorOperations

[RTA-BSW] Description	[ECUC_NvM_00518] Defines the number of retries to let the application copy data to or from the NvM module's mirror before postponing the current job. The lower multiplicity is set to 0 in contrast to NVM518_Conf. In fact, this parameter doesn't have to be configured if none of the blocks is configured to use explicit synchronization. See also the NvMBlockUseSyncMechanism parameter in the block configuration.

[AUTOSAR] Description	Defines the number of retries to let the application copy data to or from the NvM module's mirror before postponing the current job.
[RTA-BSW] Lower Multi-plicity	0
[AUTOSAR] Lower Multi-plicity	1

5.6.57 NvM/NvMCommon/NvMSetRamBlockStatusApi

[RTA-BSW] Description	[ECUC_NvM_00502] Enables/disables the NvM_SetRamBlockStatus API. If enabled, NvM_WriteAll only writes blocks to memory which have been explicitly marked as changed by using this API. If disabled, this API is not available and NvM_WriteAll writes blocks to memory regardless of whether they have changed or not.
[AUTOSAR] Description	Preprocessor switch to enable the API NvM_SetRamBlockStatus.
[REMOVED] Introduction	true: API NvM_SetRamBlockStatus enabled. false: API NvM_SetRamBlock-Status disabled.

5.6.58 NvM/NvMCommon/NvMSizeImmediateJobQueue

[RTA-BSW] Description	[ECUC_NvM_00503] Defines the number of queue entries for the immediate priority job queue. This parameter is only applicable if the NvMJobPrioritization parameter is set to true.
[AUTOSAR] Description	Defines the number of queue entries for the immediate priority job queue. If NVM_JOB_PRIORITIZATION is switched OFF this parameter shall be out of scope.
[RTA-BSW] Maximum Value	255
[AUTOSAR] Maximum Value	65535
[ADDED] Default Value	1

5.6.59 NvM/NvMCommon/NvMSizeStandardJobQueue

[RTA-BSW] Description	[ECUC_NvM_00504] Defines the number of queue entries for the standard job queue.
[AUTOSAR] Description	Defines the number of queue entries for the standard job queue.
[RTA-BSW] Maximum Value	255
[AUTOSAR] Maximum Value	65535

5.6.60 NvM/NvMCommon/NvMVersionInfoApi

[RTA-BSW] Description	[ECUC_NvM_00505] Enables/disables the NvM_GetVersion-Info API.
[AUTOSAR] Description	Pre-processor switch to enable / disable the API to read out the modules version information].
[REMOVED] Introduction	true: Version info API enabled. false: Version info API disabled.
[REMOVED] Default Value	false

5.6.61 NvM/NvMRbBlockDefault/NvMRbBlockDefaultValues/NvMBlock-CrcType

[RTA-BSW] Parent Container	NvM/NvMRbBlockDefault/NvMRbBlockDefaultValues/NvMBlockCrcType
[AUTOSAR] Parent Container	NvM/NvMBlockDescriptor/NvMBlockCrcType
[RTA-BSW] Description	[ECUC_NVM_00476] Defines the CRC data width for the NVRAM block. Due to Autosar this parameter shall define the CRC data width for data in NV memory and for data in permanent RAM block. Currently this parameter defines the CRC data width only for data in permanent RAM block (see parameter NvMCalcRamBlockCrc). For data in NV memory the CRC will be always calculated by lower layers independently from this parameter.
[AUTOSAR] Description	Defines CRC data width for the NVRAM block. Default: NVM_CRC16, i.e. CRC16 will be used if NVM_BLOCK_USE_CRC==true
[RTA-BSW] Enumeration Literals	NVM_CRC8, NVM_CRC16, NVM_CRC32
[AUTOSAR] Enumeration Literals	NVM_CRC16, NVM_CRC32, NVM_CRC8

5.6.62 NvM/NvMRbBlockDefault/NvMRbBlockDefaultValues/NvMBlockE-cucPartitionRef

[RTA-BSW] Parent Container	NvM/NvMRbBlockDefault/NvMRbBlockDefaultValues/NvMBlockEcuc PartitionRef
[AUTOSAR] Parent Container	NvM/NvMBlockDescriptor/NvMBlockEcucPartitionRef
[RTA-BSW] Description	[ECUC_NvM_00564] Maps the NV block to zero or one ECUC partition to limit the access to this NV block. The ECUC partition referenced is within the subset of the ECUC partitions where the NvM is mapped to.

[AUTOSAR] Description	Maps the NV block to zero or one ECUC partition to limit the access to this NV block. The ECUC partition referenced is within the subset of the ECUC partitions where the NvM is mapped to.

5.6.63 NvM/NvMRbBlockDefault/NvMRbBlockDefaultValues/NvMBlock-HeaderInclude

[RTA-BSW] Parent Container	NvM/NvMRbBlockDefault/NvMRbBlockDefaultValues/ NvMBlockHeaderInclude
[AUTOSAR] Parent Container	NvM/NvMBlockDescriptor/NvMBlockHeaderInclude

[RTA-BSW] Description: [BSW_SWCS_AR_NVRAMManager_Ext-2997] Defines a header file that is included. Only the header file name without the extension has to be entered here. For example “MyHeader” instead of “MyHeader.h”. If a header file is defined the declarations of the RAM data block (NvMRamBlockDataAddress) and the ROM data block (NvMRomBlockDataAddress) and all used callback-functions have to be privided within this header-file by the user. The inclusion of the header file is needed for the two mutually exclusive features: 1. Compile time check of RAM and ROM block length. 2. Auto Block Length for “non RTE blocks”. (“RTE blocks” do not need the include as Rte_Type.h is always inlcuded.) Compile Time Check: If a header file is defined for a specific NvM block, a compile time check is carried out to check if the configured NvM block length (NvMNvBlockLength) matches the size of the data given in in the RAM data block (NvMRamBlockDataAddress) and the ROM data block (NvMRomBlockDataAddress). This check is done using the sizeof operator and causes a compile time error in case of a mismatch. Note that the compile time check just can be carried out if for the NvM block the following two conditions are fulfilled: 1. The values entered in the RAM data block (NvMRamBlockDataAddress) and the ROM data block (NvMRomBlockDataAddress) are names of a linker symbol, pointers or subelements of data structures declared in the header (e.g. MyVar, MyStruct.MyMember, MyArray[3], &MyVar, …) but are not addresses in hexadecimal notation (e.g. 0x8000A000). 2. Auto Block Length is not active for this block.

[AUTOSAR] Description: Defines the header file where the owner of the NVRAM block has the declarations of the permanent RAM data block, ROM data block (if configured) and the callback function prototype for each configured callback. If no permanent RAM block, ROM block or callback functions are configured then this configuration parameter shall be ignored.

5.6.64 NvM/NvMRbBlockDefault/NvMRbBlockDefaultValues/NvM-BlockUseAutoValidation

[RTA-BSW] Parent Container	NvM/NvMRbBlockDefault/NvMRbBlockDefaultValues/ NvMBlockUseAutoValidation
[AUTOSAR] Parent Container	NvM/NvMBlockDescriptor/NvMBlockUseAutoValidation
[REMOVED] Introduction	true: if auto validation mechanism is used, false: otherwise

[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1
[REMOVED] Default Value	false

[RTA-BSW] Description: [NVM477_Conf] Defines the job priority for a NVRAM block. The current version of NvM supports two job priorities:
 0 : Immediate priority,
 1-255 : Standard priority. (higher value leads to lower prio)
 If no explicit standard priority is required then the priority 255 can be used, so that other blocks can define higher priority. If priority jobs handling is disabled then all blocks are handled with the same priority, i.e. in FCFS order.
 Immediate priority can be enabled only if priority jobs handling is enabled via the NvMJobPrioritization configuration paramter.
 Immediate priority blocks shall participate neither in multi-block-read (see also NvMSelectBlockForReadAll) nor in multi-block-write (see also NvMSelectBlockForWriteAll) operations. If an immediate block is selected for ReadAll or WriteAll, then it is not possible to queue an immediate write request for this block while the multiblock operation is being executed (as the state of this block would be PENDING (NVM385)).

[AUTOSAR] Description: Defines whether the RAM Block shall be auto validated during shutdown phase.

5.6.65 NvM/NvMRbBlockDefault/NvMRbBlockDefaultValues/NvMBlockUseCRCCompMechanism

[RTA-BSW] Parent Container	NvM/NvMRbBlockDefault/NvMRbBlockDefaultValues/NvMBlockUseCRCCompMechanism
[AUTOSAR] Parent Container	NvM/NvMBlockDescriptor/NvMBlockUseCRCCompMechanism
[RTA-BSW] Description	[ECUC_NvM_00556] Defines whether the CRC of the permanent RAM Block shall be compared during a write job with the CRC which was calculated during the last successful read or write job. This parameter can be set to true only if parameter NvMCalcRamBlockCrc is also true.
[AUTOSAR] Description	Defines whether the CRC of the RAM Block shall be compared during a write job with the CRC which was calculated during the last successful read or write job.
[REMOVED] Introduction	true: if compare mechanism is used, false: otherwise
[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1

[REMOVED] Default Value	false

5.6.66 NvM/NvMRbBlockDefault/NvMRbBlockDefaultValues/NvM-BlockUseCompression

[RTA-BSW] Parent Container	NvM/NvMRbBlockDefault/NvMRbBlockDefaultValues/NvMBlockUse Compression
[AUTOSAR] Parent Container	NvM/NvMBlockDescriptor/NvMBlockUseCompression
[RTA-BSW] Description	[ECUC_NvM_00563] Defines whether the data is compressed before written. RTA-CAR NvM provides currently block split as compression type. The user has also to define the size of block splits if using compression, see also container NvMRbSplitBlock. - true: data compression activated (takes more time to read and write) - false: no compression
[AUTOSAR] Description	Defines whether the data is compressed before written. Tags: atp.Status=draft
[REMOVED] Introduction	true: data compression activated (takes more time to read and write) false: no compression
[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1
[REMOVED] Default Value	false

5.6.67 NvM/NvMRbBlockDefault/NvMRbBlockDefaultValues/NvM-BlockUseCrc

[RTA-BSW] Parent Container	NvM/NvMRbBlockDefault/NvMRbBlockDefaultValues/NvMBlockUseCrc
[AUTOSAR] Parent Container	NvM/NvMBlockDescriptor/NvMBlockUseCrc
[REMOVED] Introduction	true: CRC will be used for this NVRAM block. false: CRC will not be used for this NVRAM block. Note: Configuring CRC for a block with immediate priority is not recommended, since the CRC calculation may extend over more than one NvM main function and this could increase the time of writing the immediate data significantly, thus defeating the purpose of immediate priority.

[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1

[RTA-BSW] Description: [ECUC_NVM_00036] Enables/disables CRC usage for the NVRAM block in NV memory. If enabled a CRC is calculated before a write operation which is then stored in the NV memory along with the block data. During a read operation this CRC is then checked again to ensure data integrity. Independant of parameter NvMBlockUseCrc the MemStack always uses CRCs to ensure data integrity, but this is done on lower software layers (not NVM level) and it is not possible to deactive it. Due to this reason the parameter NvMBlockUseCrc has to be globally enabled by setting the common parameter NvMRbNvCrcEnable to TRUE if a CRC also should be used at NVM level. Just if both parameters NvMBlockUseCrc and NvMRbNvCrcEnable are set to true the CRC will be used for this block on NVM level. Hint: For the case that the complete MemStack this does not affect any CRC useage on lower layers. Hint: If not the complete MemStack is used then the common parameter NvMRbNvCrcEnable should be set to TRUE to ensure that CRCs are used at NVM level.

[AUTOSAR] Description: Defines CRC usage for the NVRAM block, i.e. memory space for CRC is reserved in RAM and NV memory.

5.6.68 NvM/NvMRbBlockDefault/NvMRbBlockDefaultValues/NvMBlockWriteProt

[RTA-BSW] Parent Container	NvM/NvMRbBlockDefault/NvMRbBlockDefaultValues/NvMBlockWriteProt
[AUTOSAR] Parent Container	NvM/NvMBlockDescriptor/NvMBlockWriteProt
[RTA-BSW] Description	[ECUC_NVM_00033] Enables/disables an initial write protection of the NV block.
[AUTOSAR] Description	Defines an initial write protection of the NV block
[REMOVED] Introduction	true: Initial block write protection is enabled. false: Initial block write protection is disabled.
[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1

5.6.69 NvM/NvMRbBlockDefault/NvMRbBlockDefaultValues/NvMCalcRamBlockCrc

[RTA-BSW] Parent Container	NvM/NvMRbBlockDefault/NvMRbBlockDefaultValues/NvMCalcRamBlockCrc

[AUTOSAR] Parent Container	NvM/NvMBlockDescriptor/NvMCalcRamBlockCrc
[RTA-BSW] Description	[ECUC_NVM_00119] Enables/disables the CRC (re)calculation for the permanent RAM block or NVRAM blocks which are configured to use explicit synchronization mechanism.
[AUTOSAR] Description	Defines CRC (re)calculation for the permanent RAM block or NVRAM blocks which are configured to use explicit synchronization mechanism.
[REMOVED] Introduction	true: CRC will be (re)calculated for this permanent RAM block. false: CRC will not be (re)calculated for this permanent RAM block.

5.6.70 NvM/NvMRbBlockDefault/NvMRbBlockDefaultValues/NvMMaxNumOfReadRetries

[RTA-BSW] Parent Container	NvM/NvMRbBlockDefault/NvMRbBlockDefaultValues/NvMMaxNumOfReadRetries
[AUTOSAR] Parent Container	NvM/NvMBlockDescriptor/NvMMaxNumOfReadRetries
[RTA-BSW] Description	[ECUC_NVM_00533] Defines the maximum number of read retries. Read retries are implemented for MemIf failure and NvCrc failure. Not for Static Block ID check. If NvM is used with RTA-CAR Fee and Ea than this value should be 0. The RTA-CAR Fee and Ea perform already retries. The failure reaction can be more sophisticated on Fee/Ea level, because the retries can there depend on the failure type.
[AUTOSAR] Description	Defines the maximum number of read retries.
[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1
[REMOVED] Default Value	0

5.6.71 NvM/NvMRbBlockDefault/NvMRbBlockDefaultValues/NvMMaxNumOfWriteRetries

[RTA-BSW] Parent Container	NvM/NvMRbBlockDefault/NvMRbBlockDefaultValues/NvMMaxNumOf WriteRetries

[AUTOSAR] Parent Container	NvM/NvMBlockDescriptor/NvMMaxNumOfWriteRetries
[RTA-BSW] Description	[ECUC_NVM_00499] Defines the maximum number of write retries. Write retries a implemented for MemIf failure. Not for Write Verification (Read Back). If NvM is used with RTA-CAR Fee and Ea than this value should be 0. The RTA-CAR Fee and Ea perform already retries. The failure reaction can be more sophisticated on Fee/Ea level, because the retries can there depend on the failure type.
[AUTOSAR] Description	Defines the maximum number of write retries for a NVRAM block with [ECUC_NvM_00061]. Regardless of configuration a consistency check (and maybe write retries) are always forced for each block which is processed by the request NvM_WriteAll and NvM_WriteBlock.
[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1

5.6.72 NvM/NvMRbBlockDefault/NvMRbBlockDefaultValues/NvMNvramDeviceId

[RTA-BSW] Parent Container	NvM/NvMRbBlockDefault/NvMRbBlockDefaultValues/NvMNvramDeviceId
[AUTOSAR] Parent Container	NvM/NvMBlockDescriptor/NvMNvramDeviceId
[RTA-BSW] Description	[ECUC_NVM_00035] Identification of the device where the NVRAM block is located. In contrast to AUTOSAR's NvM specification, this is not a parameter derived from the block configuration on the Ea/Fee module configuration, but the device identifier is directly specified here (and the Ea/Fee block configuration is completely auto-generated). Attention: If the NVRAM block shall be located on a Ea device then user shall use the Ea device ID defined in the container NvMRbEaDevices. If ID used here does not correspond to an ID defined in the container NvMRbEaDevices then the NVRAM block will not be located on a Ea device. Attention: Device ID 0 is fixed to address Fee. If Fee is used then all corresponding blocks shall use device ID 0. If there are more than 1 Fee then first Fee (typically Fee for internal flash) shall use device ID 0 and all other Fees can use other device IDs.
[AUTOSAR] Description	Defines the NVRAM device ID where the NVRAM block is located.

[REMOVED] Introduction	Calculation Formula: value = TargetBlockReference.[Ea/Fee]DeviceIndex
[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1
[RTA-BSW] Maximum Value	255
[AUTOSAR] Maximum Value	1

5.6.73 NvM/NvMRbBlockDefault/NvMRbBlockDefaultValues/NvMResistantToChangedSw

[RTA-BSW] Parent Container	NvM/NvMRbBlockDefault/NvMRbBlockDefaultValues/NvMResistantToChangedSw
[AUTOSAR] Parent Container	NvM/NvMBlockDescriptor/NvMResistantToChangedSw
[RTA-BSW] Description	[ECUC_NVM_00483] Defines whether a NVRAM block shall be treated as resistant to configuration changes or not (see also the NvMCompiledConfigId and NvMDynamicConfiguration parameters for a description of this mechanism).
[AUTOSAR] Description	Defines whether a NVRAM block shall be treated resistant to configuration changes or not. If there is no default data available at configuration time then the application shall be responsible for providing the default initialization data. In this case the application has to use NvM_GetErrorStatus()to be able to distinguish between first initialization and corrupted data.
[REMOVED] Introduction	true: NVRAM block is resistant to changed software. false: NVRAM block is not resistant to changed software.
[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1

5.6.74 NvM/NvMRbBlockDefault/NvMRbBlockDefaultValues/NvMSelectBlockForReadAll

[RTA-BSW] Parent Container	NvM/NvMRbBlockDefault/NvMRbBlockDefaultValues/NvMSelectBlockForReadAll
[AUTOSAR] Parent Container	NvM/NvMBlockDescriptor/NvMSelectBlockForReadAll

[RTA-BSW] Description	[ECUC_NVM_00117] Defines whether a NVRAM block shall be processed during NvM_ReadAll or not. This configuration parameter is only allowed to be set for NVRAM blocks which are configured to have a permanent RAM block or which are configured to use explicit synchronization mechanism. Additionally, dataset blocks (configurable via NvMBlockManagementType parameter) as well as immediate priority blocks (configurable via NvMBlockJobPriority parameter) are not allowed to be selected for NvM_ ReadAll.
[AUTOSAR] Description	Defines whether a NVRAM block shall be processed during NvM_ReadAll or not. This configuration parameter has only influence on those NVRAM blocks which are configured to have a permanent RAM block or which are configured to use explicit synchronization mechanism.
[REMOVED] Introduction	true: NVRAM block shall be processed by NvM_ ReadAll false: NVRAM block shall not be processed by NvM_ReadAll

5.6.75 NvM/NvMRbBlockDefault/NvMRbBlockDefaultValues/NvMSelectBlockForWriteAll

[RTA-BSW] Parent Container	NvM/NvMRbBlockDefault/NvMRbBlockDefaultValues/NvMSelectBlock ForWriteAll
[AUTOSAR] Parent Container	NvM/NvMBlockDescriptor/NvMSelectBlockForWriteAll
[RTA-BSW] Description	[ECUC_NVM_00549] Defines whether a NVRAM block shall be processed during NvM_WriteAll or not. This configuration parameter is only allowed to be set for NVRAM blocks which are configured to have a permanent RAM block or which are configured to use explicit synchronization mechanism.
[AUTOSAR] Description	Defines whether a NVRAM block shall be processed during NvM_WriteAll or not. This configuration parameter has only influence on those NVRAM blocks which are configured to have a permanent RAM block or which are configured to use explicit synchronization mechanism.
[REMOVED] Introduction	true: NVRAM block shall be processed by NvM_WriteAll false: NVRAM block shall not be processed by NvM_WriteAll

5.6.76 NvM/NvMRbBlockDefault/NvMRbBlockDefaultValues/NvMStaticBlockIDCheck

[RTA-BSW] Parent Container	NvM/NvMRbBlockDefault/NvMRbBlockDefaultValues/NvMStaticBlockIDCheck
[AUTOSAR] Parent Container	NvM/NvMBlockDescriptor/NvMStaticBlockIDCheck
[RTA-BSW] Description	[ECUC_NVM_00532] Enables/disables the static block ID check for this block. The current version of NvM does not support this feature, so this parameter has currently always to be set to false.
[AUTOSAR] Description	Defines if the Static Block ID check is enabled.
[REMOVED] Introduction	false: Static Block ID check is disabled. true: Static Block ID check is enabled.
[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1
[REMOVED] Default Value	false

5.6.77 NvM/NvMRbBlockDefault/NvMRbBlockDefaultValues/NvMWriteBlockOnce

[RTA-BSW] Parent Container	NvM/NvMRbBlockDefault/NvMRbBlockDefaultValues/NvMWriteBlockOnce
[AUTOSAR] Parent Container	NvM/NvMBlockDescriptor/NvMWriteBlockOnce
[REMOVED] Introduction	true: Defines write protection after first write is enabled. false: Defines write protection after first write is disabled.
[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1

[RTA-BSW] Description: [ECUC_NVM_00072] Enables/disables automatic write protection of this block once it has been successfully read from or written to the memory media. This protects the block data to be overwritten once it has been successfully written. Please note that the fact that the block data has been successfully written is lost after NvM_Init; you typically recover this information by trying to read the block via NvM_ReadAll or NvM_ReadBlock before attempting any write/invalidate/erase operation on blocks configured to be written only once. An NvM block configured with the “Write Once” is allowed to be initialized at layout change (please see NvMRbInitBlockAtLayoutChange configuration param-

eter).If the block is initialized (i.e. its recovery data is written to the medium after ReadAll), attempting any write/invalidate/erase operation on this block will fail.

[AUTOSAR] Description: Defines write protection after first write. The NVRAM manager sets the write protection bit either after the NV block was written the first time or if the block was already written and it is detected as valid and consistent during a read for it.

5.6.78 NvM/NvMRbBlockDefault/NvMRbBlockDefaultValues/NvMWriteVerification

[RTA-BSW] Parent Container	NvM/NvMRbBlockDefault/NvMRbBlockDefaultValues/NvMWriteVerification
[AUTOSAR] Parent Container	NvM/NvMBlockDescriptor/NvMWriteVerification
[RTA-BSW] Description	[ECUC_NVM_00534] Enables/disables write verification. The current version of NvM does not support this feature, so this parameter has currently always to be set to false.
[AUTOSAR] Description	Defines if Write Verification is enabled.
[REMOVED] Introduction	false: Write verification is disabled. true: Write Verification is enabled.
[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1
[REMOVED] Default Value	false

5.7 API Definition

List of supported API

`NvM_Init`	Service for resetting all internal variables
`NvM_SetDataIndex`	Service to get the NvM block ID using the persistent ID input parameter
`NvM_GetDataIndex`	Service for getting the currently set DataIndex of a dataset NVRAM block
`NvM_SetBlockProtection`	Service for setting/resetting the write protection for a NV block
`NvM_GetErrorStatus`	Service to read the block dependent error/status information
`NvM_SetRamBlockStatus`	Service for setting the RAM block status of an NVRAM block
`NvM_SetBlockLockStatus`	Service for setting the lock status of a permanent RAM block of an NVRAM block

NvM_GetVersionInfo	Service to get the version information of the NvM module
NvM_ReadBlock	Service to copy the data of the NV block to its corresponding RAM block
NvM_WriteBlock	Service to copy the data of the RAM block to its corresponding NV block
NvM_RestoreBlockDefaults	Service to restore the default data to its corresponding RAM block
NvM_EraseNvBlock	Service to erase an NV block
NvM_InvalidateNvBlock	Service to invalidate a NV block
NvM_ReadPRAMBlock	Service to copy the data of the NV block to its corresponding permanent RAM block
NvM_WritePRAMBlock	Service to copy the data of the RAM block to its corresponding permanent RAM block.
NvM_RestorePRAMBlockDefaults	Service to restore the default data to its corresponding permanent RAM block
NvM_ReadAll	Initiates a multi block read request
NvM_WriteAll	Initiates a multi block write request
NvM_CancelWriteAll	Service to cancel a running NvM_WriteAll request
NvM_ValidateAll	Initiates a multi block validation request
NvM_MainFunction	Service for performing the processing of the NvM jobs

For further information please refer to AUTOSAR_SWS_NVRAMManager.pdf

5.8 Configuration Advice

NOTE

Before configuring this module, please note that some of the default values used by ConfGen to produce the default ECU Configuration can be changed, and this is done by either adding a Conf-Gen Enhancer Mapping (CEM) or by adding a rule to an *algo.properties* file. For more information, see the “Configuration Generation” section of the RTA-CAR User Guide.

5.8.1 Setting PositionInTask field of schedule events

The PositionInTask field of the schedule events of NvM, FEE, FLS must be set in the following order:

1. NVM
2. FEE
3. FLS

Fee_MainFunction and Fls_MainFunction are called by MemIf_Rb_MainFunction therefore the user must not schedule Fee_MainFunction and Fls_MainFunction if MemIf_Rb_MainFunction is scheduled and vice-versa.

5.8.2 Configuring NvM Blocks

Typically one or more NvM blocks will be defined with a given size for persistent storage. This definition is done via ECU configuration. NvM has no knowledge about the inner structure of the block contents.

For every configured NvM block, NvM has to create administrative data structures with a fixed size in both RAM and ROM. Although dependent on the microcontroller and compiler settings, a single NvM block typically requires 5 bytes of RAM and 36 bytes of ROM in NvM alone. Ea/Fee also require a certain amount of administrative data per block.

Startup/shutdown are dependent on the number of NvM blocks, so it is therefore recommended, wherever feasible, to create middle-sized NvM blocks (ideally in range 60 to some 4000 bytes) rather than many small ones. Very big blocks introduce handling disadvantages in Fee/Ea.

Each NvM block consists of at least one memory-area of persistent data on EEPROM or flash memory ("NV block") and one memory-area of the same size in RAM ("RAM block"). Everything related to the NV block is automatically created and managed by the NvM module and its delegates MemIf, Ea and Fee. The RAM block, however, is created by the NvM user who owns the NvM block.

RAM blocks can be either permanent (i.e. known already at ECU configuration time) or temporary (i.e. communicated to NvM with every NvM service call which involves a RAM block).

The maximum block length (NvMNvBlockLength) is restricted to 65407 Bytes because NvM retains 128 Bytes as reserved space.

5.8.3 Saved RAM Zone

It is strongly recommended that you allocate each RAM block in a RAM-area which has the initialization policy POWER-ON-CLEARED, which means that the RAM-area is only cleared automatically after a power-on of the system. In the context of the NvM, the term "saved RAM zone" is used.

When using AUTOSAR mechanisms for memory allocation, the memory class <MODULE>_SEC_VAR_SAVED_ZONE<anyNamePart>_<SIZE> or <MODULE>_SEC_VAR_SAVED_ZONE<anyNamePart>_<safety>_<SIZE> should be used.

5.8.4 Multi-block configuration

If a block is configured to be part of the ReadAll multi-block operation, then it will only be read from persistent storage during that operation if it is marked as invalid. Blocks of type NVM_BLOCK_DATASET can not be configured to be part of the ReadAll multi-block operation.

If a block is configured to be part of the WriteAll multi-block operation, then it will only be written to persistent storage during that operation if it has been marked

as having been changed by the NvM user, and if it is regarded as being valid. This is done by calling the NvM_SetRamBlockStatus API, or by invoking the client/server operation SetRamBlockStatus. These conditions do not apply to the FirstInitAll multi-block operation.

A block of type NVM_BLOCK_DATASET which has been configured to be part of the WriteAll multi-block operation is skipped if its dataset-index does not point to an NV block. Therefore, the NvM user owning the block has to correctly set the dataset-index to the intended NV block before the invocation of the NvM_WriteAll service. This is done by using the NvM_SetDataIndex service.

If a block is configured with block length null, the requests of multi-block operations are blocked - they are neither queued nor operated.

In a multi-partition configuration, multi-block operations can be issued only from the Master Partition.

5.8.5 NvMRbSwcdToEcucValuesEnable

The NvMRbSwcdToEcucValuesEnable extension parameter should be set to FALSE. ConfGen is the recommended way of generating NvMBlockDescriptors.

5.8.6 UseNewMappingCreateNvMBlock

ConfGen has two methods of creating NvMBlockDescriptors:

1. When configuring the NvM Service Dependency, if there are multiple Role-Based Data Assignments (RBDA) that are assigned the role “ramBlock” within a single Service Dependency, the Confgen process will generate NvBlocks for each RBDA.

This approach is commonly referred to as the legacy method for configuring the NvM Service Dependency. To employ this method for a project, you can either set the property “manprop_UseNewMappingCreateNvMBlock=false” in the project’s algo.properties file explicitly or leave this property unset.

The method will create NvMBlockDescriptors for each RoleBasedDataAssignment; whether the role’s value is “ramBlock” or the “defaultValue” is of no consequence. If the user sets it to “ramBlock” only when configuring the SWCs, and then updates the Rom address (when configuring the NvMBlockDescriptors), there will be no effect on the existing project.

2. To align with the AUTOSAR specification, the user can use the new mapping to create NvMBlockDescriptors by setting “manprop_UseNewMappingCreateNvMBlock=true”. The ConfGen process will create NvBlock for each Service Dependency. The quantities of “ramBlock” and “defaultValue” are determined based on the specific interactions a software component can have with the NVRAM Manager service.

It then generates the NvMBlockDescriptors and, for each one, updates the Rom block address with the name of the ParameterDataPrototype.

By default, the old mapping is used when creating a new project and will also apply to existing projects. To use the new mapping, the user must set “man-

prop_UseNewMappingCreateNvMBlock=true"(the default is false) in the project's algo.properties file.

> **NOTE**
>
> When using the new mapping, ConfGen will create NvMBlockDescriptors for each Service Dependency and update both the NvMRamBlockDataAddress and NvMRomBlockDataAddress. ConfGen will also use the first RoleBased-DataAssignment, with the role value set to "ramBlock", to update NvMRam-BlockDataAddress. It will ignore the remainder (which will each also have the role set to "ramBlock").

(Detailed use cases are described in the 'Service Dependencies and Service Use Cases' chapter of the AUTOSAR_TPS_SoftwareComponentTemplate documentation.)

5.8.7 ConfigId configuration

The NvM block descriptor NvM_ConfigId is created automatically by RTA-BSW with some specific default values, one of which is NvMRamBlockDataAddress, which is generated with the default value of NvM_Prv_idConfigStored_u16. As this default depends strictly on the relevant version of RTA-BSW implementation, if the default is overridden via manual configuration it may cause a compiling error.

5.8.8 Immediate Requests

In order to configure NvM blocks with a priority of "immediate" (via the NvMBlock-JobPriority parameter) the Immediate Requests feature must first be activated via the NvMJobPrioritization parameter.

NvM blocks configured with immediate priority cannot participate in ReadAll or WriteAll multi-block operations and they should not be selected for these operations during configuration. This is because it is not possible to queue an immediate write request for a block while the multi-block operation is being executed (the state of the block would be PENDING). Allowing an immediate block to participate in a multi-block operation would inhibit its "immediate" nature.

This is not the case for the multi-block operation FirstInitAll, as this service is intended to be performed once during factory initialization (see "Initialization of blocks in non-volatile memory" below). However, if NvM_Rb_FirstInitAll is invoked like any other NvM service after factory initialization, and if an immediate write for an immediate block (which is also selected for the FirstInitAll operation) is requested, then the write request for this block will be either rejected (if the block has not yet been processed by the multi-block operation, i.e it's still in PENDING state) or it will wait until the entire multi-block operation is finished (if the immediate write request is queued after the block was processed by the multi-block operation).

5.8.9 Write Request Monitoring

NvM allows the monitoring of all write requests. See the Supported Features

section for more details. Write Request Monitoring is activated by setting the NvMRbReqMonitor common parameter to NVM_RB_REQ_MON_WRITE.

You also need to define where the write monitoring data will be stored, and this is done through the NvMRbReqMonDevId common parameter.

5.8.10 Write Counter

NvM includes a Write Counter feature. See the Supported Features section for more details.

This feature is enabled with the NvMRbWriteCounter parameter. Additional flexibility is provided via the NvMRbCommonWriteCounter parameter, which defines the default value for the block specific NvMRbWriteCounter parameter, and this allows you to configure write counters for several blocks at once:

- With NvMRbCommonWriteCounter set to TRUE, the default value for NvMRbWriteCounter will be TRUE for all NVRAM blocks except those whose NvMRbWriteCounter is explicitly set to FALSE.
- With NvMRbCommonWriteCounter set to FALSE, the default value for NvMRbWriteCounter will be FALSE for all NVRAM blocks except those whose NvMRbWriteCounter is explicitly set to TRUE.

5.8.11 NvMMainFunctionCycleTime and NvMMainFunctionPeriod

The NvMMainFunctionCycleTime parameter was replaced in AR4.2 by NvMMainFunctionPeriod, and so it has now been removed from this latest RTA-BSW implementation of NvM. Only NvMMainFunctionPeriod should now be used.

A new extension parameter NvMRbMainFunctionScheduling has also been introduced which can be configured together with NvMMainFunctionPeriod.

The following table shows the necessary configuration changes to switch over to using NvMMainFunctionPeriod and NvMRbMainFunctionScheduling.

NvMMainFunction CycleTime	NvMMainFunction Period	NvMRbMainFunction Scheduling	Notes
0	0	NVM_RB_MAINFUNC_ SCHEDULING_RTE_ BACKGROUND	NvM_MainFunction is called in back-ground-task via RTE.
>0	>0	NVM_RB_MAINFUNC_ SCHEDULING_RTE_ TIMED	NvM_MainFunction is called in a timed-task via RTE.
n/a	0	NVM_RB_MAINFUNC_ SCHEDULING_MANUAL	NvM_MainFunction is called manually, no RTE event is generated in SWCD file.

5.8.12 Notification of Start/Completion of an NvM service

NvM can be configured to provide notifications when block-related jobs are started and finished.

5.8.12.1 Notification on Job Started

A callback function has to be defined according to the NvM_RbSingleBlockStart-CallbackFunction signature and the name of this callback function has to be communicated to NvM via the NvMRbSingleBlockStartCallback parameter.

If the user component is located above the RTE (in systems which have an RTE), the callback function name has to be set in the NvM block configuration as Rte_-Call_<BlockName>_Started_JobStarted, and a provide port has to be created for the user component and be connected to the <BlockName>_Started require port generated by NvM.

5.8.12.2 Notification on Job Finished

A callback function has to be defined according to the NvM_SingleBlockCallback-Function signature and the name of this callback function has to be communicated to NvM via the NvMSingleBlockCallback/NvMSingleBlockCallbackFnc parameter.

If the user component is located above the RTE (in systems which have an RTE), the callback function name has to be set in the NvM block configuration as Rte_-Call_<BlockName>_Finished_JobFinished, and a provide port has to be created for the user component and be connected to the <BlockName>_Finished require port generated by NvM.

5.8.13 Default Values for Implicit Recovery and NvM_RestoreBlockDefaults

In order to ensure valid data when working with an NvM block, either Implicit Recovery should be used (see the Supported Features section) or NvM_Restore-BlockDefaults should be called directly.

The possible sources for the default data are either the configured ROM block of the related NvM block and/or an initialization callback belonging to this NvM block. If both a ROM block and an initialization callback exist for an NvM block, initialization with ROM block data is executed by NvM before the initialization callback is invoked.

5.8.14 Initialization of blocks in non-volatile memory

NvM does not generally initialize the non-volatile memory. If there is no data to be read, NvM returns the error NvM_REQ_INTEGRITY_FAILED.

Initial values can, however, be written into the non-volatile memory with NvM_WriteBlock. Alternatively, you can use the NvM_Rb_FirstInitAll extension API to (re-)initialize many blocks at once. To use this feature, you need to set NvMRbSelectBlockForFirstInitAll to TRUE and provide the name of the Initialization callback.

The InitBlockCallback is also invoked for an implicit recovery or an explicit restore, so the NvM_Rb_GetActiveService extension API can be called from within the InitBlockCallback to check if the service that is currently being processed is indeed NvM_Rb_FirstInitAll (and not NvM_RestoreBlockDefaults, NvM_ReadBlock, or NvM_ReadAll). This allows you to provide initialization data that is different from the default recovery data (i.e. the data used in implicit recovery and explicit restore).

5.8.15 Increasing robustness of rarely written blocks

If an NvM block is stored in the Flash (i.e. Fee is used) and if it is written only rarely (i.e. less than once per driving cycle) then it is possible that the data in this block can be lost during an interrupted reorganization of the Fee. To avoid this data loss, the NvMBlockManagementType parameter should be set to REDUNDANT.

5.8.16 Automatically reading a block at system start

An NvM block can be configured to be read automatically from the non-volatile memory at system start. This is done by setting the "Read Block During ReadAll" option to TRUE or enabling Explicit Synchronization.

Automatic read is executed only for NvM blocks with a RAM status of INVALID.

If successful, manual operations such as NvM_ReadBlock, NvM_WriteBlock and NvM_RestoreBlockDefaults set the RAM status of an NvM block to VALID, which implicitly indicates that the current data in the RAM block is newer than the data saved in the non-volatile memory. If a manual operation has been called for a particular NvM block before an automatic read has finished, the automatic read will skip the NvM block and, in doing so, will avoid overwriting the newer data in RAM with the older data from the non-volatile memory.

If a reset occurs while an automatic read is being executed, and if the reset keeps the saved zone unaffected, consideration should be made for the possibility that a configured single block callback (end and start) is not invoked, even when the read job for this block finishes successfully.

5.8.17 Automatically storing a block at system shut-down

To avoid manually writing a simple NvM block it can be configured to be automatically written to the non-volatile memory at system shut-down. This is done by setting the "Write Block During WriteAll" option to TRUE, or by enabling Explicit Synchronization.

The NvM block will not be automatically written to non-volatile memory unless it has been marked as having been modified (by invoking NvM_SetRamBlockStatus with the BlockChanged argument set to TRUE).

5.8.18 Explicit Synchronization

Explicit Synchronization is configured at the NvM block level through the NvMBlockUseSyncMechanism parameter.

If Explicit Synchronization is enabled for an NvM block, both read and write Explicit Synchronization callbacks have to be configured for the block (NvMReadRam-

BlockFromNvCallbackand NvMWriteRamBlockToNvCallback).

The maximal number of retries on the callback routine copy operation from RAM to NvM mirror (or vice-versa) is configurable via the NvMRepeatMirrorOperations parameter.

5.8.19 API Overloading

This feature can be enabled separately for each NvM service by providing the name of the user function to be called in the body of the corresponding NvM service. This is done in the NvMRbApiOverloading container inside the NvMCommon section of NvM's configuration. The parameters and return types of the user function must be identical to the original NvM service.

See the Integration Advice section for more details on API Overloading.

5.8.19.1 NvM_Rb_SetDataIndexBase

NvM_Rb_SetDataIndexBase is only available if you have configured an overloaded API for the current request. It is your responsibility to invoke this function within the overloaded API to make sure that dataset index is set as required.

5.8.19.2 SetBlockProtection, SetRamBlockStatus and SetWriteAllTrigger

The SetBlockProtection, SetRamBlockStatus and SetWriteAllTrigger requests are initiated only if following conditions are fulfilled:

- All common pre-conditions are fulfilled excluding preconditions for asynchronous single-block services (NvM_Prv_Service_CheckPreconditions)
- All validation checks for all significant user-configured parameters are fulfilled (NvM_Prv_SetBlockProtection_CheckParameter)
- All conditions required to finalize the request initiation are fulfilled (NvM_Prv_Service_CheckBlockData)

These requests are currently implemented in the private functions NvM_Prv_SetBlockProtection, NvM_Prv_SetRamBlockStatus and NvM_Prv_SetWriteAllTrigger.

If you have configured an overloaded API for the request, then these functions invoke the overloaded API and then within the overloaded API you are responsible for invoking the NvM_Rb_SetBlockProtectionBase, NvM_Rb_SetRamBlockStatusBase and NvM_Rb_SetWriteAllTriggerBase functions. If you have NOT configured an overloaded API for the request, the private function is invoked directly.

5.8.20 RAM Block CRC

To monitor the integrity of data in RAM, you can enable the CRC recalculation for permanent RAM blocks or blocks which are configured to use the Explicit Synchronization mechanism. If this feature is enabled, NvM will recalculate the CRC for user data in RAM block every time it has been changed (by a read, write or restore operation).

The Ram Block CRC feature is enabled by setting NvMCalcRamBlockCrc to TRUE,

and you then set the CRC type that will be used via the NvMBlockCrcType parameter.

The RAM Block CRC feature can also be used to skip write operations to NV memory if the RAM block has not been updated since the last read or write operation. In this case, if the RAM block CRC calculated during a write job is equal to the CRC calculated during last successful read or write job, NvM will skip the writing and consider the job as successfully finished. To enable this functionality, set NvMBlockUseCRCCompMechanism to TRUE.

Normally the RAM Block CRC recalculation (NvMCalcRamBlockCrc) and the skipping of write operations (NvMBlockUseCRCCompMechanism) are enabled for each block separately, but you can set them as default enabled for all blocks via NvMRbCommonCalcRamBlockCrc. If you don't explicitly set NvMCalcRamBlockCrc and NvMBlockUseCRCCompMechanism, NvM will use the value set in NvMRbCommonCalcRamBlockCrc as the default.

Note that NvMRbCommonCalcRamBlockCrc of TRUE is used as the default value only for blocks which can use RAM Block CRC recalculation (i.e. blocks with permanent RAM blocks, or which are configured to use the Explicit Synchronization mechanism). For NVRAM blocks which have not configured a CRC type, NvM will use the default CRC type of 16 bit (NVM_CRC16).

5.8.21 CRC in Nonvolatile Memory (Nv CRC)

A CRC checksum can be optionally appended to user data stored on the storage media in order to check its integrity.

- CRC usage first has to be globally enabled via NvMRbNvCrcEnable
- Then the type of CRC to be used can be defined by the block-specific parameter NvMBlockCrcType.

5.8.22 Cryptographic services

To further increase the security of data stored in the non-volatile memory, you can use the following extension parameters (in NvMBlockDescriptor/NvMBlockCipheringRef) to enable the usage of the indicated cryptographic services, which NvM will automatically execute while reading or writing the NVRAM block:

- Encryption: NvMRbAddEncryption
- Signature: NvMRbAddSignature
- Authentication: NvMRbAddAuthentication

The NvM always uses the length of encrypted data for encryption/decryption i.e. the user must configure the corresponding CSM jobs for encryption/decryption so that both CSM jobs accept this length.

If MAC (Message Authentication Code) is configured and the user wants to use truncated MAC then they shall configure the corresponding MAC length to fit the truncated length. Also, the user shall make sure that CSM jobs for MAC generation/verification can work with the configured length.

5.8.22.1 Csm Job Configuration

NvM uses all Csm jobs in the operation mode for single calls (CRYPTO_OPERATIONMODE_SINGLECALL), which means that for the NvM all Csm jobs must be executed as asynchronous jobs. After starting a Csm job, NvM waits until Csm provides the result via a callback (NvM_Csm_Callback) and you can use this callback for the configuration of all Csm jobs used by NvM.

5.8.22.2 Encryption and Decryption Configuration

The AEAD and Standard algorithms are both supported for Encryption and Decryption:

- If you want to use the Standard algorithm for Encryption and Decryption, you only need to configure the NvMNvBlockNVRAMDataLength parameter.
- If you want to use the AEAD algorithm, the configuration of the NvMRbTagLength, NvMRbAssociatedDataLength and NvMRbAssociatedDataPtr parameters is also required.

5.8.22.3 NvMBlockCiphering configuration

When NvMNvBlockNVRAMDataLength is greater than 0 (user data encryption is enabled), NvM will store the following on the media:

- NvMRbInitVectorLength (if user data encryption or authentication is configured).
- NvMRbTagLength (if user data authentication is configured).
- NvMRbSignatureLength (if signing user data).

5.8.22.4 Signature generation and Signature verification additional Bytes

NvM will add a block persistent ID (2 bytes) into Nv data, so the size of additional bytes will be the size of the persistent ID + length of the generated signature.

5.8.23 Port Configuration

5.8.23.1 Provided Ports

By default, NvM's generated SWC description file contains one service P-Port for each NvM block.

The creation of ports can be switched off per NvM block by setting the optional configuration parameter NvMBlockUsePort to false. This disables the creation of any RTE port for the corresponding NvM block. If enabled for a particular NvM block by NvMBlockUsePort, one admin P-Port is also created for the block.

The short names of these P-Ports are derived from the short names of the corresponding NvM blocks, as defined in ECU configuration values. See "Identification of Blocks" in the Supported Features section.

5.8.23.2 Required ports

Unless disabled by setting the configuration parameter NvMBlockUsePort to false, NvM's generated SWC description file may also contain certain R-Ports for each NvM block. As with P-Ports, the short names of these R-Ports are also derived from the short names of the corresponding NvM blocks.

5.8.24 High-Speed Mode

The user can configure the high-speed-mode for individual memory devices. In high-speed-mode, the NvM provides exclusive processing of user requests with immediate priority only for blocks located on configured memory devices.

To configure a memory device for high-speed-mode, the user has to define the common container NvMRbDeviceProperty with the following mandatory parameters:

NvMRbNvramDeviceId	Defines the NVRAM-device-ID for which properties in this container are defined. The ID shall match one of the existing NVRAM-device-IDs used to define the location of the block (see the block-dependent parameter NvMNvramDeviceId).
NvMRbEnableHighSpeedMode	Defines whether the corresponding NVRAM device shall store NVRAM blocks with immediate priority in high-speed-mode.

5.8.25 Hints for robust configuration

The table below shows the least robust and most robust configuration value for some of the key configuration parameters for NvM. For a full list of all parameters, see the Configuration Parameters section.

The intention of this table is to assist you in building an appropriately robust configuration for NvM, based on the system impact when data is lost - the more critical the impact, the more robust the configuration should be. Please note, however, that enabling some of the optional features will introduce more complexity, require more resources, and may not be always required or provide any tangible benefit. For this reason, you should configure NvM based on your own application needs.

1	Configuration Interface	NvMBlockCrcType
	Least robust configuration	(omit)
	Most robust configuration	Feature not currently supported
	Comments	

2	Configuration Interface	NvMBlockJobPriority
	Least robust configuration	1
	Most robust configuration	Basic or robust configuration is independent of the job priority (higher job priority is to be used for writing data related to software crashes only).
	Comments	

3	Configuration Interface	NvMBlockManagementType
	Least robust configuration	NVM_BLOCK_NATIVE
	Most robust configuration	NVM_BLOCK_REDUNDANT
	Comments	NATIVE block type will have only 1 copy in memory media, where as REDUNDANT block type will have 2 copies in the memory media. However, using Redundant data type will consume more memory.

4	Configuration Interface	NvMBlockUseCrc
	Least robust configuration	True
	Most robust configuration	Feature not currently supported
	Comments	

5	Configuration Interface	NvMBlockUseSyncMechanism
	Least robust configuration	False
	Most robust configuration	True
	Comments	Enabling this feature allows to ensure that no concurrent accesses to the RAM block contents occur from both application and NvM side.

6	Configuration Interface	NvMBlockWriteProt
	Least robust configuration	False
	Most robust configuration	No robustness impact regarding data loss.
	Comments	Enabling this feature could improve robustness related to unwanted write operations

7	Configuration Interface	NvMCalcRamBlockCrc
	Least robust configuration	False
	Most robust configuration	True

	Comments	Enabling this feature improves robustness of data in RAM by CRC calculation.

8	Configuration Interface	NvMInitBlockCallback
	Least robust configuration	(omit)
	Most robust configuration	(optional)
	Comments	Having init call back is optional if ROM block values are configured.

9	Configuration Interface	NvMMaxNumOfReadRetries
	Least robust configuration	0
	Most robust configuration	Retries are handled by Ea and Fee. Therefore, set it to 0.
	Comments	

10	Configuration Interface	NvMMaxNumOfWriteRetries
	Least robust configuration	0
	Most robust configuration	Feature currently not supported.
	Comments	Ea and Fee internally retry 3 times at page layer even if this parameter is set to 0.

11	Configuration Interface	NvMNvBlockBaseNumber
	Least robust configuration	(omit)
	Most robust configuration	Feature currently not supported.
	Comments	Should not be configured.

12	Configuration Interface	NvMNvBlockLength
	Least robust configuration	(1-65535)
	Most robust configuration	It is very important to ensure that this parameter is configured correctly.
	Comments	This parameter must match the physical RAM block (if one is configured)

13	Configuration Interface	NvMNvBlockNum
	Least robust configuration	1
	Most robust configuration	2
	Comments	1 is for native blocks, 2 for redundant blocks and 1..255 for dataset blocks.

14	Configuration Interface	NvMNvramBlockIdentifier
	Least robust configuration	(omit)
	Most robust configuration	
	Comments	These IDs are automatically generated during BCT, don't configure this tag.

15	Configuration Interface	NvMNvramDeviceId
	Least robust configuration	(omit)
	Most robust configuration	To be omitted by ASW developers.
	Comments	This needs to be specified at project level by integrators.

16	Configuration Interface	NvMRamBlockDataAddress
	Least robust configuration	(RAM block name)
	Most robust configuration	
	Comments	This is the buffer where data is read or written.

17	Configuration Interface	NvMRbBlockPersistentId
	Least robust configuration	(omit)
	Most robust configuration	
	Comments	No robustness impact with respect to data loss.

18	Configuration Interface	NvMBlockUsePort
	Least robust configuration	(optional)
	Most robust configuration	
	Comments	No robustness impact.

20	Configuration Interface	NvMRbInitBlockAtLayoutChange
	Least robust configuration	False
	Most robust configuration	True
	Comments	Using this feature makes the application more robust against software change especially when this new block is added to the software layout. However, there are certain dependencies and complexities in using this feature. Applications should understand this and then use this. If disabled, applications must be prepared to get a read error until the data is written (e.g. in first driving cycle).

21	Configuration Interface	NvMRbNoFallback
	Least robust configuration	(optional)
	Most robust configuration	
	Comments	This feature should be chosen depending on whether getting old data or getting no/replacement data is preferred.

22	Configuration Interface	NvMRbResistantToLayoutRemoval
	Least robust configuration	(optional)
	Most robust configuration	
	Comments	No robustness impact with respect to data loss.

23	Configuration Interface	NvMRbSelectBlockForFirstInitAll
	Least robust configuration	False
	Most robust configuration	True
	Comments	Using this feature will provide proper values to start with. However, there are some dependencies on how to use this feature, please read documentation for further information.

24	Configuration Interface	NvMReadRamBlockFromNvCallback
	Least robust configuration	NULL_PTR
	Most robust configuration	(Call back function name)
	Comments	Please refer to comment in line 5.

25	Configuration Interface	NvMResistantToChangedSw
	Least robust configuration	(dynamic configuration is currently unsupported)
	Most robust configuration	
	Comments	Functional developer should enable this feature to ensure that the block is not lost when layout gets changed and dynamic configuration is turned on.

26	Configuration Interface	NvMRomBlockDataAddress
	Least robust configuration	NULL_PTR
	Most robust configuration	(ROM block name)

	Comments	Having an init call back or ROM values makes the application more robust when the data is lost.

27	Configuration Interface	NvMRomBlockNum
	Least robust configuration	0
	Most robust configuration	1
	Comments	Please refer to comment in line 26.

28	Configuration Interface	NvMSelectBlockForReadAll
	Least robust configuration	(optional)
	Most robust configuration	
	Comments	Basic or robust configuration is independent of fact if the application is using single block or multi block operations. But most generic use case is to read and store the data automatically.

29	Configuration Interface	NvMSelectBlockForWriteAll
	Least robust configuration	(optional)
	Most robust configuration	
	Comments	Please refer to comment in line 27.

30	Configuration Interface	NvMSingleBlockCallback/NvMSingleBlockCallbackFnc
	Least robust configuration	(optional)
	Most robust configuration	
	Comments	No robustness impact.

31	Configuration Interface	NvMRbSingleBlockStartCallback
	Least robust configuration	(optional)
	Most robust configuration	
	Comments	No robustness impact.

32	Configuration Interface	NvMStaticBlockIDCheck
	Least robust configuration	False
	Most robust configuration	
	Comments	Feature currently not supported.

33	Configuration Interface	NvMWriteBlockOnce
	Least robust configuration	False
	Most robust configuration	No robustness impact regarding data loss.
	Comments	Enabling this feature could improve robustness related to unwanted write operations

34	Configuration Interface	NvMWriteRamBlockToNvCallback
	Least robust configuration	NULL_PTR
	Most robust configuration	(Call back function name).
	Comments	Please refer to comment in line 5.

35	Configuration Interface	NvMWriteVerification
	Least robust configuration	False
	Most robust configuration	
	Comments	Feature currently not supported (verification is already performed at lower layers)

36	Configuration Interface	NvMWriteVerificationDataSize
	Least robust configuration	0
	Most robust configuration	
	Comments	Please refer to comment in line 34.

5.8.26 Multi-Partitioning

In order to configure NvM for a multi-partition set-up, the following steps should be taken:

NvBlock and SWC assignment

The configured assignment of each NvBlock to a dedicated Partition shall correspond to the EcucPartition assignments of the NvBlock owner (which could also be the RTE).

This shall ensure that all operations performed on the data of the block have access to this data structure. In other words, all SWCs executed from another Partition should not have write access to this memory.

> **NOTE**
>
> An overview of Block to Partition assignment is visible in the generated *NvM_Report.txt*.

Memory Protection Unit (MPU)

For each Partition, NvM configures/requires a dedicated set of MemMap sections. They have to be placed into memory-protected regions. The Memory Protection Unit shall fulfil the following requirements:

- Read access shall be possible at least from the Block-own Partition and from the Master Partition (there is no safety issue if every Partition has read access).
- Write access shall be possible at least from the Block-own Partition and from the Master Partition. Make sure that code executed from Partitions having lower ASIL safety levels than the Block-own one *do not* have write access.

- The MPU shall react suitably if an unauthorized write access is performed on the memory of the Block-own Partition i.e. further code execution within the NvM is stopped permanently until next system/partition reset.

Pointer Access Check

In some situations, the SWC passes pointers to the storage in which the Block data is stored. NvM, therefore, needs to check whether the pointer is pointing to a location within the intended Partition by implementing the function `NvM_<Partition>_IsAdrRangeInPartition`.

The function shall return TRUE only if the passed "start pointer" and "complete length" is within the storage of the addressed ‹Partition›. Else it shall return FALSE.

> **NOTE**
>
> `NvM_<Partition>_IsAdrRangeInPartition` is called from the NvM Master only.

Timer and Timeout Set-up

In a multi-partition configuration, the Master triggers the Satellite Partition to call all the callback functions within it. It is possible, however, that a Partition is stalled. To avoid the *freedom from interference* issue, i.e. that requests from other Partitions are blocked, the NvM prepares timeout monitoring of the callback handling.

The User must therefore ensure:

1. That the timeout parameters are suitably configured i.e. long enough that the round-trip communication (Master triggers Satellite/Satellite calls call-back to the User/User performs actions and requires retries) can be realized. But also short enough to prevent any system issues caused by the blocked NvM during the timeout period.
2. That a system timer is correctly configured and integrated into the *rba_MemLib* component. It is important to check that the measured time is fitting relative to the real time i.e. check/measure the variable `NvM_Prv_Main_st.tiRef_u32` (1 us resolution).

e.g. if exactly 100s are measured, then `NvM_Prv_Main_st.tiRef_u32` shall change by 100 000 000 us = 100000000 increments (+- 1%).

> **Hint** This verification can be done with the debugger or a measurement system. If a long measurement period (›= 100s) is used, even a hand-stopped measurement from a debugger-stopped state can be taken (i.e. measuring from manual "run" until the time is achieved and then pressing "pause"). Compare the hand-stopped time and difference of *tiRef_u32*.

Use of NvMRbRamInitAddSavedZone

In case multi-partition and saved-zone markers are combined, ensure that the latter are split on the memory's physical needs and the initialization strategy of

the different partitions. The memory might be allocated to different partitions, but will be written by the NvM Master only.

Further Configuration Limitations

Deactivate Det-error reporting in NvM (parameter `NvMDevErrorDetect` = FALSE). When using multi-partition configuration in a series production context, the Development Error reporting will be de-activated in NvM. This is because the Det-component is not yet prepared for multi-partition handling in series quality (see Limitations Section 5.4). In a development context, it might be possible to activate the Det feature in NvM (check with the safety responsible).

5.8.27 Auto Block Length

A distinction is made between NvM blocks that are used via RTE (RTE blocks) and NvM blocks not used via RTE (non-RTE blocks).

Non-RTE blocks

For non-RTE blocks, the developer has to provide the declarations of the RAM block, ROM block, and callbacks via a header file that is included in NvM.

To enable Auto Block Length for a non-RTE block:

- Configure the block-specific parameter `NvMRbAutoBlockLength` = TRUE.
- Configure the block-specific parameter `NvMBlockHeaderInclude` (which holds the declarations of RAM block/ROM block/callbacks, if they are used).
- Configure at least one of the block-specific parameters `NvMRamBlockDataAddress`, `NvMRomBlockDataAddress`, or `NvMRbNvBlockLengthString`.

RTE blocks

For RTE blocks, the RTE is providing the declarations, so no header file needs to be included in NvM. In this case, NvM automatically includes *Rte_Type.h*.

To enable Auto Block Length for an RTE block:

- Configure the block-specific parameter `NvMRbAutoBlockLength` = TRUE.
- Configure at least one of the block-specific parameters `NvMRamBlockDataAddress`, `NvMRomBlockDataAddress`, or `NvMRbNvBlockLengthString`.

5.9 Integration Advice

5.9.1 Init Functions

NvM is initialized by a call to `NvM_Init` at system startup (typically done by invocation from one of EcuM's *DriverInit* call-outs). `NvM_Init` should be called before any other service, or the `NvM_MainFunction`, are invoked. Before invoking `NvM_Init`, all underlying layers should have been initialized.

When NvM is initialized:

- All NvM queues are cleared (any pending service requests are lost).

- All request results are set to NVM_REQ_OK.
- The dataset-index is set to 0 for each NvM block of type NVM_BLOCK-_DATASET.
- If the saved zone is lost after a reset, then all RAM block states are set to unchanged and invalid (any possibly still existing indications that a block shall be written as part of `NvM_WriteAll` are lost), and all write protection states are reset to their configured state.
- All data set index settings are reset to zero.

Re-initializations of NvM are generally possible, but with the above results, which might not be desired.

If any block is configured for `NvMSelectBlockForReadAll`, then `NvM_ReadAll` must be called in the startup process (after `NvM_Init`). Note that ReadAll will not occur until `NvM_MainFunction` is called for the first time, and that until the ReadAll has occurred, the values in the RAM blocks must not be used as they are invalid.

If Cryptographic services is configured for any block, Inject key and Set key valid must be done before `NvM_ReadAll`.

When scheduling NvM without an operating system (e.g. in a “loop”), it is essential to call all parts of the NvM and the dependent underlying components. To do so, call `NvM_Rb_MainFunctionAndDependency` instead of the standard `NvM_MainFunction`.

5.9.2 DeInit Functions

There is no dedicated NvM shutdown functionality; as soon as the scheduling of `NvM_MainFunction` ends, NvM simply ceases to operate.

Note, however, that `NvM_WriteAll` is typically called during system shutdown. This writes all blocks to persistent storage which have been marked accordingly.

`NvM_WriteAll` requires that the `NvM_MainFunction` is either still being scheduled by the OS, or is being continuously called by some other means if the OS is no longer running. If neither case is true, the request for the WriteAll operation will be accepted and queued, but it will not be executed until `NvM_MainFunction` is scheduled or called again.

If Cryptographic services is configured for any block, `NvM_WriteAll` also requires that `Csm_MainFunction` and `rba_Crypto_MainFunction` are continuously called.

If `NvM_WriteAll` is to be called during system shutdown, worst case profiling of the WriteAll should be performed to ensure that the shutdown timing constraints are not violated.

When scheduling NvM without an operating system (e.g. in a “loop”), it is essential to call all parts of the NvM and the dependent underlying components. To do so, call `NvM_Rb_MainFunctionAndDependency` instead of the standard `NvM_MainFunction`.

5.9.3 Scheduled Functions

NvM_MainFunction has to be called periodically. The calling period may vary, and

there are no constraints regarding the calling order of NvM_MainFunction relative to other main functions (in particular not to the main functions of other Mem stack components).

It is highly recommended, however, that NvM_MainFunction is scheduled in the same task or container as the main functions of MemIf, Ea, Eep and Fee. Since all the main activities of NvM are carried out as part of the NvM_MainFunction, NvM's performance will directly correlate to the invocation frequency of this function.

The NvMRbMainFunctionScheduling extension parameter controls whether NvM_MainFunction is called in a background task via RTE, in a scheduled task via RTE, or if it is not called via RTE at all and therefore has to be scheduled manually. See the Configuration Advice for more details.

Scheduling for the Mem stack can be determined flexibly by the integrator. All Mem stack scheduling is handled by MemIf and NvM by default. The required scheduling only has to be set for NvM. Scheduling parameters set for NvM will be automatically forwarded to MemIf during code generation. To disable this automatic forwarding, you can set MemIfRbDisableForwardingScheduleInfo=true. If this parameter is set, then scheduling for each component except MemIf must be set in the <component>RbMainFunctionScheduling parameters. For most use cases it is recommended to retain the default scheduling and only set scheduling parameters for NvM.

Scheduling for NvM is set by the parameter NvMRbMainFunctionScheduling = NVM_RB_MAINFUNC_SCHEDULING_MANUAL or NVM_RB_MAINFUNC_SCHEDULING_RTE_TIMED or NVM_RB_MAINFUNC_SCHEDULING_BACKGROUND. Scheduling mode RTE_TIMED will configure NvM_MainFunction to be periodically called by RTE with a time period set by the parameter NvMMainFunctionPeriod. This parameter must be set if timed scheduling is used. Scheduling mode BACKGROUND will configure NvM_MainFunction to be called by RTE in the background task. Scheduling mode MANUAL will not configure any calling of NvM_MainFunction with any RTE events or entries. NvM_MainFunction will have to be manually called by the integrator.

5.9.4 API Overloading

If the API Overloading feature is enabled, the original NvM service consists only of a call to the user function, and the original functionality of the NvM service is made available under a different name whereby the original name is extended by an "Rb" prefix and a "Base" suffix (e.g. NvM_ReadBlock becomes NvM_Rb_ReadBlockBase).

No header file has to be provided to NvM to access the overloading user function as NvM generates the corresponding extern statement itself. The overloading mechanism will work regardless of whether the C interface or the RTE interface of a particular NvM service is used.

The following example is based on the overloading of the WriteBlock service. The WriteBlock service's functionality is inhibited based on a special user flag flgWriteInhibit_b:

```
Std_ReturnType MyWriteBlockOverload(NvM_BlockIdType BlockId, const void
*NvM_SrcPtr){if (FALSE == flgWriteInhibit_b){return
```

```
NvM_Rb_WriteBlockBase(BlockId, NvM_SrcPtr);}else{return E_NOT_OK;}}
```

This special code (with a configurable name) is then called whenever NvM's WriteBlock service is invoked. The NvM implementation of NvM_WriteBlock would look like the following:

```
Std_ReturnType NvM_WriteBlock(NvM_BlockIdType BlockId, const void
*NvM_SrcPtr){return MyWriteBlockOverload(BlockId, NvM_SrcPtr);}
```

Note: No published property of the overloaded NvM service should be changed as part of the API overloading, as this can lead to dangerous side effects - e.g. if the overloading user code behaves synchronously even though all NvM users assume an asynchronous service, or if non-reentrant functions (such as main functions) are called in the body of the overloading function even though the overloaded service is promised to be reentrant. Avoiding such dangerous side effects is the responsibility of the provider of the overloading function.

5.9.5 RAM Block Datatypes

In the configuration of NvM, the only information that it knows about a RAM block is the BlockLength. It does not know the data type of the RAM mirror (structure or array of uint8/uint16/uint32…).

For this reason, for all NvMRamBlockDataAddress, NvM declares extern variables of type uint8 array (in NvM_Cfg.c) instead of the correct RAM block data type. To avoid possible compilation errors, you should disable the compiling/linking option to deactivate the Lint checker.

For structure variables, padding and packing should be considered. You need to know the exact compiled size of the structure variable and specify the correct NvMNvBlockLength for it.

5.9.6 NvMSingleBlockCallbackFnc container

If the user component is located above the RTE (in systems which have an RTE), the container NvMSingleBlockCallbackFnc needs to be available and empty in order to automatically create a callback function Rte_Call_PNJF_<Block-Name>_JobFinished.

The dedicated requesting port PNJF_<BlockName> generated by NvM is then typically connected to a provided port of the user component during integration.

5.9.7 RTE Configuration

If ports for C/S communication in RTE are used for interaction between ASW and NvM, the RTE must be configured with –text-value-spec-policy=symbolic-pdav-always, and NvM_cfg.h must be included in Rte_UserCfg.h. The reason is that the PortDefinedArgumentValue is generated as TexttableValueSpecification instead of NumericalValueSpecification in order to avoid the unnecessary regeneration of RTE.

As the type definitions are now made directly within *NvM_Types.h* and no longer generated via the RTE, *NvM_Types.h* must be included in *Rte_UserCfg.h*

5.9.8 Multi-Partition Integration

Implementation of a multi-partition setup is achieved by splitting all internal RAM cells into Partition-specific sections.

NvM does not enforce or provide a technical protection mechanism, but it is expected that the integrator provides a suitable MPU set-up and configures it according to the steps listed in Configuration Advice (Section 5.8.26). All of them should be completed (this is the assumption-of-use in a safety-related context).

5.9.8.1 Satellite MainFunction

NvM generates a `MainFunction` for each Partition which blocks are assigned to. It is necessary to call these functions in a suitable Os Task period within the related Partition. The task period shall be >= that of the Master `NvM_MainFunction`, which is integrated in the Master Partition. There is no need to integrate a Satellite `MainFunction` assigned to the Master Partition.

NOTE

- Another aspect is Satellite `MainFunction` handling in situations *where the Os Task system is not running* (i.e. initialization or shutdown). As no Os Task system is scheduling the `MainFunctions`, this must be done manually (see `NvM_Rb_MainFunctionAndDependency`). This is also needed in standard-/non-multi-partition situations.
- With multi-partitioning, the access rights must normally be switched by the Os. `NvM_Rb_MainFunctionAndDependency` is expected to be called from the Master Partition with the related access rights. When calling the Satellite function, the access rights are switched to the access-rights level of the Satellite's Partition. Only then can the Satellite `MainFunction` be called, before the access rights are switched back to those of the Master Partition.
- Handling in the absence of an Os Task system must be implemented by the integrator. A template is provided in the NvM template folder (the interfaces are in the generated file *NvM_Prj_MainFunctionWrapper.h*).
- The switching of access rights must also ensure that the stack content of NvM Master is protected: first, the Master data is stored; then, the data of the Satellite might be stored/accessed. The access protection must ensure, therefore, that the Master's stack content cannot be modified when running with the access rights of the Satellite Partition.

The integrator must also provide a suitable MPU (Memory Protection Unit) set-up and configure it according to the steps listed in Configuration Advice (see Section 5.8.26). All of them should be completed (this is the assumption-of-use in a safety-related context).

5.9.9 OS Resources

The number of the active OS resources, and their usage, depends on the user configuration of the AUTOSAR System and Basic Software. It is the user's responsibility to ensure that the `ENTER` and `EXIT` resource API are implemented to disable/enable interrupts as required. Alternatively they must be configured in the OS with the correct task/ISR allocation.

The following is a list of all exclusive areas that this module might use regardless of the user configuration. It is the job of the integrator to ensure that the implementation of these exclusive areas is correctly defined. This implementation can be done using the RTE exclusive area configuration wizard or by manual implementation of the corresponding BSW API calls to ensure exclusivity.

The NvM module uses the following OS resources:

- NvM_Main

5.10 Hardware Requirements

Table 5.302.NvM Hardware Requirements

Support	Usage
FLASH	14354 bytes
RAM	898 bytes

Usage calculated based on a sample configuration.

6 rba_EaFs2

Module name	rba_EaFs2
Package	RTA-MEM
AUTOSAR Module ID	N/A
AUTOSAR Version	23-11

6.1 Introduction

rba_EaFs2 is a sub-module of Ea, and it implements the Migration concept to handle EEPROM layout changes.

rba_EaFs2 is part of the RTA-MEM stack.

Module Configuration

This module's configuration parameters are included in the configuration of another module. See the Configuration Advice section for more details.

6.2 Supported Features

6.2.1 Migration

In addition to the standard features supported via the Ea module, the rba_EaFs2 sub-module also supports a migration feature.

Whenever new software is flashed into the ECU, if there is a change in the Ea layout (due to the addition or removal of Ea blocks) the migration process will start. This involves arranging Ea blocks present in EEPROM as per the Ea_Cfg.c file layout configuration without losing the block data that is present in both the old and new software. Blocks are identified by the BlkPersistentID, which persists on the storage media and is used to match blocks in old and new layouts.

Migration will respect blocks with the survival attribute set, even if they are not included in the new layout. Survival blocks which are not included in the new layout will be preserved in the EEPROM and will be accessible if the device is flashed with new software that now includes that block in Ea_Cfg.c. Survival blocks can be configured by setting NvMRbResistantToLayoutRemoval = true. Check the documentation for NvM for further information on this parameter.

Once a block has been configured as survival, conversion of this block back to non-survival in a new layout is not supported. This configuration should be avoided as this could lead to a migration failure or "Migration Not Possible" errors in some scenarios.

If a migration failure is reported, further user requests placed in the same power cycle will be rejected with E_NOT_OK. This is done to avoid block loss.

Interrupting the ongoing migration and flashing a new software with different Ea layout should be avoided as this could lead to "Migration Not Possible" errors in some scenarios.

6.3 Extensions

NOTE

rba_EaFs2 provides the implementation of the Ea functions. Therefore, the list of extensions is the same and can be taken from Ea module reference guide.

6.4 Limitations

See the main section on the Ea module for any limitations.

6.5 Exclusions

NOTE

rba_EaFs2 provides the implementation of the Ea functions. Therefore, the list of exclusions is the same and can be taken from Ea module reference guide.

6.6 Deviations

NOTE

rba_EaFs2 provides the implementation of the Ea functions. Therefore, the list of deviations is the same and can be taken from Ea module reference guide.

6.7 API Definition

NOTE

rba_EaFs2 provides the implementation of the Ea functions. Therefore, the list of supported API is the same and can be taken from Ea module reference guide.

6.8 Configuration Advice

NOTE

rba_EaFs2 provides the implementation of the Ea functions. Therefore, the configuration advice is the same and can be taken from Ea module reference guide. The notes below are specific to the Migration feature.

6.8.1 EaRbMigEraseBufferSize

Migration involves the erasure of unwanted blocks, and if the migration is done within Ea_Init, each block erase consumes 4ms. In order to reduce the Ea_Init time, erasing is moved to cyclic migration, but if the size of the array (Ea_Rb_MigEraseBuffer_uo) is not big enough, the remaining blocks are erased in Ea_Init itself.

The EaRbMigEraseBufferSize parameter allows you to define the size of the array. The default value is 50, but for optimal Ea_Init performance, the size must be greater than or equal to the number of blocks that are to be erased.

6.8.2 Overriding Migration

EaRbSurvival/NvMRbSurvival can be used in Block Configuration to override migration even if the configured layout, and the layout present in EEPROM Hardware, is not the same.

6.8.2.1 Survival Blocks

Blocks can be set as survival with the parameter NvMRbResistantToLayoutRemoval = true at the NvM configuration level. Survival blocks will persist on the storage medium even when a migration takes place to a new layout which does not include that block in Ea_Cfg.c. To preserve this functionality, a survival block can never be set to non-survival.

Do not configure a survival block as a non-survival block in a new layout. Under certain conditions this could cause a migration error or a “Migration not possible” error.

6.9 Integration Advice

> **NOTE**
>
> rba_EaFs2 provides the implementation of the Ea functions. Therefore, the integration advice is the same and can be taken from Ea module reference guide. The notes below are specific to the Migration feature.

6.9.1 Ea_Rb_MigrationStatus

The Ea_Rb_MigrationStatus function, which returns the migration status, should not be called before Ea_Init.

- If no migration is required (the configured layout, and the layout present in EEPROM Hardware, is the same) Ea_Rb_MigrationStatus will return EA_RB_MIGRATION_NOTNEEDED, otherwise it will return the status as EA_RB_MIGRATION_RUNNING.
- Once a migration has completed, if it was successful, Ea_Rb_MigrationStatus will return EA_RB_MIGRATION_FINISHED, otherwise it will return EA_RB_MIGRATION_ERROR.

Ea_Rb_MigrationStatus is not applicable for bootmode - the migration will be started only after the call of Ea_Rb_EndInit.

During shutdown, and before calling NvM_WriteAll, unless there is a strict timing restriction you should first call Ea_Rb_MigrationStatus to check that a migration is not still running.

Ensuring that no migration is running before calling NvM_WriteAll will have the following advantages:

- Ea Initialization times are extended if a migration is ongoing (e.g. for 32KB EEPROM it takes 100msec), so allowing an ongoing migration to complete before a shutdown is executed will ensure the fastest possible re-initialization in the next power cycle.
- If NvM_WriteAll is called while a migration is ongoing, the blocks that are not present in the old software will be written into the Clipboard Area of EEPROM (a temporary area used for migration), and if that clipboard area becomes full, the migration will fail and no further migration will be possible in that power cycle.

6.9.2 Migration Overhead

If a migration is required during Ea initialization, this will involve reading EEPROM, so it should be noted that the overall time required for the Ea Initialization will be higher than if no migration were required.

Here are some approximate times for Ea Initialization with SPI frequency 10MHz and Eep Page Size 64bytes, and while a migration is ongoing:

- 128k EEPROM : 225milliseconds
- 64K EEPROM : 110milliseconds
- 32K EEPROM : 55milliseconds

6.9.3 OS Resources

The number of the active OS resources, and their usage, depends on the user configuration of the AUTOSAR System and Basic Software. It is the user's responsibility to ensure that the `ENTER` and `EXIT` resource API are implemented to disable/enable interrupts as required. Alternatively they must be configured in the OS with the correct task/ISR allocation.

The Rba_EaFs2 module does not use any exclusive OS resources.

6.10 Hardware Requirements

Information relating to the hardware resources required by this module is not currently available.

7 rba_FeeFs1x

Module name	rba_FeeFs1x
Package	RTA-MEM
AUTOSAR Module ID	N/A
AUTOSAR Version	23-11

7.1 Introduction

The Flash EEPROM Emulation 1.x module (rba_FeeFs1x) is a sub-component of the Fee module that defines Fee 1.0 layout and behaviour, and it forms part of the RTA-MEM stack. This is an implementation variant with a D-Flash layout 1.0, in which management and user data of NvRAM blocks are written one after the other.

When the `FeeRbFsSelect` extension parameter (in */Fee/FeeGeneral*) is configured to "FS1X", the Fee module source code is disabled and rba_FeeFs1x provides the implementation of the Fee functions.

rba_FeeFs1x supports the Fee module's key features, including reading and writing blocks to/from non-volatile memory. It also supports the Sector Reorganization process (see the Supported Features section (Section 7.2 for more details).

Module Configuration

This module's configuration parameters are included in the configuration of another module. See the Configuration Advice section for more details.

7.2 Supported Features

The Fee's key features are described in Fee module reference guide. This section focuses on the Data Flash organization and management by rba_FeeFs1x module.

7.2.1 Sector management

For all Flash devices managed by rba_FeeFs1x, there needs to be at least two logical sectors, this is independent from the type of flash (e.g. internal Data Flash, external Flash, internal Program Flash etc.). Every logical sector has a dedicated Sector Header Area. Several physical sectors can be combined to one logical sector.

Depending on the hardware, Data Flash characteristics can differ (e.g. page size, sector size, specified erase cycles).

rba_FeeFs1x uses sector states for sector management, which are stored in a Sector Header marker at the beginning of every logical sector. A sector's state can be changed between the following states:

Sector State	Description
UNDEF	No Sector Header available
ERASED	Sector was erased and can be used

USED	Sector was used to write data
FULL	Sector is full
READY_TO_GET_ERASED	Sector can be erased

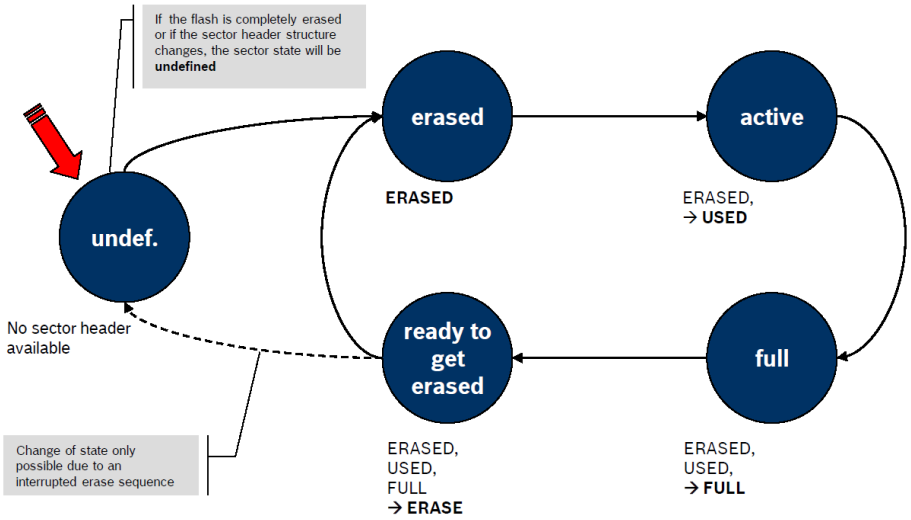


Figure 7.1.Change of Sector State

- Structure of sector header markers:

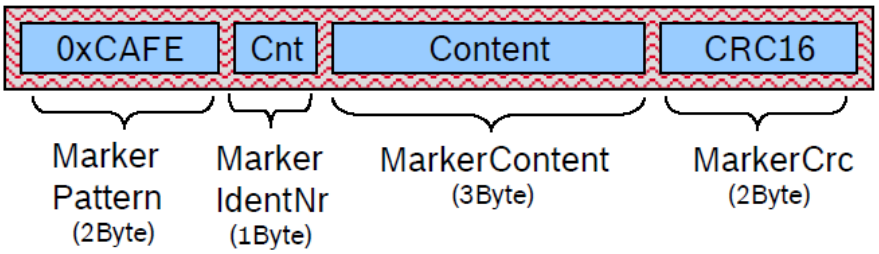


Figure 7.2.Markers Structure

- In which:
  - Marker Pattern: 0xCAFE by default
  - Marker IdentNr: Sector State (0x00: UNDEF, 0x01: ERASED, 0x02: USED, 0x03: FULL, 0x04: ERASED_REQUEST, 0x05: START)
  - Marker Content: Sector change counter (ScCnt)
  - Marker Crc: 16 bit checksum calculated over 6 previous bytes of Sector Header marker

7.2.2 Block management

Block management information is used to identify the user data for each NVRAM

block (FeeBlock) within the data flash or external flash. This “Block Overhead” needs 14 bytes and is located directly in front of the user data. The latest data of a block is always written directly after the last-written block.

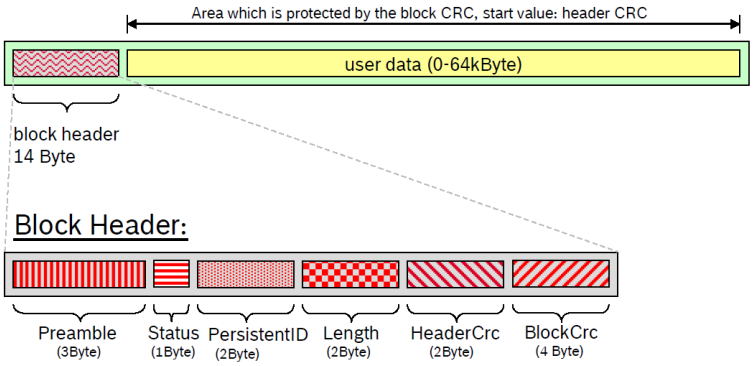


Figure 7.3.Block Header Structure

- In which:
  - Preamble: 0xA53C96. Fix pattern which helps to identify new block headers. Every half byte contains two “1” Bits and two “0” Bits
  - Status: The block status is used for the description of block characteristics in the flash.
  - FeeIdx: “Persistent ID” of the block configuration. This is defined in `FeeRbBlockPersistentId` parameter, which can be automatically calculated by NvM.
  - Length: User data length of the FeeBlock
  - HeaderCrc: CRC16 checksum to verify the block header.
  - BlockCrc: CRC32 checksum protecting the block data in order to detect programming interruptions and bit errors.

7.2.3 Sector Reorganization

The Sector Header, Block Header and last valid Block Data are filled continuously from the start until the end of each sector, and from the first to the last available sector. When the Data Flash is almost full and the Reorganization threshold is reached, Sector Reorganization is triggered to copy of all latest valid copies which are only located within the oldest FULL sector into the active (“USED”) sector. Afterward the reorganization has completed, the oldest FULL sector is erased in order to get additional space for new data.

- SSR: Soft Sector Reorganization threshold:
  - A check of this threshold is performed during the IDLE state of the emulation.
  - The copy process itself is interruptible in order to able to execute user orders in between (highest priority).
  - User orders are accepted and executed.
- HSR: Hard Sector Reorganization threshold:
  - A check of this threshold is performed before any write order

is started.

- The highest priority has the process of finishing the copy procedure for all relevant blocks.
- User orders are still accepted but executed only after the reorganization (and the following erase procedure) was finished.

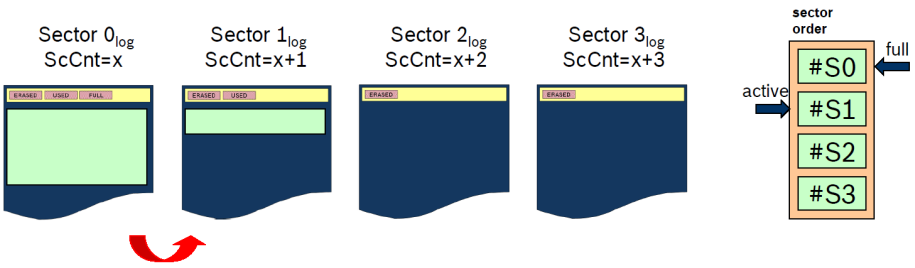
The table below shows the calculation of the thresholds for HSR and SSR. Note that the threshold is rounded up to the nearest multiple of page size.

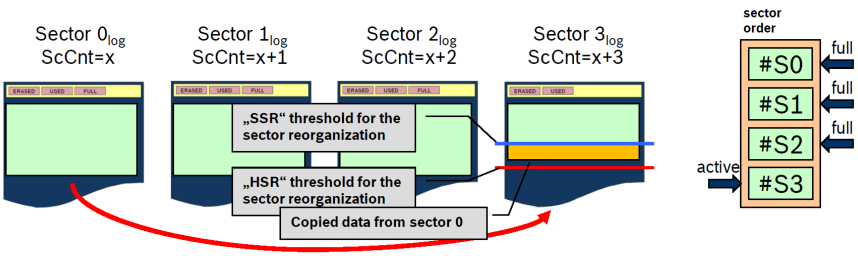
Layout Size	Threshold type	Algorithm
Layout < smallest logical Sector	HSR	smallest Sector size [Bytes] + 1
	SSR	HSR * 1.5
Layout >= smallest logical Sector	HSR	(smallest Sector size [Bytes] + 1) + (2 * biggest Block [Bytes])
	SSR	HSR * 1.5

The Layout Size is the total size of all configured NVRAM blocks (FeeBlocks), which includes the Block Header for each block.

In a 2 sector system, the threshold calculation follows:

1. HSR = smallest Sector size[Bytes] + 1
2. SSR = HSR





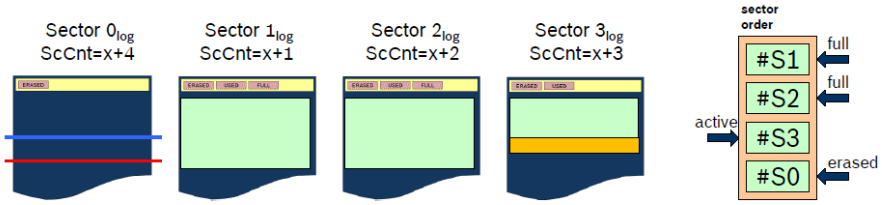


Figure 7.4.Sector Reorganization Sequence

7.2.3.1 Afterburner

The rba_FeeFs1x reorganisation also supports a "search-for-and-copy routine" of unknown blocks (i.e. blocks that are not part of the current layout). This feature, known as an "afterburner", can be activated for all blocks in the flash (FeeRbActUnknownBlockReorg) or it can be activated partially and block-specific (FeeRbActSurvivalEval).

7.3 Extensions

> **NOTE**
>
> rba_FeeFs1x provides the implementation of the Fee functions. Therefore, the list of extensions is the same and can be taken from Fee module reference guide (see Section 2.3).

7.4 Limitations

There are currently no known limitations with the rba_FeeFs1x module.

7.5 Exclusions

> **NOTE**
>
> rba_FeeFs1x provides the implementation of the Fee functions. Therefore, the list of exclusions is the same and can be taken from Fee module reference guide (see Section 2.5).

7.6 Deviations

> **NOTE**
>
> rba_FeeFs1x provides the implementation of the Fee functions. Therefore, the list of deviations is the same and can be taken from Fee module reference guide.

7.7 API Definition

NOTE

rba_FeeFs1x provides the implementation of the Fee functions. Therefore, the list of supported APIs is the same and can be taken from Fee module reference guide (see Section 2.7).

7.8 Configuration Advice

NOTE

rba_FeeFs1x provides the implementation of the Fee functions. Therefore, the configuration advice is the same and can be taken from Fee module reference guide. The notes below are specific to rba_FeeFs1x.

7.8.1 Configuring for rba_FeeFs1x

The actual layout of the data on the flash can be configured in the parent Fee module using the FeeRbFsSelect extension parameter (in /Fee/FeeGeneral).

If "FS1X" is configured, rba_FeeFs1x must be selected from the RTA-BSW CodeGen Run Configuration window.

Note that the "FS1" option is also available, but rba_FeeFs1 has now been deprecated in RTA-BSW, and so should not be selected.

Note that the "FS2" option is also available, but rba_FeeFs2 is currently not supported in RTA-BSW, and so should not be selected.

7.8.2 Configuring Critical Data as Double Storage

Blocks configured with single storage have a probability (albeit very low) of getting lost, even during regular operation. If an abort (reset or power loss) happens during physically charging the cells of the last page of a block, then the data can be lost during later operation. For this reason, it is recommended that critical data is configured as double storage.

7.9 Integration Advice

NOTE

rba_FeeFs1x provides the implementation of the Fee functions. Therefore, the integration advice is the same and can be taken from Fee module reference guide.

7.9.1 Special care for DFLASH layout creation

Some projects may use separate tools to flash DFLASH area (e.g. when creating initial DFLASH dump). When such operations are done, it is important that the sector headers are handled correctly.

- Only the markers that are required should be programmed (e.g. Erase Marker, Used Marker, Start Marker and Full Marker).
- These markers should be programmed only in the first 4 pages of the logical sector. All other pages in the sector header (8 pages) must be left unprogrammed, otherwise the complete sector could be treated as invalid and the information stored in that sector would be erased, leading to a permanent loss of blocks. Programming with a default value, such as inverse erase pattern or all 0s, is not allowed.

7.9.2 Init Functions

No specific handling necessary.

7.9.3 DeInit Functions

No specific handling necessary.

7.9.4 Scheduled Functions

There are no scheduled functions.

7.9.5 OS Resources

The number of the active OS resources, and their usage, depends on the user configuration of the AUTOSAR System and Basic Software. It is the user's responsibility to ensure that the `ENTER` and `EXIT` resource API are implemented to disable/enable interrupts as required. Alternatively they must be configured in the OS with the correct task/ISR allocation.

The Rba_FeeFs1x module does not use any exclusive OS resources.

7.10 Hardware Requirements

Information relating to the hardware resources required by this module is not currently available.

8 rba_FeeFs2

Module name	rba_FeeFs2
Package	RTA-MEM
AUTOSAR Module ID	N/A
AUTOSAR Version	23-11

8.1 Introduction

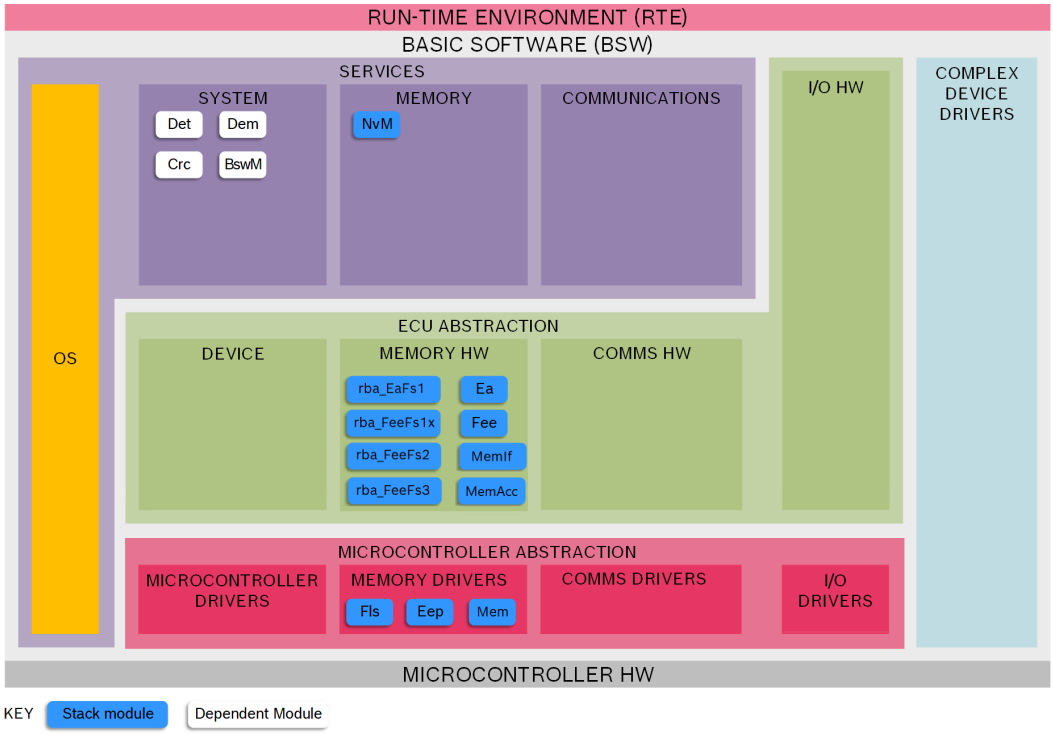


Figure 8.1.Memory Stack

The Flash EEPROM Emulation 2.0 module (rba_FeeFs2) emulates the possibilities of a serial EEPROM inside a flash memory and allows for the usage of further features. This also allows it to be used to save different types of data to a non-volatile memory.

The Fee is a BSW component with interfaces and a behavior standardized by AUTOSAR that can organize and save different types of data to a non-volatile memory.

The Fee 2.0 version module is an implementation variant with a certain D-Flash layout 2.0; therefore, the documentation of the generic Fee applies.

See the separate section on Flash EEPROM Emulation for more details.

8.2 Supported Features

8.2.1 Supported Hardware Devices

rba_FeeFs2 supports the following hardware devices:

- IFX Device 3 Host
- IFX Device 4 Host
- IFX TC33X, Device S 40nm Host A-Step - Complementary/Single Ended
- IFX TC33X, Device S 40nm HSM A-Step - Complementary/Single Ended
- IFX TC35X, Device M 40nm Host A-Step - Complementary/Single Ended
- IFX TC35X, Device M 40nm HSM A-Step - Complementary/Single Ended
- IFX TC37X, Device 3 40nm Host A-Step - Complementary/Single Ended
- IFX TC37X, Device 3 40nm HSM A-Step - Complementary/Single Ended
- IFX TC38X, Device 4 40nm Host A-Step - Complementary/Single Ended
- IFX TC38X, Device 4 40nm HSM A-Step - Complementary/Single Ended
- IFX TC39X, Device 5 40nm Host B-Step - Complementary/Single Ended
- IFX TC39X, Device 5 40nm HSM B-Step - Complementary/Single Ended
- IFX TC39X, Device 5 40nm Host A-Step - Complementary
- IFX TC39X, Device 5 40nm HSM A-Step - Complementary
- IFX TC3EX, Device 4 Evo 40nm Host A-Step - Complementary/Single Ended
- IFX TC3EX, Device 4 Evo 40nm HSM A-Step - Complementary/Single Ended
- ST Device 3 Cut3.1 Bernina Host
- Renesas P1xC with external flash MX25L32 from Macronix
- Renesas R-Car V3H R7 core with external flash S25FS256 from Cypress
- Renesas R-Car V3H R7 core with external flash S25FS512 from Cypress
- Renesas R-Car V3H R7 core with external flash S70FS01GS from Cypress
- Renesas R-Car V3H HSM core with external flash S70FS01GS from Cypress
- Zynq UltraScale with external flash S70FS01GS from Cypress
- Zynq UltraScale with external flash MT25QU01G from Micron
- Zynq UltraScale with external flash MT25QU02G from Micron
- Zynq UltraScale with external flash W25Q128JV from Winbond
- Zynq UltraScale with external flash W25Q256JV from Winbond
- Zynq UltraScale with external flash MX66U1G from Macronix
- Zynq UltraScale with external flash MX66U2G from Macronix
- NXP S32 with external flash MT35XU512 from Micron
- Limited: IFX Device 1 Host

- Limited: IFX Device 2 Host
- Limited: JDP Device 2 Host
- Limited: JDP Device 3 Host
- Limited: JDP Device 4 Host
- Limited: SPC56EL70
- Limited: Zynq UltraScale with external flash S25FS512S from Cypress
- Limited : Renesas R-Car V3H R7 core with external flash S25HS01GT from Cypress
- Limited : Renesas R-Car V3H HSM core with external flash S25HS01GT from Cypress

8.3 Extensions

NOTE

rba_FeeFs2 provides the implementation of the Fee functions. Therefore, the list of extensions is the same and can be taken from Fee module reference guide (see Section 2.3).

8.4 Limitations

8.4.1 Unsupported Hardware Devices

The following hardware devices are currently not supported or have limited functionality when used with rba_FeeFs2:

- IFX Device 1 Host not tested - not released for series.
- IFX Device 2 Host not tested - not released for series.
- IFX TC39X, Device 5 40nm Host A-Step - could be used in series without using HSM DFLASH in parallel.
- IFX TC39X, Device 5 40nm HSM A-Step - could be used in series without using Host DFLASH in parallel.
- JDP Device 2 Host only compiled and basic functional tests - not released for series.
- JDP Device 3 Host not tested - not released for series.
- JDP Device 4 Host only compiled and basic functional tests - not released for series.
- SPC56EL70 not tested - not released for series.
- Zynq Ultrascale with external flash W25Q128JV from Winbond - This device is sensitive for erase aborts. Any powerloss/reset during the erase operation in a sector can disturb the data within the related 512 kB (4 Mbit) super-block.

- Zynq Ultrascale with external flash W25Q256JV from Winbond - This device is sensitive for erase aborts. Any powerloss/reset during the erase operation in a sector can disturb the data within the related 512 kB (4 Mbit) super-block.
- Zynq UltraScale with external flash S25FS512S from Cypress never tested - not released for series.

8.5 Exclusions

> **NOTE**
>
> rba_FeeFs2 provides the implementation of the Fee functions. Therefore, the list of exclusions is the same and can be taken from Fee module reference guide (see Section 2.5).

8.6 Deviations

8.6.1 Configuration Deviations

Configuration parameters which differ from the AR v22-11 specification are listed below.

- There are no known deviations in this version.

8.7 API Definition

> **NOTE**
>
> rba_FeeFs2 provides the implementation of the Fee functions. Therefore, the list of supported APIs is the same and can be taken from Fee module reference guide (see Section 2.7).

8.8 Configuration Advice

> **NOTE**
>
> Before configuring this module, please note that some of the default values used by ConfGen to produce the default ECU Configuration can be changed, and this is done by either adding a Conf-Gen Enhancer Mapping (CEM) or by adding a rule to an *algo.properties* file. For more information, see the “Configuration Generation” section of the RTA-CAR User Guide.

8.8.1 General

- To enable rba_FeeFs2: Set *FeeRbFsSelect* to FS2.

- To select the configuration for each device, configure the *FeeRbMachineType* parameter.
- To configure FeeFs2 manually, Set *FeeRbFsSelect* to FEE_RB_FULL_MANUAL_CFG.

8.8.2 Required Configuration Changes

- When in the case the following three conditions listed are satisfied:
  - DMA is writing into the buffer (mainly in the case of external flash).
  - Data cache controller (CC) is enabled.
  - Buffer is linked to a cached memory region.

The buffer has to be fully cache line aligned (start address has to be aligned to the start of a cache line and the buffer size has to be a multiple of the cache line size). The DMA is constantly bypassing the cache controller which has to invalidate the cache lines before writing to it, in order to keep cache coherence between DMA (direct writer without CC) and the core (reader via CC). Otherwise if the cache line is shared with the data of another application, the data may get corrupted.

The buffer will be marked by the MemMap concept using *#defines*. The used *#defines* will be added to the component related to the MemMap file, and it will include the corresponding compiler directive directly e.g *"__attribute__((section("bufferCacheLineAligned")))"*, as listed in Bsw_MemMap.h.

- When a full manual configuration is used (FeeRbMachineType = FEE_RB_FULL_MANUAL_CFG) the parameter *FeeRbDeviceFs2IsPreprogm* has to be replaced with two new parameters *FeeRbDeviceFsXIsDifferential* and *FeeRbDeviceFsXIsOverprogmForInvldOk*.
- The rba_FeeFs2 specific configuration parameter *FeeRbFatCacheSize* has been removed as this is no longer used. In the current release, separated caches for known and unknown blocks have been introduced. The size of the known cache corresponds to the number of configured blocks whereas the size of the unknown cache is configurable by the Fee configuration parameter *FeeRbNrOfUnknownBlockCacheEntries*.

In case the features *FeeRbActUnknownBlockReorg* and/or *FeeRbActSurvivalEvalare* are activated, it is strongly recommended to configure the cache size according to your needs. The provided default value may increase the RAM consumption or decrease the performance unnecessarily. When *FeeRbActSurvivalEval* is exclusively enabled, *FeeRbNrOfUnknownBlockCacheEntries* has to be configured, otherwise the build process aborts. More detailed configuration hints can be found in the parameter description.

8.8.3 Configuration Prescriptions

Block Configuration Prescriptions:

- Fee/NvM logical blocks which are written only rarely (less than once per

driving cycle) shall be configured in a way that the data is stored redundantly.

- Blocks configured with single storage have a very low probability to be lost even during a regular operation. Background: When an abort (reset or power loss) happens during the charging of the cells for the last page of a block then the data can be left weakly programmed and be lost during a later operation.
- Because of this, it is recommented that critical data is configured as double storage.

Partial Reconfiguration Prescriptions:

For dedicated devices the EEPROM Emulation layout can be changed manually:

- Set the BCT parameter sector size (*FeeRbDeviceFs2SectorSize*) to the required value.
- Setup the sector descriptors (*FeeRbDeviceFs2SectorDesc*) for the BCT parameter sector list (*FeeRbDeviceFs2SectorList*).
- Refer to the Supported Features (Section 8.2) to confirm if this feature is available for the device being used.

Full Manual Configuration for First Prototyping Prescriptions:

This allows the complete Fee configuration to be setup manually:

- Set the FeeMachine Type (*FeeRbMachineType*) parameter to full manual configuration for first prototyping.
- Set all parameters of the FeeFs2 Flash Parameter (*FeeRbDeviceFs2*) container.

8.9 Integration Advice

8.9.1 Special Care for DFLASH Layout Creation

Some projects may use separate tools to flash DFLASH area (e.g. when creating initial DFLASH dump). When such operations are done, it is important that the sector headers are handled correctly.

- Only the markers that are required should be programed (e.g. ERASE marker, FAT_PREPARE marker, FAT_ACTIVE marker, DAT marker and RDY4ERASE marker).
- Store the FAT_ACTIVE and the DAT marker twice, the markers should be programmed into the first pages of the sector header of the logical sector. The last two pages of the sector header must be left unprogrammed, even programming with a default value, such as an inverse erase pattern or all 0s is not permitted.

If these pages are programmed, the complete sector will be treated as invalid and the information stored in that sector will be erased leading to a permanent loss of

blocks.

8.9.2 Fee Ininitialization Routine

During the runtime of Fee_Init it is possible that the initialization routine may write to the data flash. Therfore, it is highly recommended to avoid reading or writing operations from parallel running cores (e.g. HSM) to the host data flash or to any memory areas belonging to the same physical flash partition (e.g. Infineon TC3xx DFLASH0: EEPROM and UCB area).

8.9.3 Important Notes for rba_FeeFs2

The following points are essential and should be considered for the rba_FeeFs2 component:

- For the first startup outside of production, the information in section "Initialization" of this component documentation is to be considered. Depending on the project, this could mean that the handling within the production needs to be adjusted.
- Confirm the configuration made for the data blocks (within NvM or Fee directly) to ensure that critical blocks are configured as double storage, (Refer to the Configuration Advice section for more details).
- If the number of sectors is configured as 3, then the Soft Sector reorganization mechanism will not be available. Consequently, when incoming requests cannot fit into the remaining free space in the sector, a Hard Sector reorganization is triggered, which may require a long time to complete.

8.9.4 Init Functions

The function Fee_Init checks whether the DFLASH is already formatted for handling by the Fs2 component. If this is not the case then the Fs2 will perform a blocking format of the DFLASH. Essentially, this means one sector is erased for usage as the FAT sector and one sector is erased for usage as the data sector.

> **NOTE**
>
> Since the erasing of two sectors is a slow operation (typically between 100 ms and 1s), the initial startup will be significantly delayed. With possible system effects of error messages due to the delayed init time or even watchdog resets.

All subsequent startups do not need to repeat this operation as the formatting is retained in the DFLASH with the sector markers. NOTE: If the markers are physically lost then the formatting would be repeated, and in this case all data stored in DFLASH will be lost.

As a counter measure it is also possible to initialize the DFLASH during production.

The recommented procedure to prepare a valid initial dump is (within a debugging environment):

- Fully erase the DFLASH
- Set a breakpoint on Fee_Init
- Step over Fee_Init
- Take a DFLASH dump => the created dump can be flashed to prevent further delayed startups.

8.9.5 DeInit Functions

No specific handling necessary.

8.9.6 Scheduled Functions

No specific handling necessary.

8.9.7 Additional Hints & Tips

- Limit data layout size and writing frequency of data blocks into flash to a necessary minimum. This maximizes flash lifetime and data retention.
- Avoid very high proportion of static data, written only once in the plant, combined with a very high writing frequency of the remaining data. Permanent writes prevent early cleanup in background. This may lead to a very high writing execution time due to inevitable and more complex reorganization, and in the worst case could be an infinite process.
- Each project shall verify that targeted driving cycles are met on HSM/Host with the project specific Mem configuration.
- For the best timing behavior it is recommented to locate the Fee and rba_FeeFs2 to the core local RAM from where the code is executed.
- With this release, the driver behaves differently when entering the idle state. The debouncing mechanism which delays the start of background activities (e.g. sector erase, soft reorganization) by 100 main function calls has been removed. Pending background tasks will now start immediately with the next main function call just after reaching the idle state. The delay mechanism is no longer required, since the cost in terms of time to switch between the soft reorganization and higher prioritized user operations is significantly reduced. The main reason for the increased speed is that cache buildups are no longer required in this context.

```
a_u32 = 0;
while((MEMIF_IDLE == Fee_GetStatus()) &amp;amp;&amp;amp; (a_u32 &lt;
NR_MAINFUNCTION_CALLS_BEFORE_BACKGROUND_ACTION))
{
Fee_MainFunction(); // Do not handle background operations(erase, soft reorganization)
a_u32++;
}

while(MEMIF_IDLE == Fee_GetStatus()
{
Fee_MainFunction(); // Perform pending background operations(erase, soft reorganization)
}
```

- Before this Release: NR_MAINFUNCTION_CALLS_BEFORE_BACKGROUND_ACTION = 100
- From this Release on: NR_MAINFUNCTION_CALLS_BEFORE_BACKGROUND_ACTION = 0

8.9.8 OS Resources

The number of the active OS resources, and their usage, depends on the user configuration of the AUTOSAR System and Basic Software. It is the user's responsibility to ensure that the `ENTER` and `EXIT` resource API are implemented to disable/enable interrupts as required. Alternatively they must be configured in the OS with the correct task/ISR allocation.

The Rba_FeeFs2 module does not use any exclusive OS resources.

8.10 Hardware Requirements

Information relating to the hardware resources required by this module is not currently available.

9 rba_FeeFs3

Module name	rba_FeeFs3
Package	RTA-MEM
AUTOSAR Module ID	N/A
AUTOSAR Version	23-11

9.1 Introduction

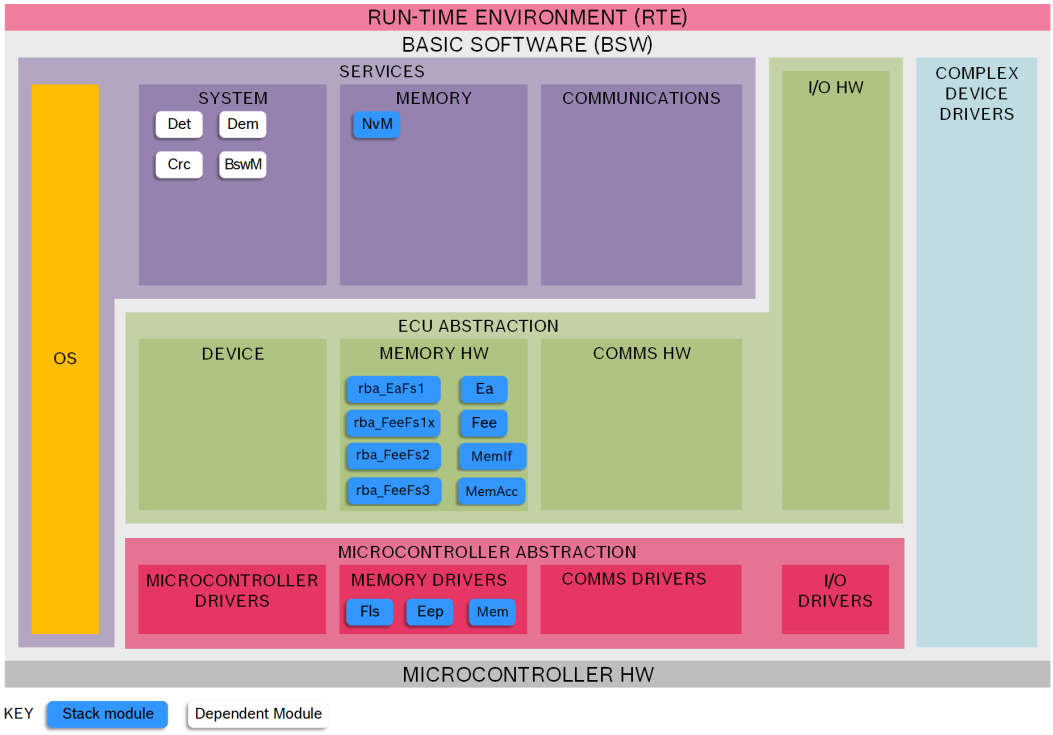


Figure 9.1.Memory Stack

The *rba_FeeFs3* module, aka *Fee3*, is an implementation variant of the Flash EEPROM Emulation (Fee) module. Its data layout is optimized for overwritable memories like *Phase-Change* or *Resistive Random Access Memories (PCM/RRAM)*. The Fee documentation (see Chapter 2) applies to this version as well, but with some restrictions (e.g. there are no sectors and no sector erases/reorganizations).

9.2 Supported Features

Since rba_FeeFs3 is a variant of Fee, the latter's features (see Section 2.2) apply in addition to the following:

9.2.1 Sector Erase/Reorganization

Since rba_FeeFs3 is a driver for overwritable memories, there is no need to erase or reorganize sectors. An existing data instance can just be overwritten with a

newer one, i.e. there is no need to append data instances as happens in flash memories.

9.2.2 Wear Leveling

In flash memories, where data is erased sector-wise and appended block-wise, the stress (i.e. the wear-out) is distributed equally. In overwritable memories, however, dedicated methods become necessary to balance the stress of frequently and seldomly written data. rba_FeeFs3 organizes the data-NvM in fixed units called *rows*, which are rotated/moved from time to time. Therefore, the data is not located at fixed/static addresses but changes dynamically.

9.2.3 Sectors vs Sections

The term *sector* denotes the smallest physically erasable portion of flash memory. As mentioned above, there is no need for sectorization in rba_FeeFs3. However, the module still divides the memory into two logical *sections*. The concept is different, however, in that the two sections provide for the redundancy of block management information (i.e. block headers) and data (in case of double-storage blocks). The data of single-storage blocks is stored alternating in both sections i.e. there is no danger of losing single-storage blocks by a power loss. Two sections are also required for defragmentation.

9.2.4 Defragmentation

If a software update changes the Fee's block configuration, some stored blocks might become obsolete, while their space is needed to add new ones. The creation of space by removing obsolete blocks is called *defragmentation*. Traditional Fees do this as part of the reorganization, but since there is no reorganization in rba_FeeFs3, it is a dedicated feature. Like a reorganization, it is executed in the background; however, its trigger is different: a defragmentation is usually only necessary once after a software change (which includes modifications of the block configuration), while a sector reorganization is triggered whenever the appended data has filled the memory beyond some threshold (i.e. much more frequently).

9.3 Extensions

NOTE

rba_FeeFs3 provides the implementation of the Fee functions. Therefore, the list of extensions is the same and can be taken from Fee module reference guide (see Section 2.3).

9.4 Limitations

There are currently no known limitations with the rba_FeeFs3 module.

9.5 Exclusions

> **NOTE**
>
> rba_FeeFs3 provides the implementation of the Fee functions. Therefore, the list of exclusions is the same and can be taken from Fee module reference guide (see Section 2.5).

9.6 Deviations

9.6.1 Configuration Deviations

Below are listed Fee module parameters with behaviors that are restricted when implemented in rba_FeeFs3:

`FeeRbNoFallback`	Not supported, but can alternatively be realized by enabling redundant data storage (`FeeRbSecLevel` = 2) for the block.
`FeeRbActUnknownBlockReorgCacheSize`	Meaningless (i.e. no sector reorganization).
`FeeRbConfirmSingleStorageBlockloss`	A single storage block's data instance is stored alternating in two logical sections, i.e. there is always a valid instance and the confirmation is thus meaningless.
`FeeRbConfirmUseGetSystemJobResult`	Meaningless (for rba_FeeFs3, the stop mode support and function `Fee_Rb_GetSystemJobResult` have been introduced at the same time).
`FeeRbGetFreeSpaceApi`	The function `Fee_Rb_GetNrFreeBytes` doesn't return the physically available free space, but the corresponding logical value. The logical value is about half of the physical one, because the memory is split into two logical sections.
`FeeRbGetNrFreeBytesAndFatEntriesApi`	Meaningless (i.e. no FAT). Use function `Fee_Rb_GetNrFreeBytes` instead.
`FeeRbGetNrSectorErases`	Meaningless (i.e. no sectors/no sector erase).
`FeeRbNrOfUnknownBlockCacheEntries`	Meaningless (i.e. no sector reorganization).

`FeeRbPageBufferSize`	As already mentioned, rba_FeeFs3 organizes the data-NvM in rows. This concept has a direct impact on the maximum possible length for any kind of memory access. A value greater than this length would be a waste of RAM, but a smaller value decreases the driver's performance. Therefore, rba_FeeFs3 automatically selects the appropriate value; the integrator shall not configure any value.
`FeeRbSecondPageBufferSize`	A second buffer is only beneficial for defragmentation, i.e. it is not supported in customer boot mode (where `FeeRbActUnknownBlockReorg` is *false*). No value or *0* will disable the second buffer, any other value will enable it. The configuration processor will throw an error and inform about the correct value if the selected one doesn't fit to the row concept.
`FeeRbSetAndGetJobModeApi`	Only the default mode (FEE_RB_ALLJOBS_ALLSTEPS_E) is supported, i.e. the feature is meaningless for rba_FeeFs3. Use `Fee_SetMode` instead.
`FeeRbTriggerReorgApi`	Meaningless (i.e. no sector reorganization).
`FeeVirtualPageSize`	rba_FeeFs3 automatically selects the appropriate value depending on the data-NvM's ECC width; the integrator shall set the value '0'.
`FlsMaxReadFastMode`, `FlsMaxReadNormalMode`, `FlsMaxWriteFastMode`, `FlsMaxWriteNormalMode`	Since the data-NvM is organized in rows, any Fls request will have a maximum length of the logical row size (e.g. 992 bytes for ST SR6P6, see RBA_FEEFS3_PRV_CFG_ROW_SIZE_LOGL in *rba_FeeFs3_Prv_Cfg.h*). A value > the logical row size would be a waste of RAM, but a smaller value decreases the driver's performance. If the MemStack Fls driver is used, do NOT set any of these parameters as doing so will automatically select the appropriate values.
`MemMaxReadSize`	The same restriction as that for the above four parameters also applies in the case of a third-party MemAcc plus Mem driver.

9.7 API Definition

> **NOTE**
> rba_FeeFs3 provides the implementation of the Fee functions. Therefore, the list of supported APIs is the same and can be taken from Fee module reference guide (see Section 2.7).

9.8 Configuration Advice

> **NOTE**
> Before configuring this module, please note that some of the default values used by ConfGen to produce the default ECU Configuration can be changed, and this is done by either adding a Conf-Gen Enhancer Mapping (CEM) or by adding a rule to an *algo.properties* file. For more information, see the "Configuration Generation" section of the RTA-CAR User Guide.

9.8.1 Dependencies

rba_FeeFs3 requires some features in addition to AUTOSAR Fls requirements. It can only be applied to memory technologies which support a direct write to memory; there is no need for a previous erase operation (as used with standard flash technologies). Therefore, the related Fls functions `Fls_Erase` and `Fls_BlankCheck`, and the MemAcc functions `MemAcc_Erase` and `MemAcc_BlankCheck` are not used.

> **NOTE**
> This section is only relevant for projects where a third-party Fls/MemAcc driver is used along with a Fee driver.

Additional AUTOSAR requirements for Fls:

SW Requirement	Description
SWS_Fls_00250 , SWS_Fls_00371	If Fls is used (instead of MemAcc), neither `Fls_Erase` nor `Fls_BlankCheck` are used/required by rba_FeeFs3.

9.8.2 Use Cases

Software Updates/FOTA

If a software update changes the Fee's block configuration (e.g. by removing blocks or changing lengths), the next application software startup will trigger a defragmentation, which removes obsolete blocks and creates space for new ones. The defragmentation is time-consuming and executed in the background.

Incoming job requests are accepted and processed during any ongoing defragmentation, but the processing might be delayed (approximately by the time it takes to write the biggest stored block).

During an ongoing defragmentation, new blocks can also be added if there is sufficient space. If not, the request's processing is delayed (with MEMIF_JOB_PENDING) until the defragmentation has removed all obsolete blocks (which can be time-consuming). If the caller cannot handle this (e.g. due to diagnostic service timeouts), the layout size must be reduced to a level such that there is always space for all previously stored, plus all new blocks.

Plant/Production

In plant, there is no need to consider wear leveling or data retention aspects respectively. Instead, low production cycle times are essential. In order to achieve the fastest programming times, the corresponding features inside the driver can be switched off by operating the driver in fast mode (i.e. call function `Fee_SetMode(MEMIF_MODE_FAST)` before requesting any other job).

In plant, too, a defragmentation is triggered in the application software whenever the Fee's block configuration changes. The above-mentioned impacts (potential processing delays, required spare space for new blocks) shall also be considered during production. The functions `Fee_Rb_DisableBackgroundOperation` and `Fee_Rb_EnableBackgroundOperation` could be used to control the defragmentation time.

9.8.3 Common Hints

When reading all relevant blocks during startup (e.g. via `NvM_ReadAll`), processing time can be minimized by operating the driver in fast mode (call function `Fee_SetMode(MEMIF_MODE_FAST)` beforehand). This suppresses potential block header corrections (i.e. programming operations), which are performed where the read has fallen back to old/redundant data due to inconsistent latest data. When the startup read is done, return the driver into slow mode again (call function `Fee_SetMode(MEMIF_MODE_SLOW)` afterwards).

Unless there is the essential need to minimize the job processing times (e.g. during startup or in production), the driver shall be operated in slow mode. The driver's features for wear leveling and data retention are only active in slow mode.

Limit the data layout size and write frequency of all data blocks as much as possible. Doing both maximizes the data-NvM's lifetime and its data retention. A small layout size also keeps space free to add new blocks during an ongoing defragmentation, and leaves spare rows to replace those with hardware defects.

9.9 Integration Advice

9.9.1 Initialization

`Fee_Init` checks if the data-NvM is already formatted for rba_FeeFs3 and the given device. If this is not so, the driver formats the data-NvM during initialization: all row headers/footers are erased; and two new rows with initial section headers are programmed. During this synchronous operation, the executing uC core is blocked (e.g. around 150~200ms on SR6P6, with 512kB data-NvM separated into

512 rows). This means the very first startup is delayed; a countermeasure is to initialize the data-NvM during production.

All subsequent startups do not need to repeat this operation, since the format is persisted inside the data-NvM by means of row headers/footers and section headers.

> **NOTE**
>
> If both section headers are lost, the formatting will be repeated and all data stored in the data-NvM is also lost.

Even if the data-NvM has been formatted for rba_FeeFs3, `Fee_Init` might trigger a (few) programming operation(s) in order to repair lost management information (i.e. row headers/footers, section or block headers).

9.9.2 How To Integrate

As mentioned, rba_FeeFs3 works only in combination with the Fee module (see Chapter 2), which provides all APIs and paramdefs.

The following points are essential to consider for the integration of this module:

- For the first startup outside production, the details in the Initialization section above (see Section 9.9.1 should be considered. Depending on the project, the handling within production might need to be adjusted.
- `Fee_Init` might trigger a (few) programming operation(s) in order to repair lost management information (i.e. row headers/footers, section or block headers). If different cores (like host and HSM) access the same hardware resource during initialization, a sychronization mechanism becomes necessary.
- For the sake of wear leveling and data retention, the very first call of `Fee_Write`, `Fee_InvalidateBlock` or `Fee_Rb_BlockMaintenance` in a driving cycle will trigger a row rotation (if in normal mode). Therefore, there should be such a request from time to time (e.g. one per driving cycle).
- As already mentioned, the data-NvM is organized in rows. During `Fee_Init`, the management information of all these rows (i.e. row headers/footers) is read and essential data is cached. Reading and caching the row's management information requires a significant amout of stack. If, for example, an ST SR6P6 host device is being used, there are 512 (= RBA_FEEFS3_PRV_CFG_FLS_SIZE/RBA_FEEFS3_PRV_CFG_ROW_SIZE_PHY, see *rba_FeeFs3_Prv_Cfg.h*) rows; for each of them, 6 bytes are pushed on to the stack. i.e. when the row initialization is entered, more than 3kB of stack must be available.
- The redundancy of double storage block data is not automatically repaired by the driver. It requires the external trigger via the function `Fee_Rb_BlockMaintenance`, preferably during shutdown.

9.9.3 Scheduling

It is important to consider all scheduling aspects described in the Fee documentation (see Chapter 2). rba_FeeFs3 has no sector reorganization, but nevertheless depends on a very fast background scheduling of the application software to execute a defragmentation after software updates.

9.9.4 OS Resources

The number of the active OS resources, and their usage, depends on the user configuration of the AUTOSAR System and Basic Software. It is the user's responsibility to ensure that the `ENTER` and `EXIT` resource API are implemented to disable/enable interrupts as required. Alternatively they must be configured in the OS with the correct task/ISR allocation.

The Rba_FeeFs3 module does not use any exclusive OS resources.

9.10 Hardware Requirements

Information relating to the hardware resources required by this module is not currently available.

10 rba_MemLib

Module name	rba_MemLib
Package	RTA-MEM
AUTOSAR Module ID	N/A
AUTOSAR Version	23-11

10.1 Introduction

The rba_MemLib module is not defined by the AUTOSAR standard. It is a module that is specific to RTA-BSW, and it consolidates into a central place the common functionality and utilities used by other RTA-Mem components.

The benefit of this centralization of common functions in the generated C code is that it improves the quality of code structure by avoiding duplication.

The rba_MemLib module does not require any configuration (because its configuration is forwarded), but it must be selected in CodeGen whenever any other RTA-Mem stack modules have been selected.

Module Configuration

This module can be generated by the RTA-BSW Configuration Generator.

For any unsupported automatic configuration, the module can be manually configured by the user.

10.2 Supported Features

10.2.1 General

rba_MemLib works as a library to store common functions for RTA-Mem stack modules.

10.2.2 Mem helper functionality

Helper functions for memory copy, compare and filling operations:

- rba_MemLib_MemCopy
- rba_MemLib_MemSet
- rba_MemLib_MemCompare

10.2.3 Timer functionality (low-level)

- rba_MemLib_GetTimerTicks
- rba_MemLib_ConvertTicksToUs
- rba_MemLib_ConvertUsToTicks

10.2.4 Timer functionality (higher-level)

To provide a free-running 1us-resolution 32bit timer.

- rba_MemLib_GetTimerUs
- rba_MemLib_UpdateTimerUs

10.2.5 Prototype switch

To enable/disable usage of prototypes in the RTA-Mem stack.

10.2.6 General Helper Functions (as inline)

- Min/Max
- Limit-Functions
- Conversion-Functions between Big/low-Endinaness

10.2.7 Measurement Support

The components of the Memstack provide the possibility to measure important RAM cells via an calibration/measurement tool. Only the step to generate the "raw" measurement information according to Autosar is considered. The receipt of an A2L file requires steps that are outside the scope of the Memstack (e.g. Rte run, ice_ara2l-tooling). See the Integration Advice and Configuration Advice for more details.

10.3 Extensions

10.3.1 Configuration Extensions

The following configuration parameters are available in version v12.7.0 in addition to those defined in AR v22-11

10.3.1.1 rba_MemLib

Short Name	rba_MemLib
Long Name	Memory: Library
Description	Configuration of the rba_MemLib component
Multiplicity	0..1
Default Value	-
Type	ECUC-MODULE-DEF

10.3.1.2 rba_MemLib/rba_MemLibCentral

Short Name	rba_MemLibCentral
Long Name	Memory Library Central Settings
Description	Central configuration parameters applying to the whole Mem stack
Multiplicity	0..1
Default Value	-

Type	ECUC-PARAM-CONF-CONTAINER-DEF

10.3.1.3 rba_MemLib/rba_MemLibCentral/rba_MemLibCentralEnableMeas

Short Name	rba_MemLibCentralEnableMeas
Long Name	Memory Library Enable Measurements
Description	Enable or disable the measurement capability for the Mem stack components (via calibratrion tooling). Please note that it is required to set parameter rba_MemLibEnumFlexible correctly.IF function is enabled.
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

10.3.1.4 rba_MemLib/rba_MemLibCentral/rba_MemLibCentralEnablePrototype

Short Name	rba_MemLibCentralEnablePrototype
Long Name	Memory Library Enable Prototypes
Description	Enable or disable prototypes in the Memstack. IF THIS IS SET TO TRUE THEN THE MEM PACKAGE CAN NO LONGER BE USED FOR SERIES PRODUCTION!
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

10.3.1.5 rba_MemLib/rba_MemLibCentral/rba_MemLibCrcSupport

Short Name	rba_MemLibCrcSupport
Long Name	Memory Library Crc Support
Description	Enable or disable the Crc calculation support function. Automatic Detection when Fee is available.
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

10.3.1.6 rba_MemLib/rba_MemLibCentral/rba_MemLibEnableBypassMem

Short Name	rba_MemLibEnableBypassMem
Long Name	Memory Library Enable MemSet/Copy Bypass
Description	Enable or disable the bypass functionality of the MemLib internal MemSet/MemCopy/MemCompare functions. If enabled then the integrator has to provide the mapping in rba_MemLib_Cfg_Mem_Callout.h.

Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

10.3.1.7 rba_MemLib/rba_MemLibCentral/rba_MemLibEnableCodeSizeOptimize

Short Name	rba_MemLibEnableCodeSizeOptimize
Long Name	Memory Library Enable CodeSize optimization
Description	Enable or disable the code size optimization (versus speed) of the internal routines
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

10.3.1.8 rba_MemLib/rba_MemLibCentral/rba_MemLibEnableExclusiveArea

Short Name	rba_MemLibEnableExclusiveArea
Long Name	Memory Library Enable Exclusive Area Generation via Rte
Description	Enable or disable the generation of Exclusive Area definitions via the Rte in the SchM header files (if disabled the related header has to be derived from the provided templates). Default: inverse of EcuCR-bLegacySchMInUse. Note: only possible to activate if Rte-usage is activated.
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

10.3.1.9 rba_MemLib/rba_MemLibCentral/rba_MemLibEnableMemAccBuild

Short Name	rba_MemLibEnableMemAccBuild
Long Name	Memory Library Enable MemAcc Build
Description	Enable or disable the use of MemAcc in the build instead of Fls/Eep. If enabled: Fls will be bypassed/eventually simulated by MemAcc. If disabled: Fls will be build, MemAcc is not used
Introduction	This switch is intended for the Fls/MemAcc layer. If enabled, the user of this layer still has the possibility to selected between the simulated Fls or the MemAcc interfaces

Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

10.3.1.10 rba_MemLib/rba_MemLibCentral/rba_MemLibEnableMemAccUsage

Short Name	rba_MemLibEnableMemAccUsage
Long Name	Memory Library Select MemAcc Usage
Description	Enable or disable the usage of the MemAcc on higher-level modules (Fee/Ea etc). If enabled: Higher-Level modules should use MemAcc interface. If disabled: Higher-Level modules shall NOT use MemAcc interface.
Introduction	This switch is intended for the Fee/Ea layer. It is evaluated only if rba_MemLibEnableMemAcc=true.
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

10.3.1.11 rba_MemLib/rba_MemLibCentral/rba_MemLibEnableMemMapGeneration

Short Name	rba_MemLibEnableMemMapGeneration
Long Name	Memory Library Enable MemMap Generation
Description	Enable or disable the generation of MemMap-files for the complete MemStack (if disabled the related MemMap files have to be manually be derived from provided templates). Default: enabled
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

10.3.1.12 rba_MemLib/rba_MemLibCentral/rba_MemLibEnableReEntryProtHigh

Short Name	rba_MemLibEnableReEntryProtHigh
Long Name	Memory Library Enable Reentrance Protection High

Description	Enable or disable the protection against re-entry in the MainFunction of the same component instance. If enabled, ensure that the MainFunction is always completed before switching Os-Modes (details in Docu). This parameter is used for NvM, MemIf, Fee,Ea layer. Default: enabled.
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

10.3.1.13 rba_MemLib/rba_MemLibCentral/rba_MemLibEnableReEntryProtLow

Short Name	rba_MemLibEnableReEntryProtLow
Long Name	Memory Library Enable Reentrance Protection Low
Description	Enable or disable the protection against re-entry in the MainFunction of the same component instance. If enabled, ensure that the MainFunction is always completed before switching Os-Modes (details in Docu). This parameter is used for MemAcc, Fls, Eep layer. Default: enabled
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

10.3.1.14 rba_MemLib/rba_MemLibCentral/rba_MemLibEnableSpecialModulo

Short Name	rba_MemLibEnableSpecialModulo
Long Name	Memory Library Enable Special Modulo Calculation
Description	Enable or disable the special modulo calculation instead of the standard C-function "%". Attention: this is only useful in some special uC architectures.
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

10.3.1.15 rba_MemLib/rba_MemLibCentral/rba_MemLibEnableTimer

Short Name	rba_MemLibEnableTimer
Long Name	Memory Library Enable Timer Usage

Description	Enable or disable of Timer integration in MemStack. This is necessary to enable for some timeout functions within the MemStack. If enabled, the integrator has to provide the interface to the physical timer via rba_MemLib_Cfg_Timer_Callout.h
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

10.3.1.16 rba_MemLib/rba_MemLibCentral/rba_MemLibEnableVecuCycle

Short Name	rba_MemLibEnableVecuCycle
Long Name	Memory Library Enable Vecu Cycle Count features
Description	Enable or disable cycle count features of VECU per default. This has no impact on the non-VECU variant. Default: disabled (except internally)
Introduction	This switch is for the VECU environment only. Even if activated it is still possible to individually selecet a different setting per device.
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

10.3.1.17 rba_MemLib/rba_MemLibCentral/rba_MemLibEnableVecuError

Short Name	rba_MemLibEnableVecuError
Long Name	Memory Library Enable Vecu Error features
Description	Enable or disable the error simulation features of VECU per defaut. This has no impact on the non-VECU variant. Default: disabled (except internally)
Introduction	This switch is for the VECU environment only. Even if activated it is still possible to individually selecet a different setting per device.
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

10.3.1.18 rba_MemLib/rba_MemLibCentral/rba_MemLibEnumFlexible

Short Name	rba_MemLibEnumFlexible
Long Name	Memory Library use flexible enum size (1/2/4byte)

Description	Enum can have a compiler or even option specific behavior.In order to define the behavior it is necessary to select between: FALSE = fixed size=4byte (C-Standard) TRUE = flexible size (1/2/4 byte) - Note: This setting is for a target compiler and not for a simulation environment. Note: parameter only needed if the measurement feature is enabled.
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

10.3.1.19 rba_MemLib/rba_MemLibCentral/rba_MemLibForcedArRelease

Short Name	rba_MemLibForcedArRelease
Long Name	Memory Library Forced AR Version
Description	Per default the AR version is determined from EcuC component. However it is also possible to overrule this setting for the MemStack by using this selection.
Multiplicity	0..1
Default Value	-
Enumeration Literals	AR42_Rev2, AR45_Rev0, AR46_Rev0, AR47_Rev0, AR48_Rev0, AR49_Rev0, AR410_Rev0
Type	ECUC-ENUMERATION-PARAM-DEF

10.3.1.20 rba_MemLib/rba_MemLibCentral/rba_MemLibInternalUsage

Short Name	rba_MemLibInternalUsage
Long Name	Memory Library Internal usage information
Description	(do not activate)
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

10.3.1.21 rba_MemLib/rba_MemLibCentral/rba_MemLibVersionInfoApi

Short Name	rba_MemLibVersionInfoApi
Long Name	Memory Library Enable Version Info API
Description	Enable or disable version information
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

10.3.1.22 rba_MemLib/rba_MemLibPublished

Short Name	rba_MemLibPublished
Long Name	Memory Library Published Parameters
Description	Generated configuration parameters which can be used by other Mem components
Multiplicity	0..1
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

10.3.1.23 rba_MemLib/rba_MemLibPublished/rba_MemLibPublishedARMajorVersion

Short Name	rba_MemLibPublishedARMajorVersion
Long Name	MemLib Published AR Release Major Version
Description	Generated AR release major version info
Multiplicity	0..1
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

10.3.1.24 rba_MemLib/rba_MemLibPublished/rba_MemLibPublishedARMinorVersion

Short Name	rba_MemLibPublishedARMinorVersion
Long Name	MemLib Published AR Release Minor Version
Description	Generated AR release minor version info
Multiplicity	0..1
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

10.3.1.25 rba_MemLib/rba_MemLibPublished/rba_MemLibPublishedARRevisionVersion

Short Name	rba_MemLibPublishedARRevisionVersion
Long Name	MemLib Published AR Release Revision Version
Description	Generated AR release revision version info
Multiplicity	0..1
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

10.3.1.26 rba_MemLib/rba_MemLibPublished/rba_MemLibPublishedArxmlVersion

Short Name	rba_MemLibPublishedArxmlVersion
Long Name	MemLib Published arxml header version
Description	String used to generate the arxml files with a dedicated DTD version. Example: 4-0-3, 4-2-2
Multiplicity	0..1
Default Value	-
Type	ECUC-STRING-PARAM-DEF

10.3.1.27 rba_MemLib/rba_MemLibPublished/rba_MemLibPublishedEnableExclusiveArea

Short Name	rba_MemLibPublishedEnableExclusiveArea
Long Name	MemLib Published Exclusive Area Generation
Description	Enable or disable the generation of Exclusive Area handling for the Memstack. If enabled: Rte tool will generate the SchM header files. If disabled: The exclusive-area definition files have to be manually derived from provided templates.
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

10.3.1.28 rba_MemLib/rba_MemLibPublished/rba_MemLibPublishedEnableMeas

Short Name	rba_MemLibPublishedEnableMeas
Long Name	MemLib Published Enable Measurement
Description	Enable or disable of Measurement in MemStack components (via calibration tooling)
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

10.3.1.29 rba_MemLib/rba_MemLibPublished/rba_MemLibPublishedEnableMemAccBuild

Short Name	rba_MemLibPublishedEnableMemAccBuild
Long Name	MemLib Published Enable MemAcc Build

Description	Enable or disable the use of MemAcc in build instead of Fls/Eep layer. If enabled: Fls will be bypassed/eventually simulated by MemAcc. If disabled: Fls will be build, MemAcc is not used
Introduction	This switch is intended for the Fls/MemAcc layer. If enabled, the user of this layer still has the possibility to selected between the simulated Fls or the MemAcc interfaces
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

10.3.1.30 rba_MemLib/rba_MemLibPublished/rba_MemLibPublishedEnableMemAccUsage

Short Name	rba_MemLibPublishedEnableMemAccUsage
Long Name	MemLib Published Select MemAcc Usage
Description	Enable or disable the usage of the MemAcc on higher-level modules (Fee/Ea etc). If enabled: Higher-Level modules should use MemAcc interface. If disabled: Higher-Level modules shall NOT use MemAcc interface.
Introduction	This switch is intended for the Fee/Ea layer.
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

10.3.1.31 rba_MemLib/rba_MemLibPublished/rba_MemLibPublishedEnableMemMapGeneration

Short Name	rba_MemLibPublishedEnableMemMapGeneration
Long Name	MemLib Published MemMap Generation
Description	Enable or disable the generation of MemMap-files for the complete MemStack (if disabled the related MemMap files have to be manually be derived from provided templates). Default: enabled
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

10.3.1.32 rba_MemLib/rba_MemLibPublished/rba_MemLibPublishedEnablePrototype

Short Name	rba_MemLibPublishedEnablePrototype

Long Name	MemLib Published Enable Prototype
Description	Enable or disable prototype handling for the Memstack
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

10.3.1.33 rba_MemLib/rba_MemLibPublished/rba_MemLibPublishedEnableReEntryProtHigh

Short Name	rba_MemLibPublishedEnableReEntryProtHigh
Long Name	MemLib Published Enable Reentrance Protection High
Description	Enable or disable the protection against re-entry in the MainFunction of the same component instance. If enabled, ensure that the MainFunction is always completed before switching Os-Modes (details in Docu). This parameter is used for NvM, MemIf, Fee,Ea layer. Default: enabled.
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

10.3.1.34 rba_MemLib/rba_MemLibPublished/rba_MemLibPublishedEnableReEntryProtLow

Short Name	rba_MemLibPublishedEnableReEntryProtLow
Long Name	MemLib Published Enable Reentrance Protection Low
Description	Enable or disable the protection against re-entry in the MainFunction of the same component instance. If enabled, ensure that the MainFunction is always completed before switching Os-Modes (details in Docu). This parameter is used for MemAcc, Fls, Eep layer. Default: enabled
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

10.3.1.35 rba_MemLib/rba_MemLibPublished/rba_MemLibPublishedEnableRte

Short Name	rba_MemLibPublishedEnableRte
Long Name	MemLib Published Enable Rte
Description	Indication whether the Rte is considered to be enabled or not.

Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

10.3.1.36 rba_MemLib/rba_MemLibPublished/rba_MemLibPublishedEnableRteFound

Short Name	rba_MemLibPublishedEnableRteFound
Long Name	MemLib Published Enable Rte Switch Found
Description	Indication whether the Rte is actively enabled or disabled via EcuC
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

10.3.1.37 rba_MemLib/rba_MemLibPublished/rba_MemLibPublishedEnableTimer

Short Name	rba_MemLibPublishedEnableTimer
Long Name	MemLib Published Enable Timer Usage
Description	Enable or disable of Timer integration in MemStack. If enabled, the integrator has provided the timer low-level interfaces as integration code.
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

10.3.1.38 rba_MemLib/rba_MemLibPublished/rba_MemLibPublishedEnableVecu

Short Name	rba_MemLibPublishedEnableVecu
Long Name	MemLib Published Force Enable Vecu
Description	true=Forced unconditional enable of the Vecu-support within the Memstack (via Ecuc), false=per default not actived, but activatable via compiler switch RBA_VECU_ENABLED.
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

10.3.1.39 rba_MemLib/rba_MemLibPublished/rba_MemLibPublishedEoLStyle

Short Name	rba_MemLibPublishedEoLStyle

Long Name	MemLib Published Eol Style
Description	End-of-line character code of ftl. 0=Unix(\n), 1=Win (\r\n), 2=Mac (\r)
Multiplicity	0..1
Value Range	0..2
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

10.3.1.40 rba_MemLib/rba_MemLibPublished/rba_MemLibPublishedInternalUsage

Short Name	rba_MemLibPublishedInternalUsage
Long Name	MemLib Published Internal usage information
Description	Internal Usage information
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

10.3.1.41 rba_MemLib/rba_MemLibPublished/rba_MemLibPublishedSizeEnum16

Short Name	rba_MemLibPublishedSizeEnum16
Long Name	MemLib Published enum16 size
Description	Size of an 8bit enum (1byte or 4 bytes) of the target controller
Multiplicity	0..1
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

10.3.1.42 rba_MemLib/rba_MemLibPublished/rba_MemLibPublishedSizeEnum8

Short Name	rba_MemLibPublishedSizeEnum8
Long Name	MemLib Published enum8 size
Description	Size of an 16bit enum (2byte or 4 bytes) of the target controller
Multiplicity	0..1
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

10.3.1.43 rba_MemLib/rba_MemLibPublished/rba_MemLibPublishedSwMajorVersion

Short Name	rba_MemLibPublishedSwMajorVersion

Long Name	MemLib Published Sw Major Version
Description	Generated software major version info
Multiplicity	0..1
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

10.3.1.44 rba_MemLib/rba_MemLibPublished/rba_MemLibPublishedSwMinorVersion

Short Name	rba_MemLibPublishedSwMinorVersion
Long Name	MemLib Published Sw Minor Version
Description	Generated software minor version info
Multiplicity	0..1
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

10.3.1.45 rba_MemLib/rba_MemLibPublished/rba_MemLibPublishedSwPatchVersion

Short Name	rba_MemLibPublishedSwPatchVersion
Long Name	MemLib Published Sw Patch Version
Description	Generated software patch version info
Multiplicity	0..1
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

10.3.1.46 rba_MemLib/rba_MemLibValidated

Short Name	rba_MemLibValidated
Long Name	Memory Library Validation Parameters
Description	Generated configuration parameters which can be used by other Mem components
Multiplicity	0..1
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

10.3.1.47 rba_MemLib/rba_MemLibValidated/rba_MemLibValidatedSwMajorVersion

Short Name	rba_MemLibValidatedSwMajorVersion
Long Name	Validated Mem-SwMajorVersion

Description	Validated software major version to check differences from an old to the current software version. The current SW-version is available in parameter rba_MemLibPublishedSwMajorVersion.
Multiplicity	0..1
Value Range	0..100
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

10.3.2 Additional Services

The following API are available in version v12.7.0 in addition to those defined in AR v22-11

- There are no additional API defined in this version.

10.4 Limitations

There are no known limitations in the rba_MemLib module.

10.5 Exclusions

The following configuration parameters from AR v22-11 are not supported in version v12.7.0.

- There are no known exclusions in this version.

10.6 Deviations

Configuration parameters which differ from the AR v22-11 specification are listed below.

- There are no known deviations in this version.

10.7 API Definition

The rba_MemLib module does not expose a public API.

10.8 Configuration Advice

10.8.1 General

Modules in the RTA-Mem stack use functions in rba_MemLib, so rba_MemLib must be selected when generating other modules in the RTA-Mem stack.

Most of the configuration of rba_MemLib has been moved from the callout functions to configuration parameters. The callout header files are now optional and are only necessary if the related feature is activated.

If in the past you have used the timer or memory-operation bypass functionality, you will need to create new header files from the provided template files.

10.8.2 Measurement Support

The following configuration is required for Measurement Support:

- Set rba_MemLibCentralEnableMeas as TRUE to enable the measurement functionality.
- The rba_MemLibEnumFlexible parameter must be set depending on your compiler setting.

10.9 Integration Advice

10.9.1 Init Functions

rba_MemLib does not have an Init function.

10.9.2 DeInit Functions

rba_MemLib does not have a DeInit function.

10.9.3 Scheduled Functions

rba_MemLib does not have a scheduled function, but it does have an optional interface (rba_MemLib_UpdateTimerUs) which can be integrated into RTA-Mem stack functions as well as in any cyclic tasks. Calling rba_MemLib_UpdateTimerUs will ensure that the internal timers are updated.

In the rare case when timer measurement within rba_MemLib is activated and the RTA-Mem stack functionality is not serviced for more than 500ms, an update of the rba_MemLib_UpdateTimerUs functionality should be scheduled from a cyclic Os-task (e.g. 100ms) to ensure that the internal timers are updated before an overflow occurs.

10.9.4 Code Generation dependency

When integrating the RTA-Mem stack, the rba_MemLib module must be selected during CodeGen.

10.9.5 Implementation of Timer Callout

Timer Callout is implemented in rba_MemLib_Cfg_Timer_Callouts.h

1. Select a suitable hardware timer and modify the SW interface (“rba_MemLib_Callout_GetTimerTicks”) to access the timer by fulfilling the following requirements:

- The Timer-reading has to behave like a free-running, incrementing, wrap-around hardware timer (incremented with every timer tick and wrapping around of the highest value to 0). If the system has a decrementing free-running timer, then you might be able to service the interface by inverting the timer values. The highest value has to be a power-of-2 value minus 1.

- Timer-reading has to have a resolution of 1us or smaller.
- The value range until the Timer-reading wraps around has to be >= 1000000us = 1s.
- Timer-reading has to have a value range of at least 20bits (maximum 32bits). The value needs to be set in the "RBA_MEMLIB_TIMERBITSIZE" macro.

2. From the above Timer-reading properties, you have to provide the following conversion functions:

- rba_MemLib_Callout_ConvertTimerTicksToUs
- rba_MemLib_Callout_ConvertUsToTimerTicks.

3. There is no need to perform an overflow/limitation handling in these functions if the above requirements are met.
4. Remember to configure the selected timer to start within the project integration code or within another function.

5 If you do not need any timer functionality in the context of the RTA-MEM stack, you can:

- Set RBA_MEMLIB_USE_TIMER to STD_OFF (the default is STD_ON).
- Always return 0u in the Timer callout interfaces.

10.9.5.1 Checking the Implementation

In order to check that the implementation of the Timer callout has been successful, it is possible within the rba_MemLib_UpdateTimerUs function to activate a timer configuration check during the initialization. This is done by setting RBA_MEMLIB_USE_TIMER_CHECK to STD_ON.

If the check is successful and the timer is running, then the rba_MemLib internal 1us-timer will start to increment, otherwise it will remain at 0.

The following checks are performed in SW:

- Timer tick resolution has to be <= 1us (based on rba_MemLib_Callout_ConvertUsToTimerTicks)
- Timer tick resolution has to be >= 1/4000s (based on rba_MemLib_Callout_ConvertUsToTimerTicks)
- Timer tick value range has to ensure that 1s can be displayed (based on rba_MemLib_Callout_ConvertUsToTimerTicks)
- Timer has to be started running (value reading > 0, and is changing)

You should check the values of rba_MemLib_tiTimerUs (which should increment) to verify the correct initialization (e.g. with 10 seconds, the timer value will change by 10000000).

10.9.6 Implementation of other TimerUs

rba_MemLib supports the direct use of an externally available 1us/LSB free-running up-counting timer (32bit).

To activate this, you need to do the following within the rba_MemLib_Cfg_Callout.h header:

- Set the #define RBA_MEMLIB_BYPASS_TIMER_US to STD_ON
- Provide the timer value in 1us resolution (full 32bit value range) within LOCAL_INLINE uint32 rba_MemLib_Callout_BypassTimerUs(void)

10.9.7 Measurement Support Activation

The following is required for the activation of Measurement Support:

- The project must include rba_ArxmlGen.
- Measured components have to be registered within the Rte (otherwise they are ignored). This is done via the EcucModuleConfigurationValues structure in the Rte*EcucValues_SWCD.arxml file.

10.9.8 rba_MemLib_Cfg_Sync.h

The "rba_MemLib_Cfg_Sync.h" header file needs to be included (template available), which provides an interface to the project/compiler specific realization of a memory barrier used by synchronization functions.

RBA_MEMLIB_MSYNC() needs to be defined in a way that the CPU's memory pipeline is completed after this macro.

If the rba_BswSrv component is available in the project, there can be a direct mapping:

- #define RBA_MEMLIB_MSYNC() RBA_BSWSRV_MSYNC()

If the project does not use the RTA-MEM stack in a multi-core configuration, it is possible to define the macro as empty:

- #define RBA_MEMLIB_MSYNC()

10.9.9 OS Resources

The number of the active OS resources, and their usage, depends on the user configuration of the AUTOSAR System and Basic Software. It is the user's responsibility to ensure that the `ENTER` and `EXIT` resource API are implemented to disable/enable interrupts as required. Alternatively they must be configured in the OS with the correct task/ISR allocation.

The following is a list of all exclusive areas that this module might use regardless of the user configuration. It is the job of the integrator to ensure that the implementation of these exclusive areas is correctly defined. This implementation can be done using the RTE exclusive area configuration wizard or by manual imple-

mentationof the corresponding BSW API calls to ensure exclusivity.

The Rba_MemLib module uses the following OS resources:

- rba_MemLib_TimerUs

10.10 Hardware Requirements

Information relating to the hardware resources required by this module is not currently available.

11 rba_MemWrprMemAcc

Module name	rba_MemWrprMemAcc
Package	RTA-MEM
AUTOSAR Module ID	N/A
AUTOSAR Version	23-11

11.1 Introduction

The *rba_MemWrprMemAcc* module provides a wrapper/decoupling layer to abstract and harmonize the behavior of different low-level Mem drivers. This is required because the interpretation of the AUTOSAR specification in third-party MemAcc and/or Mem drivers is not always consistent, and it might also differ from the corresponding RTA-BSW interpretation/implementation.

When enabled, rba_MemWrprMemAcc provides extensions for the MemAcc interfaces. Inside these wrapper functions, third-party MemAcc/Mem driver implementations can be adapted to the needs of the higher-level Mem stack. An implementation for the standard needs is provided, but the integrator is free to modify this low-complexity wrapper to solve specific incompatibilities.

When disabled, rba_MemWrprMemAcc routes its function calls directly to the MemAcc interfaces (without any overhead).

rba_MemWrprMemAcc interacts between higher-level Mem components like Fee/Ea etc., and the lower-level components MemAcc/Mem driver.

11.2 Supported Features

According to AUTOSAR, a Mem stack using a MemAcc/Mem driver must be able to handle (uncorrectable) ECC errors for various jobs. However, the RTA-BSW Mem stack assumes that ECC detection (or the corresponding core exception/trap) is *disabled*. If this assumption cannot be met (i.e. that ECC detection is enabled), the RTA-BSW implementation of MemAcc users (e.g. the Fee module) require that any MemAcc read request provides as much data as possible, *including that of uncorrectable ECC errors*.

Therefore, MemAcc shall continue to request any remaining unread data from the Mem driver after such an event has occurred (otherwise, block losses might happen). Incorrect data, which is potentially falsified by ECC errors, will be detected by CRC methods inside the higher-level components. When enabled, rba_MemWrprMemAcc can be used to avoid missing/wrong ECC error handling.

rba_MemWrprMemAcc provides wrapper functions for MemAcc APIs, with the same function parameters and the same return value (see Section 11.7). The wrappers are re-entrant for different users/address areas, but not so for the same user/address area.

If `rba_MemWrprMemAcc_Read` is called (with length `nrBytTot_u32`) and reports an uncorrectable ECC error, the call is separated and repeated in smaller portions (of length `nrBytPrtn_u32`) until the error location is reached. The affected parts of the buffer, which are potentially falsified by the error, are then filled by a replace-

mentvalue (defined by `dataRepl_u8`). The re-read then continues until the overall MemAcc read is complete (i.e. the buffer is filled with the complete length `nrBytTot_u32`).

11.2.1 Mapping of MemAcc_GetJobResult Return Values

The following table shows the return values of `MemAcc_GetJobResult` in case of an ECC error detection:

MemAcc API	Return value for MEMACC_MEM_ECC_CORRECTED	Return value for MEMACC_MEM_ECC_UNCORRECTED
`MemAcc_Write`	MEMACC_MEM_OK	MEMACC_MEM_FAILED
`MemAcc_Erase`	MEMACC_MEM_OK	MEMACC_MEM_FAILED
`MemAcc_Compare`	MEMACC_MEM_OK	MEMACC_MEM_INCONSISTENT
`MemAcc_BlankCheck`	MEMACC_MEM_OK	MEMACC_MEM_INCONSISTENT
`MemAcc_Read`	MEMACC_MEM_OK	MEMACC_MEM_ECC_UNCORRECTED (*)

(*) - If `rba_MemWrprMemAcc_Read` reports an uncorrectable ECC error, `rba_MemWrprMemAcc_GetJobResult` returns *MEMACC_MEM_ECC_UNCORRECTED*, unless the error disappears during re-read (in which case *MEMACC_MEM_OK* is returned). Depending upon the User's needs, the call may be considered a success, or further error handling can follow.

11.3 Extensions

11.3.1 Configuration Extensions

The following configuration parameters are available in version v12.7.0 in addition to those defined in AR v22-11

11.3.1.1 rba_MemWrprMemAcc

Short Name	rba_MemWrprMemAcc
Long Name	Memory: Memory Access Wrapper
Description	Configuration of the MemAcc wrapper
Multiplicity	0..1
Default Value	-
Type	ECUC-MODULE-DEF

11.3.1.2 rba_MemWrprMemAcc/rba_MemWrprMemAccGeneral

Short Name	rba_MemWrprMemAccGeneral
Long Name	General Parameters
Description	Container for general configuration parameters of the MemAcc wrapper
Multiplicity	0..1
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

11.3.1.3 rba_MemWrprMemAcc/rba_MemWrprMemAccGeneral/ rba_MemWrprMemAccEna

Short Name	rba_MemWrprMemAccEna
Long Name	Enable MemAcc wrapper

Description	Pre-processor switch to enable/disable the wrapper on top of MemAcc. When enabled, the wrapper provides rba_MemWrprMemAcc extensions for the MemAcc interfaces. Inside these wrapper functions, systems and/or 3rd party MemAcc/Mem driver implementations can be adapted to the needs of the ETAS memstack. A typical use case is the handling of ECC errors, which are reported for different MemAcc jobs. The ETAS memstack assumes that ECC detection (or the corresponding core exception/trap) is disabled. If this assumption can't be met, it is important to know that the ETAS implementation of MemAcc users (like the Fee) requires that any MemAcc read request provides as much data as possible - even in case of uncorrectable ECC errors! In particular, MemAcc shall continue to request any remaining unread data from the Mem driver after such an event has occurred (otherwise, block losses might happen).
Introduction	true: Wrapper functions are provided. False: Wrapper functions are not provided/are mapped to the MemAcc interfaces.
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

11.3.2 Additional Services

The following APIs are available in version v12.7.0 in addition to those defined in AR v22-11

11.3.2.1 rba_MemWrprMemAcc_Read

Syntax	`Std_ReturnType rba_MemWrprMemAcc_Read(MemAcc_AddressAreaIdType idAdrArea_o, MemAcc_AddressType adrSource_uo, MemAcc_DataType *data_pu8, uint32 nrBytTot_u32, uint32 nrBytPrtn_u32, uint8 dataRepl_u8)`
Parameters In	`idAdrArea_o`: Numeric identifier of address area. `adrSource_uo`: Read address in logical address space. `nrBytTot_u32`: Read length in bytes. `nrBytPrtn_u32`: Length of portion to re-read when searching uncorrectable ECC error. `dataRepl_u8`: Replacement value for bytes affected by uncorrectable ECC error.
Parameter Out	`data_pu8`: Destination memory pointer/buffer to store the read data.

Return Values	`E_OK`: The requested job has been accepted by MemAcc. `E_NOT_OK`: The requested job has not been accepted by MemAcc.
Description	Triggers a MemAcc read job to copy data from the source address into the referenced destination data buffer.

11.4 Limitations

11.5 Exclusions

11.6 Deviations

11.7 API Definition

Table 11.7.rba_MemWrprMemAcc supported API functions

`rba_MemWrprMemAcc_Cancel`	Mapped to `MemAcc_Cancel`.
`rba_MemWrprMemAcc_Write`	Mapped to `MemAcc_Write`.
`rba_MemWrprMemAcc_Erase`	Mapped to `MemAcc_Erase`.
`rba_MemWrprMemAcc_Compare`	Mapped to `MemAcc_Compare`.
`rba_MemWrprMemAcc_BlankCheck`	Mapped to `MemAcc_BlankCheck`.
`rba_MemWrprMemAcc_GetJobStatus`	Mapped to `MemAcc_GetJobStatus`.
`rba_MemWrprMemAcc_GetJobResult`	Mapped to `MemAcc_GetJobResult`.

11.8 Configuration Advice

NOTE

Before configuring this module, please note that some of the default values used by ConfGen to produce the default ECU Configuration can be changed, and this is done by either adding a Conf-Gen Enhancer Mapping (CEM) or by adding a rule to an *algo.properties* file. For more information, see the “Configuration Generation” section of the RTA-CAR User Guide.

11.9 Integration Advice

rba_MemWrprMemAcc is not complex in how it resolves potential incompatibility in known/potential ECC error handling. Where necessary, this (tested) implementation can be used as provided.

However, there might be completely different and/or additional adaptation needs. Therefore, the integrator may take responsibility/ownership of the wrapper in order to change/extend its features. rba_MemWrprMemAcc’s configuration options and signatures are fixed, but the C source code may be changed.

There is no need to enable (or even integrate) rba_MemWrprMemAcc if there are no adaptation needs (and the ECC detection or the corresponding core exception/trap is disabled).

11.9.1 Initialization

No initialization is required/possible.

11.9.2 OS Resources

The number of the active OS resources, and their usage, depends on the user configuration of the AUTOSAR System and Basic Software. It is the user's responsibility to ensure that the `ENTER` and `EXIT` resource API are implemented to disable/enable interrupts as required. Alternatively they must be configured in the OS with the correct task/ISR allocation.

The Rba_MemWrprMemAcc module does not use any exclusive OS resources.

11.10 Hardware Requirements

Information relating to the hardware resources required by this module is not currently available.

12 Glossary

ASIL
Automotive Safety Integrity Level

CRC
Cyclic Redundancy Check

Dem
AUTOSAR Diagnostic Event Manager module

Ea AUTOSAR EEPROM Abstraction module

ECC
Error Correction Code

EcuM
AUTOSAR ECU State Manager module

Eep
AUTOSAR EEPROM Driver module

EEPROM
Electrically Erasable and Programmable ROM

Fee
AUTOSAR Flash EEPROM Emulation module

Fls AUTOSAR Flash Driver module

FOTA
Firmware Over The Air

LSB
Least Significant Bit

MemIf
AUTOSAR Memory Abstraction Interface module

NvBlock
Basic Storage Object and a mandatory part of an NVRAM Block

NvM
AUTOSAR NVRAM Manager module

NvMBlock
Separately manageable unit in an ECU's persistent data storage facility organized by NvM

NVRAM
Non-Volatile RAM

NVRAM Block
Management unit represented in the NvM module as an NvMBlock.

RAM
Random Access Memory

rba_EaFs2
RTA-BSW module, a sub-module of Ea that implements the Migration concept to handle EEPROM layout changes

rba_FeeFs1x
RTA-BSW module, a sub-module of Fee that defines Fee 1.0x layout and behavior

rba_MemLib
RTA-BSW module, consolidates into a central place the common functionality and utilities used by other RTA-Mem components

ROM
Read Only Memory

SPI Serial Peripheral Interface

us Microsecond